

Mechanical Application Programming Interface (API) Developer's Guide

Product Development
MCAD Market Group
May 2001

Copyright © 1997-2001 Autodesk, Inc.

All Rights Reserved

This publication, or parts thereof, may not be reproduced in any form, by any method, for any purpose.

AUTODESK, INC. MAKES NO WARRANTY, EITHER EXPRESSED OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE, REGARDING THESE MATERIALS AND MAKES SUCH MATERIALS AVAILABLE SOLELY ON AN “AS-IS” BASIS.

IN NO EVENT SHALL AUTODESK, INC. BE LIABLE TO ANYONE FOR SPECIAL, COLLATERAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES IN CONNECTION WITH OR ARISING OUT OF PURCHASE OR USE OF THESE MATERIALS. THE SOLE AND EXCLUSIVE LIABILITY TO AUTODESK, INC., REGARDLESS OF THE FORM OF ACTION, SHALL NOT EXCEED THE PURCHASE PRICE OF THE MATERIALS DESCRIBED HEREIN.

Autodesk, Inc. reserves the right to revise and improve its products as it sees fit. This publication describes the state of this product at the time of its publication, and may not reflect the product at all times in the future.

Autodesk Trademarks

The following are registered trademarks of Autodesk, Inc., in the USA and/or other countries: 3D Plan, 3D Props, 3D Studio, 3D Studio MAX, 3D Studio VIZ, 3DSurfer, ActiveShapes, Actrix, ADE, ADI, Advanced Modeling Extension, AEC Authority (logo), AEC-X, AME, Animator Pro, Animator Studio, ATC, AUGI, AutoCAD, AutoCAD Data Extension, AutoCAD Development System, AutoCAD LT, AutoCAD Map, Autodesk, Autodesk Animator, Autodesk (logo), Autodesk MapGuide, Autodesk University, Autodesk View, Autodesk WalkThrough, Autodesk World, AutoLISP, AutoShade, AutoSketch, AutoSurf, AutoVision, Biped, bringing information down to earth, CAD Overlay, Character Studio, Design Companion, Drafix, Education by Design, Fire, Flame, Flint, Frost, Generic, Generic 3D Drafting, Generic CADD, Generic Software, Geodyssey, Heidi, HOOPS, Hyperwire, Inferno, Inside Track, Kinetix, MaterialSpec, Mechanical Desktop, Mountstone, Multimedia Explorer, NAAUG, ObjectARX, Office Series, Opus, PeopleTracker, Physique, Planix, Powered with Autodesk Technology, Powered with Autodesk Technology (logo), RadioRay, Rastation, Riot, Softdesk, Softdesk (logo), Solution 3000, Stone, Stream, Tech Talk, Texture Universe, The AEC Authority, The Auto Architect, TinkerTech, Vapour, VISION*, WHIP!, WHIP! (logo), Wire, Woodbourne, WorkCenter, and World-Creating Toolkit.

The following are trademarks of Autodesk, Inc., in the USA and/or other countries: 3D on the PC, ACAD, Advanced User Interface, AEC Office, AME Link, Animation Partner, Animation Player, Animation Pro Player, A Studio in Every Computer, ATLAST, Auto-Architect, AutoCAD Architectural Desktop, AutoCAD Architectural Desktop Learning Assistance, AutoCAD Learning Assistance, AutoCAD LT Learning Assistance, AutoCAD Simulator, AutoCAD SQL Extension, AutoCAD SQL Interface, Autodesk Animator Clips, Autodesk Animator Theatre, Autodesk Device Interface, Autodesk Inventor, Autodesk PhotoEDIT, Autodesk Software Developer's Kit, Autodesk View DwgX, AutoFlix, AutoPAD, AutoSnap, AutoTrack, Built with ObjectARX (logo), ClearScale, Combustion, Concept Studio, Content Explorer, cornerStone Toolkit, Dancing Baby (image), Design 2000 (logo), DesignCenter, Design Doctor, Designer's Toolkit, DesignProf, DesignServer, Design Your World, Design Your World (logo), Discreet, DWG Linking, DWG Unplugged, DXF, Extending the Design Team, FLI, FLIC, GDX Driver, Generic 3D, Heads-up Design, Home Series, Kinetix (logo), Lightscape, ObjectDBX, Ooga-Chaka, Photo Landscape, Photoscape, Plugs and Sockets, PolarSnap, ProjectPoint, Pro Landscape, QuickCAD, SchoolBox, Simply Smarter Diagramming, SketchTools, Suddenly Everything Clicks, Supportdesk, The Dancing Baby, Transform Ideas Into Reality, Visual LISP, Visual Syllabus, Volo, and Where Design Connects.

Third Party Software Credits

ACIS is a registered trademark of SPATIAL TECHNOLOGY, INC.

Portions licensed from D-Cubed Ltd. DCM-2D is a trademark of D-Cubed LTD. DCM-2D Copyright D-Cubed Ltd. 1989-1997

The license management portion of this product is based on Élan License Manager © 1997, Élan Computer Group, Inc. All rights reserved. Élan License Manager is a trademark of Élan Computer Group, Inc.

International CorrectSpell™ English spelling correction system © 1993 by INSO Corporation. All rights reserved. Reproduction or disassembly of embodied algorithms or database prohibited.

SmartHeap memory manager Copyright © 1991-1996 by Arthur D. Applegate. All rights reserved. All other brand names, product names or trademarks belong to their respective holders.

GOVERNMENT USE

Use, duplication, or disclosure by the U. S. Government is subject to restrictions as set forth in FAR 12.212 (Commercial Computer Software-Restricted Rights) and DFAR 227.7202 (Rights in Technical Data and Computer Software), as applicable.

TABLE OF CONTENTS

ALPHABETICAL LISTING OF FUNCTIONS 23

CHAPTER 1 OVERVIEW 34

API Goals 34

Interaction with Other Libraries 35
AcBr Library 35
AcGe Library 36
ObjectARX™ Libraries 36

Supported Platforms 36

CHAPTER 2 MCAD API FEATURES 37

Uniform Object Referencing 37

Unified API for All Modules 37

Powerful Geometric Abstraction 37

Support for Geometry Change Notification 37

Referential Integrity Across Parametric Regenerations 38

Attribute Support 38

Synergistic Integration with Other Autodesk APIs 38

Support for Asynchronous Releases of All Components 38

Utilities for ObjectARX 38

Mechanical Desktop Part and Surface Modeling 38
Object Keys 39
Object Key Hierarchy 39

Using the MCAD API Referencing Mechanism 41

Summary 41

CHAPTER 3 MCAD API GUIDELINES 43

Attribute Creation in the MCAD API 43
Basic Functionality 43
Attribute Creation, Interoperability, and Customizability 43
Attribute Management 44
Attribute Ownership 44
Attribute Associativity 44
The AmiAttribute Class 44
Public Methods 44

Integration of MDT System Attributes and MCADAPI Attributes 46

MCAD API Instantiable Attributes 47
Notation 47
Problem 47
Instantiable Attributes 47
Defining Attribute "Classes" 47
Example 48
Attributes From Other Applications 49
Attributes To and From an Attribute Definition File 49
Example 49
Creating Instantiable Attribute Instances 49
Transactions 50
Accessing Attribute Fields 50
Standard Attributes 50
Function Reference 50

MCAD API Object Descriptors 52
AmiDescrip 52
AmiValue 53
Basic Functionality for Feature Descriptors 53
Descriptor Creation 54
Informers 54
AmiSelection 55
Drawing Manager Descriptors 55
Scene Component Descriptors 56
Constraint Descriptors 56
File Descriptors 56
Event Reaction Descriptors 56
Class Hierarchy 58
Class Hierarchy: Chart Two 59
Class Hierarchy: Chart Three 60
Composite Features 61

AmiStatus Overview 62
CLASS AmiStatus 63

File Structures 64
MDT Part Only Files 64

Coding Practices 66
Return Values 66
Memory Allocation & Freeing 66
MCADAPI Object Memory Management 66

Coding Examples **67**

CHAPTER 4 MCAD API FUNCTIONS 68

Detailed Description of Functions **68**
Common **68**
Handling Pick Objects **68**
AmiStatus amiFillPickObj 68
AmiStatus amiGetPickInfo 68
AmiStatus amiPick 69
Creating Keys **69**
AmiStatus amiGetInferGeom 69
AmiStatus amiGetKeyFromId 70
AmiStatus amiGetKeyFromPath 70
AmiStatus amiGetKeyFromPick 71
AmiStatus amiGetGeomKey 71
AmiStatus amiInferGeomKey 71
Handling Keys **72**
AmiStatus amiAreKeysEquivalent 72
AmiStatus amiCopyKey 72
AmiStatus amiGetKeyType 73
AmiStatus amiGetAssociatedDbs 73
AmiStatus amiIsKeyKindOf 73
AmiStatus amiIsKeyNull 74
AmiStatus amiReadKey 74
AmiStatus amiRefocusKey 74
AmiStatus amiWriteKey 75
Entity Highlighting **75**
AmiStatus amiHighlight 75
Handling Geometry **76**
AmiStatus amiGetGeomData 76
AmiStatus amiGetSketchedSplineSegments 76
AmiStatus amiLineIsSilhouette 76
AmiStatus amiSetGeometry 77
Handling Change Notification **77**
AmiStatus amiDisableActiveNotification 77
AmiStatus amiEnableActiveNotification 77
AmiStatus amiEnableActiveNotification2 77
AmiStatus amiObjectHasChanged 78
amiObjectWasErased 78
B-rep API Integration **78**
AmiStatus amiGetBrepFromKey 78
AmiStatus amiGetKeyFromBrep 79
Handling Attributes **79**
AmiStatus amiAddAttribute 79
AmiStatus amiGetAllAttributes 79
AmiStatus amiGetAttributeHolders 80
AmiStatus amiGetAttributes 80
AmiStatus amiRemoveAttribute 81
Instantiable Attributes **81**
AmiStatus amiDefineAttClass 81
AmiStatus amiGetAttClasses 81

AmiStatus amiGetAttClassFields ... 81
AmiStatus amiGetAttClassName ... 82
AmiStatus amiGetAttField ... 82
AmiStatus amiGetAttField ... 82
AmiStatus amiMakeInstAtt ... 83
AmiStatus amiReadAttClassesFromFile ... 83
AmiStatus amiSetAttField ... 83
AmiStatus amiSetAttField ... 83
AmiStatus amiUndefineAttClass ... 84
AmiStatus amiWriteAttClassToFile ... 84
AmiStatus amiWriteAttClassesToFile ... 84
Miscellaneous ... 84
AmiStatus amiApplyColorOverride ... 84
AmiStatus amiCreateSystemAttribute ... 85
AmiStatus amiEndExternalEdit ... 85
AmiStatus amiEraseObject ... 85
AmiStatus amiGetAssociatedDoc ... 85
AmiStatus amiGetColorOverride ... 85
AmiStatus amiGetCurrentDatabase ... 86
AmiStatus amiGetDbIdsFromKey ... 86
AmiStatus amiGetExecVersion ... 86
AmiStatus amiGetFileVersion ... 86
AmiStatus amiGetObjectName ... 86
AmiStatus amiGetObjectState ... 86
AmiStatus amiGetObjectVisibility ... 87
AmiStatus amiGetSystemAttributeName ... 88
AmiStatus amiGetSysVar ... 88
AmiStatus amiHasColorOverride ... 88
AmiStatus amiIsDirectlyEditable ... 89
AmiStatus amiRemoveColorOverride ... 89
AmiStatus amiSetObjectName ... 89
AmiStatus amiSetObjectVisibility ... 89
AmiStatus amiSetSystemAttributeName ... 90
AmiStatus amiSetSysVar ... 90
AmiStatus amiStartExternalEdit ... 90
Parametric Modeling ... 91
Parameter Handling ... 91
AmiStatus amiCreateParam ... 91
AmiStatus amiEvalParamExpress ... 91
AmiStatus amiGetGlobalParams ... 91
AmiStatus amiGetParamComment ... 92
AmiStatus amiGetParamDeps ... 92
AmiStatus amiGetParamExpress ... 92
AmiStatus amiGetParamName ... 92
AmiStatus amiGetParamUsers ... 93
AmiStatus amiGetParamValue ... 93
AmiStatus amiSetParamComment ... 93
AmiStatus amiSetParamExpress ... 93
AmiStatus amiSetParamValue ... 93
Part Handling ... 94
AmiStatus amiCreateEmptyPart ... 94
AmiStatus amiGetActivePart ... 94
AmiStatus amiGetContainingPart ... 94
AmiStatus amiGetNumPartFeats ... 94

AmiStatus amiGetPartParams ... 95
AmiStatus amiGetPartWorkAxes ... 95
AmiStatus amiGetPartWorkPlanes ... 95
AmiStatus amiGetPartWorkVertices ... 96
AmiStatus amiGetUnusedSketchesFromPart ... 96
AmiStatus amiRollbackPart ... 96
Regeneration Control ... 96
AmiStatus amiRegen ... 96
AmiStatus amiRegenAllOfAType ... 97
AmiStatus amiSetNeedsRegen ... 97
Suppression Control ... 97
AmiStatus amiIsFeatSuppressed ... 97
AmiStatus amiSuppressFeat ... 97
AmiStatus amiSuppressFeatsByType ... 97
AmiStatus amiUnsuppressFeat ... 98
AmiStatus amiUnsuppressFeatsByType ... 98
AmiStatus amiUnsuppressPartFeats ... 98
Component Definition Handling ... 98
AmiStatus amiAddCompToCompDef ... 98
AmiStatus amiCopyCompDef ... 99
AmiStatus amiCreateCompDef ... 99
AmiStatus amiCreateCompDefFromFile ... 99
AmiStatus amiExternalize ... 100
AmiStatus amiGetActiveCompDef ... 100
AmiStatus amiGetAllCompDefs ... 100
AmiStatus amiGetAllInstances ... 100
AmiStatus amiGetBodyFromCompDef ... 101
AmiStatus amiGetCompDefChildren ... 101
AmiStatus amiGetCompDefName ... 101
AmiStatus amiGetMasterCompDef ... 101
AmiStatus amiGetPartDefinition ... 102
AmiStatus amiIsCompDefExternal ... 102
AmiStatus amiIsCompDefLeafNode ... 102
AmiStatus amiIsCompDefMaster ... 102
AmiStatus amiLocalize ... 103
AmiStatus amiReassignComps ... 103
AmiStatus amiRemoveCompFromCompDef ... 103
AmiStatus amiSetActiveCompDef ... 103
AmiStatus amiSetCompDefName ... 104
Component Handling ... 104
AmiStatus amiGetActiveAssembly ... 104
AmiStatus amiGetActiveLeafNode ... 104
AmiStatus amiGetCompChildren ... 104
AmiStatus amiGetCompDef ... 104
AmiStatus amiGetCompInDef ... 105
AmiStatus amiGetCompName ... 105
AmiStatus amiGetCompOwner ... 105
AmiStatus amiGetCompPosition ... 105
AmiStatus amiGetContainingComp ... 106
AmiStatus amiSetActiveAssembly ... 106
AmiStatus amiSetActiveCompDef ... 106
AmiStatus amiSetActiveLeafNode ... 106
AmiStatus amiSetCompName ... 106
AmiStatus amiSetCompPosition ... 107

Constraint Handling **107**
AmiStatus amiGetData 109
AmiStatus amiSetData 110
AmiStatus amiCreateConstraint 111
AmiStatus amiCreateConstraints 111
AmiStatus amiCreateConstrDescrip 111
AmiStatus amiGetConstrDescrip 112
AmiStatus amiGetConstrOperands 112
AmiStatus amiGetConstrParam 112
AmiStatus amiGetConstrType 112
AmiStatus amiGetConstrValue 112
AmiStatus amiGetDimConstrLoc 113
AmiStatus amiSetConstrExpr 113
AmiStatus amiSetConstrValue 113
Component Constraint Handling **113**
AmiStatus amiGetCompsFromConstr 113
AmiStatus amiGetConstrsActingOnComp 113
AmiStatus amiGetConstrsFromComp 114
Sketch Handling **114**
AmiStatus amiGetBrepGeomFromSketchGeom 114
AmiStatus amiGetExtSketchConstrs 114
AmiStatus amiGetFeatsFromSketch 114
AmiStatus amiGetPathStartPoint 115
AmiStatus amiGetSketchConstrs 115
AmiStatus amiGetSketchCoordSys 115
AmiStatus amiGetSketchDefPlane 115
AmiStatus amiGetSketchesFromGeom 116
AmiStatus amiGetSketchGeom 116
AmiStatus amiGetSketchParams 116
AmiStatus amiGetSketchType 116
Sketch Constraint Handling **117**
AmiStatus amiGetConstrSketches 117
Feature Handling **117**
AmiStatus amiAbortCompositeFeat 117
AmiStatus amiCreateFeature 117
AmiStatus amiEndCompositeFeat 117
AmiStatus amiGetConstrGeomDescrip 118
AmiStatus amiGetFeatDepFeats 118
AmiStatus amiGetFeatDepWorkGeom 118
AmiStatus amiGetFeatEdges 118
AmiStatus amiGetFeatFaces 119
AmiStatus amiGetFeatParams 119
AmiStatus amiGetFeatType 119
AmiStatus amiGetFeatsFromGeom 119
AmiStatus amiGetFeatSketch 120
AmiStatus amiGetFeatVertices 120
AmiStatus amiGetFeatDescrip 120
AmiStatus amiGetOwningCompositeFeat 121
AmiStatus amiIsCompToolbody 121
AmiStatus amiIsFeatKindOf 121
AmiStatus amiReorderFeat 121
AmiStatus amiSetCompositeFeatEditCallback 121
AmiStatus amiStartCompositeFeat 122
Feature Descriptors **122**

AmiStatus amiCreateFeatDescrip 122
AmiStatus amiGetCombinerType 122
AmiStatus amiGetDescripType 122
AmiStatus amiGetFeatData 123
AmiStatus amiIsDescripKindOf 123
AmiStatus amiSetCombinerType 123
AmiStatus amiSetFeatPart 123
Composite Feature Descriptors 123
AmiStatus amiGetCompositeFeatData 123
AmiStatus amiSetCompositeFeatDrivingParams 124
AmiStatus amiSetCompositeFeatOwnerInfo 124
AmiStatus amiSetCompositeFeatType 124
AmiStatus amiClearHoleTapData 124
AmiStatus amiGetCBoreHoleData 124
AmiStatus amiGetCSinkHoleData 125
AmiStatus amiGetHoleData 125
AmiStatus amiGetHoleTapData 125
AmiStatus amiSetCBoreHoleCbDepth 126
AmiStatus amiSetCBoreHoleCbDiam 126
AmiStatus amiSetCSinkHoleCsAngle 126
AmiStatus amiSetCSinkHoleCsDiam 126
AmiStatus amiSetHoleDiameter 126
AmiStatus amiSetHoleDirectionVec 126
AmiStatus amiSetHoleDrillAngle 126
AmiStatus amiSetHoleFitClass 127
AmiStatus amiSetHoleNominalSize 127
AmiStatus amiSetHolePitch 127
AmiStatus amiSetHoleTapData 127
AmiStatus amiSetHoleTapDrillDiameter 127
AmiStatus amiSetHoleUnits 128
Base Feature (Solid) Descriptors 128
AmiStatus amiGetBaseData 128
AmiStatus amiSetBaseData 128
Extrusion Descriptors 128
AmiStatus amiGetExtrusionData 128
AmiStatus amiGetThinExtrusionData 129
AmiStatus amiSetExtrusionDirectionVec 129
AmiStatus amiSetExtrusionDraftAngle 129
AmiStatus amiSetExtrusionProfile 129
AmiStatus amiSetThinExtrusionExtend 129
AmiStatus amiSetThinExtrusionThickness 129
Revolution Descriptors 130
AmiStatus amiGetRevolveData 130
AmiStatus amiSetRevolveAxis 130
AmiStatus amiSetRevolveDirectionVec 130
AmiStatus amiSetRevolveProfile 130
Chamfer Descriptors 130
AmiStatus amiGetChamferData 130
AmiStatus amiSetChamferAngle 131
AmiStatus amiSetChamferAngleFace 131
AmiStatus amiSetChamferDist1 131
AmiStatus amiSetChamferDist1Face 131
AmiStatus amiSetChamferDist2 131
AmiStatus amiSetChamferEdges 132

AmiStatus amiSetChamferType ... 132
Fillet Descriptors ... 132
AmiStatus amiGetCubicFilletData ... 132
AmiStatus amiGetFixedFilletData ... 132
AmiStatus amiGetIndFilletData ... 132
AmiStatus amiGetLinearFilletData ... 133
AmiStatus amiGetUniFilletData ... 133
AmiStatus amiSetCubicFilletRadii ... 133
AmiStatus amiSetFilletEdges ... 133
AmiStatus amiSetFixedFilletChordLength ... 133
AmiStatus amiSetIndFilletRadii ... 134
AmiStatus amiSetLinearFilletRadii ... 134
AmiStatus amiSetUniFilletRadius ... 134
Sweep Descriptors ... 134
AmiStatus amiGetSweepData ... 134
AmiStatus amiSetSweepDraftAngle ... 134
AmiStatus amiSetSweepMethod ... 135
AmiStatus amiSetSweepPath ... 135
AmiStatus amiSetSweepProfile ... 135
Feature Array Descriptors ... 135
AmiStatus amiGetArrayData ... 135
AmiStatus amiGetPolarArrayData ... 135
AmiStatus amiGetRecArrayData ... 136
AmiStatus amiSetArrayFeature ... 136
AmiStatus amiSetPolarArrayAngle ... 136
AmiStatus amiSetPolarArrayNumInstances ... 136
AmiStatus amiSetPolarArrayRotateAsCopied ... 137
AmiStatus amiSetPolarArrayPolarAngleType ... 137
AmiStatus amiSetRecArrayColData ... 137
AmiStatus amiSetRecArrayRowData ... 137
AmiStatus amiSetRecArrayXVector ... 137
AmiStatus amiSetRecArrayYVector ... 138
Surf Cut Descriptors ... 138
AmiStatus amiGetSurfCutData ... 138
AmiStatus amiSetSurfCutDirectionVec ... 138
AmiStatus amiSetSurfCutSurf ... 138
Shell Descriptors ... 138
AmiStatus amiGetShellData ... 138
AmiStatus amiSetShellExcludes ... 139
AmiStatus amiSetShellOverrides ... 139
Parametric Boolean Descriptors ... 139
AmiStatus amiGetParametricBoolData ... 139
AmiStatus amiSetParametricBoolComp ... 139
Constrained Geometry Descriptors ... 140
AmiStatus amiCreateConstrGeom ... 140
AmiStatus amiGetConstrGeomData ... 140
AmiStatus amiGetWorkPlaneData ... 140
Loft Descriptors ... 140
AmiStatus amiGetLoftData ... 140
AmiStatus amiSetLoftAngles ... 141
AmiStatus amiSetLoftMinimizeTwist ... 141
AmiStatus amiSetLoftType ... 141
AmiStatus amiSetLoftWeights ... 142

AmiStatus amiSetLoftXSections ... 142
Face Draft Descriptors ... 142
AmiStatus amiGetDraftData ... 142
AmiStatus amiGetEdgeDraftData ... 142
AmiStatus amiGetPlaneDraftData ... 142
AmiStatus amiGetShadowDraftData ... 143
AmiStatus amiSetDraftAngle ... 143
AmiStatus amiSetDraftTangents ... 143
AmiStatus amiSetDraftPlane ... 143
AmiStatus amiSetEdgeDraftFaces ... 143
AmiStatus amiSetPlaneDraftFaces ... 144
AmiStatus amiSetShadowDraftFaces ... 144
Split Descriptors ... 144
AmiStatus amiGetFaceSplitData ... 144
AmiStatus amiGetPartSplitData ... 144
AmiStatus amiSetFaceSplitFaces ... 145
AmiStatus amiSetPartSplitFlip ... 145
AmiStatus amiSetPartSplitName ... 145
AmiStatus amiSetSplitCombinerPartName ... 145
Sketch Descriptors ... 145
AmiStatus amiCreateSketchDescrip ... 145
AmiStatus amiCreateSketch ... 146
AmiStatus amiGetData ... 146
AmiStatus amiSetData ... 146
Sketch Plane Descriptors ... 148
AmiStatus amiGetSketchPlaneDescrip ... 148
AmiStatus amiCreateSketchPlane ... 148
AmiStatus amiGetCurrentSketchPlane ... 148
AmiStatus amiGetSketchPlaneFromSketch ... 148
Array Descriptors ... 149
AmiStatus amiArrayHasIndependentInstances ... 149
AmiStatus amiIsArrayInstanceIndependent ... 149
Bend Descriptors ... 149
AmiStatus amiGetBendData ... 149
AmiStatus amiSetBendAngle ... 149
AmiStatus amiSetBendArcLen ... 150
AmiStatus amiSetBendDirection ... 150
AmiStatus amiSetBendProfile ... 150
AmiStatus amiSetBendRadius ... 150
AmiStatus amiSetBendSide ... 150
AmiStatus amiSetBendType ... 150
Rib Descriptors ... 151
AmiStatus amiGetRibData ... 151
AmiStatus amiSetRibDirection ... 151
AmiStatus amiSetRibProfile ... 151
AmiStatus amiSetRibThickness ... 151
Pattern Descriptors ... 151
AmiStatus amiGetPatternData ... 151
AmiStatus amiSetPatternFeatures ... 152
AmiStatus amiSetPatternSuppressedInstances ... 152
Axial Pattern Descriptors ... 152
AmiStatus amiGetAxialPatternData ... 152
AmiStatus amiSetAxialPatternNRevolutions ... 153

AmiStatus amiSetAxialPatternOffsetDirection 153
AmiStatus amiSetAxialPatternOffsetHeight 153
AmiStatus amiSetAxialPatternOffsetSpacingType 153
Polar Pattern Descriptors 153
AmiStatus amiGetPolarPatternData 153
AmiStatus amiSetPolarPatternAngle 154
AmiStatus amiSetPolarPatternAngleSpacingType 154
AmiStatus amiSetPolarPatternNInstances 154
AmiStatus amiSetPolarPatternOrientation 154
AmiStatus amiSetPolarPatternRotationDirection 154
Rectangular Pattern Descriptors 155
AmiStatus amiGetRectangularPatternData 155
AmiStatus amiSetRectangularPatternAlignmentAngle 155
AmiStatus amiSetRectangularPatternColumnData 155
AmiStatus amiSetRectangularPatternColumnDirection 156
AmiStatus amiSetRectangularPatternRowData 156
AmiStatus amiSetRectangularPatternRowDirection 156
AmiStatus amiSetRectangularPatternSketchPlane 156
Locators 156
AmiStatus amiAddAtomLocator 156
AmiStatus amiCreateAtomLocator 157
AmiStatus amiCreateLocator 157
AmiStatus amiGetAbsoluteLocData 157
AmiStatus amiGetAngledLocData 157
AmiStatus amiGetAtomLocators 157
AmiStatus amiGetAtomLocData 158
AmiStatus amiGetAtomLocType 158
AmiStatus amiGetCoordSysLocData 158
AmiStatus amiGetDistFromLocData 158
AmiStatus amiGetLocator 158
AmiStatus amiGetOffsetLocData 159
AmiStatus amiGetOnPlaneLocData 159
AmiStatus amiGetTangentLocData 159
AmiStatus amiSetAbsoluteLocData 159
AmiStatus amiSetAngledLocData 159
AmiStatus amiSetAtomLocData 160
AmiStatus amiSetCoordSysLocData 160
AmiStatus amiSetDistFromLocData 160
AmiStatus amiSetLocator 160
AmiStatus amiSetOffsetLocData 160
AmiStatus amiSetOnPlaneLocData 161
AmiStatus amiSetTangentLocData 161
Terminators 161
AmiStatus amiCreateTerminator 161
AmiStatus amiGetBlindTermData 161
AmiStatus amiGetFromToTermData 161
AmiStatus amiGetInsideTermData 162
AmiStatus amiGetMidPlaneTermData 162
AmiStatus amiGetOutsideTermData 162
AmiStatus amiGetTerminator 162
AmiStatus amiGetTermType 162
AmiStatus amiGetToFaceTermData 163
AmiStatus amiGetToPlaneTermData 163
AmiStatus amiSetBlindTermData 163

AmiStatus amiSetFromToTermData 163
AmiStatus amiSetInsideTermData 163
AmiStatus amiSetMidPlaneTermData 164
AmiStatus amiSetOutsideTermData 164
AmiStatus amiSetTerminator 164
AmiStatus amiSetToFaceTermData 164
AmiStatus amiSetToPlaneTermData 164
Table-Driven Versions 164
AmiStatus amiGetCurrentVersion 164
AmiStatus amiGetLinkInfo 165
AmiStatus amiRemoveLink 165
AmiStatus amiSetCurrentVersion 165
AmiStatus amiBuildAndLinkFile 165
AmiStatus amiBuildFile 165
Event Handling 166
AmiStatus amiReactDescrip::make 166
AmiStatus amiGetData 169
AmiStatus amiRegisterApp 169
AmiStatus amiRegisterEventReaction 169
AmiStatus amiRemoveEventReaction 170
AmiStatus amiSetData 170
Miscellaneous 170
AmiStatus amiGetDesignMode 170
AmiStatus amiGetKeyFromName 170
AmiStatus amiSetDesignMode 171
Analysis 171
AmiStatus amiCheckInterference 172
AmiStatus amiCheckInterferenceTrans 172
AmiStatus amiGetData 173
AmiStatus amiGetDataArray 173
AmiStatus amiGetDOFDescrip 174
AmiStatus amiGetMassProps 174
AmiStatus amiMinDistance 174
Mechanical Desktop Surface Modeling 175
Geometry Creation 175
AmiStatus amiCreateSurfFromGeom 175
AmiStatus amiCreateSurfArrayFromGeom 175
Drawing Manager 176
Introduction 176
Views 176
AmiStatus amiAddDataArray 183
AmiStatus amiCreateDwView 183
AmiStatus amiCreateDwVwDescrip 184
AmiStatus amiDwEditView 184
AmiStatus amiDwVwDescripIsKindof 184
AmiStatus amiFreezeLayers 184
AmiStatus amiGetAllDwVws 184
AmiStatus amiGetData 185
AmiStatus amiGetDataArray 185
AmiStatus amiGetDwVwDescrip 186
AmiStatus amiGetDwVwModelObjs 186
AmiStatus amiGetDwVwParent 186
AmiStatus amiGetNumDwVws 186
AmiStatus amiSetData 187

AmiStatus amiSetDataArray ... 187
AmiStatus amiThawLayers ... 188
Annotations ... 188
AmiStatus amiCreateDwAnnot ... 189
AmiStatus amiCreateDwAnnotDescrip ... 189
AmiStatus amiDwAnnotDescripIsKindof ... 189
AmiStatus amiDwEditAnnot ... 189
AmiStatus amiGetData ... 189
AmiStatus amiGetDwAnnotDescrip ... 190
AmiStatus amiGetDwAnnotView ... 190
AmiStatus amiGetDwVwAnnots ... 190
AmiStatus amiMoveDwAnnot ... 190
AmiStatus amiSetData ... 191
Scenes ... 191
AmiStatus amiCreateScene ... 191
AmiStatus amiCreateSceneCompDescrip ... 192
AmiStatus amiEditSceneComp ... 192
AmiStatus amiGetAllScenes ... 192
AmiStatus amiGetData ... 192
AmiStatus amiGetSceneCompDef ... 193
AmiStatus amiGetSceneCompDescrip ... 193
AmiStatus amiGetSceneExplodeFactor ... 193
AmiStatus amiGetSceneLock ... 193
AmiStatus amiSetData ... 193
AmiStatus amiSetSceneLock ... 194
Hidden Line (AHL) Calculation ... 194
AmiStatus amiClearCustomAHL ... 194
AmiStatus amiGetCustomAHL ... 195
AmiStatus amiSetCustomAHL ... 195
Layouts ... 196
AmiStatus amiGetAllDwLayouts ... 196
User Interface Functions ... 196
Browser Display ... 196
AmiStatus amiIsBrowserDisplayed ... 196
AmiStatus amiDisplayBrowser ... 196
Browser Customization ... 197
AmiStatus amiAddBrowserTab ... 197
AmiStatus amiRemoveBrowserTab ... 197
File Descriptor Functions ... 197
AmiStatus amiAttachFile ... 197
AmiStatus amiCreateFileDescrip ... 197
AmiStatus amiGetData ... 198
AmiStatus amiGetFile ... 198
AmiStatus amiIsPDMRegistered ... 198
AmiStatus amiRemoveFile ... 198
AmiStatus amiSetData ... 199

APPENDIX A SAMPLE APPLICATIONS FOR THE MCAD API ... 202

Comprehensive Sample Application (MAISAMP) ... 202

Common 204
Handling Pick Objects 204
MAIALLOWGHOST 204
MAICREATEKEY 204
MAIGETKEYFROMID 205
MAIGETKEYFROMPATH 205
Creating Keys 205
MAICREATEINFERKEY 205
Handling Keys 205
MAIAREKEYSEQUIVALENT 205
MAICOPYKEY 205
MAIGETKEYTYPE 205
MAIERASEOBJECT 205
MAIISKEYKINDOF 205
MAIISKEYNULL 205
MAIREFOCUSKEY 206
MAIREFOCUSKEY2 206
Entity Highlighting 206
MAIHIGHLIGHT 206
Handling Geometry 206
MAICREATESURFFROMGEOM 206
MAICREATESURFARRAYFROMGEOM 206
MAIGETSKETCHEDSPLINESEGMENTS 206
MAIISDIRECTLYEDITABLE 206
MAISETGEOMETRY 206
Handling Change Notification 206
MAIDISABLEACTIVENOTIFICATION 206
MAIENABLEACTIVENOTIFICATION 207
MAIENABLEACTIVENOTIFICATION2 207
MAIOBJECTHASCHANGED 207
MAIOBJECTWASERASED 207
MAIRESETNOTIFICATION 207
B-rep API Integration 207
MAIBREP 207
MAIBREPFROMTO 207
Handling Attributes 207
MAIADDATTRIBUTE 207
MAIATTEDITFLAG 208
MAIATTRASSOCTYPE 208
MAICREATEATTRIBUTE 208
MAIGETALLATTRIBUTES 208
MAIGETATTRIBUTEHOLDERS 208
MAIGETATTRIBUTES 208
MAIHOLDERSFLAG 208
MAILISTINDEX 208
MAIREMOVEATTRIBUTE 208
MAIOWNERISMCADAPI 208
Instantiable Attributes 209
MAIDEFINEATTCLASS 209
MAIGETATTCLASSES 209
MAIGETATTCLASSFIELDS 209
MAIMAKEINSTATT 209
MAIREADATTCLASSESFROMFILE 209

MAISTATICFLAG ... 209
MAIUNDEFINEATTCLASS ... 209
MAIWRITEATTCLASSTOFILE ... 209
MAIWRITEATTCLASSESTOFILE ... 209
Miscellaneous ... 210
MAIENDEXTERNALEDIT ... 210
MAIGETCURRENTDATABASE ... 210
MAIGETDBIDSFROMKEY ... 210
MAIGETKEYFROMNAME ... 210
MAIGETOBJECTSTATE ... 210
MAIGETDESIGNMODE ... 210
MAISTARTEXTERNALEDIT ... 210

Parametric Modeling ... 211
Parameter Handling ... 211
MAICREATEPARAM ... 211
MAIEVALPARAMEXPRESS ... 211
MAIGETGLOBALPARAMS ... 211
MAIGETPARAMDEPS ... 211
MAIGETPARAMCOMMENT ... 211
MAIGETPARAMEXPRESS ... 211
MAIGETPARAMNAME ... 211
MAIGETPARAMUSERS ... 211
MAIGETPARAMVALUE ... 211
MAISETPARAMCOMMENT ... 211
MAISETPARAMEXPRESS ... 212
MAISETPARAMVALUE ... 212
Part Handling ... 212
MAICREATEEMPTYPART ... 212
MAIGETACTIVEPART ... 212
MAIGETCONTAININGPART ... 212
MAIGETMASSPROPS ... 212
MAIGETNUMPARTFEATS ... 212
MAIGETPARTFEATS ... 212
MAIGETPARTPARAMS ... 212
MAIGETUNUSEDSKETCHESFROMPART ... 213
Regeneration Control ... 213
MAIREGEN ... 213
MAISETNEEDSREGEN ... 213
Suppression Control ... 213
MAISUPPRESSFEAT ... 213
MAISUPPRESSFEATSBYTYPE ... 213
MAIUNSUPPRESSFEAT ... 213
MAIUNSUPPRESSFEATSBYTYPE ... 213
MAIUNSUPPRESSPARTFEATS ... 213
Component Definition Handling ... 213
MAIADDCOMPTOCOMPDEF ... 213
MAICOPYCOMPDEF ... 214
MAICREATECOMPDEFFROMFILE ... 214
MAIEXTERNALIZECOMP ... 214
MAIGETACTIVECOMPDEF ... 214
MAIGETALLCOMPDEFS ... 214
MAIGETBODYFROMCOMPDEF ... 214

MAIGETCOMPDEFCHILDREN ... 214
MAIGETCOMPDEFNAME ... 214
MAIGETMASTERCOMPDEF ... 214
MAIISCOMPDEFEXTERNAL ... 214
MAIISCOMPDEFLEAFNODE ... 215
MAIISCOMPDEFMASTER ... 215
MAILOCALIZECOMP ... 215
MAIREMOVECOMPFROMCOMPDEF ... 215
MAISETACTIVECOMPDEF ... 215
MAISETCOMPDEFNAME ... 215
Component Handling ... 215
MAICREATECOMPDEF ... 215
MAIGETACTIVELEAFNODE ... 215
MAIGETCOMPCHILDREN ... 215
MAIGETCOMPDEF ... 216
MAIGETCOMPINDEF ... 216
MAIGETCOMPNAME ... 216
MAIGETCOMPOWNER ... 216
MAIGETCOMPPOSITION ... 216
MAIGETCONTAININGCOMP ... 216
MAIISCOMPTOOLBODY ... 216
MAISETACTIVELEAFNODE ... 216
MAISETCOMPNAME ... 216
MAISETCOMPPOSITION ... 216
Constraint Handling ... 217
MAICREATEASSMCONSTR ... 217
MAICREATESKETCHCONSTR2 ... 217
MAIGETCONSTRDESCRIPDATA ... 217
MAIGETCONSTREXPR ... 217
MAIGETCONSTRPARAM ... 217
MAIGETCONSTRTYPE ... 217
MAIGETCONSTRVALUE ... 217
MAIGETCONSTROPERANDS ... 217
MAIGETDIMCONSTRLOC ... 217
MAISETCONSTREXPR ... 218
MAISETCONSTRVALUE ... 218
Component Constraint Handling ... 218
MAIGETCOMPSFROMCONSTR ... 218
MAIGETCONSTRSACTINGONCOMP ... 218
MAIGETCONSTRSFROMCOMP ... 218
Sketch Handling ... 218
MAICREATESKETCH ... 218
MAIGETCURRENTSKETCHPLANE ... 218
MAIGETEXTSKETCHCONSTRS ... 218
MAIGETFEATSFROMSKETCH ... 218
MAIGETSKETCHCONSTRS ... 218
MAIGETPATHSTARTPOINT ... 219
MAIGETSKETCHCOORDSYS ... 219
MAIGETSKETCHDEFPLANE ... 219
MAIGETSKETCHESFROMGEOM ... 219
MAIGETSKETCHGEOM ... 219
MAIGETSKETCHPARAMS ... 219
MAIGETSKETCHPLANE ... 219
MAIGETSKETCHPLANEDESCRIPDATA ... 219

MAIGETSKETCHPLANEFROMSKETCH ... 219
Sketch Constraint Handling ... 220
MAICREATESKETCHCONSTR ... 220
MAIGETCONSTRSKETCHES ... 220
Feature Handling ... 220
MAIGETFEATDESCRIPDATA ... 220
MAIGETFEATDEPFEATS ... 220
MAIGETFEATDEPWORKGEOM ... 220
MAIGETFEATEDGES ... 220
MAIGETFEATFACES ... 220
MAIGETFEATPARAMS ... 220
MAIGETFEATSFROMGEOM ... 220
MAIGETFEATSKETCH ... 220
MAIGETFEATTYPE ... 221
MAIGETFEATVERTICES ... 221
MAIISFEATKINDOF ... 221
MAIREORDERFEAT ... 221
MAIRETURNFEATKEY ... 221
MAIRETURNINFORMERS ... 221
MAIRETURNPARAMETERS ... 221
MAISKETCHPLANE ... 221
Feature Descriptor Handling ... 221
MAIARRAY ... 221
MAIBASE ... 222
MAICHAMFER ... 222
MAICOMBINE ... 222
MAIEXTRUDE ... 222
MAIFACEDRAFT ... 222
MAIFACESPLIT ... 222
MAIFILLET ... 222
MAIGETPARTWORKAXES ... 222
MAIGETPARTWORKPLANES ... 222
MAIGETPARTWORKVERTICES ... 223
MAIHOLE ... 223
MAILOFT ... 223
MAIMODIFIER1 ... 223
MAIMODIFIER2 ... 223
MAIPARTSPLIT ... 223
MAIREVOLVE ... 223
MAISHELL ... 223
MAISURFCUT ... 223
MAISWEEP ... 224
MAIWORKAXIS ... 224
MAIWORKPLANE ... 224
MAIWORKPOINT ... 224
Array Descriptors ... 224
MAIARRAYHASINDEPENDENTINSTANCES ... 224
MAIISARRAYINSTANCEINDEPENDENT ... 224
Table-Driven Versions ... 224
MAICREATELINK ... 224
MAIGETCURRENTVERSION ... 224
MAIGETLINKDATA ... 225
MAIREMOVELINK ... 225
MAISETCURRENTVERSION ... 225

MAISETLINKDATA ... 225
Event Handling ... 225
MAIREGISTEREVENTREACTION ... 225
MAIREMOVEEVENTREACTION ... 225
Miscellaneous ... 225
MAIGETDESIGNMODE ... 225
MAISETDESIGNMODE ... 225

Analysis ... 225
MAICHECKINTERFERENCE ... 225
MAIGETDOFDESCRIPDATA ... 226
MAIMINDISTANCE ... 226

Mechanical Desktop Surface Modeling ... 226
MAICREATESURFFROMGEOM ... 226

Drawing Manager ... 226
MAICREATEDWANNOT ... 226
MAICREATEDWVW ... 226
MAICREATEINFERGEOMKEY ... 226
MAIGETALLDWVWS ... 226
MAIGETDWANNOTDATA ... 226
MAIGETDWVWANNOTS ... 226
MAIGETDWANNOTVIEW ... 227
MAIGETDWVWATTRIBUTE ... 227
MAIGETDWVWPARENT ... 227
MAIISDWANNOTKEYKINDOF ... 227
MAIISDWVWKEYKINDOF ... 227
MAISETDWANNOTDATA ... 227
MAISETDWVWATTRIBUTE ... 227
Scenes ... 228
MAICREATESCENE ... 228
MAIGETALLSCENES ... 228
MAIGETSCENELOCK ... 228
MAISETSCENELOCK ... 228
Hidden Line (AHL) Calculation ... 228
MAICLEARCUSTOMAHL ... 228
MAICUSTOMAHLON ... 228
MAIGETCUSTOMAHL ... 228
MAISETCUSTOMAHL ... 228
Layouts ... 228
MAIGETALLDWLAYOUTS ... 228

User Interface Functions ... 229
MAIISBROWSERDISPLAYED ... 229
MAIDISPLAYBROWSER ... 229

File Descriptor Functions ... 229
MAIISPDMREGISTERED ... 229
MAIREGISTERPDM ... 229
MAIUNREGISTERPDM ... 229

Utility **229**
MAIDELETEKEY 229
MAIDELETEATTRIBUTE 229
MAIGETSYSVAR 229
MAILISTATTRIBUTES 229
MAILISTKEYS 229
MAIPRINTATTRIBUTE 229
MAIPRINTKEY 230
MAIPRINTSTATUSINFO 230
MAIRESETATTRIBUTEDICT 230
MAIRESETKEYDICT 230
MAISETSYSVAR 230

SmartHole Sample Application **231**

Commands **231**

SHCreate **231**

SHEdit **231**

SHErase **231**

SHMatch **231**

SHAudit **231**

Known Issues **231**

Hole Projection Sample Application **233**
Simple User Interface **233**
Known Issues **233**
Methods and Functionality Exercised **233**
Files **234**

MfcBrepTrav Sample Application **235**

MyBrowser Sample Application **236**

CountAssemblies Sample Application **237**

Which_MDT Sample Application **238**

APPENDIX B HOW TO USE THE MCAD API **239**

Understanding the SDK Directory Content **239**
The *ObjectARX* Subdirectory **239**
The *mcadapi* Subdirectory **239**

Installing the Application **241**

The Registry Structure **241**
Running the Application **241**
Guidelines for Building Your Application **241**

APPENDIX C LISP/ADS SUPPORT FOR THE AUTODESK MECHANICAL DESKTOP 242

Autodesk Mechanical Desktop Commands **242**

Invoking Mechanical Desktop Commands From LISP/ADS **242**
Integer (acedGetInt) **242**
Selection set (acedSSGet) **243**

Notes **244**
Examples Invoking Mechanical Desktop Commands **244**

System Variables **245**

Integrating the MCAD API with Mechanical Desktop Commands **247**
Calling Mechanical Desktop Commands **247**
Example **247**
Benefits **247**
Usage Details **247**
Passing Additional Points **248**
Sample Code **248**

APPENDIX D DEVELOPER PARTNER GUIDELINES 251

Introduction **251**

Part 1 ..... **251**
Product Development Guidelines **251**

Part 2 ..... **255**
Installation Development Guidelines **255**

REFERENCE: YOUR MOST FREQUENTLY ASKED QUESTIONS 256

REFERENCE: ALPHABETICAL LISTING OF ENUM TYPES 259

REFERENCE: GLOSSARY 272

Active (leaf node, part) **272**
Active Notification **272**
API **272**
Associativity (Attributes) **272**

Atomic Locator 272
Attribute 272
Attribute Class 272
B-rep 272
B-rep API 272
Derived Attribute 273
Feature Descriptor 273
Filer 273
GeLib 273
Geometry Key 273
Ghost 273
Inferred Geometry 273
Informer 273
Instantiable Attribute 273
Key 273
Key Copying 273
Leaf Node 273
Locator 274
Notification 274
Object Key 274
Owner (Attributes) 274
Passive Notification 274
Pick Object 274
Refocus 274
Scope 274
Terminator 274
Unused Sketch 274

Alphabetical Listing of Functions

The following table lists MCAD API library functions alphabetically. It also briefly describes the purpose of each MCAD API function and lists the page number on which the function is located.

Function	Description	Page
amiAbortCompositeFeat	Handles a feature	117
amiAddAtomLocator	Handles a feature descriptor	156
amiAddAttribute	Handles attributes	79
amiAddBrowserTab	Manipulates a user interface	197
amiAddCompToCompDef	Handles a component definition	98
amiAddDataArray	Drawing Manager	183
amiApplyColorOverride	Handles object color	84
amiAreKeysEquivalent	Handles a key	72
amiArrayHasIndependentInstances	Handles array descriptors	149
amiAttachFile	Handles file descriptors	197
amiBuildFile	Table-Driven Versions	165
amiBuildAndLinkFile	Table-Driven Versions	165
amiCheckInterference	Handles analysis	172
amiCheckInterferenceTrans	Handles analysis	172
amiClearCustomAHL	Handles hidden line calculations	194
amiClearHoleTapData	Handles a feature descriptor	124
amiCopyCompDef	Handles a component definition	99
amiCopyKey	Handles a key	72
amiCreateAtomLocator	Handles a feature descriptor	157
amiCreateCompDef	Handles a component definition	99
amiCreateCompDefFromFile	Handles a component definition	99
amiCreateConstraint	Handles a constraint	111
amiCreateConstraints	Handles a constraint	111
amiCreateConstrDescrip	Handles a constraint	111
amiCreateConstrGeom	Handles a feature descriptor	140
amiCreateDwAnnot	Drawing Manager	189
amiCreateDwAnnotDescrip	Drawing Manager	189
amiCreateDwView	Drawing Manager	183
amiCreateDwVwDescrip	Drawing Manager	184
amiCreateEmptyPart	Handles a part	94
amiCreateFeatDescrip	Handles a feature descriptor	122
amiCreateFeature	Creates a feature	117
amiCreateFileDescrip	Handles file descriptors	197
amiCreateLocator	Handles a feature descriptor	157
amiCreateParam	Creates a parameter	91
amiCreateScene	Handles a scene	191
amiCreateSceneCompDescrip	Handles a scene	192
amiCreateSketch	Handles a sketch descriptor	146
amiCreateSketchDescrip	Handles a sketch descriptor	145
amiCreateSketchPlane	Handles a sketch plane	148
amiCreateSurfFromGeom	Creates a bounded surface	175
amiCreateSurfArrayFromGeom	Creates a set of bounded surfaces	175

Function	Description	Page
amiCreateSystemAttribute	Creates an MDT system attribute	85
amiCreateTerminator	Handles a feature descriptor	161
amiDefineAttClass	Handles an instantiable attribute	81
amiDisableActiveNotification	Handles change notification	77
amiDisplayBrowser	Handles browser display	196
amiDwAnnotDescripIsKindOf	Drawing Manager	189
amiDwEditAnnot	Drawing Manager	189
amiDwEditView	Drawing Manager	184
amiDwVwDescripIsKindOf	Drawing Manager	189
amiEditSceneComp	Handles a scene	192
amiEnableActiveNotification	Handles change notification	77
amiEnableActiveNotification2	Handles change notification	77
amiEndCompositeFeat	Handles a feature	117
amiEndExternalEdit	Handles external editing	85
amiEraseObject	Erases an object	85
amiEvalParamExpress	Handles a parameter	91
amiExternalize	Handles a component definition	100
amiFillPickObj	Creates a pick object	68
amiFreezeLayers	Handles a view	184
amiGetAbsoluteLocData	Handles a feature descriptor	157
amiGetActiveAssembly	Handles a component	104
amiGetActiveCompDef	Handles a component definition	100
amiGetActiveLeafNode	Handles a component	104
amiGetActivePart	Queries a part	94
amiGetAllAttributes	Handles attributes	79
amiGetAllCompDefs	Handles a component definition	100
amiGetAllInstances	Handles a component definition	100
amiGetAllScenes	Handles scenes	192
amiGetAngledLocData	Handles a feature descriptor	157
amiGetArrayData	Queries a feature	135
amiGetAssociatedDoc	Queries associated documents	86
amiGetAssociatedDbs	Handles a key	73
amiGetAtomLocators	Queries a feature	157
amiGetAtomLocData	Queries a feature	158
amiGetAtomLocType	Queries a feature	158
amiGetAttClasses	Handles an instantiable attribute	81
amiGetAttClassFields	Handles an instantiable attribute	81
amiGetAttClassName	Handles an instantiable attribute	82
amiGetAttField	Handles an instantiable attribute	82
amiGetAttributeHolders	Handles attributes	80
amiGetAttributes	Handles attributes	80
amiGetAxialPatternData	Handles a feature descriptor	152
amiGetBaseData	Handles a feature descriptor	128
amiGetBendData	Handles a feature descriptor	149
amiGetBlindTermData	Queries a feature	161
amiGetBodyFromCompDef	Queries a component definition	101
amiGetBrepFromKey	B-rep API integration	78

Function	Description	Page
amiGetBrepGeomFromSketchGeom	Handles sketch geometry	114
amiGetCBoreHoleData	Queries a feature	124
amiGetChamferData	Queries a feature	130
amiGetColorOverride	Handles object color	85
amiGetCombinerType	Queries a feature	122
amiGetCompChildren	Handles a component	104
amiGetCompDef	Handles a component	104
amiGetCompDefChildren	Handles a component definition	101
amiGetCompDefName	Handles a component	101
amiGetCompInDef	Handles a component	105
amiGetCompName	Handles a component	105
amiGetCompositeFeatData	Handles a feature	123
amiGetCompOwner	Handles a component	105
amiGetCompPosition	Handles a component	105
amiGetCompsFromConstr	Handles a constraint	113
amiGetConstrDescrip	Handles a constraint	112
amiGetConstrGeomData	Handles a feature descriptor	140
amiGetConstrGeomDescrip	Handles a feature	118
amiGetConstrOperands	Handles a constraint	112
amiGetConstrParam	Queries a constraint	112
amiGetConstrsActingOnComp	Handles a component	113
amiGetConstrsFromComp	Handles a constraint	114
amiGetConstrSketches	Handles a sketch constraint	117
amiGetConstrType	Handles a constraint	112
amiGetConstrValue	Handles a constraint	112
amiGetContainingComp	Handles a component	106
amiGetContainingPart	Queries a part	94
amiGetCoordSysLocData	Handles a feature descriptor	158
amiGetCSinkHoleData	Queries a feature	125
amiGetCubicFilletData	Queries a feature	132
amiGetCurrentDatabase	Queries the current database	86
amiGetCurrentSketchPlane	Handles a sketch	148
amiGetCurrentVersion	Table-Driven Versions API	164
amiGetCustomAHL	Handles hidden line calculations	195
amiGetData	Handles a constraint	109
amiGetData	Handles an event	169
amiGetData	Performs analysis	173
amiGetData	Drawing Manager	191
amiGetData	Drawing Manager	187
amiGetData	Handles a scene	192
amiGetData	Handles a file descriptor	198
amiGetData	Handles a sketch descriptor	146
amiGetDataArray	Performs analysis	173
amiGetDataArray	Drawing Manager	185
amiGetDbIdsFromKey	Database integration	86
amiGetDescripType	Queries a feature	122
amiGetDesignMode	Queries design mode	170

Function	Description	Page
amiGetDimConstrLoc	Handles a constraint	113
amiGetDistFromLocData	Queries a feature	158
amiGetDOFDescrip	Handles DOF descriptor	174
amiGetDraftData	Handles a feature descriptor	142
amiGetDwAnnotDescrip	Drawing Manager	190
amiGetDwAnnotView	Drawing Manager	190
amiGetDwVwAnnots	Drawing Manager	190
amiGetDwVwDescrip	Drawing Manager	186
amiGetDwVwModelObjs	Drawing Manager	186
amiGetDwVwParent	Drawing Manager	186
amiGetEdgeDraftData	Handles a feature descriptor	142
amiGetExecVersion	Queries a feature	86
amiGetExtrusionData	Queries a feature	128
amiGetExtSketchConstrs	Queries a sketch	114
amiGetFaceSplitData	Handles a split descriptor	144
amiGetFeatEdges	Queries a feature	118
amiGetFeatFaces	Queries a feature	119
amiGetFeatData	Handles a feature descriptor	123
amiGetFeatDepFeats	Handles a feature	118
amiGetFeatDepWorkGeom	Handles a feature	118
amiGetFeatDescrip	Queries a feature	120
amiGetFeatParams	Queries a feature	119
amiGetFeatsFromGeom	Queries a feature	119
amiGetFeatsFromSketch	Queries a sketch	114
amiGetFeatSketch	Queries a feature	120
amiGetFeatType	Queries a feature	119
amiGetFeatVertices	Queries a feature	120
amiGetFile	Handles file descriptors	198
amiGetFileVersion	Queries a feature	86
amiGetFixedFilletData	Queries a feature	79
amiGetFromToTermData	Queries a feature	161
amiGetGeomData	Handles geometry	76
amiGetGeomKey	Creates a geometric key	71
amiGetGlobalParams	Queries a parameter	91
amiGetHoleData	Queries a feature	125
amiGetHoleTapData	Queries a feature	125
amiGetIndFilletData	Queries a feature	132
amiGetInferGeom	Creates a key	69
amiGetInsideTermData	Queries a feature	162
amiGetKeyFromBrep	B-rep API integration	79
amiGetKeyFromId	Creates a key	70
amiGetKeyFromName	Creates a key	170
amiGetKeyFromPath	Creates a key	70
amiGetKeyFromPick	Creates a key	71
amiGetKeyType	Handles a key	73
amiGetLinearFilletData	Queries a feature	133
amiGetLinkInfo	Table-Driven Versions API	165

Function	Description	Page
amiGetLocator	Queries a feature	158
amiGetLoftData	Handles a feature descriptor	140
amiGetMassProps	Performs analysis	174
amiGetMasterCompDef	Handles a component definition	101
amiGetMidPlaneTermData	Queries a feature	162
amiGetNumDwVws	Drawing Manager	186
amiGetNumPartFeats	Queries a part	94
amiGetObjectName	Queries an object name	86
amiGetObjectState	Queries an object's state	86
amiGetObjectVisibility	Queries an object's visibility	87
amiGetOffsetLocData	Handles a feature descriptor	159
amiGetOnPlaneLocData	Handles a feature descriptor	159
amiGetOutsideTermData	Queries a feature	162
AmiGetOwningCompositeFeat	Handles a composite feature	121
amiGetParamComment	Handles a parameter	92
amiGetParamDeps	Queries a parameter	92
amiGetParametricBoolData	Handles a feature descriptor	139
amiGetParamExpress	Gets a parameter	92
amiGetParamName	Queries a parameter	92
amiGetParamUsers	Queries a parameter	93
amiGetParamValue	Queries a parameter	93
amiGetPartDefinition	Handles a component definition	102
amiGetPartFeats	Queries a part	94
amiGetPartParams	Queries a part	95
amiGetPartSplitData	Handles a split descriptor	144
amiGetPartWorkAxes	Queries a part	95
amiGetPartWorkPlanes	Queries a part	95
amiGetPartWorkVertices	Queries a part	96
amiGetPathStartPoint	Handles a sketch	115
amiGetPatternData	Handles a feature descriptor	151
amiGetPickInfo	Queries a pick object	68
amiGetPlaneDraftData	Handles a feature descriptor	142
amiGetPolarArrayData	Queries a feature	135
amiGetPolarPatternData	Handles a feature descriptor	153
amiGetRecArrayData	Queries a feature	136
amiGetRectangularPatternData	Handles a feature descriptor	155
amiGetRevolveData	Queries a feature	130
amiGetRibData	Handles a feature descriptor	151
amiGetSceneCompDef	Handles a scene	193
amiGetSceneCompDescrip	Handles a scene	193
amiGetSceneExplodeFactor	Handles a scene	193
amiGetSceneLock	Handles a scene	193
amiGetShadowDraftData	Handles a feature descriptor	143
amiGetShellData	Queries a feature	138
amiGetSketchConstrs	Queries a sketch	115
amiGetSketchCoordSys	Queries a sketch	115
amiGetSketchDefPlane	Queries a sketch	115

Function	Description	Page
amiGetSketchedSplineSegments	Handles geometry	76
amiGetSketchesFromGeom	Queries a sketch	116
amiGetSketchGeom	Queries a sketch	116
amiGetSketchParams	Queries a sketch	116
amiGetSketchPlaneDescrip	Handles a sketch plane	148
amiGetSketchPlaneFromSketch	Handles a sketch plane	148
amiGetSketchType	Queries a sketch	116
amiGetSurfCutData	Queries a feature	138
amiGetSystemAttributeName	Queries the name of a specified system attribute	88
amiGetSweepData	Queries a feature	134
amiGetSysVar	Queries a system variable	88
amiGetTangentLocData	Handles a feature descriptor	159
amiGetTerminator	Queries a feature	162
amiGetTermType	Queries a feature	162
amiGetThinExtrusionData	Queries a feature	129
amiGetToFaceTermData	Queries a feature	163
amiGetToPlaneTermData	Queries a feature	163
amiGetUniFilletData	Queries a feature	133
amiGetUnusedSketchesFromPart	Queries a part	95
amiGetWorkPlaneData	Handles a feature descriptor	140
amiHasColorOverride	Handles object color	88
amiHighlight	Controls entity highlighting	75
amiInferGeomKey	Creates a key	71
amiIsBrowserDisplayed	Handles browser display	196
amiIsArrayInstanceIndependent	Handles an array descriptor	149
amiIsCompDefExternal	Handles a component definition	102
amiIsCompDefLeafNode	Handles a component	102
amiIsCompDefMaster	Handles a component definition	102
amiIsCompToolbody	Handles a feature	121
amiIsDescripKindOf	Queries a feature	123
amiIsDirectlyEditable	Handles objects	89
amiIsFeatKindOf	Queries a feature	121
amiIsFeatSuppressed	Controls suppression	97
amiIsKeyKindOf	Handles a key	73
amiIsKeyNull	Handles a key	74
amiIsPDMRegistered	Handles file descriptors	198
amiLineIsSilhouette	Handles geometry	76
amiLocalize	Handles a component definition	103
amiMakeInstAtt	Handles an instantiable attribute	83
amiMinDistance	Handles analysis	174
amiMoveDwAnnot	Drawing Manager	190
amiObjectHasChanged	Handles change notification	78
amiObjectWasErased	Handles change notification	78
amiPick	Creates a pick object	69
amiReactDescrip::make	Event handling	166
amiReadAttClassesFromFile	Handles an instantiable attribute	83

Function	Description	Page
amiReadKey	Handles a key	74
amiReassignComps	Handles a component definition	103
amiRefocusKey	Handles a key	74
amiRegen	Controls regeneration	96
amiRegenAllOfAType	Controls regeneration	97
amiRegisterApp	Event handling	169
amiRegisterEventReaction	Event handling	169
amiRemoveAttribute	Handles attributes	81
amiRemoveColorOverride	Handles object color	89
amiRemoveEventReaction	Event Handling	170
amiRemoveBrowserTab	Manipulates a user interface	197
amiRemoveCompFromCompDef	Handles a component definition	103
amiRemoveFile	Handles file descriptors	198
amiRemoveLink	Table-Driven Versions API	165
amiReorderFeat	Handles a feature	121
amiRollbackPart	Handles a part	96
amiSetAbsoluteLocData	Handles a feature descriptor	159
amiSetActiveAssembly	Handles a component	106
amiSetActiveCompDef	Handles a component definition	106
amiSetActiveLeafNode	Handles a component definition	174
amiSetAngledLocData	Handles a feature descriptor	159
amiSetArrayFeature	Handles a feature descriptor	136
amiSetAtomLocData	Handles a feature descriptor	160
amiSetAttField	Handles an instantiable attribute	83
amiSetAxialPatternNRevolutions	Handles a feature descriptor	153
amiSetAxialPatternOffsetDirection	Handles a feature descriptor	153
amiSetAxialPatternOffsetHeight	Handles a feature descriptor	153
amiSetAxialPatternOffsetSpacingType	Handles a feature descriptor	153
amiSetBaseData	Handles a feature descriptor	128
amiSetBendAngle	Handles a feature descriptor	149
amiSetBendArcLen	Handles a feature descriptor	150
amiSetBendDirection	Handles a feature descriptor	150
amiSetBendProfile	Handles a feature descriptor	150
amiSetBendRadius	Handles a feature descriptor	150
amiSetBendSide	Handles a feature descriptor	150
amiSetBendType	Handles a feature descriptor	150
amiSetBlindTermData	Handles a feature descriptor	163
amiSetCBoreHoleCbDepth	Handles a feature descriptor	126
amiSetCBoreHoleCbDiam	Handles a feature descriptor	126
amiSetChamferAngle	Handles a feature descriptor	131
amiSetChamferAngleFace	Handles a feature descriptor	131
amiSetChamferDist1	Handles a feature descriptor	131
amiSetChamferDist1Face	Handles a feature descriptor	131
amiSetChamferDist2	Handles a feature descriptor	131
amiSetChamferEdges	Handles a feature descriptor	132
amiSetChamferType	Handles a feature descriptor	132
amiSetCombinerType	Handles a feature descriptor	123

Function	Description	Page
amiSetCompDefName	Handles a component definition	104
amiSetCompName	Handles a component	106
amiSetCompositeFeatDrivingParams	Handles a feature descriptor	124
amiSetCompositeFeatEditCallback	Handles a composite feature	121
amiSetCompositeFeatOwnerInfo	Handles a feature descriptor	124
amiSetCompositeFeatType	Handles a feature descriptor	124
amiSetCompPosition	Handles a component	107
amiSetConstrExpr	Handles a constraint	113
amiSetConstrValue	Handles a constraint	113
amiSetCoordSysLocData	Handles a feature descriptor	160
amiSetCSinkHoleCsAngle	Handles a feature descriptor	126
amiSetCSinkHoleCsDiam	Handles a feature descriptor	126
amiSetCubicFilletRadii	Handles a feature descriptor	133
amiSetCurrentVersion	Table-Driven Versions API	165
amiSetCustomAHL	Handles hidden line calculations	195
amiSetData	Handles a constraint	110
amiSetData	Handles a sketch descriptor	146
amiSetData	Handles a file descriptor	148
amiSetData	Handles an event	170
amiSetData	Drawing Manager	187
amiSetData	Drawing Manager	191
amiSetData	Handles a scene	193
amiSetData	Handles a feature descriptor	170
amiSetDesignMode	Handles design mode	171
amiSetDistFromLocData	Handles a feature descriptor	160
amiSetDraftAngle	Handles a feature descriptor	143
amiSetDraftPlane	Handles a feature descriptor	143
amiSetDraftTangents	Handles a feature descriptor	143
amiSetDwAnnotData	Drawing Manager	191
amiSetDwVwData	Drawing Manager	187
amiSetDwVwDataArray	Drawing Manager	187
amiSetEdgeDraftFaces	Handles a feature descriptor	143
amiSetExtrusionDirectionVec	Handles a feature descriptor	129
amiSetExtrusionDraftAngle	Handles a feature descriptor	129
amiSetExtrusionProfile	Handles a feature descriptor	129
amiSetFaceSplitFaces	Handles a split descriptor	145
amiSetFeatPart	Handles a feature descriptor	123
amiSetFilletEdges	Handles a feature descriptor	133
amiSetFixedFilletChordLength	Handles a feature descriptor	133
amiSetFromToTermData	Handles a feature descriptor	163
amiSetGeometry	Handles geometry	77
amiSetHoleDiameter	Handles a feature descriptor	126
amiSetHoleDirectionVec	Handles a feature descriptor	126
amiSetHoleDrillAngle	Handles a feature descriptor	126
amiSetHoleFitClass	Handles a feature descriptor	127
amiSetHoleNominalSize	Handles a feature descriptor	127
amiSetHolePitch	Handles a feature descriptor	127

Function	Description	Page
amiSetHoleTapData	Handles a feature descriptor	127
amiSetHoleTapDrillDiameter	Handles a feature descriptor	127
amiSetHoleUnits	Handles a feature descriptor	128
amiSetIndFilletRadii	Handles a feature descriptor	134
amiSetInsideTermData	Handles a feature descriptor	163
amiSetLinearFilletRadii	Handles a feature descriptor	134
amiSetLocator	Handles a feature descriptor	160
amiSetLoftAngles	Handles a feature descriptor	141
amiSetLoftMinimizeTwist	Handles a feature descriptor	141
amiSetLoftType	Handles a feature descriptor	141
amiSetLoftXSections	Handles a feature descriptor	142
amiSetMidPlaneTermData	Handles a feature descriptor	164
amiSetObjectName	Sets an object name	89
amiSetObjectVisibility	Sets an object's visibility	89
amiSetOffsetLocData	Handles a feature descriptor	159
amiSetOnPlaneLocData	Handles a feature descriptor	161
amiSetOutsideTermData	Handles a feature descriptor	164
amiSetParamComment	Sets a parameter	93
amiSetParametricBoolComp	Handles a feature descriptor	139
amiSetParamExpress	Sets a parameter	93
amiSetParamValue	Edits a parameter	93
amiSetPartSplitFlip	Handles a split descriptor	145
amiSetPartSplitName	Handles a split descriptor	145
amiSetPatternFeatures	Handles a feature descriptor	152
amiSetPatternSuppressedInstances	Handles a feature descriptor	152
amiSetPlaneDraftFaces	Handles a feature descriptor	144
amiSetPolarArrayAngle	Handles a feature descriptor	136
amiSetPolarArrayNumInstances	Handles a feature descriptor	136
amiSetPolarArrayPolarAngleType	Handles a feature descriptor	137
amiSetPolarArrayRotateAsCopied	Handles a feature descriptor	137
amiSetPolarPatternAngle	Handles a feature descriptor	154
amiSetPolarPatternNInstances	Handles a feature descriptor	154
amiSetPolarPatternOrientation	Handles a feature descriptor	154
amiSetPolarPatternRotationDirection	Handles a feature descriptor	154
amiSetRecArrayColData	Handles a feature descriptor	137
amiSetRecArrayRowData	Handles a feature descriptor	137
amiSetRecArrayXVector	Handles a feature descriptor	137
amiSetRecArrayYVector	Handles a feature descriptor	137
amiSetRectangularPatternAlignmentAngle	Handles a feature descriptor	155
amiSetRectangularPatternColumnData	Handles a feature descriptor	155
amiSetRectangularPatternColumnDirection	Handles a feature descriptor	156
amiSetRectangularPatternRowData	Handles a feature descriptor	156
amiSetRectangularPatternRowDirection	Handles a feature descriptor	156
amiSetRectangularPatternSketchPlane	Handles a feature descriptor	156
amiSetRevolveAxis	Handles a feature descriptor	130
amiSetRevolveDirectionVec	Handles a feature descriptor	130
amiSetRevolveProfile	Handles a feature descriptor	130

Function	Description	Page
amiSetRibDirection	Handles a feature descriptor	151
amiSetRibProfile	Handles a feature descriptor	151
amiSetRibThickness	Handles a feature descriptor	151
amiSetSceneLock	Handles a scene	194
amiSetShadowDraftFaces	Handles a feature descriptor	144
amiSetShellExcludes	Handles a feature descriptor	139
amiSetShellOverrides	Handles a feature descriptor	139
amiSetSplitCombinerPartName	Handles a feature descriptor	145
amiSetPatternSuppressedInstances	Handles a feature descriptor	152
amiSetSurfCutDirectionVec	Handles a feature descriptor	138
amiSetSurfCutSurf	Handles a feature descriptor	138
amiSetSweepDraftAngle	Handles a feature descriptor	134
amiSetSweepMethod	Handles a feature descriptor	135
amiSetSweepPath	Handles a feature descriptor	135
amiSetSweepProfile	Handles a feature descriptor	135
amiSetSystemAttributeName	Handles an MDT system attribute name	90
amiSetSysVar	Handles a system variable	90
amiSetTangentLocData	Handles a feature descriptor	161
amiSetTerminator	Handles a feature descriptor	164
amiSetThinExtrusionExtend	Handles a feature descriptor	129
amiSetThinExtrusionThickness	Handles a feature descriptor	129
amiSetToFaceTermData	Handles a feature descriptor	164
amiSetToPlaneTermData	Handles a feature descriptor	164
amiSetUniFilletRadius	Handles a feature descriptor	134
amiStartCompositeFeat	Handles a feature	122
amiStartExternalEdit	Handles external file edits	90
amiSuppressFeat	Controls suppression	97
amiSuppressFeatsByType	Controls suppression	97
amiThawLayers	Handles a view	188
amiUndefineAttClass	Handles an instantiable attribute	84
amiUnsuppressFeat	Controls suppression	98
amiUnsuppressFeatsByType	Controls suppression	98
amiUnsuppressPartFeats	Controls suppression	98
amiWriteAttClassesToFile	Handles an instantiable attribute	84
amiWriteAttClassToFile	Handles an instantiable attribute	84
amiWriteKey	Handles a key	75

List of Tables

Below is a list of tables in this document, along with the page number on which it is found.

Table Name	Page
Constraint Attribute Keywords	107
Constraint Attribute Keywords - Assembly Constraints	107
Constraint Attribute Keywords - Sketch Constraints	107
Constraint Attribute Keywords - Sketch Dimensions	107
Event Reaction Attributes	107
Sketch Attribute Keywords	146
Event Keywords	166
Callback Function Pointer Typedefs	168
: Degrees of Freedom Attribute Keywords	171
View Attribute Keywords	176
View Attributes: kBaseVw	179
View Attributes: kOrthoVw	180
View: kIsoVw	180
View Attributes: kDetVw	180
View Attributes: kAuxVw	181
View: kBrokenVw	181
Drawing Manager View Section Types	182
Annotation Attribute Keywords	188
Scene-Comp Attribute Keywords	191
File Attribute Keywords	199
File Attribute Keywords - continued	200

Chapter 1
Overview

This document introduces the Autodesk Mechanical Application Programming Interface (MCAD API), an application development tool for creating associative mechanical applications. The MCAD API enhances and extends the ObjectARX™ development environment.

Autodesk mechanical applications have quickly become the desktop standard mechanical design automation platform with hundreds of thousands of users worldwide. The MCAD API provides a high level of direct access to AutoCAD mechanical data objects and is intended to support the development of a broad base of interoperable design automation, manufacturing and visualization products.

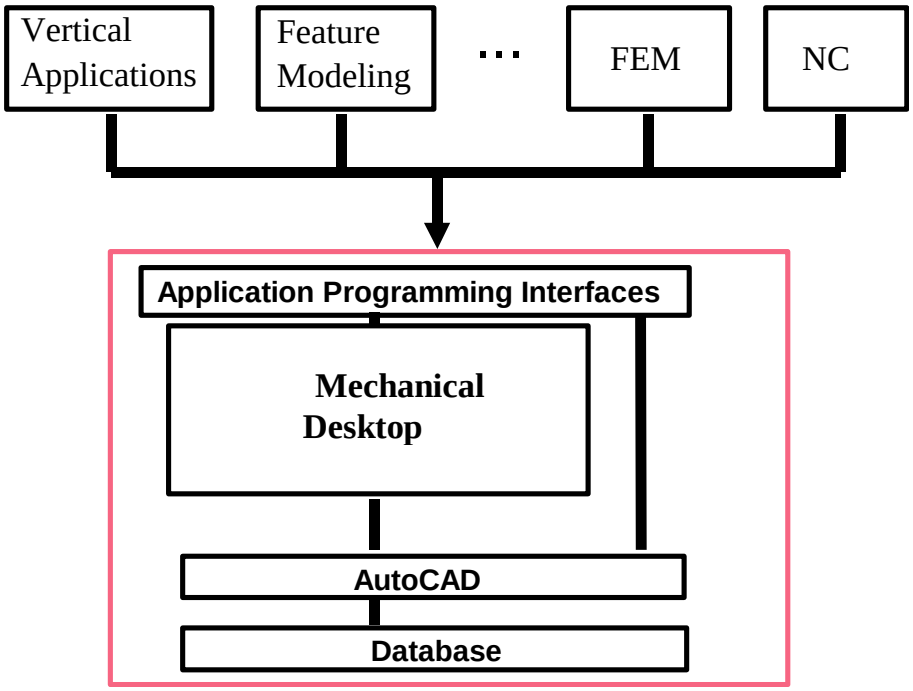
With this API, third-party developers can create applications that feature parametric editing and associativity—requirements that have become baseline features in today's modern mechanical CAD marketplace. Using the MCAD API, developers of mechanical applications have more time to focus on higher level modeling techniques and new modeling and analysis paradigms.

The MCAD API program creates opportunities for developers to create and market applications in the following areas (among others):

- Manufacturing and tooling, driven by free-form surface or solid models
- Structural and plastic molding analysis
- Finite element analysis with associative mesh and boundary conditions
- Design productivity tools, such as symbol libraries or parametric mechanical features
- Tolerance analysis
- Parametric content distribution
- Form feature toolkit through user-defined features
- Sheet metal design
- Assembly modeling extensions, BOM enhancement, and SQL links
- Kinematic or fully dynamic analysis
- Bill of materials input/output
- Electronic PCB geometry interface

API Goals

Applications communicate through a common compiled-language interface, so a high level of integration is achieved between third-party applications and the Mechanical Desktop. From the developer's point of view, integrated applications eliminate the time-consuming and often inaccurate task of translating data. Developers can improve the functionality of their designs, bring products to market faster, and make their products more attractive to the large AutoCAD user base.



The MCAD API puts programmatic access to the Mechanical Desktop and AutoCAD under one common layer. The MCAD API provides access not only to multiple modeling paradigms and unrestricted modeling environments (manifold solids and surface modeling), but also a unified access mechanism for AutoCAD and Mechanical Desktop geometry. This mechanism, geometric keys, allow uniform references to all types of model geometry, including: wireframe, surface, solid and parametric part geometry; geometry that is in the local database or in an external database; geometry that is on the model or that is inferred from existing geometry, etc.

Extensive mechanisms for attribute creation, attachment and editing are provided. Users of the API have full access to all the data defining parts, assemblies, drawing views and tables used, as well as a comprehensive suite of modeling event notifications and the possibility of customizing the browser and the appearance of components in drawing views.

Interaction with Other Libraries

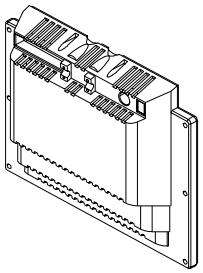
The MCAD API interacts with the various ObjectARX™ libraries to achieve a high degree of integration among AutoCAD-based applications.

AcBr Library

This library is the interface to query topological information from a geometric item, when that information is present. AutoCAD solid models and Mechanical Desktop part models are B-rep solids, which consist of a collection of topological and geometric entities.

The MCAD API interacts with the boundary representation query library, AcBr, to access topological and geometric data. From a geometry key a B-rep entity can be returned, initialized to that location. The entity can then be used to traverse the B-rep to get connectivity information and to create new keys. The following solid entities can be queried without a geometry key just by using methods in the AcBr library:

- Solids (AcDb3dSolid)
- Bodies created by exploding solids (AcDbBody)
- Regions (AcDbRegion)



The AcBr library transfers read-only data about a model into your application for display, analysis, or manipulation. It locates particular features of interest in the model and queries them for information, such as their geometry and connectivity.

AcGe Library

The MCAD API interacts with the Geometry Library to retrieve geometry. The AcGe library consists of utility classes for performing common 2D and 3D algebra, such as vectors and matrices. It provides geometric objects such as points, curves, and surfaces. AcGe objects returned by the MCAD API are copies of application objects that are returned by geometry queries. Additional AcGe objects can also be directly created by the library user. AcGe objects are taken in as the data specification in geometric editing functions. The AcGe library supports geometry query and manipulation.

ObjectARX™ Libraries

ObjectARX™ libraries offer the following general functionality:

- Creating menus and registering commands
- Handling user input/output
- System queries, database operations, and graphics display

Please note that you cannot open a Mechanical Desktop file from the Mechanical Desktop using ObjectARX.

Supported Platforms

Applications may be developed for Windows 98, Windows NT, Windows 2000 and Windows Me.

Chapter 2
MCAD API FEATURES

This chapter discusses MCAD API features and benefits for third-party applications, object keys, and the MCAD API architecture. The Mechanical Desktop modules referred to in the items below include part modeling, assembly modeling, drawing management, etc.

Uniform Object Referencing

Users refer to application objects uniformly, regardless of which module creates and owns the referred object. This approach reduces the number of exported objects, allows tighter integration across modules, and is one of the technologies necessary for a unified API.

Unified API for All Modules

The MCAD API is unified across all Mechanical Desktop modules. The same interface allows users to interact with all currently offered Mechanical Desktop modulesand with new ones that might be released in the future. The unified interface has the following advantages:

- Reduces the number of exported functions
- Eliminates redundancies and inconsistencies in exporting similar functionality Mechanical Desktop modulesMakes the API easier to learn
- Reduces the amount of system documentation required
- Promotes development of applications that work uniformly across models from the different Mechanical Desktop modules

All Mechanical Desktop modules subscribe to the same API, so the breadth of third-party applications can be increased in an automatic, transparent manner without being recompiled by the third-party developer. Today, for example, Mechanical Desktop surface geometry cannot be constrained. The MCAD API offers methods that deal with geometrical constraints, but at this time these methods are effective only when the parametric part modeler owns the geometry. In the future, the Mechanical Desktop might support constrained surface geometry. If and when that happens, the calls in the API that handle constraints will be honored by the Desktop. Third-party applications that made use of these calls and used to work only on parametric solid models will automatically start to work on Mechanical Desktop surface models as well.

Powerful Geometric Abstraction

With the MCAD API geometry keys, developers refer to geometric objects uniformly, without concern for the details of how that geometry was created or is stored. For example, an application refers to a line in the same manner, regardless of whether that line is a first-class database entity, a linear edge of a solid, or even inferred from other objects. This mechanism greatly facilitates writing code that is simple and robust.

Support for Geometry Change Notification

Application developers can get active (immediate) and passive (upon request) notifications of changes affecting any geometry used by any Mechanical Desktop module. Developers can use the type of notification that best fits their application needs. The two forms of notification can also be mixed and matched without limitations.

Referential Integrity Across Parametric Regenerations

Any reference to a Mechanical Desktop part entity remains valid across parametric regenerations (if a mapping exists from the entity in the model before regeneration to some entity in the new model). The system takes care of managing the mapping. With this capability, sophisticated applications can be written that are associative to the underlying model and make full use of the benefits of parametric, feature-based solid modeling. For example cutter paths or loads can be associated to faces in the model, and such relationships remain valid across model edits.

Attribute Support

The MCAD API supports attribute creation, customization, attachment, query, and edits. In addition, attributes that are attached to entities in a parametric model remain attached across parametric regenerations even though the actual model entity is deleted and recreated in the process. This functionality is very important for a variety of applications that need to attach attributes to parts of the underlying model. Custom attribute classes can be created and managed using the MCAD API.

Synergistic Integration with Other Autodesk APIs

The MCAD API makes use of other Autodesk APIs to enhance and facilitate third-party application interaction with the underlying model. The Geometry Library provides surface and curve modeling information and functionality. The B-rep API allows topological information to be obtained, and ObjectARX handles database, graphics, and user interface issues.

The MCAD API focuses on those aspects that are specific to mechanical applications, and relies on the other APIs for the additional functionality required by third-party applications. As more functionality is added to the other Autodesk APIs, users of the MCAD API benefit automatically.

Support for Asynchronous Releases of All Components

The MCAD API application and the different Mechanical Desktop modulescan be released asynchronously without forcing recompilation of existing third party applications. The third party applications might immediately benefit from some of the new enhancements without having to recompile. Other new features that would require code changes or recompilation can be taken advantage of at a later time.

This gives a lot of flexibility to third-party developers in terms of coordinating their enhancements with new releases of the MCAD API and of Mechanical Desktop modules.

Utilities for ObjectARX

The MCAD API provides utilities that enhance capabilities, such as picking and highlighting, that traditionally have been hard to use in the context of ObjectARX. The developer is not required to deal with and traverse low-level data structures if that level of flexibility is not required. Users can concentrate more on their application problems and less on peripheral issues.

Mechanical Desktop Part and Surface Modeling

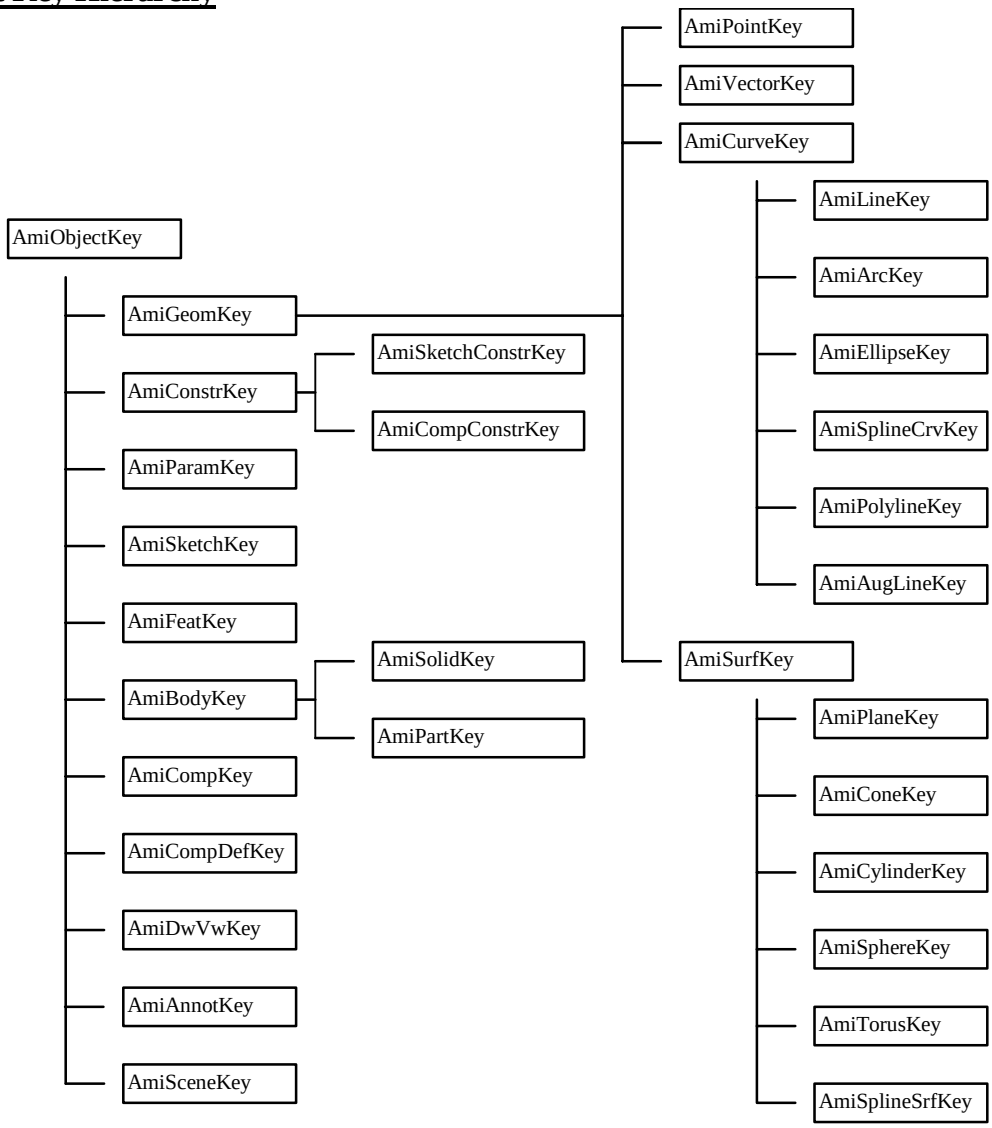
The modeling power and sophistication formerly offered by the Mechanical Desktop, such as feature and constraint handling or complex surface modeling capabilities, are made directly available to third-party developers with the MCAD API.

Object Keys

Third-party applications use keys to refer to application objects. Keys are persistent and uniform across applications. All the necessary information to resolve the key to the referred application object is stored in the key. An application identifier that allows the system to quickly identify which application manages the key is also stored in the key. When a key is passed to an MCAD API function, the system knows what application created the key, so it can route the call to the proper piece of code.

Keys are C++ objects grouped in a terse hierarchy that provides users with type checking and hierarchical characterization. Because the MCAD API uses a functional interface to export functionality, methods are not present in the object key hierarchy. Keys are passed and returned as arguments of API functions. This approach insulates users from changes in the MCAD API implementations that might cause applications to break. The functions fully use the key hierarchical characterization, so that at compile time inappropriate key types cannot be passed into the function.

Object Key Hierarchy



The following key classes are defined by the MCAD API:

Abstract Classes	Parent of
AmiObjectKey	key classes
AmiCurveKey	curve key classes
AmiGeomKey	geometric key classes
AmiSurfKey	surface key classes
AmiBodyKey	body key classes

Instantiable Classes	Refers to a
AmiConstrKey	constraint
AmiParamKey	parameter
AmiSketchKey	sketch
AmiFeatKey	feature
AmiPartKey	part
AmiSolidKey	AutoCAD solid
AmiSketchConstrKey	sketch constraint
AmiCompConstrKey	component constraint
AmiCompDefKey	component definition
AmiCompKey	component
AmiPointKey	point
AmiVectorKey	vector
AmiLineKey	line
AmiArcKey	full or partial circular arc
AmiSplineCurveKey	spline curve
AmiEllipseKey	full or partial elliptical arc
AmiPolylineKey	polyline
AmiAugLineKey	augmented line*
AmiPlaneKey	plane
AmiConeKey	cone
AmiCylinderKey	cylinder
AmiSphereKey	sphere
AmiTorusKey	torus
AmiSplineSrfKey	spline surface
AmiDwVwKey	drawing view
amiDwAnnotKey	drawing annotation
amiSceneKey	scene

***** A polyline with a vector associated with each vertex

API users do not need to be concerned with the internal information stored in the key, what application created it, or how to handle the key. API functions construct a key, check if a key is null, save and restore a key, and release the key. The following paradigm is normally followed by the MCAD API user in working with keys:

1. Obtain an object key to a mechanical application object by calling appropriate API functions.
2. Use the key to identify the object when calling functions in the MCAD API.
3. Check the status returned by functions.
4. Save or restore the key if necessary.
5. Release the key when it is no longer needed.

Using the MCAD API Referencing Mechanism

The MCAD API provides a persistent, uniform referencing mechanism. This mechanism allows application developers to refer to objects in models without having to account for the details of which application created the object being referred to or the particular representation details for that object (e.g., whether the object being referred to is a first class database entity, is inside a block reference, is a subentity inside another object, or is inferred from other existing objects). An application refers to an application object by holding onto a key to the object. Keys to application objects are created at the third party developer's request.

Currently, in addition to geometric keys, keys to Mechanical Desktop parts, parameters, assembly constraints, components, component definitions, scenes, sketches, and features, drawing views, drawing annotations, as well as AutoCAD solids, are supported. The geometric coverage available encompasses the following types of geometric entities:

- AutoCAD points
- AutoCAD lines
- AutoCAD arcs
- AutoCAD circles
- AutoCAD ellipses
- AutoCAD splines
- AutoCAD Polylines (2D or 3D) except non-simple polylines
- Any vertex on a Mechanical Desktop part or surface, or an AutoCAD solid, body, or region
- Any edge on a Mechanical Desktop part or surface, or an AutoCAD solid, body, or region
- Any face on a Mechanical Desktop part or an AutoCAD solid or body
- Mechanical Desktop surfaces
- Mechanical Desktop augmented lines
- Mechanical Desktop workpoints, workaxis, and workplanes

In addition to these direct references, the following inferences, on top of these references, are also supported:

- Point as the endpoint of an open curve
- Point as the midpoint of a linear segment
- Point as the center point of a circle, circular arc, or ellipse
- Line as the center axis of a circle or circular arc
- Plane as the plane of a circle or circular arc

Summary

The following points highlight some key features of the MCAD API architecture:

- The MCAD API is based on a flexible architecture that is designed to be extensible.
- Object keys are used in most interactions between the client application and the MCAD API.
- Most API functions delegate processing to different Mechanical Desktop modules such as part, assembly or surface API implementation modules.
- The MCAD API uses information in the object keys to determine how to delegate the processing.
- Some API functions, such as the selection and highlighting utilities, interface directly with AutoCAD.
- The modular design allows Autodesk to ship updates of the API modules as well as the Mechanical Desktop without affecting released 3rd party applications.

- Future Mechanical Desktop modulescan be easily integrated into the existing MCAD API architecture. Existing client applications of the MCAD API are automatically supported by the new Mechanical Desktop modules

Chapter 3
MCAD API Guidelines

Attribute Creation in the MCAD API

Basic Functionality

The MCAD API allows attributes to be created and attached to any geometry for which a key can be obtained. That includes both geometry directly present in the model and geometry inferred from existing geometry. For the specific function calls that can be made to handle attributes, see the descriptions for *amiAddAttribute*, *amiRemoveAttribute*, *amiGetAttributes*, and *amiGetAttributeHolders* in chapter 3.

The attributes are attached *through* the key, not *to* the key. In other words, the attribute can be thought of as being attached to the entity being referred to by a key, as opposed to being an attribute of the key itself. So, suppose attribute *a1* is attached through *key1* and attribute *a2* is attached through *key2*. If *key1* and *key2* are equivalent (i.e., resolve to the same entity), asking for all attributes attached through *key1* will yield both attributes *a1* and *a2*. Identical results would have been obtained if the query for attached attributes had been done through *key2*. Furthermore, erasing *key1*, *key2,* or both has no effect on the attribute or its attachments.

One attribute object can be simultaneously attached to several geometric items; thus, changes to one attribute object can simultaneously affect several other objects.

Removing an attribute from an object does not cause the attribute to be erased. If an attribute is explicitly erased by an application it will not be returned as part of any attribute query.

The attribute attachment is persistent across model regenerations if the geometry that the attribute is attached to is part of a Mechanical Desktop part model.

Attribute Creation, Interoperability, and Customizability

Application developers using the MCAD API will be able to create attributes in two different ways. They can define their own attribute class, which should be derived from the abstract base class *AmiAttribute*, or, alternatively, they can use the general instantiable attribute class that will be provided by the MCAD API.

A few considerations must be made when choosing whether to derive your own class or use the built-in instantiable mechanism. An intrinsic trade-off between maximizing interoperability or customizability takes place.

If you decide to derive your own class, you can customize your implementation to suit your needs precisely. Because the new class must derive from *AmiAttribute*, it must support the virtual protocol defined at that level, which automatically provides some level of interoperability as it allows other applications to query the attribute content. Data not appropriate to be shared by another application can be stored privately, without allowing other applications access by the query routines, declared at the *AmiAttribute* level. The code for the new attribute class will live with the application that created the attribute (the creating application). For another application to access the attribute, the creating application must be loaded. Otherwise, the attribute is not accessible and is indeed a proxy object, as defined by ObjectARX™.

On the other hand, if you decide to use the general instantiable attribute class provided by the MCAD API, your attribute objects that have been saved to a DWG file can be accessed without the application that originally created the attribute being present. In this scenario you can increase the interoperability of your attribute objects, but you forgo the possibility of customizing the attribute implementation. Using instantiable attributes also saves you time because you don't have to provide an implementation for your attribute class.

Attribute Management

Attributes attached to an entity must be properly managed when that entity is copied, exported to another database or imported from another database. There are two ways to control how attributes are managed: attribute ownership, which is specified when the attribute is created, and attribute associativity, which is specified when an attribute is attached to an entity.

Attribute Ownership

The attribute owner is specified when the attribute is created and cannot be changed during the lifetime of the attribute. Either the MCAD API "owns" and controls the attribute, or the application does. When the MCAD API manages the attribute, it ensures that the attribute is cloned, exported or imported along with the entity to which it is attached. If the API is not the owner, then the application is responsible for triggering a clone of the attribute and ensuring that the owner of the attribute is appropriately cloned as well. Instantiable attribute objects are always owned by the MCAD API, but instances of custom attribute classes can be set to be owned by the API on an instance-by-instance basis.

If the attribute is not owned by the MCAD API, the attribute can be ignored automatically, but the copy behavior depends on the owning application. If the application is not properly cloning the attributes it controls, then after a copy the attribute will be shared. If the entity is exported (using the AutoCAD **WBLOCK** command) then the attribute will be dropped, because the API can no longer access the object that owns the attribute and therefore cannot clone the attribute.

Attribute Associativity

The associativity of an attribute is specified when the attribute is attached to a specific object. It controls what should happen to the attachment of this particular attribute when the object is copied, exported or imported. The current options are copy, share, or ignore for all actions. "Copy" means to copy the attribute and attach the new copy to the new object. "Share" means to attach the original attribute to the new object. "Ignore" means don't include this attribute on the new object. More precise control over attribute behavior (for example, the option to share during an internal copy but ignore when exporting) is planned for future releases of the MCAD API.

The AmiAttribute Class

The AmiAttribute class is the abstract base class for all attribute classes. It defines a set of query methods that allow any application to get information about the content of an attribute created by any other application. The following methods are provided:

Public Methods

virtual AmiStatus getOwningApp (const char*& *appName*) const

Gets the name of the application that created the attribute.

Output

appName Application name.

AmiStatus getName (const char*& *attributeName*) const

Gets the attribute name.

Output

attributeName Attribute name.

AmiStatus getNumFields (int& *numFields*) const

Gets the number of fields in the attribute.

Output

numFields Number of attribute fields.

AmiStatus getFieldName (int *index*, const char*& *fieldName*) const

Gets the field name corresponding to the given index.

Input

index Index for the desired field (starts at zero).

Output

fieldName Field name.

AmiStatus getFieldDesc (int *index*, const char*& *fieldDesc*) const

Gets a string that gives a description of the meaning of the specified field.

Input

index Index for the desired field (starts at zero).

Output

fieldDesc Description of field meaning.

AmiStatus getFieldType (int *index*, AmiFieldType& *type*) const

Gets the data type contained in the specified field.

Input

index Index for the desired field (starts at zero).

Output

type Type of the data contained in the field.

AmiStatus getFieldCont (int *index*, const void*& *pContent*) const

Gets the data stored in the specified field by giving the caller a pointer to the data location.

Input

index Index for the desired field (starts at zero).

Output

pContent Pointer to data location for the field.

AmiStatus setFieldCont (int *index*, const void*& *pContent*) const

Sets the data stored in the specified field by giving the caller a pointer to the data location from where to copy the new data.

Input

index Index for the desired field (starts at zero).

Output

pContent Pointer to data location for the data to be used in updating the field.

Applications defining their own attribute classes must implement these methods. Pointers set by any of the public methods are assumed to point to memory that does *not* need to be freed by the caller.

Privacy of some of the fields in an attribute can be obtained by not including them in the set of fields that can be accessed through the query methods defined by AmiAttribute. Thus, if an attribute class holds on to three public attributes and two private ones, *getNumFields* could be implemented to return 3 as the number of fields in the attribute and valid field indices for the other methods would be 0, 1, and 2 only.

Integration of MDT System Attributes and MCADAPI Attributes

In Mechanical Desktop5, the attributes assigned to a component definition via the AMASSIGN command have been unified with MCADAPI attributes. Using AMSSSIGN, MDT users are allowed to create and attach attributes to a component definition. The attributes created via the AMASSIGN command are a special type of AmiAttribute called a "system attribute". The system attribute contains a single field. The data type of that field can be a string, a single double value, or a single integer value.

With a key to a component definition in hand, any attributes attached to that component definition via AMASSIGN will be returned from a call to "amiGetAttributes" along with any other kinds of attributes attached to that definition via the API. With a pointer to a system attribute, the methods defined on AmiAttribute can be used to set or retrieve data from the attribute field.

With the addition of these system attributes, the MDT5 API provides additional functions allowing the API user to create the special system attributes. Using the API, a system attribute can be created, named, and assigned a value. If and when one of these attributes is added to a component definition via the API, that attribute becomes indistinguishable from an attribute attached to the component definition via AMASSIGN.

The function to create a system attribute is:

```
AmiStatus amiCreateSystemAttribute(const char* pName,
                                   Ami::AttrDataType dataType,
                                   AmiAttribute*& pAttrib);
```

The first argument is the system attribute name. Any system attribute attached to a component definition must be named and that name must be unique within the component definition. The second argument is the data type for the single field. Valid types are Ami::kString, Ami::kLong, and Ami::kDouble. The third argument is an attribute pointer that will be filled in by the function assuming the call is successful. Once an attribute is created using this function, it can be attached to the component definition by calling amiAddAttribute on the component definition key.

There are two additional functions that were added for working with system attributes. As was mentioned previously, system attribute must have unique names within their component definition. Although the name is specified at the time the attribute is created, additional functions have been added to allow the name to be changed on an existing attribute and also for the name to be queried.

```
AmiStatus amiSetSystemAttributeName(AmiAttribute* pAttrib, const char*
pName);

AmiStatus amiGetSystemAttributeName(AmiAttribute* pAttrib, std::string&
name);
```

MCAD API Instantiable Attributes

The general instantiable attribute class allows attributes to be defined by developers without the need to implement C++ classes. An arbitrary number of fields are allowed in an attribute. Attribute schemas are definable at runtime. You can manipulate the attribute even when the application that created it is not loaded. Efficient access time and small memory footprints are provided.

Notation

The word "class" in double quotes is used to indicate an MCAD API run-time "class" as opposed to a C++ class.

Problem

An application can define its own attribute class by deriving from the abstract class AmiAttribute, implementing the custom methods required for that attribute class, and implementing the virtual functions in the base class. These attributes can be saved in the DWG file; however, if one of these attributes is read and the application that defined this attribute is not loaded, the attribute will be a proxy.

Instantiable Attributes

The instantiable attribute class is maintained by the MCAD API and can be used by all client applications regardless of whether the defining application is loaded or not. This new attribute class has the following characteristics:

- Attribute objects contain an arbitrary number of fields of different types.
- Attribute objects can be exchanged freely through DWG files regardless of whether or not the application that defined the attribute is loaded at the time the attribute is accessed.
- An application will be able to instantiate an attribute whose "class" was defined by another application that may not currently be loaded.
- Attribute "classes" can be defined at runtime.
- The application defining a new attribute "class" does not need to implement any methods for the new "class."
- Static data can be defined for a given attribute "class." Such data is shared by all instances of that "class."
- Efficient data access time and small memory footprint for each instance are provided.

Defining Attribute "Classes"

An instantiable attribute "class" can be defined in one of three ways.

- An application can programmatically define a new attribute "class" and then create instances of the new attribute.
- An application can acquire attribute "classes" by loading a DWG file containing attributes defined by another application.
- Instantiable attribute "classes" can be imported/exported via an ASCII attribute definition file.

Attribute "Classes" Defined Programmatically

To programmatically define a new attribute "class," specify descriptions of all the fields to a single MCAD API function. The file *miattrib.h* includes a field description structure defined as follows:

```
struct AmiAttFieldDesc
{
    const char*    mFieldName;
    const char*    mFieldDesc;
```

```
    AmiFieldType   mFieldType;
    Adesk::Boolean mStaticFlag;
};
```

The field description members are as follows:

```
mFieldName  - Text string containing the name of the attribute field.
mFieldDesc  - Text string containing a description of the field.
mFieldType  - The data type of the field. This can be one of the following values:
                kAmiShort
                kAmiLong
                kAmiDouble
                kAmiChar
                kAmiShortArray
                kAmiLongArray
                kAmiDoubleArray
                kAmiCharArray
mStaticFlag - A flag stating whether or not the field is static.  This can be one
              of the following values:
                Adesk::kFalse
                Adesk::kTrue
            A static field means that this value is shared by all attributes of
              this type. (Changing the value for one attribute changes the value
              for all the attributes of this type.)
```

To define an instantiable attribute “class,” an array of these field structures are filled in, one for each field, and passed into a new MCAD API function, which is prototyped as follows:

```
amiDefineAttClass(const char* className, int nFields,
                  const AmiAttFieldDesc* pFields);
```

When this function is called, a “class” record is created and maintained by the system. This does not necessarily mean that the “class” record is saved into the DWG file. The record only exists in memory until an instance of the described attribute is created. At that time, a copy of the “class” record is put into the database. Each subsequent creation of an instance of the described attribute references the same “class” record and increments a reference count within the record of how many instances are in the current database.

Conversely, as attribute instances are erased from the database, the reference count in the record is decremented. If and when that reference count reaches zero, the “class” record is erased from the database. At this point, the in-memory record still exists so new instances of the attribute can still be created without having to re-register the attribute “class.” The in-memory record lasts for the length of the session.

Example

As an example of how to define an instantiable attribute, suppose you want to attach a material attribute to a displayable object to control how the object looks when it is rendered. The following is a code snippet showing how that can be done.

```
AmiAttFieldDesc fields[4];

fields[0].mFieldName   = “ambient”;
fields[0].mFieldDesc   = “Ambient Coefficient”;
fields[0].mFieldType   = kAmiDouble;
fields[0].mStaticFlag  = FALSE;

fields[1].mFieldName   = “diffuse”;
fields[1].mFieldDesc   = “Diffuse Coefficient”;
fields[1].mFieldType   = kAmiDouble;
fields[1].mStaticFlag  = FALSE;

fields[2].mFieldName   = “specular”;
fields[2].mFieldDesc   = “Specular Coefficient”;
fields[2].mFieldType   = kAmiDouble;
fields[2].mStaticFlag  = FALSE;

fields[3].mFieldName   = “exponent”;
fields[3].mFieldDesc   = “Specular Exponent”;
fields[3].mFieldType   = kAmiLong;
fields[3].mStaticFlag  = FALSE;
```

```
if (amiDefineAttClass("material", 4, fields) == Ami::eOk)
  .. success, continue
```

Attributes From Other Applications

Whenever an application defines an instantiable attribute "class," a single "class" record containing that definition is acquired by the system. As soon as a single attribute of that type is instanced and put in the current database, that class record is also put into the database. Similarly, if a DWG file is loaded that contains class records of instantiable attributes defined by another application, those class records are detected by the system. The result is that the current session knows how to read these attributes from the DWG file without the defining application being loaded. In addition, with those class records loaded, any application can create new instances of these attributes even though the defining application is not loaded.

Attributes To and From an Attribute Definition File

At any time in an application, all of the attribute "classes" currently known by the system can be written to an ASCII attribute definition file using the MCAD API function *amiWriteAttClassesToFile*. This function writes out an ASCII representation of all current attribute "classes" to a file that can later be read in by any application using the MCAD API function *amiReadAttClassesToFile*.

The format of the attribute definition file is essentially the same format used to define the attribute "class" programmatically. The first line of the file contains the number of attribute "classes" in the file followed by the individual "class" definitions. For each "class" definition, there displays the number of fields in the "class" followed by the field data as it appears in the *AmiAttFieldDesc* structure.

Example

An attribute definition file containing only a "material" attribute would look as follows:

```
1
material
4
ambient
Ambient Coefficient
double
FALSE
FALSE
diffuse
Diffuse Coefficient
double
FALSE
FALSE
specular
Specular Coefficient
double
FALSE
FALSE
exponent
Specular Exponent
long
FALSE
FALSE
```

Creating Instantiable Attribute Instances

Once an attribute "class" is defined in the system (either programmatically or by loading a DWG file), new instances can be created by calling another MCAD API function and specifying the attribute "class" name. Using the previous example of a material attribute, take a look at another example:

```
AmiInstAtt *pAtt;
if (amiMakeInstAtt("material", pAtt) == Ami::eOk)
{
  amiSetAttFieldDouble(pAtt, "ambient",  0.3);
  amiSetAttFieldDouble(pAtt, "diffuse",  0.7);
  amiSetAttFieldDouble(pAtt, "specular", 0.8);
  amiSetAttFieldLong  (pAtt, "exponent", 12);
```

```
}
```

In this example, the MCAD API function *amiMakeInstAtt* is called to create an instance of the instantiable attribute whose "class" name is "material." Once the instance is successfully created, the MCAD API function *amiSetAttFieldDouble* is called to set the double value on the material attribute fields. The MCAD API contains several functions for setting and getting field values based on the field type. See the "Function Reference" section for a complete listing of these functions.

Transactions

In the previous section's example of creating an attribute instance, a pointer to the newly created instance is returned from the function *amiMakeInstAtt*. This pointer will only be valid if there is an open database transaction when the create function is called. When an API function returns a pointer to an attribute, you need to make sure there is a database transaction open during the entire time you are working with the pointer. If these functions are called within the scope of ObjectARX™ commands (as they are in the sample app), then a transaction will already be open. If the functions are called in another scope, however (such as an event handling function of a modeless dialog box), then care must be taken to make sure that a transaction is opened and stays open as long as use of the returned pointer is required. This can be done via the transaction manager methods *startTransaction* and *endTransaction*.

Accessing Attribute Fields

To enable the most efficient access to fields inside attribute objects, the MCAD API provides for each function that gets and sets a field's content by name a corresponding function that gets and sets the field's content by the field's index. Accessing a field's content by the field's index results in faster data access, though the caller must keep track of appropriate indices. All functions that access attribute fields by index have their name derived from the corresponding function that accesses attribute fields by name by adding a capital "I" to the function name. For example: *amiGetAttFieldShort* to access by name and *amiGetAttFieldShortI* to access by index.

Standard Attributes

Another benefit of the instantiable attributes mechanism is the ability of third parties and Autodesk to supply libraries of standard attribute definitions. Instead of having several applications defining attributes for things like material properties, an instantiable attribute definition can be agreed upon and the definition could be added to the MCAD API. These standard definitions will be available in future releases.

Function Reference

The functionality provided by the set of classes that constitute the instantiable attribute subsystem is exported in a functional form in the MCAD API. The functions are grouped in related groups as follows.

Class Definition Functions

These functions allow for definitions of an attribute "class" based on all of the pertinent information, including "class" name and field descriptions.

- `amiDefineAttClass(const char* className, int nFields, const AmiAttFieldDesc* pFields);`
- `amiUndefineAttClass (const char* className);`

Instance Creation Functions

These functions allow you to create attribute instances of a specified "class."

```
amiMakeInstAtt(const char* className, AmiInstAtt*& pAtt);
```

Data Access Functions

These functions allow you to the query and set data in a given attribute instance either by field name or field index.

By Field Name:

- `amiGetAttField(AmiInstAtt* pAtt, const char* fieldname, <input Type> value);`
- `amiSetAttField(AmiInstAtt* pAtt, const char* fieldName, <input Type> value);`

By Index:

- `amiGetAttField(AmiInstAtt* pAtt, int index, <input Type> value);`
- `amiSetAttField(AmiInstAtt* pAtt, int index, <input Type> value);`

System Query Functions

These functions allow you to query global information, such as finding all of the attribute "classes" registered with the system.

- `amiGetAttClasses(int& nClasses, const char**& classNames);`

Attribute Query Functions

These functions allow you to query data in a given attribute class.

- `amiGetAttClassFields(const char*, int& nFields, AmiAttFieldDesc*& pFields);`

Attribute ASCII I/O Functions

These functions allow you to write or read attribute definitions from an ASCII attribute definition file.

- `amiWriteAttClassToFile(const char* className, const char* fileName);`
- `amiWriteAttClassesToFile(const char* fileName);`
- `amiReadAttClassesFromFile(const char* fileName);`

MCAD API Object Descriptors

AmiDescrip

The AmiDescrip class is used to hold attribute values for various types of Mechanical Desktop objects. These descriptors serve as a snap-shot description of a database object. They hold a collection of attribute values that describe the settings for an object. These attribute values can be altered and then applied via a key to any object of the same type.

Read/write operations on values in AmiDescrip objects is done by using keywords to denote specific attributes. For example, to read the scale-factor for a view, you must first obtain the view descriptor from a view key, then use *amiGetData()* with the keyword 'vwAttScale'. The scale-factor is returned as a 'double'. Some attributes can be represented in multiple formats, and the descriptor access functions are overloaded appropriately.

In addition to being overloaded on the return type, these get/set functions are overloaded on the descriptor type. Thus, the same function names are used for getting/setting attribute values in descriptors for views, annotation, scene components, files, etc. Those functions are amiGetData(), amiSetData(), amiGetDataArray(), and amiSetDataArray(). The separate versions for the various descriptor types are described separately in the appropriate sub-sections in Chapter 3 (MCAD API Functions—Detailed Description of Functions).

Some attributes are "read-only." Typically, the values for read-only attributes can be altered within a descriptor, but when that descriptor is applied to a database object the value must match the setting already stored in the database. Thus, these attributes are read-only with respect to database and not with respect to descriptors. Other attributes are "write-once," meaning that they can be written to the database when a descriptor is used to create a new database object, but they cannot be used to edit the state of an existing database object.

There is a special value that can be used for any attribute in any descriptor. This value is not a legitimate value in the domain of any of the data types for any of the attributes – it is 'Ami::attNAV' and stands for "Not A Value."

If any attributes in the descriptor are set to Ami::attNAV, then those attributes won't be applied to the database when the descriptor is used in an edit operation on a key (e.g. *amiDwEditView*). These NAV values will be skipped – this is not an error condition and the return status is eOk. Thus, NAV can be used as a "don't care" condition when applying AmiDescrip descriptors to keys.
If any read-only attributes in the descriptor have a value that differs from the value already in affect for the given key, and if that attribute is not NAV, then the edit operation on the key will return an error status (the status is eAttReadOnly). Some attributes are mandatory attributes and must have a non-NAV value when a descriptor is used to create a new database object.

An "empty" descriptor is a descriptor where all values are NAV. Using an empty descriptor to edit an object referenced by a key has no effect on the object (all of the values are "don't care").

This semantics for NAV and editing operations makes it possible to use a partially set descriptor in an editing operation on a key to set a small number of attributes while leaving the others unchanged. Similarly, you can obtain a descriptor from a key, alter one or two of the attribute values, apply the descriptor to the same key, and the altered settings will be applied while the others remain unaffected.

AmiValue

Once a descriptor is obtained, it can be used in a handful of new API functions to query for information about the descriptor. Numeric information is returned in the form of *AmiValue* objects.

```
AmiFeatKey *pKey;   Assume this is a key to a hole.
AmiFeatDescrip *pDescrip;
AmiStatus stat;

stat = amiGetFeatDescrip(pKey, pDescrip, Adesk::kTrue, Adesk::kFalse);

Adesk::Boolean isHole;
amiIsDescripKindOf(pDescrip, Ami::kHole, isHole);

if (isHole)
{
  AmiHoleDescrip *pHole = (AmiHoleDescrip*) pDescrip;
  AmiValue diameter, drillAngle;

  amiGetHoleData(pHole, diameter, drillAngle);

  acutPrintf("Hole diameter is %f\n", diameter.getValue());
}

pDescrip->release();
```

In the example above, the function *amiIsDescripKindOf* is called to see if the descriptor returned from *amiGetFeatDescrip* refers to a hole feature (drilled, counter bore, or counter sink). If it does, the pointer is cast to the appropriate type and passed into the function *amiGetHoleData*. The values returned from that call are *AmiValue* objects. The *AmiValue* object is designed to be lean and constructable on the stack. The value object will contain a *double* or a *AmiParamKey** depending on the value of *fReturnParameters* in the call to *amiGetFeatDescrip*. Once a value object is obtained, a numeric value can be obtained via the object's *getValue* method regardless of what is really inside. The two main *AmiValue* methods that are of interest are:

```
double       getValue() const;
AmiParamKey* getParam() const;
```

The *getValue* method is simple. Regardless of whether the value object contains a double or a parameter key, a call to *getValue* will return a valid numeric value. The *getParam* method is a little different though. If the value object contains a double, a call to *getParam* will return a NULL pointer. If, on the other hand, a value object contains a parameter key, a call to *getParam* will return a copy of the contained key. This is an important thing to remember. An *AmiValue* owns its data. In other words, an *AmiValue* object only exposes copies of what it contains. This means that if you query for a parameter key, you get a copy. Furthermore, if you make a copy of an *AmiValue* (which you can do) and that value contains a parameter key, the key is copied. Lastly, when a value object is destroyed, the key is destroyed with it.

Basic Functionality for Feature Descriptors

The MCAD API allows Mechanical Desktop part features to be queried through *feature descriptors* which can be obtained from a feature using a feature *key*. A single function call *amiGetFeatDescrip* will take a feature key, construct a feature descriptor object containing information about that feature, and return a pointer to the new descriptor object. Once a pointer to a descriptor object is obtained, it can be passed into the appropriate API function to extract pieces of information from the descriptor object. Once the descriptor has outlived its usefulness, it is released by the programmer. In addition to general feature information (i.e. hole diameter, fillet radius, etc.), a feature descriptor also contains location and termination information in the form of *AmiInformer* objects which can be obtained from any feature descriptor. Lastly, feature descriptor objects are also used in the API for communicating feature information for purposes of creating and editing features as well.

Descriptor Creation

Looking first at an example:

```
AmiFeatKey *pKey;   Assume this is a valid feature key obtained by the user.
AmiFeatDescrip *pDescrip;
AmiStatus stat;

stat = amiGetFeatDescrip(pKey, pDescrip, Adesk::kTrue, Adesk::kFalse);
```

The function call above simply passes in a valid feature key and gets back a pointer to a new feature descriptor object. The second and third parameters are booleans which tell the API how lean you want your descriptor to be. The first boolean parameter *fReturnInformers* specifies whether or not informer information (location and termination info) should also be included with the descriptor. Although this information is obviously very useful, it can also be expensive so if its not needed, it is best left out. The second boolean parameter *fReturnParams* specifies whether value information should come back in the form of real numbers (*double*) or in the form of parameter keys (*AmiParamKey**) again for optimization purposes.

Informers

Location and termination information is encapsulated into its own set of objects called *informers*. Once a pointer to a feature descriptor is obtained, the termination information can be obtained from the descriptor via a call to *amiGetTerminator*. Once a terminator is obtained, the terminator type can be queried and then the appropriate API function can be called to obtain the termination information.

```
AmiFeatDescrip *pDescrip;   A hole descriptor (for example).
AmiTerminator *pTerm;

amiGetTerminator(pDescrip, pTerm);

AmiTerminatorType tType;
amiGetTermType(pTerm, tType);

if (tType == Ami::kBlindTerm)
{
  AmiBlindTerm *pBlindTerm = (AmiBlindTerm*) pTerm;
  AmiValue      depth;

  amiGetBlindTermData(pBlindTerm, depth);

  acutPrintf("Hole depth = %f\n", depth.getValue());
}

pTerm->release();
```

Similarly, location information can be obtained from a feature descriptor. A feature descriptor pointer can be passed to the function *amiGetLocator* and a pointer to the descriptor's locator object will be passed back. The difference here is that a locator by itself has no type. A locator is made up of sub-objects called *atomic locators*. Once a pointer to a locator is obtained, it can be passed into the function *amiGetAtomLocators* which will return a list of the atomic locators that make up the location information for the original feature. As an example, suppose there is a hole descriptor that was located at a distance from two edges. The locator for the hole would be made up of two atomic locators of type *AmiDistFromLoc*. Each of these sub-objects will contain a geometry key to a line representing an edge and a corresponding distance value.

```
//
have a feature descriptor pointer (AmiFeatDescrip *) called pDescrip...
//
AmiLocator *pLoc = NULL;
amiGetLocator(pDescrip, pLoc);

int nLocators;
AmiAtomLocator **pLocators = NULL;

if(amiGetAtomLocators(pLoc, nLocators, pLocators) != Ami::eOk)
  return;
```

```
for (int ii = 0; ii < nLocators; ii++)
{
  AmiAtomLocatorType locType;
  if(amiGetAtomLocType(pLocators[ii], locType) == Ami::eOk)
    if(locType == Ami::kDistFromLoc)
    {
      AmiDistFromLoc *pDistLoc = (AmiDistFromLoc*) pLocators[ii];
      AmiValue dist;
      if(amiGetDistFromLocData(pDistLoc, dist) == Ami::eOk)
        acutPrintf("Distance from edge = %f\n", dist.getValue());
    }

pLocators[ii]->release();
}

delete[] pLocators;
pLoc->release();
```

New data types and functions have been added for pattern creation and query of pattern data. These functions are documented in mifeat.h. In addition, locators may be used for patterns as follows. Rectangular patterns use an AmiAlignedLoc when the columns are aligned to an edge. This locator should contain a key to an edge or a work axis.Polar (and axial) patterns use locators to specify the rotation center. The following types of locators may be used: an AmiAlignedLoc (containing a key to a work axis), an AmiCoincidentLoc (containing a key to a work point), or an AmiConcentricLoc (containing either an AmiCurveKey to a cylindrical edge, or an AmiSurfKey to a cylindrical face).

AmiSelection

The class AmiSelection is found in the file miselect.h and is used for passing data to the sketch creation functions. When creating 2D sketches and 3D polyline paths, the geometry used to create the sketch can either be MDT geometry data (edges) or AutoCAD data. The AmiSelection class provides a single object to encapsulate a geometry selection. A selection object can be created using either a geometry key (AmiGeomKey*) or an object ID (AcDbObjectId). When a collection of selection objects is used to create a 2D path sketch, the path start point is specified by also specifying a pick point (AcGePoint3d) in addition to the geometry on the first selection.

The AmiSelection class contains several methods for setting and getting the underlying geometry. Any method can be called to reset the geometry on a selection. When getting the geometry, however, the appropriate method must be called based on whether the selection is holding a geometry key or an object id. The type of the underlying geometry can be queried by calling the "getType" method.

AmiSelection::Type getType() const;

The return value will be on of the following three values.

AmiSelection::Unset - No geometry has been set on this selection
AmiSelection::GeomKey - Selection is a geometry key
AmiSelection::DbId - Selection is an object id

Drawing Manager Descriptors

The Drawing Manager functionality uses AmiDescrip objects for access to properties of views and annotations. The functions *amiGetData(), amiSetData(), amiGetDataArray(),* and *amiSetDataArray()* are overloaded for all of the data types appropriate for view attribute values. The supported overloads are listed in the tables in the Drawing Manger API section The attribute keywords are defined in the AmiViewAttribute and AmiAnnotAttribute enums.

Scene Component Descriptors

AmiDescrip objects are used for access to properties of model components in the context of a scene. These descriptors describe the position and visibility of the component within the scene. The current set of attribute keywords are defined in the AmiSceneCompAttribute enum.

Constraint Descriptors

AmiDescrip objects are used for access to properties of either sketch or assembly constraints. These descriptors describe the operands and the relationship between them. Any available display information is also provided. The current set of attribute keywords are defined in the AmiConstrAttribute enum.

File Descriptors

amiFileDescrip, in general, is used to encapsulate information about files used by Mechanical Desktop. It is used by the Table-Driven Versions functions to describe the versioning spreadsheet which defines either part variables and feature suppression or global design variables across versions. Future APIs will support registering an application's input or output files with the registered PDM using file descriptors.

File descriptors use amiSetData / amiGetData to access data members. This mechanism is similar to that used in Drawing Manager View and Section Descriptors (See the Table-Driven Versions section of Chapter 3 for more information).

File Descriptors have their own enumerator to define the available attributes (AmiFileAttribute) and support file specific data types. There are overloaded versions of the accessor functions for each data type required, such as int, char* and AmiSectionStructureSee the table in the description of amiSetData and amiGetData for information on the appropriate data types for the different attributes. This table will expand as additional attributes become available on file descriptors.

Event Reaction Descriptors

There are many interesting events happening in the Mechanical Desktop as a design is being developed, which an application may need to be aware of. These events include creating a new part, deleting a component, or changing the current version of global design variables.

The event reaction API allows an application to associate a function with a specific event, and that function is called every time the event occurs in the Desktop. The event prescribes a function signature to which the callback must conform. Typically the object(s) which is affected by the event is passed to the registered function using MCAD API keys. For instance, the new Part event requires a function which takes a pointer to a key and an event context. When a new part is introduced, the function is called, passing as its argument, a key to that newly created part. The version activated event takes a pointer to a key (which may be null in the case of global design variables), a string (char *) for the version and an event context. In order to avoid the combinatorial number of function types which may be required, any time a key is returned the signature takes a pointer to an AmiObjectKey, not a pointer to the specific key type which could be returned. It is the callback's responsibility to type check the key if it is handling only specific key types.

AmiReactDescriptor is used to build up an event reaction description. Construction of this object ensures that the function used as the event call back conforms with the event signature imposed by the event specified (see AmiReactDescriptor::make function description). There are overloaded make functions for each function signature supported. If the specified event keyword does not support the function type given by the overload, an eInvalidArg error is returned.

The keys which are passed to a callback may either be free copies or shared copies. The callback is responsible for releasing free copies, and it must never release or erase shared copies. The reaction descriptor has an attribute which controls whether the keys are free or shared. **It is critical that an**

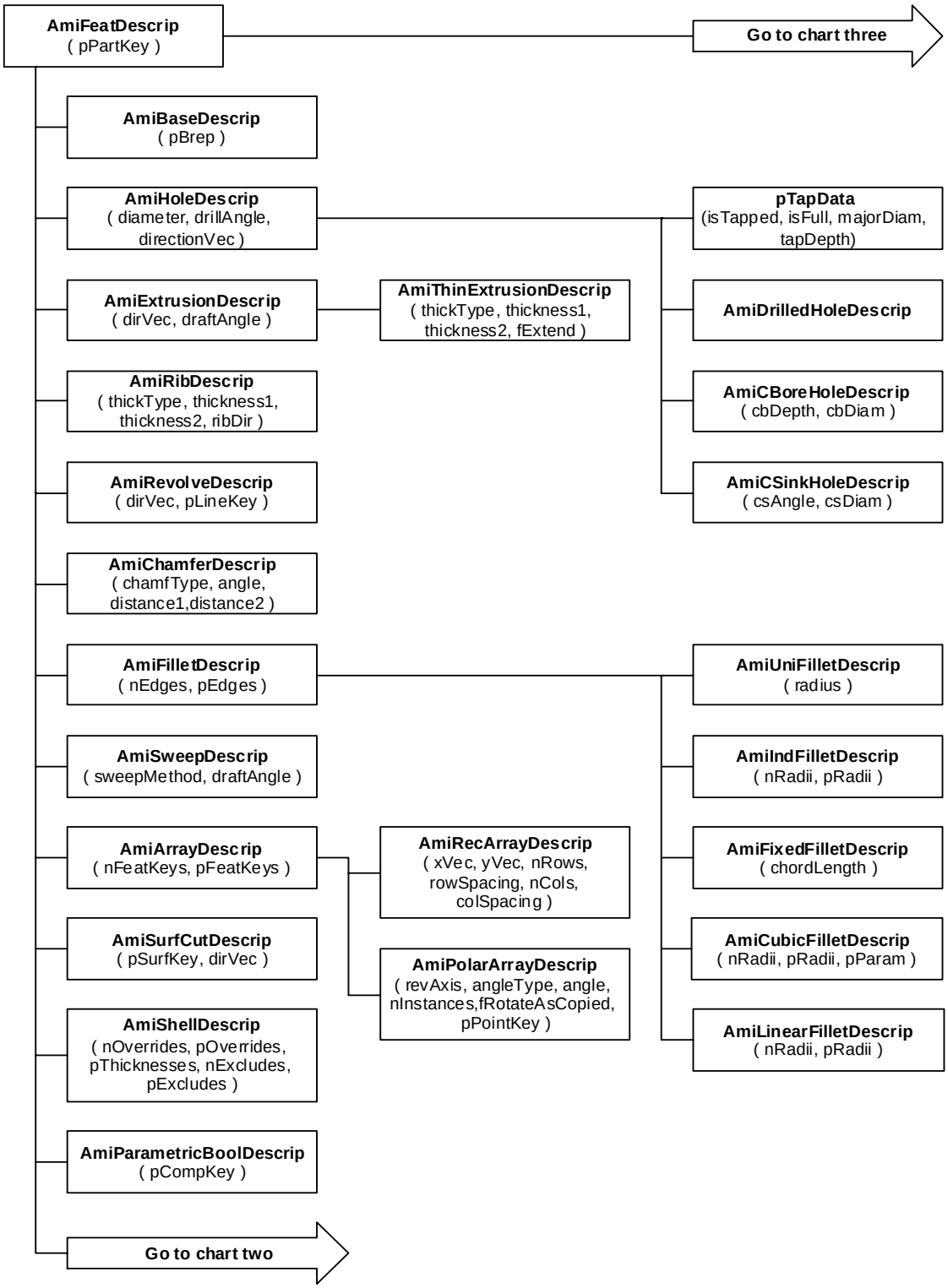
application provide the correct value to this attribute as required by the callback to avoid memory leaks or memory stomping.

Once the descriptor is finalized, the function amiRegisterEventReaction takes the information and starts the notification process. From that point forward, the callback function is called each time the event occurs. This continues until either the session is ended or the callbacks are unregistered. The event reaction descriptor may be deleted after calling amiRegisterEventReaction.

Each event imposes a specific callback function signature. Along with the description of amiRegisterEventReaction there is a table of currently supported function pointer types, the prototype they define and a brief description. New function types will be added to this table as needed.

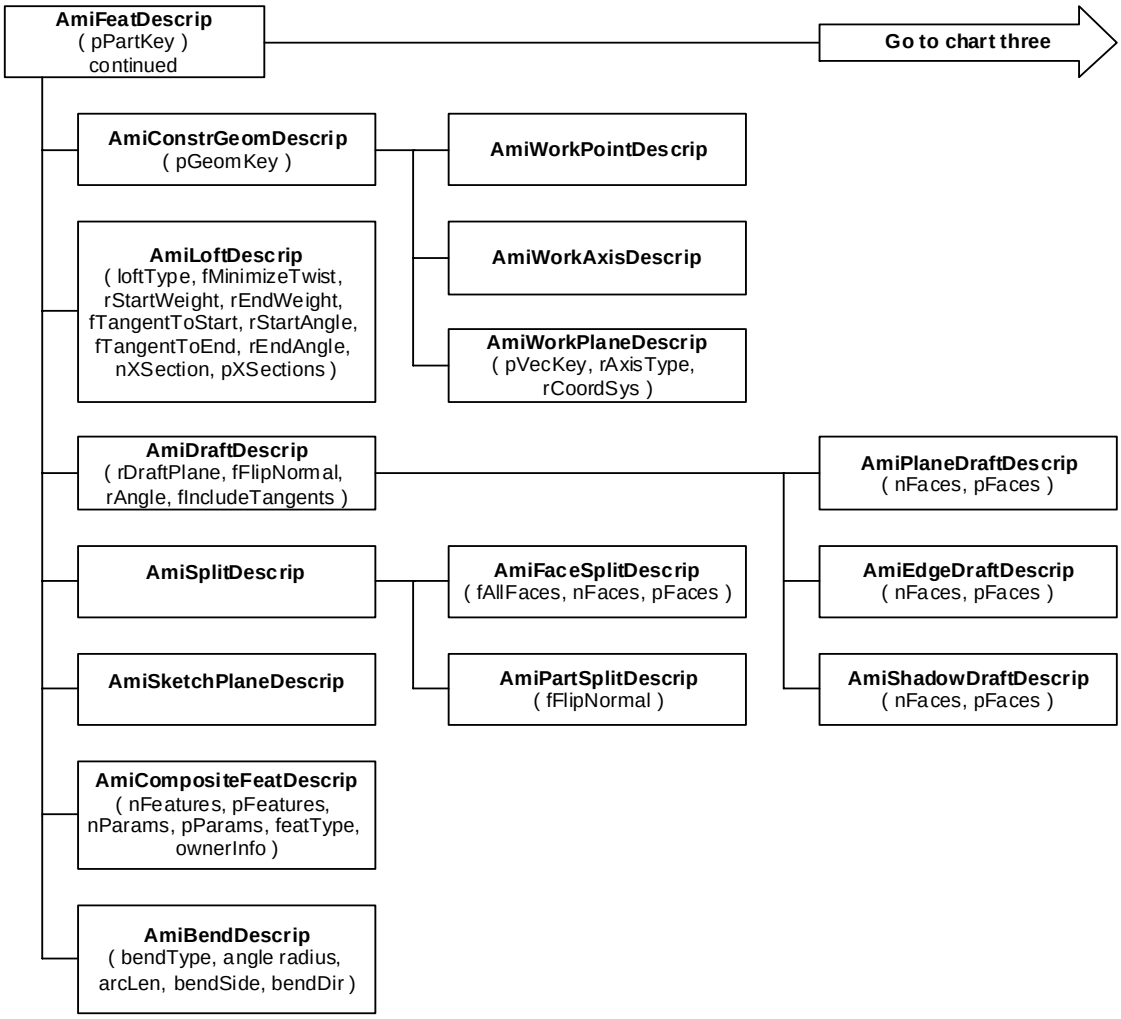
Class Hierarchy

The class chart shows all of the feature descriptor and informer classes, their class hierarchy, and the information that can be obtained from each class.



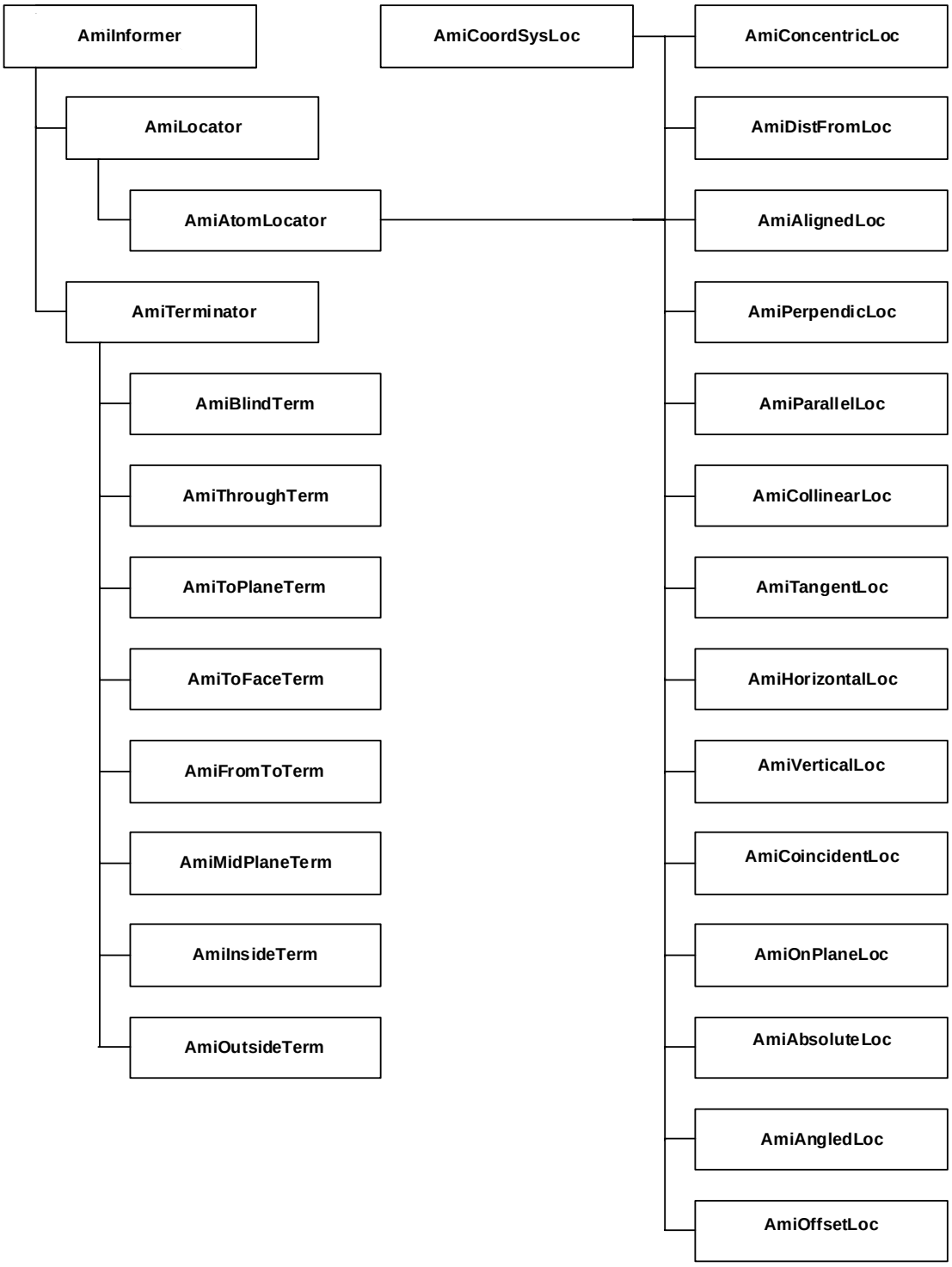
Class Hierarchy: Chart Two

This is a continuation of the class hierarchy chart.



Class Hierarchy: Chart Three

This is a continuation of the class hierarchy chart.



Composite Features

As of MDT5, the MCADAPI allows third parties to create composite features. A composite feature is simply a feature made up of other features. Using these composites, a third party can create a custom feature made up of any number of standard MDT features and hide the MDT features within their own custom (composite) feature. The browser displays the resulting feature
as one item and the features contained within the composite are highlighted and selectable as one item. A third party can even register a callback so that they can handle editing of the composite when it is selected for edit.

Composite features are only available through the MCADAPI and, more specifically, are handled by the feature API. Many similarities exist between how these features are handled and how standard MDT features are handled but there are also many important differences.

1. Feature query is the same as with any feature. Given a feature key, the API can be used to obtain a descriptor to that feature where the individual elements that make up the composite can be queried.

2. Feature create is where the big difference comes in. A composite feature in not created but, instead, it is "started." When a composite feature is started, a transaction starts up internally and any features created from that point on are added to the composite until such time that the composite feature is "ended." Once the composite feature is ended, the internal transaction is closed and MDT reverts back to its regular state. Optionally, a composite feature can also be "aborted" which aborts the internal transaction erasing the composite and any features that were added to it.

The fact that an internal transaction is started when a composite feature is started is very important to remember. If a third party starts their own transaction before starting the composite and then ends their transaction before the composite is ended, the composite's transaction will also be closed. This is very undesirable behavior. Therefore, it is recommended (and almost mandatory) that the composite feature be started, defined, and ended within a single area of code so that the result looks like a "sandwich" where the individual features are sandwiched between the start and end composite calls.

3. Only one composite feature can be in the process of creation within a single document at any given time. As described above, starting a composite puts the current into a "state" so, until the composite is ended, another cannot be started.

Two of the composite feature descriptor items are mandatory in order to begin creation of a composite feature. Those items are the feature type string and the owner info string.

1. The feature type string is provided as a way for third parties to keep track of what this feature is in their application. If a third party application has several different types of features that are represented in MDT as composites, they may want to perform edit operations differently depending on the type. This string provides a key for that application to differentiate between their own types of features.

2. The owner info string servers several purposes. First, the string itself is a null terminated string made up of three tokens separated by semicolons. The first token should be the owner application's company name. The second token should be the name of the creating application. The third (optional) token should be a URL where a user can go to the third party for information or downloads. Mostly, the layout of this string is for future use. Today, however, we use the owner info string to register the edit callback function described below.

When a composite feature is created, the creator has the option of specifying a list of driving parameters to the composite. These are pre-created parametersthat are used to control the appearance

of the feature. By default, if a composite feature is selected for edit, nothing happens. If these driving parameters are specified at create time, selecting the composite for edit will bring up the standard MDT part/global variables dialog which will contain the list of the specified driving parameters allowing the user to edit them. Another option for editing is for the third party to register a callback function to be called whenever the composite is selected for edit. The callback is registered using the composite's owner info string. This would mean that one edit callback is registered per application. When a composite feature with a matching owner info string is selected for edit, the registered function is called. It is then up to the application to determine the type of feature that was selected (using the feature type specified at create time) and act accordingly.

Lastly, the introduction of composite features influences the behavior of some of the functions already in place that deal with feature keys. For example, functions such as amiSuppressFeat and amiEraseObject will act on the composite and all of the features contained within the composite. Functions returning features like amiGetPartFeats will return a single key to the composite excluding keys to the features contained within the composite. The functions amiGetFeatsFromSketch and amiGetFeatsFromGeom have been modified to take a boolean flag allowing the caller to specify whether to return the composite or the innermost contained feature when a found feature is contained within a composite.

AmiStatus Overview

All MCADAPI functions return an AmiStatus object on the stack. In the past AmiStatus has been an enum datatype -- it has been redefined to be a class in the Ami namespace (Ami::Status). We have multiple status values for success, as well as all the previously supported values for errors.

There are four severity levels to MCADAPI status values. These are used to distinguish errors from warnings, as well as to provide two degrees of severity for errors. The meanings of these four severity levels are:

kInfo	normal outcome
kWarning	successful outcome, but unusual in some way (e.g. incomplete result)
kError	failure; all output arguments are undefined
kSevere	severe failure (e.g. file corruption or other chronic problem)

When an MCADAPI function returns failure (kError or kSevere), any output parameters are left undefined. When an MCADAPI function returns success (kInfo or kWarning), the output parameters can be expected to contain valid data and the caller is responsible for proper clean-up (e.g. calling release() on keys and descriptors, freeing storage for traditional C strings, etc).

Typically, a warning status is returned when the operation was at least partly successful and all of the output parameters hold valid data. Returning a warning instead of an error allows MCADAPI functions to produce meaningful results in the face of minor errors, rather than imposing an all-or-nothing semantics.

For example, an MCADAPI function that produces an object descriptor when given a key, might return a warning status if there is an error while populating one of the attributes in the descriptor. In this situation, it is preferable to produce the incomplete descriptor than to return failure, which would imply that the output parameter is unset and no descriptor is produced. The incomplete descriptor can't be used as-is to create a new object in MDT, but the non-offending attributes can be queried, and the offending attribute can be set by the external application. Until it is set by the application, that attribute will be "Not A Value" (NAV; see the discussion of AmiDescrip.)

AmiStatus objects hold additional information to better pinpoint the nature of an error or warning. When the problem can be attributed to a particular input parameter to the MCADAPI function, the argIndex() method will return the parameter index for the offending argument (numbering begins at 0). When the problem can be attributed to a particular attribute in a descriptor, the attrKeyword() method returns the keyword for that attribute. These methods return integers, with negative values used to indicate that the data is not available.

It should be emphasized that the new implementation of AmiStatus is fully compile-time compatible with the old implementation. There is no need to re-code applications. The old enum symbols will continue to be supported in future releases. They serve as a convenient short-hand for specific AmiStatus values by coupling an Ami::StatusCode with an Ami::Severity.

CLASS AmiStatus

All MCADAPI functions return AmiStatus objects. The AmiStatus holds a simple status to convey the outcome of the function. This includes error codes, warning codes, and success codes.

Query access:

success()
: Return true if this is not an error status.

failure()
: Returns true if this is an error status.

severity()
: Returns the severity of the status.

symbol()
: Returns an enum value which represents the basic type of status.

argIndex()
: Returns the position of the function argument that gave rise to the status. Numbering begins with 0, and a negative number is returned if no argument has been singled-out as the cause. arrayIndex() When argIndex() indicates that an array input argument is the cause of the status, and the problem can be further pinpointed to a particular element, then this function returns the index of that element. Numbering begins at 0, and a negative number is returned if no element is singled out.

attrKeyword()
: When argIndex() indicates that a descriptor input argument is the cause of the status, and the problem can be further isolated to a single attribute, then this function returns the keyword for that attribute. Note: the keyword is passed back as an 'int' and not as an enum. This is because there is no single enum datatype for attributes of all descriptor types (the enum values are distinct, however, so there is no loss of information in using an 'int' here).

text()
: Returns a string holding a short text description of the status.

operator ==

operator !=
: Comparisons are possible between two AmiStatus objects, between an AmiStatus and an enum symbol, etc.

operator <
: This comparison returns true if the LHS is less severe than the RHS. If both operands have severity kInfo, then this comparison will still return true if the RHS has a non-kOk symbol while the LHS is kOk.

Edit access:

setSeverity()

Sets the severity to the given level.

setSymbol()

Sets the symbol to the given StatusCode value.

setArgIndex()

Sets the arg-index to the given value.

setArrayIndex()

Sets the array-index to the given value. This will overwrite the attribute-keyword if it is set. The attribute-keyword and array-index cannot be used together in the same AmiStatus instance.

setAttrKeyword()

Sets the attribute-keyword to the given value. This will overwrite the array-index if it is set. The attribute-keyword and array-index cannot be used together in the same AmiStatus instance.

operator =

The assignment operator.

operator +=

This operator performs an assignment if and only if LHS<RHS holds. It can be used to "accumulate" status values.

Notes:

AmiStatus objects can be compared directly to Ami::FlatStatus enum values. The Ami::FlatStatus value will be promoted to an AmiStatus before the comparison takes place, with the severity of the promoted object set to kError. There is ONE exception: Ami::eOk is promoted to AmiStatus(kOk, kInfo).

The same promotion rule is used when an AmiStatus variable is assigned an Ami::FlatStatus value. For example, the following equalities hold:
AmiStatus(Ami::kInvalidArg, Ami::kError) == Ami::eInvalidArg
AmiStatus(Ami::kOk, Ami::kInfo) == Ami::eOk
AmiStatus status; (status = Ami::eInvalidArg)== Ami::eInvalidArg

When an AmiStatus object is constructed or assigned a value using an Ami::StatusCode, the same promotion rules apply -- the severity is assumed to be kError except for kOk which has severity kInfo. So, for example,
AmiStatus(Ami::kInvalidArg, Ami::kError) == Ami::kInvalidArg
AmiStatus(Ami::kOk, Ami::kInfo) == Ami::kOk

While Ami::StatusCode values often correspond to Ami::FlatStatus values, there is no guarantee. For example, the following equality is not guaranteed to hold in future releases:
Ami::eInvalidArg == Ami::kInvalidArg

File Structures

MDT supports two basic file structures: Assembly files and Part files. Part files are useful for situations in which the full DWG structure, full UI, and full API are not needed because the user is only interested in modeling a single part.

MDT Part Only Files

Part Only Files are DWG files that contain a single part and do not allow normal assembly-modeling operations. However, because a single part can be comprised of multiple solids using parametric-boolean part-modeling features, and because these other solids (toolbodies) can be positioned and constrained using inserts and 3D constraints, a limited "assembly modeling" API is supported in Part Only Files.

Thus, a Part Only File is structured much like a "flat assembly," where the part itself, along with any unconsumed toolbodies, can be referenced as a component inserted into the master assembly. This can lead to confusion because conceptually there is only one part in a Part Only File and therefore it is easy to overlook the insert of that part-definition into the master component definition of the Part Only File. It is easy to confuse an AmiCompKey that references the entire Part Only File with an AmiCompKey that references the leaf-node component for the base part in that file. This distinction is especially easy to overlook in the case where the Part Only File has no toolbodies (yet).

The flat assembly hierarchy is enforced -- it is not possible to create a subassembly in a Part Only File. All component definitions other than the master component definition will be a definition for a single solid (e.g. the part, the toolbodies).

Unlike a normal assembly DWG, unconsumed toolbodies in a Part Only File are tagged as toolbodies from the moment they are created. In an assembly file, an unconsumed toolbody is no different than any other component -- it does not take on the role of toolbody until it is used in a parametric-boolean feature. For Part Only Files, where there can be only one part, there is no ambiguity in the role of subsequent solids: they are toolbodies. This makes is easy to distinguish the grounded base part from all other AmiPartKey objects and AmiCompKey objects for the Part Only File.

In the case of an XRef'd Part Only File, the Browser presents users with a single part (hiding any unconsumed toolbodies). The MCADAPI, however, allows the Xref'd Part Only File to be used as a flat assembly. Thus, if you obtain an AmiCompKey from a pick on an Xref'd Part Only File then you can use amiGetCompChildren() to obtain keys for any unconsumed toolbodies in that Part Only File (in addition to a key for the grounded base part). It is important to realize that the AmiCompKey obtained from a pick on an Xref'd Part Only File is an AmiCompKey that refers to the entire Part Only File and not just to the leaf-node component that is the grounded base part.

When you use amiPick() with Xref'd Part Only Files, the AcDbFullSubentPath encapsulated in the AmiPick object has the following characteristics:

- If the Part Only File is not open for Xref Edit, and the user clicks on graphics on the grounded base part, then the AmiPick object will reference the entire Part Only File. If this AmiPick object is used to create an AmiCompKey then the comp will be for the entire (albeit flat) assembly in the Part Only File.

- If the Part Only File is not open for Xref Edit, and the user clicks on graphics on an unconsumed toolbody, then the amiPick() function will cycle between two choices: one choice yields an AmiPick object that references the entire Part Only File (as with picks on the grounded base part); and the other choice yields an AmiPick object that references the specific toolbody instance picked. If the user chooses the second of these options, and if this AmiPick object is used to create an AmiCompKey, then the component will be for that leaf-node toolbody instance, in the scope of the active component definition in the host database.

- If the Part Only File is open for Xref Edit, then the amiPick() function will behave as it does when the Part Only File is loaded directly. The AmiPick object will reference the leaf-node that was picked, in the context of the master definition in the Part Only File.

Thus, to obtain an AmiCompKey object that references an Xref'd grounded base part in the context of the master assembly using amiPick(), you must use amiGetCompChildren() with the (assembly) component key obtained from the pick object. If the user clicks on an unconsumed toolbody instead of

on the grounded base part, then you must use amiGetContainingComp() first and then amiGetCompChildren() to get the corresponding grounded base part leaf-node comp key.

When using amiGetActiveLeafNode(), the AmiCompKey returned is always a leaf-node component. Thus, when an Xref'd Part Only File is open for Xref Edit, the output of amiGetActiveLeafNode() refers to the grounded base part in the Xref'd file, in the context of the active component definition (in the host database).

Coding Practices

The MCAD API observes the following conventions:

Return Values

All functions return a status that the program must check to verify that the function executed successfully.

Memory Allocation & Freeing

Any time an array of pointers to keys is returned by a function, you must free the associated memory when you're finished using it. That involves releasing the keys and deleting the array itself.

If a pointer to a key is part of a persistent application object, the subErase method on the object needs to erase the key.

MCADAPI Object Memory Management

MCADAPI objects (keys, descriptors, locators) support reference counting to allow for efficient copying and sharing of objects.

Applications should use addRef() and release() to increment and decrement the reference counts respectively. When the reference count hits 0, the object immediately deletes itself; pointers referencing that object then point to garbage and must not be dereferenced. Thus, in practice, release() can be used much like delete. External applications are barred from deleting MCADAPI objects directly (the destructor is protected).

MCADAPI keys are not persistent, but external applications might have persistent objects which hold onto keys and these applications file MCADPI keys in file filers. The MCADAPI has always supported filing AmiObjectKey objects. With the introduction of reference counting, the number of times a given key is held by persistent objects is maintained and updated as keys are filed in/out and erased. Furthermore, this reference count is itself persistent.

Therefore, a separate reference count is maintained for "persistently held" MCADAPI keys. When an MCADAPI is assigned to a data member of a custom database object, the persistent count should be incremented. When that custom object is erased, the subErase() method on that object should call erase() on data members referencing MCADAPI keys. This will cause the persistent-count to be decremented.

With one notable exception, MCADAPI assumes that the external variables used as arguments for output parameters in MCADAPI functions are "transient holders". In other words, before returning a pointer to a key, MCADAPI functions call addRef() on those keys.

The exception is amiReadKey(), which restores the persistent reference count from the filer. For file filers, this will normally reflect the number of times the key was filed out, with 0 for the number of transient holders. For undo filers, this will reflect the state of the key at the time it was written to the undo filer. Only the portion of the count maintained by addPersistentRef()/erase() is stored in the

undo filer; the transient count is unaffected by undo filing (i.e. it is not restored when reading from the undo filer).

The following rules describe MCADAPI behaviour with respect to reference counts:

- All MCADAPI functions except amiReadKey() call addRef() on any output arguments that are pointers to MCADAPI objects.
- The erase() method on AmiObjectKey increments/decrements the persistent reference count in accordance with the bool argument. The persistent count is decremented for erase(true) and incremented for erase(false).

We recommend the following practices:

- Calls to addRef() should be paired with calls to release().
- Calls to MCADPI functions that pass back pointers to keys should be paired with calls to release().
  - **Except** for amiReadKey(), which is the only MCADAPI function which should be paired with calls to erase() instead of release().
- Calls to addPersistentRef () should be paired with calls to erase().
- In the subErase() method for any custom objects that hold pointers to MCADAPI keys, call erase() on those keys.
- Use addRef() and release() accordingly when using variable aliasing (data sharing).
- Call the release() method before allowing a pointer to an MCADAPI object to go out of scope.
- When assigning the value of a local variable that points to an AmiObjectKey to a non-static data member of a custom database object, call addPersistentRef() followed by release().

Coding Examples

For examples of how to make calls to the MCAD API, see the sample applications in Appendix A.

Chapter 4
MCAD API Functions

This chapter describes MCAD API functions.

Detailed Description of Functions

This chapter describes MCAD API functions. Function descriptions are organized by types: common (which includes basic AutoCAD), parametric modeling, Mechanical Desktop surfaces functions, Drawing Manager, User Interface, and File Descriptor functions. Function names appear in alphabetical order within those categories. Each function description begins with the library function's calling scheme, so the number and type of parameters can be seen at a glance.

Common

The following functions are useful for AutoCAD or Mechanical Desktop models.

Handling Pick Objects

The following functions facilitate the task of creating a pick object by providing different ways to construct one. The pick object encapsulates all the information needed to create a key and, once obtained, can be passed as an argument to functions that create keys.

AmiStatus amiFillPickObj

(struct resbuf* *pickResBuf*, **AmiKeyType** *type*, **AmiPickObj&** *pickObj*, **Adesk::Boolean** *allowGhost* **= Adesk::kFalse)**

Fills data fields of *pickObj* given a result buffer in the format returned by acedSSNameX() and the type of key to be created using *pickObj*. Note that, when set to **Adesk::kTru**e, the *allowGhost* flag is effective only for Mechanical Desktop part edges or vertices.

Input

pickResBuf	A resbuf that contains pick information as formatted and returned by acedSSNameX().
type	The type of the key that the caller creates. An abstract type can be used. Not all concrete types are supported (e.g. compKey, dwvwKey, dwLayoutKey, and sceneKey are not supported.
allowGhost	A value of *true* when the caller wants resolution even if the entity has disappeared, for example, resolution to an edge that has been chamfered.

Output

pickObj	An object with all of its data fields filled that represents the object picked by the user. This construct can then be used to create a key to the object selected by the user.

AmiStatus amiGetPickInfo

(AmiPickObj* *pPickObj*, **AcDbFullSubentPath&** *subentPath*, **AcGePoint3d&** *pickPoint*, **AcGeVector3d&** *pickVector*, **AcGeMatrix3d&** *viewTrans*, **Adesk::Boolean&** *isGhost*)

Returns the information associated with the specified pick object.

Input

pPickObj	Pick object to retrieve information from.

Output

subentPath	Full sub entity path of pick.

pickPoint	Pick point.
pickVector	Pick vector.
viewTrans	View transformation matrix.
isGhost	True if geometric resolution is possible even after the entity has disappeared.

AmiStatus amiPick

(const char* *prompt*, **AmiKeyType** *type*, **struct resbuf*** *entmask*, **AmiPickObj&** *pickObj*, **Adesk::Boolean** *allowGhost*, **Adesk::Boolean** *activeInstance*, **const char*** *keyWords*, **const char** *defaultKeyWord*, **Adesk::Boolean** *includeInList*, **char**** *keyWordUsed* **)**

Prompts the user to pick a single object and fills the pick object data fields. The pick object (*pickObj*) can be used in creating a key to the object picked by the user. Note that, when set to **Adesk::kTrue**, the allowGhost flag is effective only for Mechanical Desktop part edges or vertices.

Input

prompt	The prompt displayed to the user.
type	The type of key that the caller intends to create. An abstract type can be used.
entmask	The kind of result-buffer list that acedSSGet() uses to filter a drawing to select particular entities.
allowGhost	A value of *true* when the caller wants resolution even if the entity has disappeared, for example, resolution to an edge that has been chamfered.
activeInstance	A value of *true* restricts the picking to the currently active part instance (if valid for the specified key type).
keyWords	A list of keywords separated by spaces. For localization support, use the AutoCAD standard format to represent the keywords. For example: “localAaa localBbb localCcc_Aaa Bbb Ccc.”
defaultKeyWord	Specifies a keyword. Use NULL for no default keyword. The localization format can ge used if necessary. For example: “localDefault_Default.”

Output

pickObj	An object with all of its data fields filled that represents the object picked by the user. This construct can then be used to create a key to the object picked by the user.
keyWordUsed	The keyword that the user accepted. If localization is used, the user-accepted keyword will be the English keyword. This argument is only filled when the return status is *kKeyWord*. The user is responsible for releasing the memory used by keyWordUsed.
includeInList	TRUE indicates that the default keyword is added to the main list, e.g. [Aaa/Bbb/Ccc/Ddd]<Ddd>. FALSE indicates that the default keyword is not added to the list, e.g. [Aaa/Bbb/Ccc]<Ddd>.

Creating Keys

Keys are created based on data encapsulated in the pick object. Keys provide MCAD API users with a uniform, persistent mechanism to refer to objects in application models. Keys are used as arguments in most MCAD API functions. The following functions provide different ways to construct keys.

AmiStatus amiGetInferGeom

(AmiGeomKey* pInferredGeomKey, int& numGeomKeys, AmiGeomKey& pGeomKeys)**

For geometry which is inferred from other geometry, for instance a (mid)point from a line, this function eturns keys to the geometry used to make that inference. In the example above, a line key to the referenced line would be returned. If the geometry is not inferred Ami::eNotApplicable is returned.

Input

pInferredGeomKey Inferred geometry key.

Output

numGeomKeys Number of keys in the above array.

pGeomKeys Array of geometry keys used to infer the geometry referenced in *pInferredGeomKey*.

AmiStatus amiGetKeyFromId

(AcDbObjectId *id*, **AmiKeyType** *type*, **AmiObjectKey*&** *pObjectKey*, **Adesk::Boolean** *inferFlag* **= Adesk::kTrue, AcDbSubentId*** *pSubId* **= NULL, AcGePoint3d*** *pPoint* **= NULL)**

Creates a geometric key from the given object id. If an error occurs a bad status is returned and *pObjectKey* is set to NULL.

NOTE: If a point key is requested and a sub id is passed in, the given sub id should represent an edge that contains the appropriate vertex subentity. It should not represent the vertex subentity. Use the point parameter to indicate which vertex in the edge to use.

Also, this function should not be used when the entity is blocked. An object key will be returned, but data retrieved from the object key will not be adjusted with respect to the entity's inserted position. Instead, use **amiGetKeyFromPath** and send in an **AcDbFullSubentPath** that lists the containing inserts. It is the caller's responsibility to release the key.

Input

id The object id from which the geom key is created.

type Indicates what kind of key to create.

inferFlag Indicates whether or not to infer a key.

pSubId An optional subentity from which to create the key.

pPoint An optional point to clarify how the key should be created.

Output

pObjectKey A pointer to the key.

AmiStatus amiGetKeyFromPath

(const AcDbFullSubentPath& *subentPath*, **AmiKeyType** *type*, **AmiObjectKey*&** *pObjectKey*, **Adesk::Boolean** *inferFlag* **= Adesk::kTrue, AcGePoint3d*** *pPoint* **= NULL)**

Creates an object key from the given subentity path. The key will refer to the subentity identified by the **subentPath**, if any. If no subentity is identified, the key will refer to the entity itself. If the entity's id is not contained in the **AcDbFullSubentPath**, an error is returned. If an error occurs, a bad status is returned and *pGeomKey* is set to NULL.

NOTE: If a point key is requested and a sub id is passed in, the sub id in the given subentity path should represent an edge that contains the appropriate vertex subentity. It should not represent the vertex subentity. Use the point parameter to indicate which vertex in the edge to use. It is the caller's responsibility to release the key.

Input

subentPath The **subentPath** from which the geom key is created.

type Indicates what kind of key to create.

inferFlag Indicates whether or not to infer a key.

pPoint An optional point to clarify how the key should be created.

Output

pObjectKey A pointer to the key.

AmiStatus amiGetKeyFromPick

(AmiPickObj* *pPickObj*, **AmiKeyType&** *type*, **AmiObjectKey*&** *pObjKey* **)**

Creates an object key from a pick object and returns the type of key created. If an error occurs, a bad status is returned and *pObjKey* is set to NULL. It is the caller's responsibility to release the key.

Input

pPickObj The pointer to the pick object.

Output

type The type of the key that was created.

pObjKey The pointer to the key.

AmiStatus amiGetGeomKey

(AmiKeyType *type*, **Adesk::Boolean** *inferFlag*, **AmiGeomKey*&** *pGeomKey*, **AmiPickObj*** *pPickObj1*, **AmiPickObj*** *pPickObj2* = **NULL)**

Creates a geometric key of a specified type using information passed in by pick objects. The function can infer geometry from other geometry. A point can be inferred as the endpoint of an open curve, the midpoint of a picked line, the center point of an arc or ellipse, or as the midpoint of a circular arc. A line can be inferred as the axis through the center of a picked arc perpendicular to the plane of the arc. A line can also be inferred from the endpoints of a spline. A plane can be inferred as the plane of a picked arc or by two coplanar lines that were picked. If the operation cannot be completed a bad status is returned and *pGeomKey* is set to NULL. It is the caller's responsibility to release the key.

Input

type The type of the geometry to be created.

inferFlag A flag set to *true* or *false* to allow or disallow geometry to be inferred from other picked geometry.

pPickObj1 Pick information.

pPickObj2 Additional pick information required to infer the final key when two picked entities are involved as in inferring a plane from two coplanar lines.

Output

pGeomKey The pointer to the new geometry key.

AmiStatus amiInferGeomKey

(AmiGeomKey** *pGeomKeys*, **const int** *numGeomKeys*, **AmiKeyType** *type*, **AcGePoint3d** *proximityPoint*, **AmiGeomKey*&** *pInferredKey* **)**

Attempts to create an inferred geom key of the given type using all the geometry keys given in *pGeomKeys*. The proximity point is used to disambiguate among choices, and the choice closest to the proximity point is returned. The same inferences made in *amiGetGeomKey* will be made in creating the key. For instance, a point key can be inferred from a line key. The appropriate end of the line is determined by which is closer to the proximity point.

Input

pGeomKeys Array of geometry keys used to infer the new key

numGeomKeys	Number of keys in the above array
type	Type of geometry key to be created.
proximityPoint	A point used to disambiguate among alternative keys which could be inferred.

Output

pInferredKey	The pointer to the new geometry key.

Handling Keys

The following functions provide general functionality to handle any key. These functions allow keys to be checked for equivalence, to be copied, to be erased, and to be written/read to/from a filer.

AmiStatus amiAreKeysEquivalent

(AmiObjectKey* *pKey1*, **AmiObjectKey*** *pKey2*, **Adesk::Boolean&** *areEquivalent,* **Adesk::Boolean** *considerComp = Adesk::kTrue,* **Adesk::Boolean*** *differsInCompOnly* = **NULL)**

Compares two keys to determine if they refer to the same entity. The *pKey1* parameter points to the first object key. The *pKey2* parameter points to second object key. When the function returns, the *areEquivalent* parameter is *true* if the keys resolve to the same entity; otherwise, it is *false*. The *considerComp* parameter allows you to determine not only whether the geometry or features of two keys are equivalent, but also whether they are located on the same component. The two keys must resolve to the same lowest-level component to be equal. The *differsInCompOnly* parameter allows you to determine whether the difference between keys is caused solely by location on different components.

Note also that *considerComp* is applicable only to component keys, geometry keys and feature keys. It is ignored (*differsInCompOnly* returns false) for part, constraint, parameter, sketch, solid and component definition keys.

Input

pKey1	The pointer to the first object key.
pKey2	The pointer to the second object key.
considerComp	If *true*, the equivalence test considers the component as a criterion for determining if the keys are equivalent. If *false*, the component is not considered. In other words, if *true*, the two keys must not only refer to the same geometry or feature to be equivalent, but they must also be on the same component. If *false*, two keys that refer to the same geometry or feature will return a value of *true* regardless of component.

Output

areEquivalent	A value of *true* if the keys resolve to the same entity; otherwise, it is *false*.
differsInCompOnly	If a pointer to a Boolean is specified here, the result will show whether the keys are not equivalent only because they are located on different components. This is independent of the *considerComp* argument and is ignored if a NULL pointer is specified.

AmiStatus amiCopyKey

(AmiObjectKey* *pObjKey*, **AmiObjectKey*&** *pCopy,* **AcDbDatabase*** *pTargetDatabase* **= NULL)**

Copies the given key and returns another key that resolves to the same application object as the original key. AmiCopyKey should be used:

- before enabling active notification

- to put the key in a different database before using AddPersistentRef.

The new key created by this function will not be persistent, regardless of whether the original key was persistent. If the new key should be persistent, then you must call addPersistentRef.

pTargetDatabase defaults to the Desktop's current database (returned from amiGetCurrentDatabase). The database does not need to contain the nested entity, but the nested entity must be accessible from the destination database. If it isn't, the key will be invalid if the destination database is opened from a different document.

The destination database must be the same as the database returned by amiGetAssociatedDBs, if one is returned. If no database is returned, then the key can be copied to any database desired.

It is the caller's responsibility to release the key.

Input

pObjKey	A pointer to the original key.
pTargetDatabase	The database into which the key is copied. Defaults to the Desktop's current database (returned from amiGetCurrentDatabase).

Output

pCopy	A pointer to the copy of the key.

AmiStatus amiGetKeyType

(AmiObjectKey* *pObjKey*, **AmiKeyType&** *type* **)**

Returns the type of the specified key.

Input

pObjKey	A pointer to a key.

Output

type	The type of the key

AmiStatus amiGetAssociatedDbs

(AmiObjectKey* *pObjKey*, **int&** *numDbs*, **AcDbDatabase**&** *pDbs* **)**

This function gets the databases used to build the key. A key may be associated with zero or more databases. If there are no database objects used in the key, *pDbs* is set to NULL and *numDbs* = 0, but the returned status is not an error status.

Input

pObjKey	Pointer to a key.

Output

numDbs	Number of databases used to build the key.
pDbs	Pointer to an array of AcDbDatabases.

AmiStatus amiIsKeyKindOf

(AmiObjectKey* *pObjKey*, **AmiKeyType** *type*, **Adesk::Boolean&** *isType* **)**

Checks if the type of the key is derived from or the same as *type*.

Input

pObjKey	The pointer to a key
type	The type to check against.

Output

isType A value of *true* if the key is of the same type as *type* or if the key type is derived from *type*; otherwise, *isType* is *false*.

AmiStatus amiIsKeyNull

(AmiObjectKey* *pObjKey*, **Adesk::Boolean&** *nullKey* **)**

Checks whether the given object keys has been initialized.

Input

pObjKey A pointer to an object key.

Output

nullKey A value of *true* if the key is null; otherwise, it is *false*.

AmiStatus amiReadKey

(AcDbDwgFiler* *filer*, **AcDbObject*** *pFilingObj*, **AmiObjectKey*&** *pObjKey* **)**

Reads a key from a given filer. This function is usually called from the dwgin method of the application object that owns the key. It is the caller's responsibility to release the key.

Input

filer A pointer to the filer.

pFilingObj The filing database object.

Output

pObjKey The pointer to the key.

AmiStatus amiRefocusKey

(AmiObjectKey* *pObjKey*, **AmiObjectKey*** *pRefKey*, **AmiObjectKey*&** *pNewObjKey* = **AMI_NULLOBJKEYREF)**

Creates a new key from pObjKey which refers to the equivalent object to that referred to by pObjKey at the scope implied by pRefKey. Five basic types of refocusing operations are supported by this function.

The first type of refocusing operation takes place between a part and an instance of that part. For example, if one has a line key to the boundary of a part, it is possible, using this function, to obtain a key to the same line in the scope of an instance of that part in an assembly and vice-versa.

The second type allows the caller that has a key to an entity on a given instance of a part to obtain a key to the equivalent entity on another instance of the same part. For example, if one has a feature key to a hole on the front left wheel of a vehicle, one could obtain the key to the same hole on the rear right wheel of the vehicle.

The third type allows the caller to map from a geometry key to AutoCAD wire-frame geometry to equivalent sketch geometry if the geometry was consumed by a sketch (the reverse operation, that is, going from the sketch geometry to the AutoCAD geometry is not supported). For example if one has a line key to an AutoCAD line that was used in a sketch, this function can return the equivalent sketch line key (which would be needed, say, to create a sketch constraint involving that line). If there is no equivalent sketch geometry in the scope of pRefKey, then an error is returned with symbol kUnmatchedLeafObject.

The fourth type allows the caller to extend or reduce the scope of a key. For example, if one has a key to the edge of a wheel in the scope of the wheel assembly, one could extend the scope of that key to refer to the same edge in the context of the overall vehicle assembly, if one passes in a key that refers to a specific instance of the wheel in the vehicle assembly.

The fifth type allows the caller to map an entity that is in the scope of some component to the scope of a given component definition for one of its parent components. For example if one has a key to the to the edge of a wheel in the scope of the vehicle, one could obtain the key to the same edge in the context of the front axle assembly by passing the component definition of the axle assembly.

If the pRefKey is incompatible with pObjKey, and therefore the key cannot be refocused, then an error is returned with status kFocusMismatch.

Input

pObjKey Pointer to key to original entity. Only geom, feature, component and sketch keys are supported.

PRefKey Pointer to key to object that specifies desired focus (currently only component, part, component definition, geometry, sketch and feature keys supported).

Output

pNewObjKey Key created which references the equivalent object as *pObjKey* in the scope of *pRefKey*. To refocus a key in place, without creating a new key, you can allow the return key to default to **AMI_NULLOBJKEYREF,** which is a null reference.

eUnmatchedLeafObject Returned if there is no equivalent sketch entity in the scope of *pRefKey*.

kFocusMismatch Returned with severity kError if the reference key is incompatible with the object key (e.g. neither is in the scope of the other).

AmiStatus amiWriteKey

(AcDbDwgFiler* *filer***, AcDbObject*** *pFilingObj***, AmiObjectKey*** *pObjKey* **)**

Writes the key to the given filer. This function is usually called from the dwgout method of the application object that owns the key.

Input

filer A pointer to the filer.

pFilingObj The filing database object.

pObjKey The pointer to the key.

Entity Highlighting

The following function provides convenient highlighting of an entity referred to by a key.

AmiStatus amiHighlight

(AmiObjectKey* *pObjKey***, Adesk::Boolean** *highlight***, double** *scaleFactor* **= 1.0)**

Highlights or turns off highlighting of the entity referred to by *pObjKey*.

Input

pObjKey Pointer to a key

highlight A value of *true* to highlight the entity referred to by *pObjKey* or *false* to turn off highlighting for the entity.

scaleFactor A value applied to the default size of graphics used in highlighting unbounded or inferred entities.

Handling Geometry

The following functions allow the geometry referred to by a key to be retrieved or set.
NOTE: Only geometry that is directly editable can be set.

AmiStatus amiGetGeomData

(AmiGeomKey* *pGeomKey*, **AcGeEntity3d*&** *pGeEntity* **)**

Returns an AcGe entity copy of the geometric object referred to by the specified key. The GeLib object that is returned is a snapshot of the geometric object referred to by the key at the time of the query. After its construction, the GeLib object is not kept up-to-date by the MCAD API. After being used, the GeLib object should be deleted.

Input

pGeomKey Pointer to geometry key that refers to the object for which an AcGe entity copy is needed.

Output

pGeEntity A pointer to the new GeEntity.

NOTE: Free the memory allocated for GeEntity when it is no longer needed.

AmiStatus amiGetSketchedSplineSegments

(AmiSplineCrvKey* *pSplineKey*, **int&** *totalNumSegs*, **int** *startIndexNum*, **int** *endIndexNum*, **int*** *numSegsReturned*, **AmiLineKey***** *pSegmentKeys* **)**

Returns geometry keys to the spline segments of the given spline. The start and end indices can be specified, -1 in either position implies the first/last index respectively. The return keys are optional, so if pSegmentKeys or numSegsReturned is null, they are not created. A subset of the segments can be specified by specifying a start and/or end index. The defaults for these arguments specify the beginning and end of the list, respectively. If the end index is beyond the physical number of segments, the end of the list is assumed. These indices are 0 based (range from 0 to the numSegsReturned-1). When a sketch contains a spline, constraints are applied to the straight line segments between the control points. There is one segment that extends past each end point of the spline, representing the tangent lines. AmiGetSketchedSplineSegments returns geometry keys to these line segments to be used when constraining the sketch through amiCreateConstraint.

Input

pSplineKey Pointer to a spline key.

startIndexNum When only a subset of the segments is to be returned, this is the first index to return.

endIndexNum When only a subset of the segments is to be returned, this is the last index to return.

Output

totalNumSegs The total number of segments in the spline, regardless of start/end index arguments.

numSegsReturned Number of segments specified by the start and end index. When pSegmentKeys is not null it contains this number of keys. (Optional)

pSegmentKeys An optional list of pointers to geometry keys to the spline segments.

AmiStatus amiLineIsSilhouette

(AmiLineKey* *pLineKey*, **Adesk::Boolean&** *isSilhouette* **)**

A line can be a silhouette or not.

Input

pLineKey Line key

Output

isSilhouette Boolean that tells whether the line is a silhouette.

AmiStatus amiSetGeometry

(AmiGeomKey* *pGeomKey*, **AcGeEntity3d*** *pGeEntity* **)**

Sets the data in the geometric object referred to by the specified key to the data in the Ge entity passed in. This call will be successful only if the geometry is directly editable as defined by the result of a call to **amiIsDirectlyEditable**.

Input

pGeomKey Pointer to geometry key to entity to be set.

pGeEntity Pointer to Ge entity to be used in setting the geometric object.

Handling Change Notification

The following functions give access to the active and passive notification mechanisms provided with the MCAD API. When active notification through a given key is requested, the system will fire the registered callback functions whenever the entity being referred to is edited or erased. Active notification through a key has to be explicitly enabled through a call to **amiEnableActiveNotification** specifying the object you want to get active notification of changes for. Passive notification, on the other hand, is always available and is good across sessions. When using passive notification, you ask whether an edit has taken place at the time that is convenient for you.

AmiStatus amiDisableActiveNotification

(AmiGeomKey* *pGeomKey* **)**

Disables active notification through the specified geometry key.
NOTE: This operation is not undoable.

Input

pGeomKey A pointer to a geometry key to application geometry.

AmiStatus amiEnableActiveNotification

(AmiGeomKey* *pGeomKey*, **AmiNotifFcnPtr** *pChangeFcn*, **AmiNotifFcnPtr** *pEraseFcn* **)**

Enables active notification through the specified geometry key. Once enabled, active notification through a key is good for as long as the DWG file containing the key is loaded. The next time that the file is loaded, another call to **amiEnableActiveNotification** must be made to reinstate active notification through the key.

Input

pGeomKey A pointer to the geometry key to application geometry.

pChangeFcn A pointer to the function to be called when geometry changes.

pEraseFcn A pointer to the function to be called when geometry is erased.

AmiStatus amiEnableActiveNotification2

(AmiGeomKey* *pGeomKey*, **AmiNotifFcnPtr2** *pChangeFcn*, **AmiNotifFcnPtr2** *pEraseFcn*, **AcDbObjectId** *id***)**

Enables active notification through the specified geometry key. A pointer to the key passed in will be returned when notifying.

This function differs from *amiEnableActiveNotification* in that this function takes a database object id from the user. This id is returned during the notification process. For example, the id might be the database id of the object that owns the key and that will have to react to the change experienced by the geometry.

Input

pGeomKey	A pointer to the geometry key to application geometry.
pChangeFcn	A pointer to the function to be called when geometry changes.
pEraseFcn	A pointer to the function to be called when geometry is erased.
id	AcDbObjectId to be returned during notification.

AmiStatus amiObjectHasChanged

(AmiGeomKey* *pGeomKey*, **Adesk::Boolean&** *changed*, **Adesk::Boolean** *resetNotification* **= Adesk::kTrue)**

Checks if the geometric object referenced by *pGeomKey* has changed since last checked. By default, notification is reset after the call, meaning that if a change is indicated in a call, a second call indicates change only if another change happened since the previous call. This resetting can be avoided by passing the value of *resetNotification* as **Adesk::kFalse**.

Input

pGeomKey	A pointer to the geometry key to application geometry.
resetNotification	If *true*, passive notification will be reset; if *false*, passive notification will not be reset.

Output

changed	A value of *true* if the application object changed; otherwise, it is *false*.

amiObjectWasErased

(AmiObjKey* *pObjKey*, **Boolean&** *erased* **)**

Checks if the object referenced by *pObjKey* has been erased.

Input

pObjKey	A pointer to the object key.

Output

erased	A value of *true* if the application object was erased; otherwise, it is *false*.

B-rep API Integration

The following two functions provide integration between the MCAD API and the B-rep API by allowing API users to go back and forth between MCAD API keys and B-rep API entities.

AmiStatus amiGetBrepFromKey

(AmiObjKey* *pObjKey*, **AcBrEntity*&** *pBrepEntity* **)**

Creates a B-rep entity from the given object key. If an error occurs, a bad status is returned and *pBrepEntity* is set to NULL. The B-rep entity can be used to create a B-rep traverser. For this call to succeed, the key being passed in must refer to an entity that is part of a B-rep. Acceptable types are geometry keys and component keys to leaf node components. It is the caller's responsibility to delete the B-rep entity returned.

Input

pObjKey	The key from which to construct a B-rep entity.

Output

pBrepEntity A pointer to the corresponding B-rep entity.

AmiStatus amiGetKeyFromBrep

(AcBrEntity* *pBrepEntity*, **AmiGeomKey*&** *pGeomKey* **)**

Creates a geometric key from the given B-rep entity. If an error occurs, a bad status is returned and *pGeomKey* is set to NULL. It is the caller's responsibility to release the geometry key returned.

Input

pBrepEntity The B-rep entity from which the geom key is created.

Output

pGeomKey A pointer to the key.

Handling Attributes

The following functions handle attribute attachment, removal and query through keys. For more information on attribute creation, see the section in chapter 4, "Attribute Creation in the MCAD API."

AmiStatus amiAddAttribute

(AmiObjKey* *pObjKey*, **AmiAttribute*** *pAttribute,* **Ami::AttrAssocType** *attrAssoc = Ami::kShareAlways* **)**

Adds an attribute to the object referred to by the specified key. Currently only *AmiGeomKey*, *AmiSolidKey*, *AmiPartKey*, *AmiFeatKey*, *AmiCompDefKey* and *AmiCompKey* are supported. At the time the attribute is added, it is possible to specify what should be the behavior if the object is copied (attribute could be ignored, shared with the copied object, or a copy of the attribute could be attached to the copy of the object).

Input

pObjKey A pointer to the object key.

pAttribute A pointer to an attribute object. The attribute object must already have been added to the database.

AttrAssoc Association type for this attribute when attached to *pObjKey*. Indicates whether the attribute should be copied, shared or ignored when the object is copied within a database or across databases.

AmiStatus amiGetAllAttributes

(int& *numAttributes*, **AmiAttribute**&** *pAttribsArray*, **AmiAttrMatchType** *type* **= Ami::kInstantiable, const char*** *className* **= NULL, AmiObjectKey*** *pObjKey* **= NULL)**

Gets all attributes (owned by the MCAD API) attached in the current database. The type argument indicates what types of attributes to return, either derived, instantiable or both. The className can filter the attributes returned to only those that are instances of a specified attribute class. ClassName is compared with the name returned from AmiAttribute::getName, which is the public class name for the attribute.. pObjKey provides a scope from which attributes are retrieved. E.G. passing a part key returns all attributes attached anywhere on that part (through geom or part feature keys). A key can be used to filter the list returned to only those attached somewhere in that scope. For instance, if a part key is passed in, only attributes attached to geometry or features of that part are returned. It is the caller's responsibility to free the array (not the attributes) when it is no longer needed.

Input

type The type of attributes to return.

className For instantiable attributes, class name to find.

pObjKey Scoping key.

Output

numAttributes	The number of attributes found.
pAttributesArray	An array of pointers to all attached attributes matching the query requirements.

AmiStatus amiGetAttributeHolders

(AmiAttribute* *pAttribute*, **int&** *numKeys*, **AmiObjectKey**&** *pKeysArray* **)**

Gets all the keys that have the specified attribute. It is the caller's responsibility to release all the keys returned and delete the array.. Array of keys is not set if no objects are associated with the specified attribute, or if any error occurs.

Input

pAttribute	Pointer to the specified attribute.

Output

numKeys	Total number of keys found.
pKeysArray	Array of keys to all objects that have the attribute attached. To be freed by the user when finished using it.

AmiStatus amiGetAttributes

(AmiObjKey* *pObjKey*, **int&** *numAttributes*, **AmiAttribute**&** *pAttribsArray*, **Adesk::Boolean** *exactMatch* **= Adesk::kFalse)**

Gets all the attributes that have been attached to the object referred to by the specified key. Currently only *AmiGeomKey*, *AmiSolidKey*, *AmiPartKey*, *AmiFeatKey*, *AmiCompDefKey* and *AmiCompKey* are supported. For geometry, component and component feature keys, attributes on the equivalent entity anywhere in the subassembly chain will be gathered and returned, if *exactMatch* = FALSE. It is the caller's responsibility to free the array (not the attributes) when it is no longer needed. The array of attributes is not set if no attributes are associated with the entity, or if any error occurs. This function is overloaded (see below) to allow attribute query for B-reps without having to create a key.

Input

pObjKey	A pointer to the object key.
exactMatch	A flag indicating whether to return attributes only when the holder is an exact match of pObjKey (TRUE).

Output

numAttributes	The number of attributes found.
pAttributesArray	An array of pointers to all attached attributes referred to by the specified key. To be freed by the user when finished using it.

(AcBrEntity* *pBrepEntity*, **int&** *numAttributes*, **AmiAttribute**&** *pAttribsArray*, **Adesk::Boolean** *exactMatch* **= Adesk::kFalse)**

This overload of *amiGetAttributes* allows attribute query without creating a key by using an equivalent B-rep representation.

Input

pBrepEntity	A pointer to a brep entity. It is the caller's responsibility to delete this brep entity.
exactMatch	A flag indicating whether to return attributes only when the holder is an exact match of pObjKey (TRUE).

Output

numAttributes	The number of attributes found.

pAttributesArray An array of pointers to all attached attributes referred to by the specified key. To be freed by the user when finished using it.

AmiStatus amiRemoveAttribute

(AmiObjKey* *pObjKey*, **AmiAttribute*** *pAttribute* **)**

Removes an attribute from the object referred to by the specified key. Currently only *AmiGeomKey*, *AmiSolidKey*, *AmiPartKey*, *AmiFeatKey*, *AmiCompDefKey* and *AmiCompKey* are supported. This function is overloaded (see below) to remove a specific attribute without having to create a key to the holding object.

Input

pObjKey A pointer to the object key.

pAttribute A pointer to an attribute object.

(AcBrEntity* *pBrepEntity*, **AmiAttribute*** *pAttribute* **)**

This overload of amiRemoveAttribute removes a specific attribute without having to create a key to the holding object. The attribute is removed from the object referenced in the B-rep.

Input

pBrepEntity A pointer to the brep entity. It is the caller's responsibility to delete this pointer.

pAttribute A pointer to an attribute object.

Instantiable Attributes

AmiStatus amiDefineAttClass

(const char* *className*, **int** *nFields*, **const AmiAttFieldDesc*** *pFields* **)**

Defines a new instantiable attribute "class."

Input

className Instantiable attribute "class" name.

nFields Number of fields in the attribute.

pFields Array of *nFields* field description structures.

AmiStatus amiGetAttClasses

(int& *nClasses*, **const char**&** *classNames* **)**

Gets the class names for all of the currently defined instantiable attribute "classes." It is the caller's responsibility to delete the array of class names when finished.

Output

nClasses Number of defined attribute "classes."

classNames Array of character buffers containing attribute "class" names.

AmiStatus amiGetAttClassFields

(const char* *className*, **int&** *nFields*, **AmiAttFieldDesc*&** *pFields* **)**

Gets the field descriptions from and instantiable attribute "class." It is the caller's responsibility to delete the array of field descriptions when finished.

Input

className Name of attribute "class."

Output

nFields Number of fields in attribute.

pFields Array of field descriptor structures.

AmiStatus amiGetAttClassName

(AmiInstAtt* *pAtt*, **const char*&** *className* **)**

Gets the "class" name of the instantiable attribute.

Input

pAtt Pointer to an instantiable attribute object.

Output

className Character buffer containing class name.

AmiStatus amiGetAttField

(AmiInstAtt* *pAtt*, **const char*** *fieldName*, **<input type>** *value* **)**

Gets the value of an instantiable attribute field by name. This function is overloaded for all output types needed to support the current set of attribute field types. These types are:

short&

long&

double&

char&

int& size, const short*& pArray

int& size, const long*& pArray

int& size, const double*& pArray

int& size, const char* pArray

Input

pAtt Pointer to an instantiable attribute object.

fieldName Name of field to get.

value The value for the field name.

AmiStatus amiGetAttField

(AmiInstAtt* *pAtt*, **int** *index*, **<input type>** *value* **)**

Gets the value of an instantiable attribute field by index. This function is overloaded for all output types needed to support the current set of attribute field types. These types are:

short&

long&

double&

char&

int& size, const short*& pArray

int& size, const long*& pArray

int& size, const double*& pArray

int& size, const char*& pArray

Input

pAtt Pointer to an instantiable attribute object.

index Index of field to get.

value The value for the field name.

AmiStatus amiMakeInstAtt

(const char* *className*, **AmiInstAtt*&** *pAtt* **)**

Creates an instantiable attribute object.

Input

className Name of previously defined instantiable attribute class.

Output

pAtt Pointer to the new attribute object.

AmiStatus amiReadAttClassesFromFile

(const char* *fileName* **)**

Reads an ASCII file containing instantiable attribute "class" definitions. If a fully qualified path name is specified, the file at that location is opened for reading. Otherwise, the current attach point is scanned for the file.

Input

fileName Name of file to read from.

AmiStatus amiSetAttField

(AmiInstAtt* *pAtt*, **const char*** *fieldName*, **<input type>** *value* **)**

Sets the value of an instantiable attribute field by name. This function is overloaded for all input types needed to support the current set of attribute field types. These types are:

short

long

double

char

int size, const short* pArray

int size, const long* pArray

int size, const double* pArray

int size, const char* pArray

Input

pAtt Pointer to an instantiable attribute object.

fieldName Name of field to set.

value The value for the field name.

AmiStatus amiSetAttField

(AmiInstAtt* *pAtt*, **int** *index*, **<input type>** *value* **)**

Sets the value of an instantiable attribute field by index. This function is overloaded for all input types needed to support the current set of attribute field types. These types are:

short

long

double

char

int size, const short* pArray

int size, const long* pArray

int size, const double* pArray

int size, const char* pArray

Input

pAtt	Pointer to an instantiable attribute object.
index	Index of field to set.
value	The value for the field name.

AmiStatus amiUndefineAttClass

(const char* *className* **)**

Undefines a defined attribute class. If an attribute object has been instantiated in the current database from the named class, the class cannot be undefined.

Input

className	Name of class to undefine.

AmiStatus amiWriteAttClassToFile

(const char* *className*, **const char*** *fileName*, **Adesk::Boolean** *fAppend* **)**

Writes the instantiable attribute "class" identified by the specified class name to an ASCII file located in the specified file. If a fully qualified path name is specified, the file is written to that location. Otherwise, the file is created at the current attach point.

Input

className	Name of the "class" to be written.
fileName	Name of file to write to.
fAppend	Specify whether the file is appended or overwritten.

AmiStatus amiWriteAttClassesToFile

(const char* *fileName*, **Adesk::Boolean** *fAppend* **)**

Writes all of the currently defined instantiable attribute "class" descriptions to an ASCII file located in the specified file. If a fully qualified path name is specified, the file is written to that location. Otherwise, the file is created at the current attach point.

Input

fileName	Name of file to write to.
fAppend	Specify whether the file is appended or overwritten.

Miscellaneous

AmiStatus amiApplyColorOverride

(AmiObjectKey* *pObjKey*, **Adesk::UInt16** *color* **)**

Applies a color override to the object specified by the object key.

NOTE: Currently this function only works for construction features.

Input

pObjKey Pointer to an object key.
color Standard AutoCAD color index.

AmiStatus amiCreateSystemAttribute

(const char* *pName*, **Ami::AttrDataType** *dataType*, **AmiAttribute*&** *pAttrib* **)**

Creates an MDT system attribute. These attributes contain a single data field that can be of type Ami::kString, Ami::kLong, or Ami::kDouble.

Input
pName The name to assign to the new system attribute.
dataType The field data type for the new attribute.

Output
pAttrib Pointer to the newly created attribute.

AmiStatus amiEndExternalEdit

(bool *bSaveChanges*, **const char*** *appName*= **Designer_Assemblies)**

Ends any XRef edit that is in progress. Has no effect if there is no XRef edit in progress.

Input
bSaveChanges If true, then any changes to the external part will be saved out to the external file. This is the default, and is normal Mechanical Desktop behavior.
appName Optional; name of application to perform the function.

AmiStatus amiEraseObject

(AmiObjectKey* *pObjKey* **)**

Erases the object referenced by the key, if allowed.

Input
pObjKey Pointer to the key.

AmiStatus amiGetAssociatedDoc

(AmiObjectBase* *pAmiBase*, **AcApDocument*&** *pDoc* **)**

This function gets the document associated with the key. Every key is associated with at most one document.

Input
pAmiBase Pointer to an MCAD API object (e.g. a key).

Output
pDoc Pointer to the document associated with the input argument.

AmiStatus amiGetColorOverride

(AmiObjectKey* pObjKey, Adesk::UInt16& color)

Gets the color override from the object specified by the object key. An error is returned if the object has no color override.

NOTE: Currently, this function only works for construction features.

Input
pObjKey Pointer to an object key.
color Standard AutoCAD color index.

AmiStatus amiGetCurrentDatabase

(AcDbDatabase*& *pDB* **)**

Returns the database that is actively being edited. This will change as the document changes and as external databases are edited locally (xref edit).

Output

pDB Database being edited.

AmiStatus amiGetDbIdsFromKey

(AmiObjectKey* *pObjKey*, **int&** *numIds*, **AcDbObjectId*&** *pIdsArray* **)**

Returns an array of AcDbObjectId's corresponding to the given object key. It is not guaranteed to work for all key types. It is the caller's responsibility to delete the array.

Input

pObjKey Pointer to key.

Output

numIds Number of DbIds in pIdsArray.

pIdsArray Array of DbIds to be deleted by the caller.

AmiStatus amiGetExecVersion

(AmiMDTVersion& *execVersion*, **const char* appName=NULL)**

Get the MDT version for the currently running executable.

Output

execVersion Version object holding MDT release number. The AmiMDTVersion class consists of three intergers representing the major, minor, and patch numbers for MDT.

AmiStatus amiGetFileVersion

(AmiMDTVersion& *fileVersion*, **const char* appName=NULL)**

Get the version for MDT that last saved the current DWG.

Output

fileVersion Version object holding MDT release number. The AmiMDTVersion class consists of three intergers representing the major, minor, and patch numbers for MDT.

AmiStatus amiGetObjectName

(AmiObjectKey* *pObjKey*, **char*&** *objName* **)**

Retrieves the name for the object specified by the object key. The user is responsible for releasing the memory used by *objName*. This function is supported for AmiCompDefKey, AmiSceneKey, AmiDwLayoutKey, AmiFeatKey, AmiParamKey, and AmiSketchKey.

Input

pObjKey Pointer to the object key.

Output

objName Name of object of the given key.

AmiStatus amiGetObjectState

(AmiObjectKey* *pObjKey*, **AmiObjectState&** *objState* **)**

Retrieves the state for the object specified by the object key. Macros can be used to retrieve the specific attributes packed into the state. The following #defines can be used to determine the states packed into the return from this function:

Parameter States

AMI_STATE_PARAM_DRIVEN 0x80000000: externally driven/controlled by a file such as a spreadsheet used for table-driven parts.

AMI_STATE_PARAM_INDEP 0x40000000: independent of other parameters.

AMI_STATE_PARAM_GLOBAL 0x20000000: is a global parameter.

Part States

AMI_STATE_PART_ACTIVE 0x00010000: updated and working.

AMI_STATE_PART_DIRTY 0x00020000: edited, needs regen.

AMI_STATE_PART_ROLLBACK 0x00040000: rolled back, feature edited.

AMI_STATE_PART_MAINTENANCE 0x00080000: in maintenance mode.

Feature States

AMI_STATE_FEAT_STABLE 0x80000000: updated and working.

AMI_STATE_FEAT_SUPPRESSED 0x40000000: user suppressed.

AMI_STATE_FEAT_VERSIONED 0x20000000: suppressed via table-driven parts.

AMI_STATE_FEAT_DIRTY 0x10000000: marked as dirty.

AMI_STATE_FEAT_COMPUTEFAILED 0x08000000: feature failed to compute.

Both Features and Parameters

AMI_STATE_USERCREATED 0x01: user-created, not system-created.

Input

pObjKey Pointer to the object key.

Output

objState Bit packed representation of the object state . The state attribute location is dependent on the type of key used.

AmiStatus amiGetObjectVisibility

(AmiObjectKey* *pObjKey*, **Adesk::Boolean&** *bVisibilityOn***)**

Gets the visibility of the object referenced by the given key. This function is not implemented for all types of keys. Currently, this function only works on:

- Components (AmiCompKey)
- Work features (AmiFeatKey, AmiGeomKey)

NOTE: For comp keys, the key must be for a "comp in def". In other words, visibility is a property of only those comps that are not contained within other comps. The function, amiGetCompInDef() can be used to convert a comp key to one that meets this criterion.

Input

pObjKey Pointer to an object key.

Output

bVisibilityOn Returns kTrue if and only if the underlying object has not had visibility turned off.

AmiStatus amiGetSystemAttributeName

(AmiAttribute* *pAttrib*, **std::string&** *name***)**

Gets the name from the specified system attribute.

Input

pAttrib The system attribute from which to get the name.

name The name currently assigned to the system attribute.

AmiStatus amiGetSysVar

(const char* *szVarName*, **char*&** *szValue***);**

Get a string value from the named system variable. It is the caller's responsibility to delete the string when finished.

Input

szVarName System variable name.

Output

szValue String value.

(const char* *szVarName*, **int&** *value***);**

Get an integer value from the named system variable.

Input

szVarName System variable name.

Output

value Integer value.

(const char* *szVarName*, **double&** *value***);**

Get a double value from the named system variable.

Input

szVarName System variable name.

Output

value Double value.

(const char* *szVarName*, **AcGePoint3d&** *value***);**

Get a 3d point value from the named system variable.

Input

szVarName System variable name.

Output

value Point value.

AmiStatus amiHasColorOverride

(AmiObjectKey* *pObjKey*, **Adesk::Boolean&** *bHasOverride***)**

Checks to see whether the object specified by the object key contains a color override.

NOTE: Currently, this function only works for construction features.

Input

pObjKey Pointer to an object key.

Output

bHasOverride True if the object contains a color override. Otherwise false.

AmiStatus amiIsDirectlyEditable

(AmiObjectKey* *pObjKey*, **Adesk::Boolean&** *editable* **)**

Checks if the object referenced by the specified key is directly editable. Returns kTrue if and only if the specified key refers to an active, working database as returned by amiGetCurrentDatabase. If the key is a geometry key, then it must not refer to a boundary of a part or a solid, nor can it be inferred.

This function accepts any type of MCADAPI key.

If this function returns false, the Amistatus::symbol() will reflect the reason the object is not editable. The following symbols are used with kFalse:

kDatabaseUnloaded (xref'd object not loaded)

kObjectNotEditable (object is outside of xref edit context)

kOk (object is of a type that can't be edited.)

Input

pObjKey Pointer to the object key.

Output

editable *True*, object is directly editable; *false*, otherwise.

AmiStatus amiRemoveColorOverride

(AmiObjectKey* *pObjKey* **)**

Removes the color override from the object specified by the object key.

NOTE: Currently, this function only works for constructionfeatures. A warning will be returned if the object has no color override.

Input

pObjKey Pointer to an object key.

AmiStatus amiSetObjectName

(AmiObjectKey* *pObjKey*, **const char*** *objName* **)**

Sets the name for the object specified by the object key. An error is returned if the name is the same as that of another object of the same type. This function is currently implemented for feature keys only.

Input

pObjKey Pointer to the object key.

Output

objName New name for the object.

AmiStatus amiSetObjectVisibility

(AmiObjectKey* *pObjKey*, **Adesk::Boolean** *bVisibilityOn* **)**

Sets the visibility of the object referenced by the given key. This function is not implemented for all types of keys. Currently, this function only works on

- Components (AmiCompKey)
- Work features (AmiFeatKey, AmiGeomKey)

NOTE: For comp keys, the key must be for a "comp in def." In other words, visibility is settable on only those comps that are not contained within other comps. The function amiGetCompInDef() can be used to convert a comp key to one that meets this criterion.

Input

pObjKey Pointer to an object key.

bVisibilityOn The desired visibility setting for the object.

AmiStatus amiSetSystemAttributeName

(AmiAttribute* pAttrib, const char* pName)

Sets the name on the specified system attribute. In order for a system attribute to be attached to a component definition, the attribute must be named and that name must be unique within the component definition. If the name is not unique or unspecified, an error will be returned when an attempt is made to attach the attribute to a component definition.

Input

pAttrib The system attribute to set the name on.

pName The name to assign to the system attribute.

AmiStatus amiSetSysVar

(const char* *szVarName***, const char*** *szValue***);**

Set a string value on the named system variable.

Input

szVarName System variable name.

szValue String value.

(const char* *szVarName***, int** *value***);**

Set an integer value on the named system variable.

Input

szVarName System variable name.

value Integer value.

(const char* *szVarName***, double** *value***);**

Set a double value on the named system variable.

Input

szVarName System variable name.

value Double value.

(const char* *szVarName***, const AcGePoint3d&** *value***);**

Set a 3d point value on the named system variable.

Input

szVarName System variable name.

value Point value.

AmiStatus amiStartExternalEdit

(AmiCompKey* *pCompKey***, bool** *bWithDimming***= kTrue)**

Starts an XRef edit on the given leaf node component for an external part. This function will fail if there is already an XRef edit in progress.

Input

pCompKey	Pointer to (leaf) comp key for an external part. This is the comp that will be become the active leaf node.
bWithDimming	If true, then UI "dimming" of entities not in the edit will be used. This is the default, and is normal XRef editing behavior.

Parametric Modeling

The following functions are Mechanical Desktop part and assembly modeling-specific functions:

Parameter Handling

AmiStatus amiCreateParam

(AmiParamKey*& *pParamKey*, **const char*** *pName*, **const char*** *comment*, **const char*** *pExp*, **AmiPartKey*** *pPartKey* **= NULL** , **const char*** *appName* = **Designer)**

Creates either a global or local parameter with the information passed in.

Input

pName	Name of the parameter.
comment	Comment of the parameter.
pExp	Expression for the parameter.
pPartKey	Pointer to part key to which to attach parameter when creating a local parameter. A global parameter is created if *pPartKey* is NULL.
appName	Name of related application.

Output

pParamKey	Pointer to the parameter key.

AmiStatus amiEvalParamExpress

(const char* *pExpression*, **double&** *result*, **AmiPartKey*** *pPartKey* **= NULL, const char*** *appName* **= Designer)**

Evaluates a given expression for validity.

Input

pExpression Pointer to an expression.

pPartKey	If NULL, the expression is checked in the global scope only. If non NULL, the expression is checked in both the global scope and in the scope of the part.

Output

result	Resulting value from expression.

Return

Ami::eOk	Expression is valid.
Ami::eIllegalExpression	Expression has illegal syntax.
Ami::eExprEvalFailure	Unable to evaluate expression.
Ami::eParamNotFound	A variable in the expr was not found.

AmiStatus amiGetGlobalParams

(int& *size*, **AmiParamKey**&** *pParamKeys*, **const char*** *appName* = **Designer)**

Returns keys to all the global parameters. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

appName Name of application being queried.

Output

size Number of keys returned.

pParamKeys Array of pointers to keys to global parameters.

AmiStatus amiGetParamComment

(AmiParamKey* *pParamKey*, **char*&** *comment* **)**

Gets the comments associated with the parameter defined by the given key. It is the caller's responsibility to delete the string returned.

Input

pParamKey Pointer to the parameter key.

Output

comment Comment of the parameter. If parameter has no comment set, a null string is returned. This is not an error situation.

AmiStatus amiGetParamDeps

(AmiParamKey* *pParamKey*, **int&** *size*, **AmiParamKey**&** *pParamKeys* **)**

Returns keys to all parameter objects that affect the computation of the given parameter. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pParamKey Pointer to parameter key to application parameter.

Output

size Number of keys returned.

pParamKeys Array of pointers to keys to parameters that affect the computation of the given parameter.

AmiStatus amiGetParamExpress

(AmiParamKey* *pParamKey*, **char*&** *pExpression* **)**

Returns the current expression associated with the parameter. It is the caller's responsibility to delete the string indicated by *pExpression* that is returned.

Input

pParamKey Pointer to parameter key to application parameter.

Output

pExpression Expression associated with parameter.

AmiStatus amiGetParamName

(AmiParamKey* *pParamKey*, **char*&** *name* **)**

Returns the name associated with the specified parameter.

Input

pParamKey Pointer to parameter key to application parameter.

Output

name Pointer to parameter name (to be deleted by caller).

AmiStatus amiGetParamUsers

(AmiParamKey* *pParamKey*, **int&** *size*, **AmiParamKey**&** *pParamKeys* **)**

Returns keys to all parameter objects that depend on the specified parameter. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pParamKey Pointer to parameter key to application parameter.

Output

size Number of keys returned.

pParamKeys Array of pointers to keys to all parameter objects that depend on the specified parameter.

AmiStatus amiGetParamValue

(AmiParamKey* *pParamKey*, **double&** *value* **)**

Returns the current value associated with the parameter. The returned value is calculated by evaluating the expression associated with the parameter.

Input

pParamKey Pointer to parameter key to application parameter.

Output

value Current parameter value.

AmiStatus amiSetParamComment

(AmiParamKey* *pParamKey*, **const char*** *pComment* **)**

Sets the value of the parameter comments associated with the given key.

Input

pParamKey The parameter to modify.

pComment The new comment string for the parameter. If a null string is passed in the existing comment is removed.

AmiStatus amiSetParamExpress

(AmiParamKey* *pParamKey*, **const char*** *pExpression* **)**

Sets the current expression associated with the parameter.

Input

pParamKey Pointer to parameter key to application parameter.

pExpression Expression to be associated with parameter.

AmiStatus amiSetParamValue

(AmiParamKey* *pParamKey*, **double** *value* **)**

Sets the expression associated with the parameter to the value passed in.

Input

pParamKey Pointer to parameter key to application parameter.

value Value to be associated with the parameter.

Part Handling

AmiStatus amiCreateEmptyPart

(AmiPartKey*& *pPartKey*, **Adesk::Boolean** *fPartKey* **= Adesk::kTrue, const char*** *name* = **NULL**, **const char*** *appName* **= Designer**)

Creates a new empty part.

Input

fPartKey If true, returns a part key to the new part.

name The name of the new part (optional).

appName The name of the new application handling implementation of creation of the new part and owning key (optional).

Output

pPartKey Key to the new part (optional).

AmiStatus amiGetActivePart

(AmiPartKey*& *pPartKey*, **char*** *appName* = **Designer)**

Returns a key to the active part. It is the caller's responsibility to release the key.

Input

appName Name of application being queried.

Output

pPartKey Pointer to key to active part.

AmiStatus amiGetContainingPart

(AmiObjectKey* *pObjKey*, **AmiPartKey*&** *pPartKey* **)**

Returns key to the part which "contains" the object referred to by the key passed in. (If the object is not "owned" by any part, *pPartKey* is set to NULL.) By passing a key to boundary geometry, work geometry, constraint, feature, parameter, or sketch in a parametric part as input, a part key to the parametric part is returned. It is the caller's responsibility to release the key.

Input

pObjKey Pointer to object key.

Output

pPartKey Pointer to body key to the parametric part that "contains" the object specified, or NULL if object is not "owned" by a parametric part.

AmiStatus amiGetNumPartFeats

(AmiPartKey* *pPartKey*, **int&** *numFeats*, **Adesk::UInt32 state = Ami::kAll)**

Returns the number of features in the part.

Input

pPartKey Pointer to part key.

state Bit field allows control over the feature states allowable in the features returned by this function.

Output

numFeats Number of features in the part.

AmiStatus amiGetPartFeats

(AmiPartKey* *pPartKey*, **int&** *size*, **AmiFeatKey**&** *pFeatKeys*, **Adesk::UInt32 state = Ami::kAll)**

Returns keys to all the part's features in regeneration order. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

state Bit field allows the returned feature set to be thinned out based on each feature's current state. The argument is specified by combining (ORing) together values from the Ami::FeatState enumeration.

Output

size Number of keys returned.

pFeatKeys Array of pointers to keys to features in the part.

AmiStatus amiGetPartParams

(AmiPartKey* *pPartKey*, **int&** *size*, **AmiParamKey**&** *pParamKeys* **)**

Returns keys to all the parameters associated with a given part. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

Output

size Number of keys returned.

pParamKeys Pointers to parameter keys to part parameters.

AmiStatus amiGetPartState

(AmiPartKey* *pPartKey*, **AmiModelStateType&** *mState* **)**

Returns the current state of the part passed in.

Input

pPartKey Pointer to part key.

Output

mState State of the part referenced by the key passed in. It may be active and stable, active edited, inactive but affected by toolbody editing, possibly affected by current toolbody editing, or in a maintenance mode.

AmiStatus amiGetPartWorkAxes

(AmiPartKey* *pPartKey*, **int&** *size*, **AmiGeomKey**&** *pGeomKeys* **)**

Returns geometry keys to all of the work axes associated with a given part. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

Output

size Number of keys returned.

pGeomKeys Array of pointers to geometry keys referring to all of the work axes associated with the part.

AmiStatus amiGetPartWorkPlanes

(**AmiPartKey*** *pPartKey*, **int&** *size*, **AmiGeomKey**&** *pGeomKeys*)

Returns geometry keys to all of the work planes associated with a given part. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

Output

size Number of keys returned.

pGeomKeys Array of pointers to geometry keys referring to all of the work planes associated with the part.

AmiStatus amiGetPartWorkVertices

(**AmiPartKey*** *pPartKey*, **int&** *size*, **AmiGeomKey**&** *pGeomKeys*)

Returns geometry keys to all of the work points associated with a given part. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

Output

size Number of keys returned.

pGeomKeys Array of pointers to geometry keys referring to all of the work points associated with the part.

AmiStatus amiGetUnusedSketchesFromPart

(AmiPartKey* *pPartKey*, **int&** *numSketches*, **AmiSketchKey**&** *pSketchKeys* **)**

Finds all sketches associated with the specified part that have not been consumed and returns an array of keys to them It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pPartKey Pointer to part key.

Output

numSketches Number of sketch keys returned.

pSketchKeys Pointer to an array of pointers to keys to all the unused sketches found.

AmiStatus amiRollbackPart

(AmiPartKey ***pPartKey*, **AmiFeatKey ***pFeatKey*)**

Rolls the part back to the point just before the specified feature is created. Use "amiRegen" to roll the part all the way forward.

Input

pPartKey Pointer to part key refers to the part that is to be rolled back.

pFeatKey Pointer to feature key refers to the reference feature. The part is rolled back to the point just before this feature is created.

Regeneration Control

AmiStatus amiRegen

(AmiObjectKey* *pKey*, **Adesk::Boolean** *fForce* **= Adesk::kFalse)**

Regenerates the specified Mechanical Desktop object. If the object has been marked for update, then this function will regen the object. The force-flag can be used to force a regen. This function does

not necessarily update the display to bring it into sync with the object (update of the displayed graphics is often done at end-of-command following a regen). Not all objects can be regenerated directly. For example, amiRegen() returns eNotApplicable for feature keys and constraint keys. Instead, the part or assembly should be regenerated with respect to these objects.

Input

pKey Pointer to an object key.

fForce Force regen even if internal states indicates that regeneration is not necessary (i.e. object not marked for update).

AmiStatus amiRegenAllOfAType

(Ami::KeyType *objType*, **Adesk::Boolean** *fForce* **= Adesk::kFalse)**

Regenerates all objects of the given type.

Input

objType The type of objects to be regenerated, represented as a key type.

fForce Force regen even if not necessary.

AmiStatus amiSetNeedsRegen

(AmiObjectKey* *pKey* **)**

Tags the internal state of the specified Mechanical Desktop object to indicate that the object is not up-to-date and requires regeneration.. Supported only for *AmiFeatKey*, *AmiParamKey*, *AmiSketchKey* and *AmiDwVwKey*.

Input

pKey Pointer to an object key.

Suppression Control

AmiStatus amiIsFeatSuppressed

(**AmiFeatKey*** *pFeatureKey*, **Adesk::Boolean&** *isSuppressed*)

Returns a boolean stating whether or not the specified feature is currently suppressed.

Input

pFeatureKey Pointer to feature key.

Output

isSuppressed Boolean stating if the feature is suppressed.

AmiStatus amiSuppressFeat

(AmiFeatKey* *pFeatureKey* **)**

Suppresses regeneration of the specified feature and all features that directly or indirectly depend on it.

Input

pFeatureKey Pointer to feature key.

AmiStatus amiSuppressFeatsByType

(AmiFeatType *featType*, **AmiPartKey*** *pPartKey* **= NULL, char*** *appName* **= Designer)**

Suppress regeneration of all features on the given part or on all parts of the specified feature type.

Input

featType Feature type to suppress.

pPartKey Pointer to a part key (NULL means all parts).

appName Name of application to delegate to (used only if the part key is NULL).

Note The feature types that can be suppressed in this manner have some limitations. For example, drilled holes by themselves cannot by suppressed, only holes in general. Also, base features cannot be suppressed. Allowable feature types in this call are kHole, kExtrusion, kRevolve, kChamfer, kFillet, kSweep, kArray, kSurfCut, kShell, kParametricBool, kWorkAxis, kWorkPlane, kLoft, and kDraft.

AmiStatus amiUnsuppressFeat

(AmiFeatKey* *pFeatKey*, **Adesk::Boolean bWithDependents = Adesk::kTrue)**

Resumes regeneration of the specified feature.

Input

pFeatKey Pointer to feature key.

bWithDependents If true, also unsuppress dependent features.

AmiStatus amiUnsuppressFeatsByType

(AmiFeatType *featType*, **AmiPartKey*** *pPartKey* **= NULL, char*** *appName* **= Designer)**

Resume regeneration of all features on the given part or on all parts of the specified feature type.

Input

featType Feature type to unsuppress.

pPartKey Pointer to a part key (NULL means all parts).

appName Name of application to delegate to (used only if the part key is NULL).

Note The feature types that can be unsuppressed in this manner have some limitations. For example, drilled holes by themselves cannot by unsuppressed, only holes in general. Also, base features cannot be unsuppressed. Allowable feature types in this call are kHole, kExtrusion, kRevolve, kChamfer, kFillet, kSweep, kArray, kSurfCut, kShell, kParametricBool, kWorkAxis, kWorkPlane, kLoft, and kDraft.

AmiStatus amiUnsuppressPartFeats

(AmiPartKey* *pPartKey* **= NULL, char*** *appName* **= Designer)**

Resumes regeneration of any suppressed features on the given part of on all parts.

Input

pPartKey Pointer to a part key (NULL means all parts).

appName Name of application to delegate to (used only if the part key is NULL).

Component Definition Handling

AmiStatus amiAddCompToCompDef

(AmiCompKey*& *pCompKey*, **AmiCompDefKey*** *pDefKey*, **AmiCompDefKey*** *pOwnerKey*, **const AcGeMatrix3d&** *matrix* **)**

Adds a component to a component definition by providing the definition for the component to be added. The owner component definition must *not* be a leaf node nor may it be external. A matrix is to be supplied to position the component in WCS. The component key for the newly created component is created and returned. It is the caller's responsibility to release the memory associated with the component key.

Input

pDefKey Pointer to the component definition key for the component being inserted.

pOwnerKey Pointer to the component definition where the first component will be inserted.

matrix WCS position for the component.

Output

pCompKey The pointer to the new component key.

AmiStatus amiCopyCompDef

(**AmiCompDefKey*** *pCompDefKey*, **AmiCompDefKey*&** *pNewCompDefKey*, **const char*** *compDefName* = **NULL**)

Copies the component definition to a new one with the given name. Returns eInvalidName if the name is not syntactically correct and eNameAlreadyExists if there is already a component definition with that name. If the name argument is NULL, then the next available component definition name is used. Applicable only for leaf node component definitions.

Input

pCompDefKey Pointer to component definition key.

compDefName Name of component of the given key.

Output

pNewCompDefKey Key to the newly created component definition.

AmiStatus amiCreateCompDef

(AmiCompDefKey*& *pCompDefKey*, **AmiBodyKey*** *pBodyKey* **= NULL**, **const char*** *name* **= NULL**, **const char*** *appName* = **Designer_Assemblies)**

Creates a component definition from a given body key. Because Mechanical Desktop no longer allows component definitions based on solids, this function converts AutoCAD solids to Desktop Parts before creating the component definition. An error will be returned if the passed-in name is invalid. The user can provide a name for the created component definition. Otherwise, if the name is NULL, a default name will be used.

Passing a NULL value for the *pBodyKey* parameter will create an empty component definition (subassembly). This new component definition can now have components inserted into it with **amiAddCompToCompDef ()**.

The **AmiBodyKey** passed-in will be consumed by the new component definition and become invalid. The caller is responsible for erasing the key.

Input

pBodyKey The pointer to the body key.

name Name of the new component definition.

appName Name of application to perform the function.

Output

pCompDefKey The pointer to the new component definition key.

AmiStatus amiCreateCompDefFromFile

(AmiCompDefKey*& *pCompDefKey*, **const char*** *filename*, **const char*** *name* **= NULL**, **const char*** *appName* = **Designer_Assemblies**)

Creates a component definition from a given filename. The file needs to be a DWG file type. The file does not need to be a Mechanical Desktop part. A name can be provided if the filename won't be used as the component definition name. Providing a NULL value for the name will default to the

filename as the component definition name. An error status will be returned if the filename is invalid. The caller is responsible for erasing the key.

Input

filename	Name of file.
name	Name of the new component definition. If the name is NULL, the filename will be used as the component definition name.
appName	Name of the application to perform the function.

Output

pCompDefKey	Key to the new component definition.

AmiStatus amiExternalize

(AmiCompDefKey* *pCompDefKey* **const char*** *pFileName* **)**

Externalizes the given component definition.

Input

pCompDefKey	Pointer to the component definition key whose component definition is to be externalized.
pFileName	Name of the new file in which the component definition is to be stored.

AmiStatus amiGetActiveCompDef

(AmiCompDefKey*& *pCompDefKey* **const char*** *appName* **= Designer_Assemblies)**

Returns the definition of the component that is the current edit target. It is the caller's responsibility to release the key.

Input

appName	Name of the application to perform the function.

Output

pCompDefKey	Pointer to the component definition key.

AmiStatus amiGetAllCompDefs

(**long&** *size*, **AmiCompDefKey**&** *pCompDefKeys*, **const char*** *appName* **= Designer_Assemblies**)

Retrieves all the component definitions that exist in the current database. This list does NOT include the master component definition. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

appName	Name of the application to perform the function.

Output

size	Number of keys returned.
pCompDefKeys	The pointer to an array of pointers to component definition keys.

AmiStatus amiGetAllInstances

(AmiCompDefKey* *pDefKey*, **AmiCompDefKey*** *pScopeKey*, **int&** *numInstances*, **AmiCompKey**** *pInstances*)

A convenience function to get all instances (components) of a given component definition. Only those instances (components) within the given subassembly (scope) are returned. All of the components returned will be "focused" to the given scope (i.e. they will be components in the assembly defined by the component definition referenced by *pScopeKey*. If *pScopeKey* is NULL, then the master assembly is used.

Input

pDefKey	Component definition for which instances are returned.
pScopeKey	Context/scope for the search for instances. Only instances within this subassembly will be returned. If NULL, then the master assembly is used.

Output

numInstances	Number of instances returned.
pInstances	Array of pointers to keys for the instances. It is the caller's responsibility to release() the keys in the array and delete the array.

AmiStatus amiGetBodyFromCompDef

(AmiCompDefKey* *pCompDefKey*, **AmiBodyKey*&** *pBodyKey* **)**

Returns key to a body associated with the definition of a leaf node component. If the component definitions is not that of a leaf node, or if a body is not associated with the leaf node, the pointer to the body key is set to NULL. It is the caller's responsibility to release the key.

Input

pCompDefKey	The pointer to the component definition key.

Output

pBodyKey	Pointer to body key to body associated with the component definition specified or NULL if not applicable.

AmiStatus amiGetCompDefChildren

(AmiCompDefKey* *pCompDefKey*, **long&** *size*, **AmiCompKey**&** *pCompKeys* **)**

Returns keys to all components used by a component definition. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pCompDefKey	Pointer to geometry key to application geometry.

Output

size	Number of keys returned.
pCompKeys	Array of pointers to component keys used by component definition.

AmiStatus amiGetCompDefName

(AmiCompDefKey* *pCompDefKey*, **char*&** *compDefName*, **const Adesk::Boolean&** *fullPath* = **Adesk::kFalse)**

Returns the name of the component definition. If *fullPath* is *true* and the component definition is external, the full path to the file will be returned. The caller is responsible for releasing the memory for *compDefName*.

Input

pCompDefKey	The pointer to the component definition key.
fullPath	If *true*, and the component definition is external, the full path to the file will be returned.

Output

compDefName	Name of the component definition.

AmiStatus amiGetMasterCompDef

(AmiCompDefKey*& *pCompDefKey*, **AcDbDatabase*** *pDb* **= NULL, const char*** *appName* = **Designer_Assemblies**)

Returns a pointer to a component definition key that represents the database's master component definition. This function can be used to obtain the master compdef of any loaded databases for the current document (i.e. the host database and any loaded XRef'd databases.) It is the caller's responsibility to release the key.

Input

pDb The database to get the master of. If not provided, the host database is used.

appName Name of the application to perform the function. If not provided, the default application is **Designer_Assemblies.**

Output

pCompDefKey The pointer to the component definition key.

AmiStatus amiGetPartDefinition

(AmiPartKey* pPartKey,AmiCompDefKey*& pDefKey)

Creates a key to the component definition for the given part.

Input

pPartKey A key to a part.

Output

pDefKey A key to the corresponding component definition.

AmiStatus amiIsCompDefExternal

(AmiCompDefKey* *pCompDefKey*, **Adesk::Boolean&** *external* **)**

Checks if the component definition is an external definition. The output argument is true if and only if the component definition refers to an external object (with respect to the host database). If the component definition object itself is internal, but is used to reference an external subassembly or part, then a warning status of kExternalAttachment is returned (in addition to the argument being true). This warning status is used for any component definition that resides in one database but refers directly to a subassembly or part in another database.

Input

pCompDefKey The pointer to the component definition key.

Output

external *True* if external; *false* if local.

AmiStatus amiIsCompDefLeafNode

(AmiCompDefKey* *pCompDefKey*, Adesk::**Boolean&** *leaf* **)**

Checks if the component definition is that of a leaf node.

Input

pCompDefKey The pointer to the component definition key.

Output

leaf *True*, if the component definition is a leaf node; *false*, if the component definition is not a leaf node.

AmiStatus amiIsCompDefMaster

(AmiCompDefKey* *pCompDefKey*, **Adesk::Boolean&** *master* **)**

Checks if the given component definition is the master definition.

Input

pCompDefKey The pointer to the component definition key.

Output

master Returns *true* if the component definition is the master.

AmiStatus amiLocalize

(amiCompDefKey* *pCompDefKey* **)**

Localizes the given component definition.

Input

pCompDefKey	Pointer to the component definition key whose component definition is to be localized.

AmiStatus amiReassignComps

(int *numCompsToReassign*, **AmiCompKey**** *compsToReassign*, **AmiCompDefKey*** *pTargetDef* **)**

Reassigns the given components to the indicated component definition. This function is overloaded to allow for specification of the target component definition using a component key. When the target is a component instead of a component definition, then constraints on that component will be favored over others when it is it is not possible to preserve all constraints. In general, assembly constraints are preserved wherever possible, but if the target definition is instanced, then it is best to supply a target component.

Input

numCompsToReassign	The length of the array in referenced in the second argument.
compsToReassign	An array of pointers to component keys for components to be reassigned to the new owning (target) component definition.
pTargetDef	Specification of the target component definition. If a component key is supplied, then the definition of that component will be the target definition.

(int *numCompsToReassign*, **AmiCompKey**** *compsToReassign*, **AmiCompKey*** *pTargetComp* **)**

Input

numCompsToReassign	The length of the array in referenced in the second argument.
compsToReassign	An array of pointers to component keys for components to be reassigned to the new owning (target) component definition.
pTargetComp	Specification of the target component definition. If a component key is supplied, then the definition of that component will be the target definition.

AmiStatus amiRemoveCompFromCompDef

(amiCompKey* *pCompKey*, **AmiCompDefKey*** *pOwnerKey* **)**

Removes a component from a component definition. An error is returned if the component does not exist within the component definition.

Input

pCompKey	The pointer to the component key to be removed.
pOwnerKey	The pointer to the component definition key from which the component will be removed.

AmiStatus amiSetActiveCompDef

(amiCompDefKey* *pCompDefKey* **)**

Sets the current edit target component to that referenced by the given key.

Input

pCompDefKey Pointer to a component definition key.

AmiStatus amiSetCompDefName

(amiCompDefKey* *pCompDefKey*, **const char*** *compDefName* **)**

Sets the name of the component definition. An error status is returned if the name is invalid.

Input

pCompDefKey The pointer to the component definition key.

compDefName Name to be given to component definition.

Component Handling

AmiStatus amiGetActiveAssembly

(**AmiCompKey*&** *pCompKey*, **const char*** *appName*)

Returns the currently active assembly as a component key. If the master assembly is the currently active assembly, then an error of type kNullKey is returned (since there is no component for the the master assembly).

Input

appName (Optional.) Name of the app to get the active assembly.

Output

pCompKey Pointer to created comp key. Must be released by caller when done using it.

AmiStatus amiGetActiveLeafNode

(**AmiCompKey*&** *pCompKey*, **const char*** *appName* = **Designer_Assemblies**)

Returns the current active part as a component key.

Input

appName Name of the application to get the active part instance. If NULL, assume Assemblies.

Output

pCompKey Pointer to created (leaf) component key. Must be freed by caller when done using it.

AmiStatus amiGetCompChildren

(**AmiCompKey*** *pCompKey*, **long&** *size*, **AmiCompKey**&** *pCompKeys*)

Returns a pointer to an array of component key pointers to the immediate (first level) subcomponents of the given component. It is the caller's responsibility to release the keys.

Input

pCompKey The pointer to the component key.

Output

size Number of component key pointers being returned.

pCompKeys A pointer to an array of component key pointers.

AmiStatus amiGetCompDef

(AmiCompKey* *pCompKey*, **AmiCompDefKey*&** *pCompDefKey*)

Gets the component's definition. It is the caller's responsibility to release the key..

Input

pCompKey Pointer to the component key.

Output

pCompDefKey Pointer to key to the component's definition.

AmiStatus amiGetCompInDef

(AmiCompKey* *pCompKey1*, **AmiCompKey*&** *pCompKey2* **)**

Given a key to a component returns the key to the component which is its corresponding component in its space of definition. For example, suppose a component is a subassembly made of one bolt and two nuts (let's call it NB). NB could be instanced in several other subassemblies in a model. If you have a component key, N1, that refers to one of the nuts in one of the instances of NB, say NB1, you might want to go edit the definition of NB to remove the nut leaving only one nut and one bolt in NB's definition.

If *amiGetCompInDef* is called and N1 is passed in, a new key, N1A, will be returned, which is the key to that nut in NB's definition. N1A, could be used in a call to *amiRemoveCompFromCompDef* to remove that nut from NB's definition. To perform this call, you would also need to pass in NB's definition, which could be obtained by calling *amiGetCompOwner* passing N1 in. It is the caller's responsibility to release the key.

Input

pCompKey1 The pointer to the component key.

Output

pCompKey2 Pointer to the corresponding component key.

AmiStatus amiGetCompName

(AmiCompKey* *pCompKey*, **char*&** *compName* **)**

Retrieves the name for the component specified by the component key. The caller is responsible for releasing the memory used by *compName*.

Input

pCompKey The pointer to the component key.

Output

compName Name of component of the given key.

AmiStatus amiGetCompOwner

(AmiCompKey* *pCompKey*, **AmiCompDefKey*&** *pCompDefKey* **)**

Gets the definition to the component owning the component passed-in. The caller is responsible for erasing the component definition key.

Input

pCompKey The pointer to the component key.

Output

pCompDefKey The pointer to the component definition key.

AmiStatus amiGetCompPosition

(AmiCompKey* *pCompKey*, **AcGeMatrix3d&** *matrix* **)**

Returns the current WCS position of the component.

Input

pCompKey The pointer to the component key.

Output

matrix Actual position of component in current WCS space.

AmiStatus amiGetContainingComp

(AmiObjectKey* *pObjKey*, **AmiCompKey*&** *pCompKey* **)**

Returns a key to the component which "contains" the object referred to by the key passed in (if the object is not "contained" by any component, *pCompKey* is set to NULL).

Input

pObjKey Pointer to object key.

Output

pCompKey Pointer to component key referencing the component that "contains" the object specified.

AmiStatus amiSetActiveAssembly

(AmiCompKey* *pCompKey* **)**

Sets the current edit target component to that referenced by the given key. The component key should be for a non-leaf component (i.e. an assembly). Also, that assembly must be in the database that is currently open for edit (use amiStartExternalEdit / amiEndExternalEdit to start/stop Xref Edits).

Input

pCompKey Pointer to a component key.

AmiStatus amiSetActiveCompDef

(AmiCompDefKey* *pCompDefKey* **)**

Sets the current edit target component to that referenced by the given key.

Input

pCompDefKey Pointer to a component definition key.

AmiStatus amiSetActiveLeafNode

(AmiCompKey*& *pCompKey* **)**

Sets the current active part. The component key must be for a leaf component (i.e. a part instance). Furthermore, the component key must be for a component in the current active component definition. If this requirement is not met, then an error status of kUnmatchedActiveContext is returned. If an attempt is made to transition to or from an external (XRef) part, this function fails, returning kExternalDef. Separate functions are provided for starting and ending XRef edits.

Input

pCompKey Pointer to (leaf) comp key. This is the component that will become the current edit target.

AmiStatus amiSetCompName

(**AmiCompKey*** *pCompKey*, **const char*** *compName*)

Sets the name for the component specified by the component key. An error status is returned if an invalid name is passed-in.

NOTE: The name of a component that stems from an external file can be changed during the drawing session. However, the same change is not persistent and will revert during the next file loading.

Input

pCompKey The pointer to the component key.

Output

compName New name for the component.

AmiStatus amiSetCompPosition

(**AmiCompKey*** *pCompKey*, **const AcGeMatrix3d&** *matrix*)

Sets the component to the new WCS position. All instances of the component will also update accordingly.

Input

pCompKey The pointer to the component key.

matrix New WCS position for the component.

Constraint Handling

Constraint Descriptors

Constraints can be queried and created using descriptors. These descriptors maintain a snapshot of the state of the constraint when queried. When used for creation, they provide an encapsulation of all the information required to construct a constraint.

These descriptors are similar in interface to the drawing manager and event descriptors. They are accessed with two functions, amiGetData and amiSetData, which are overloaded for all appropriate data types.

The following table lists all of the attributes currently defined in the Constraint API, along with a brief description of each. These same attributes are then separated by constraint-type, as several of these attributes are applicable to only some types of view. New keywords will be added to this list as the set of constraint attributes supported by the MCAD API grows.

Table 1: Constraint Attribute Keywords

Attribute Keyword	Data Type	Read-Only	Applies to	Description
csAttConstrType	AmiConstraintType	Y	All	The type of constraint/descriptor.
csAttParametric	AmiValue		Dimensions & Assembly	The parameter controlling the constraint. May be an expression, numeric, or a parameter key.
csAttLocation	AcGePoint3d		If csAttDisplayed = kTrue	Location of the display element in WCS.
csAttDisplayable	Adesk::Boolean	Y	All	Flag indicating whether the constraint type can be depicted as an entity.
csAttDisplayed	Adesk::Boolean displayOff, displayOnInSketch, displayOn		If csAttDisplayable = kTrue	Adesk::Boolean: Flag indicating whether the constraint is currently displayed. displayOff: Dim not displayed. displayOnInSketch: Dim displayed in sketches only. DisplayOn: Dim displayed in sketches and views.
csAttDisplayId	AcDbObjectId	Y	Dimensions	AcDbObjectId of the entity used to display the constraint (e.g. dimension). Available only when the display entity exists. If the constraint is not displayed at the time the descriptor is built, querying this attribute will return NAV.
csSkAnglePoint	AcGePoint3d		Angle Dimensions	Location in WCS used to disambiguate which of the four angles is to be dimensioned.
csAttMaxOperands	Int	Y	All	Maximum number of operands the constraint can use during construction.

csAttMinOperands	Int	Y	All	Minimum number of operands the constraint requires during construction.
csAttOperand1	AmiGeomKey*		All	The first operand used by the constraint.
csAttOrientation1	AmiNormalOrientnType		All	The orientation of the normal for the first operand, whether it is reversed or based on the raw data from the model.
csAttOperand2	AmiGeomKey*		All	The second operand used by the constraint.
csAttOrientation2	AmiNormalOrientnType		All	The orientation of the normal for the second operand, whether it is reversed or based on the raw data from the model.
csAttGeometryOfSymmetry	AmiGeomKey		Mirror constraint	The geometry around which operand 2 mirrors operand 1.
csAttGeometryOfSymmOrientation	AmiNormalOrientnType		if csAttGeometryOfSymmetry has a value	The orientation of the normal for the axis of symmetry.

In the tables below, the second column describes any constraints on allowable values for the specified attribute. The third column, labeled 'Role in constraint creation' indicates if the attribute must have a non-NAV value when the descriptor is used to create a new view. If this entry is blank, then the attribute is optional and can be NAV. The fourth column indicates if the attribute is read-only, write-once, or read/write (this last case is indicated by a blank entry in column four).

Table 2: Constraint Attribute Keywords - Assembly Constraints

Attribute Name	Allowable Values	Role in constraint creation	Settable through API ?

Assembly constraints are all parametric but do not support display.

csAttConstrType	eCompConstraint (& all subtypes)	mandatory	Write-once
csAttParametric	Any expression, numeric or parameter key		
csAttDisplayable	kFalse		Read-only
csAttMaxOperands	1 or 2		Read-only
csAttMinOperands	1 or 2		Read-only
csAttOperand1	AmiGeomKey to constrained geometry	mandatory	
csAttOrientation1	AmiNormalOrientnType (kRaw, kReversed)		
csAttOperand2	AmiGeomKey to constrained geometry	Mandatory if csAttMaxOperands > 1, not allowed if csAttMaxOperands < 2	
csAttOrientation2	AmiNormalOrientnType (kRaw, kReversed)		

Sketch constraints define absolute relationships between geometry which is non-parametric in nature. These are relationships such as parallel, concentric and tangency. Thus the parametric attribute does not apply. These are also non-displayable at this point.

Table 3: Constraint Attribute Keywords - Sketch Constraints

Attribute Name	Allowable Values	Role in constraint creation	Settable through API ?

csAttConstrType	**Sketch constraints** (eSkCircleCoincidence, eSkPointCurveCoincidence, eSkCenterPoint, eSkHorizontal, eSkVertical, eSkParallel, eSkColinear, eSkPerpendicular, eSkProjection, eSkPointCoincidence, eSkCoincident, eSkConcentric, eSkTangent, eSkXEqual, eSkYEqual, eSkEqualRadius, eSkEqualLength, eSkMirror)	mandatory	Write-once
csAttDisplayable	kFalse		Read-only
csAttMaxOperands	1 or 2		Read-only
csAttMinOperands	1 or 2		Read-only
csAttOperand1	AmiGeomKey to constrained geometry	mandatory	
csAttOrientation1	AmiNormalOrientnType (kRaw, kReversed)		
csAttOperand2	AmiGeomKey to constrained geometry	Mandatory if csAttMinOperands > 1, not allowed if csAttMaxOperands < 2	
csAttOrientation2	AmiNormalOrientnType (kRaw, kReversed)		
csAttGeometryOfSymmetry	AmiGeomKey		
csAttGeometryOfSymmOrientation	AmiNormalOrientnType	Mandatory if csAttGeometryofSymmetry has a value	

Sketch dimensions specify parametric relationships between geometry. They can be displayed by a dimension object, optionally through the API. Thus most of the control for display applies to these constraints.

Table 4: Constraint Attribute Keywords - Sketch Dimensions

Name	Allowable Values	Role in constraint creation	Settable through API ?
csAttConstrType	**Sketch dimensions** (eSkDistance, eSkAngle, eSkRadius, eSkHorDistance, eSkVerDistance, eSkAlignDistance, eSkDiameter, eSkEllRadiusMajor, eSkEllRadiusMinor, eSkAlignDiamDistance, eSkOrdXBase, eSkXOrdDistance, eSkOrdYBase, eSkYOrdDistance, eSkParaDistance)	mandatory	Write-once
csAttParametric	Any expression, numeric or parameter key		
csAttLocation	AcGePoint3d location for dimension display		
csAttDisplayed	Adesk::Boolean		
csAttDisplayId	AcDbObjectId of dimension if csAttDisplayed = kTrue		Read-only
csAttDisplayable	kTrue		Read-only
csSkAnglePoint	AcGePoint3d point within the angle being dimensioned	Mandatory if csAttConstrType=eSkAngle	
csAttMaxOperands	1 or 2		Read-only
csAttMinOperands	1 or 2		Read-only
csAttOperand1	AmiGeomKey to constrained geometry	mandatory	
csAttOrientation1	AmiNormalOrientnType (kRaw, kReversed)		
csAttOperand2	AmiGeomKey to constrained geometry	Mandatory if csAttMinOperands > 1, not allowed if csAttMaxOperands < 2	
csAttOrientation2	AmiNormalOrientnType (kRaw, kReversed)		

AmiStatus amiGetData

(AmiConstrDescrip* *pConstrDescrip*, **AmiConstrAttribute** *attr*, **<Return Type>&** *value*)

Extracts the value for an attribute. This function is overloaded for all return types needed to support the current set of constraint attributes. These types are:

AmiObjectKey*

AcGePoint3d

AmiValue

double

char*

int

Adesk::Boolean

AcDbObjectId

AmiNormalOrientnType

AmiConstraintType

The keywords for attributes are defined by the *AmiConstrAttribute* enum. The Constraint Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. If the value for the attribute is NAV, meaning "Not A Value", then *eAttValueNAV* is returned as the AmiStatus and the content of the 'value' is undefined. It is the caller's responsibility to free, as appropriate, any returned value.

Input

pConstrDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).

Output

value	The value for the specified attribute.

AmiStatus amiSetData

(AmiConstrDescrip* *pConstrDescrip*, **AmiConstrAttribute** *keyword*, **<Input Type>** *value* **)**

Sets an attribute value from the given constraint descriptor. This function is overloaded for all input types needed to support the current set of constraint attributes. These types are:
AmiObjectKey*

AcGePoint3d

AmiValue

double

char*

int

Adesk::Boolean

AcDbObjectId

AmiNormalOrientnType

AmiConstraintType

The keywords for attributes are defined by the *AmiConstrAttribute* enum.The Constraint Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. In addition, the special value Ami:att_NAV can be used to set the specified attribute to "Not A Value". NAV attributes are treated as "unset" or "don't care" values when applying the descriptor to a constraint.

Input

pConstrDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).
value	The value for the specified attribute.

AmiStatus amiCreateConstraint

(AmiConstrDescrip* *pDescrip*, **Adesk::Boolean** *createKey*, **AmiConstrKey**** *pConstrKey* **)**
Creates a constraint as defined in the descriptor, using the specified type and given geometry key(s). If csAttLocation has a value that placement will be used, but a default placement will be created if one is not given. csAttDisplayed controls whether the display entity is created. If csAttParametric has a value (either a number, an expression or a parameter key) it will be used with parametric constraints. If none is given, a parameter using the system defaults (0.0 for assembly constraints, the current distance for sketch dimensions) will be created and used.

Part scope is determined from the principal and auxiliary operands on the constraint descriptors. Both keys (if two are given) must supply the same part instance, otherwise a “no common part” error is returned. Input points (e.g. placement points) are assumed to be in the coordinate space of the part instance provided. If the keys are in the scope of the part, then the input points are assumed to be in part coordinate space.

A key to the newly created constraint can optionally be returned. It is the caller's responsibility to release that key. When creating angle constraints the display location given in the descriptor is required, and is used to determine which of the possible four angles to constrain.

Input

pDescrip The pointer to the constraint descriptor.

createKey Flag indicating whether a key to the new constraint should be created and returned.

Output

pConstrKey A pointer to the constraint key.

AmiStatus amiCreateConstraints

(AmiConstrDescrip** *pDescrips*, **int** *numDescrips*, **Adesk::Boolean** *createKey* **= Adesk::kFalse**
AmiConstrKey*** *pConstrKeys* = **NULL)**

Creates many constraints based on the information given in the constraint descriptors. The constraints are added, followed by a single solve of all of them, avoiding the expense of multiple solves. In the case of failure, the key’s array will have a null pointer in any argument where the creation failed, and the status will return both a warning and the status for the last failure that occurred. Keys to the newly created constraints can optionally be returned. When creating angle constraints the display location given in the descriptor is required, and is used to determine which of the possible four angles to constrain.

Input

pDescrips The pointer to an array of descriptors describing the constraints to be created.

numDescrips Number of descriptors passed in, and number of constraint keys returned.

createKey Flag indicating whether keys to the new constraint should be created and returned.

Output

pConstrKeys A pointer to an array of constraint keys.

AmiStatus amiCreateConstrDescrip

(AmiConstraintType *constrType*, **AmiConstrDescrip*&** *pConstrDescrip*, **const char*** *appName* = **NULL)**

Returns an empty constraint descriptor of the given type. An "empty" constraint descriptor is a descriptor where all values are NAV.

Input

constrType The type for the descriptor.

appName Name of related application. If NULL is specified then a default application will be used.

Output

pConstrDescrip A pointer to an empty descriptor.

AmiStatus amiGetConstrDescrip

(AmiConstrKey* *pConstrKey*, **AmiConstrDescrip*&** *pDescrip*)

Returns a descriptor for the given constraint. The descriptor holds information on the current state of the constraint. The descriptor is de-coupled from the constraint (it does not change as the constraint changes).

Input

pConstrKey A key for a constraint.

Output

pDescrip A pointer to the descriptor.

AmiStatus amiGetConstrOperands

(AmiConstrKey* *pConstrKey*, **AmiGeomKey*&** *pOp1,* **AmiGeomKey*&** *pOp2* **)**

Returns keys to the geometric objects operated upon by the given constraint. It is the caller's responsibility to release the keys.

Input

pConstrKey The pointer to a component constraint key.

Output

pOp1 Key to first geometric object.

pOp2 Key to second geometric object.

AmiStatus amiGetConstrParam

(AmiConstrKey* *pConstrKey,* **AmiParamKey*&** *pParamKey* **)**

Returns key to the parameter associated with a given constraint. It is the caller's responsibility to release the key.

Input

pConstrKey The pointer to the constraint key.

Output

pParamKey A pointer to the parameter key for the constraint key.

AmiStatus amiGetConstrType

(AmiConstrKey* *pConstrKey*, **AmiConstraintType&** *type* **)**

Returns the constraint type. Constraint types are defined in header file *miconstr.h*.

Input

pConstrKey The pointer to the component constraint key.

Output

type Type of component constraint.

AmiStatus amiGetConstrValue

(AmiConstrKey* *pConstrKey*, **double&** *value* **)**

Returns a constraint distance or angle value, in radians.

Input

pConstrKey The pointer to the constraint key.

Output
value The distance or angle offset.

AmiStatus amiGetDimConstrLoc

(AmiConstrKey* *pConstrKey*, **AcGePoint3d&** *locPoint*)
Returns the location point used to display the dimension of a dimensional constraint.

Input
pConstrKey The pointer to the dimensional constraint key.
Output
locPoint The location point used in the display of the dimension.

AmiStatus amiSetConstrExpr

(AmiConstrKey* *pConstrKey*, **const char*** *valExpr*)
Sets the expression associated with the constraint. If the constraint does not have an associated expression (e.g., line/line mating) then an error is returned.

Input
pConstrKey Pointer to a constraint key.
valExpr Expression string.

AmiStatus amiSetConstrValue

(AmiConstrKey* *pConstrKey*, **const double&** *value* **)**
Sets the distance or angle value of the constraint. If setting a constraint value is not applicable (i.e., line/line mating), an error is returned.

Input
pConstrKey The pointer to the constraint key.
value The distance or angle (in radians) offset.

Component Constraint Handling

AmiStatus amiGetCompsFromConstr

(AmiCompConstrKey* *pConstrKey*, **AmiCompKey*&** *pCompKey1*, **AmiCompKey*&** *pCompKey2* **)**
Returns components constrained by the constraint provided. Components are returned in order of birthday. It is the caller's responsibility to release the returned keys.

Input
pConstrKey The pointer to the constraint key.
Output
pCompKey1 A pointer to the first component key (determined by birthday).
pCompKey2 A pointer to the second component key.

AmiStatus amiGetConstrsActingOnComp

(AmiCompKey* *pCompKey*, **long&** *size*, **AmiCompConstrKey**&** *pConstrKeys* **)**

Returns a pointer to an array of component constraint keys for the constraints that play an active part in positioning the component. It is the caller's responsibility to release all the keys in the array and delete the array.

Input
pCompKey The pointer to the component key.

Output

size Number of pointers to constraint keys.

pConstrKeys A pointer to an array of pointers to component constraint keys to constraints that contribute to the component position.

AmiStatus amiGetConstrsFromComp

(AmiCompKey* *pCompKey*, **long&** *size*, **AmiCompConstrKey**&** *pConstrKeys* **)**

Returns a pointer to an array of component constraint keys for the given component. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pCompKey The pointer to the component key.

Output

size Number of pointers to constraint keys.

pConstrKeys A pointer to an array of pointers to component constraint keys.

Sketch Handling

AmiStatus amiGetBrepGeomFromSketchGeom

(AmiGeomKey* *pSketchGeomKey*, **AmiGeomKey*&** *pBrepGeomKey* **)**

A sketch can reference B-rep geometry from the part. The referenced geometry may be projected onto the appropriate sketch plan. But as the B-rep geometry changes, so does the sketch. For sketch geometry, retrieved from amiGetSketchGeom, this function will return the B-rep geometry that defines that sketch geometry.

Input

pSketchGeomKey Inferred geometry key

Output

pBrepGeomKey B-rep geometry driving pSketchGeomKey

AmiStatus amiGetExtSketchConstrs

(AmiSketchKey* *pSketch*, **int&** *size*, **AmiSketchConstrKey**&** *pConstraints* **)**

Returns all constraints that act simultaneously upon geometry contained by the specified sketch and geometry not contained in the sketch. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pSketch Pointer to sketch key.

Output

size Number of keys returned.

pConstraints An array of pointers to keys to constraints that affect geometry contained by the sketch and geometry not contained in the sketch.

AmiStatus amiGetFeatsFromSketch

(**AmiSketchKey*** *pSketch*, **int&** *numFeats*, **AmiFeatKey**&** *pFeatKeys*, **Adesk::UInt32** *state*, **Adesk::Boolean** *fReturnComposite*))

Finds the features which were created from the specified sketch.

Input

pSketchKey Pointer to sketch key.

state Bit field allows control over the feature states allowable in the features returned by this function.

fReturnComposite If a feature bringing about this geometry is contained in a composite feature, a value of true will cause the composite to be returned. False will return the innermost feature.

Output

numFeats Number of keys in resulting array.

pFeatKeys Array of features created from the sketch.

AmiStatus amiGetPathStartPoint

(AmiSketchKey* *pSketch*, **AmiPointKey*&** *pPoint* **)**

Returns the start point from a 2D or 3D path.

Input

pSketch Pointer to sketch key. The sketch referred to by this key must be a 2D or 3D path.

Output

pPoint The start point of the path.

AmiStatus amiGetSketchConstrs

(AmiSketchKey* *pSketch*, **int&** *size*, **AmiSketchConstrKey**&** *pConstraints* **)**

Returns all constraints that affect geometry contained by the specified sketch. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pSketch Pointer to sketch key.

Output

size Number of keys returned.

pConstraints An array of pointers to keys to constraints that affect geometry contained by the sketch and geometry not contained in the sketch.

AmiStatus amiGetSketchCoordSys

(AmiSketchKey* *pSketchKey*, **AcGeMatrix3d&** *coordSys***)**

Returns the transformation matrix that maps from sketch coordinate system to the global coordinate system.

Input

pSketchKey Pointer to sketch key.

Output

coordSys Transformation matrix that performs the mapping.

AmiStatus amiGetSketchDefPlane

(AmiSketchKey* *pSketch*, **AmiPlaneKey*&** *pPlaneKey***)**

Returns a key to the plane of a closed profile. Any other kind of sketch is considered invalid. It is the caller's responsibility to release the plane key.

Input

pSketch Pointer to sketch key.

Output

pPlaneKey Pointer to plane key.

AmiStatus amiGetSketchesFromGeom

(AmiGeomKey* *pGeom*, **int&** *numSketches*, **AmiSketchKey**&** *pSketchKey*, **char*** *appName* = **Designer)**

Finds all sketches associated with the specified geometry. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pGeom	Pointer to geometry key.
appName	Name of application being queried.

Output

numSketches	Number of related sketches found.
pSketchKey	Array of pointers to sketch keys.

AmiStatus amiGetSketchGeom

(AmiSketchKey* *pSketch*, **int*** *pSize* = **NULL, AmiGeomKey***** *pGeom* = **NULL, int*** *pPointSize* = **NULL**, **AmiPointKey***** *pPointGeom* = **NULL, int*** *pSupportSize* **= NULL, AmiGeomKey***** *pSupportGeom* = **NULL)**

Returns all geometric objects used by the specified sketch. It is the caller's responsibility to release all the keys in the arrays and delete the arrays.

Input

pSketch	Pointer to sketch key.

Output

pSize	Number of keys returned. (Optional.)
pGeom	Array of pointers to keys to all the geometry that constitutes the profile, path or cutline. (Optional.)
pPointSize	An optional return of the number of point geometry keys returned.
pPointGeom	An optional array of pointers to keys to all of the support geometry included in the sketch, which is used to help constrain or construct the resulting feature.
pSupportSize	An optional return of the number of support geometry keys returned.
pSupportGeom	An optional array of pointers to keys to all the support geometry included in the sketch, which is used to help constrain or construct the resulting feature.

AmiStatus amiGetSketchParams

(AmiSketchKey* *pSketch*, **int&** *size*, **AmiParamKey**&** *pParams* **)**

Returns all parameter objects related to the specified sketch. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pSketch	Pointer to sketch key.

Output

size	Number of keys returned.
pParams	Array of pointers to keys to all parameters related to the sketch.

AmiStatus amiGetSketchType

(AmiSketchKey* *pSketchKey*, **AmiSketchType&** *sketchType***)**

Returns the sketch type from the given sketch key.

Input

pSketchKey Pointer to sketch key.

Output

sketchType Sketch type.

Sketch Constraint Handling

AmiStatus amiGetConstrSketches

(AmiSketchConstrKey* *pConstraintKey*, **int&** *numSketches*, **AmiSketchKey**&** *pSketchKeys* **)**

Finds all sketches that are constrained by the specified sketch constraint. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pConstraintKey Pointer to a sketch Constraint Key.

Output

numSketches Number of constrained sketches found.

pSketchKeys Array of Sketch Key pointers.

Feature Handling

AmiStatus amiAbortCompositeFeat

(const char* *appName* **= Designer)**

Aborts composite feature creation erasing the current composite feature and all features that were created from the time the current composite feature was started. An error will be returned if there was no composite feature being created.

Input

appName Name of application to delegate to.

AmiStatus amiCreateFeature

(AmiFeatDescrip* *pFeatDescrip*, **AmiFeatKey*&** *pFeatKey*, **Adesk::Boolean** *fFeatKey* = **Adesk::kFalse)**

Creates a feature on the active part using the feature descriptor. It is the caller's responsibility to release the returned key. Note: this function should not be used to create a composite feature. Instead, use amiStartCompositeFeat.

Input

pFeatDescrip Descriptor to use for feature creation.

fFeatKey Set true if a key to the new feature is to be returned.

Output

pFeatKey Output feature key if *fFeatKey* is true.

AmiStatus amiEndCompositeFeat

(AmiFeatKey*& *pFeatKey***, const char*** *appName* **)**

Ends composite feature creation closing the current composite feature and returning a key to it. An error will be returned if there was no composite feature being created.

Input

appName Name of application to delegate to.

Output

pFeatKey Pointer to the composite feature key.

AmiStatus amiGetConstrGeomDescrip

(AmiGeomKey* *pGeomKey*, **AmiConstrGeomDescrip*&** *pGeomDescrip*, **Adesk::Boolean** *fReturnParameters = Adesk::kFalse*, **Adesk::Boolean** *fRetainKey = Adesk::kFalse*, **char*** *appName* **= Designer)**

Creates a constrained geometry descriptor object from a geometry key that refers to a work entity. It is the caller's responsibility to release the descriptor object.

Input

pGeomKey	Pointer to a valid geometry key.
fReturnParameters	If true, locator values are returned in the form of parameter keys. Otherwise, they are returned as doubles.
fRetainKey	If true, the descriptor hangs onto a copy of the original geometry key.
appName	Creating application name.

Output

pGeomDescrip	Pointer to the new descriptor object.

AmiStatus amiGetFeatDepFeats

(AmiFeatKey* *pFeatureKey,* **int&** *size,* **AmiFeatKey**&** *pFeatureKeys,* **Adesk::UInt32 state = Ami::kAll)**

Returns all of the features dependent on a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey	Pointer to feature key.
state	Bit field allows control over the feature states allowable in the features returned by this function.

Output

size	Number of keys returned.
pFeatureKeys	Array of pointers to keys to all the features dependent on the given feature.

AmiStatus amiGetFeatDepWorkGeom

(AmiFeatKey* pFeatureKey, int& size, AmiGeomKey& pGeomKeys)**

Returns all of the work entities dependent on a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey	Pointer to feature key.

Output

size	Number of keys returned.
pGeomKeys	Array of pointers to keys to all the work entities dependent on the given feature.

AmiStatus amiGetFeatEdges

(AmiFeatKey* *pFeatureKey*, **int&** *size*, **AmiCurveKey**&** *pCurveKeys* **)**

Returns curve keys to all the edges associated with a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey Pointer to feature key.

Output

size Number of keys returned.

pCurveKeys Array of pointers to keys to all the edges associated with the feature.

AmiStatus amiGetFeatFaces

(AmiFeatKey* *pFeatureKey*, **int&** *size*, **AmiSurfKey**&** *pSurfKeys* **)**

Returns surface keys to all the faces associated with a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey Pointer to feature key.

Output

size Number of keys returned.

pSurfKeys Array of pointers to keys to all the faces associated with the feature.

AmiStatus amiGetFeatParams

(AmiFeatKey* *pFeatureKey*, **int&** *size*, **AmiParamKey**&** *pParamKeys* **)**

Returns parameter keys to all the parameters directly driving a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey Pointer to feature key.

Output

size Number of keys returned.

pParamKeys Array of pointers to keys to all parameters driving the feature.

AmiStatus amiGetFeatType

(AmiFeatKey* *pFeature,* **AmiFeatType&** *featTyp*e **)**

Gets the type of a feature.

Input

pFeature Pointer to a feature key.

Output

featType The feature type from the key.

AmiStatus amiGetFeatsFromGeom

(AmiGeomKey* *pGeomKey*, **int&** *size*, **AmiFeatKey**&** *pFeatureKeys,* **Adesk::UInt32** *state,*, **Adesk::Boolean** *fReturnComposite* **)**

Returns all the features that brought about the specified geometric object. The features are not returned in any specific order.

Features are returned in the same scope as the geometry key. For example, if the geometry key references the edge of a hole on a component, a component feature will be returned, defined in the space of that component. If the geometry key references geometry on the part, then part component keys will be returned.

NOTE: Any geom keys created with the *allowGhost* flag are not supported.

Input

pGeomKey Pointer to geometry key.

state Bit field allows control over the feature states allowable in the features returned by this function.

fReturnComposite If a feature bringing about this geometry is contained in a composite feature, a value of true will cause the composite to be returned. False will return the innermost feature.

Output

size Number of keys in resulting array.

pFeatureKeys Array of keys to all the features that constitute the specified geometric object.

AmiStatus amiGetFeatSketch

(AmiFeatKey* *pFeatureKey*, **int&** *size*, **AmiSketchKey**&** *pSketchKeys* **)**

Returns all the sketches that are associated with a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey Pointer to feature key.

Output

size Number of keys returned.

pSketchKeys Array of pointers to keys to all the sketches that are associated with the feature key.

AmiStatus amiGetFeatVertices

(AmiFeatKey* *pFeatureKey*, **int&** *size*, **AmiPointKey**&** *pPointKeys* **)**

Returns point keys to all the vertices associated with a given feature. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pFeatureKey Pointer to feature key.

Output

size Number of keys returned.

pPointKeys Array of pointers to keys to all the vertices associated with the feature.

AmiStatus amiGetFeatDescrip

(AmiFeatKey* *pFeature*, **AmiFeatDescrip*&** *pFeatDescrip*, **Adesk::Boolean** *fReturnInformers* **= Adesk::kFalse, Adesk::Boolean** *fReturnParameters* **= Adesk::kFalse)**

Creates a feature descriptor object from a feature key. It is the caller's responsibility to release the feature descriptor.

Input

pFeature Pointer to a feature key.

fReturnInformers If true, the full feature description is returned including all locators and terminators. Otherwise locators and terminators are not returned.

fReturnParameters If true, descriptor values are returned in the form of parameters keys. Otherwise, they are returned as doubles.

Output

pFeatDescrip Pointer to new feature descriptor object.

AmiStatus amiGetOwningCompositeFeat

(AmiFeatKey* *pFeatKey*, **AmiFeatKey*&** *pOwningComposite***)**

Returns a key to the owning composite feature for the given feature if the given feature is contained in a composite.

Input

PfeatKey Pointer to feature key.

Output

POwningComposite Pointer to a key to the owning composite feature or NULL if the feature is not contained in a composite.

AmiStatus amiIsCompToolbody

(AmiCompKey* *pCompKey*, **Adesk::Boolean&** *fIsToolbody* **)**

Returns a flag stating whether or not the given component is a toolbody.

Input

pCompKey A pointer to a component key.

fIsToolbody True if component is a toolbody.

AmiStatus amiIsFeatKindOf

(**AmiFeatKey*** *pFeature*, **AmiFeatType** *featType*, **Adesk::Boolean&** *isType*)

Checks if the specified feature either matches the type specified or is a kind of the type specified.

Input

pFeature Pointer to a feature key.

featType Feature type to check against.

Output

isType If true, feature matches or derives from the specified feature type.

AmiStatus amiReorderFeat

(AmiFeatKey* *pFeatKey*, **AmiFeatKey*** *pDestFeat* **)**

Reorders the given feature in the history tree.

Input

PFeatKey Feature to reorder.

PDestFeat Feature that the reordered feature will be placed immediately after in the tree.

AmiStatus amiSetCompositeFeatEditCallback

(const char* *pOwnerInfo*, **AmiCompFeatEditFcnPtr** *fcn*, **const char*** *appName***)**

Sets a callback function associated with an owner info string to be called when composite features created with the same owner info string are edited.

Note1: The owner info string is a null terminated string made up of three tokens separated by semicolons. The first token should be the owner app's company name. The second token is the name of the creating app. While the third (optional) token is a URL. This owner info string has to be the same string used to create the composite features for the owner's app in order for this to work.

Note2: The edit callback is not set on the document. It is global. That means for that for a single application, this call only needs to be made once per session.

Input

pOwnerInfo User supplied owner info string (see note above).

fcn Pointer to the function to be called when composites created with the same owner info string are selected for edit.

AmiStatus amiStartCompositeFeat

(AmiCompositeFeatDescrip* *pCompDescrip***)**

Starts composite feature creation. All features created from this point on are considered to be contained by this composite feature until such time that the composite feature creation is ended or aborted. This includes features created via the API, the MDT UI, and ADS.

Note: The first volume feature (base) on a part cannot be a composite. The part must have volume before the first composite is created.

Note: Only one composite feature per document can be open for create at a given time.

Input

pCompDescrip Pointer to a composite feature descriptor.

Feature Descriptors

AmiStatus amiCreateFeatDescrip

(AmiFeatType *featType*, **AmiFeatDescrip*&** *pFeatDescrip*, **char*** *appName***=Designer)**

Creates a new empty feature descriptor of the given type. It is the caller's responsibility to release the feature descriptor.

Input

featType Type of feature descriptor to create.

Output

pFeatDescrip Pointer to new feature descriptor object.

AmiStatus amiGetCombinerType

(AmiFeatDescrip* *pFeatDescrip*, **AmiCombinerType&** *combType* **)**

Gets the feature combiner type.

Input

pFeatDescrip Pointer to a feature descriptor.

Output

combType The combiner type.

AmiStatus amiGetDescripType

(AmiFeatDescrip* *pFeatDescrip*, **AmiFeatType&** *featType* **)**

Gets the type of a feature.

Input

pFeatDescrip Pointer to a feature descriptor.

Output

featType The feature type from the descriptor.

AmiStatus amiGetFeatData

(AmiFeatDescrip* *pFeatDescrip*, **AmiPartKey*&** *pPartKey* **)**

Gets the base data from a feature descriptor. It is the caller's responsibility to release the returned key.

Input

pFeatDescrip Pointer to a feature descriptor.

Output

pPartKey Part that the feature was created on.

AmiStatus amiIsDescripKindOf

(AmiFeatDescrip* *pFeatDescrip*, **AmiFeatType** *featType*, **Adesk::Boolean&** *isType* **)**

Checks if the feature descriptor refers to a feature that either matches the type specified or is a kind of the type specified.

Input

pFeatDescrip Pointer to a feature descriptor.

featType Feature type to check against.

Output

isType If true, feature matches or derives from the specified feature type.

AmiStatus amiSetCombinerType

(AmiFeatDescrip * *pFeatDescrip*, **AmiCombinerType** *combType* **)**

Sets the combiner type on a feature descriptor.

Input

pFeatDescrip Pointer to feature descriptor.

combType Combiner type.

AmiStatus amiSetFeatPart

(AmiFeatDescrip* *pFeatDescrip*, **AmiPartKey*** *pPartKey* **)**

Sets the part key on a feature descriptor. The given key is copied when this call is made.

Input

pFeatDescrip Pointer to a feature descriptor.

pPartKey Pointer to a part key.

Composite Feature Descriptors

AmiStatus amiGetCompositeFeatData

(AmiCompositeFeatDescrip* *pCompDescrip*, **int&** *nFeats*, **AmiFeatKey**&** *pFeats*, **int&** *nParms*, **AmiParamKey**&** *pParms*, **std::string&** *featType*, **std::string&** *ownerInfo* **)**

Gets the data from a composite feature descriptor. Note: The owner info string is a null terminated string made up of three tokens separated by semicolons. The first token should be the owner app's company name. The second token is the name of the creating app. While the third (optional) token is a URL. The owner info string is currently unused but reserved for future use.

Input

pCompDescrip Pointer to a composite feature descriptor.

Output

nFeats Number of feature keys in the key array.

pFeats Array of contained feature keys.
nParms Number of param keys in driving parameters array.
pParms Array of driving parameter keys.
featType The custom feature type.
ownerInfo The owner info string.

AmiStatus amiSetCompositeFeatDrivingParams

(AmiCompositeFeatDescrip * *pCompDescrip*, **int** *nParms*, **AmiParamKey**** *pParms* **)**

Sets the driving parameters on the given composite feature descriptor.

Input

pCompDescrip Pointer to a composite feature descriptor.
nParms Number of keys in driving parameters array.
pParms Array of driving parameters keys.

AmiStatus amiSetCompositeFeatOwnerInfo

(AmiCompositeFeatDescrip* *pCompDescrip*, **const char*** *pOwnerInfo* **)**

Sets the user supplied owner information string on the given composite feature descriptor. Note: The owner info string is a null terminated string made up of three tokens separated by semicolons. The first token should be the owner app's company name. The second token is the name of the creating app. While the third (optional) token is a URL. The owner info string is currently used in registration of the edit callback function.

Input

pCompDescrip Pointer to a composite feature descriptor.
pOwnerInfo The owner information string.

AmiStatus amiSetCompositeFeatType

(AmiCompositeFeatDescrip* *pCompDescrip*, **const char*** *pFeatType***)**

Sets the user supplied custom feature type on the given composite feature descriptor.

Input

pCompDescrip Pointer to a composite feature descriptor.
pFeatType The custom feature type.

Hole Descriptors

AmiStatus amiClearHoleTapData

(AmiHoleDescrip * *pHoleDescrip* **)**

Clears the hole tap data.

Input

pHoleDescrip Pointer to hole descriptor.

AmiStatus amiGetCBoreHoleData

(AmiCBoreHoleDescrip* *pHoleDescrip*, **AmiValue&** c*bDepth*, **AmiValue&** *cbDiam* **)**

Gets the data from a counter bored hole descriptor.

Input

pHoleDescrip Pointer to a counter bored hole descriptor.

Output

cbDepth The counter bore depth.

cbDiam The counter bore diameter.

AmiStatus amiGetCSinkHoleData

(AmiCSinkHoleDescrip* *pHoleDescrip*, **AmiValue&** *csAngle*, **AmiValue&** *csDiam* **)**

Gets the data from a counter sink hole descriptor.

Input

pHoleDescrip Pointer to a counter sink hole descriptor.

Output

csAngle The counter sink angle (degrees).

csDiam The counter sink diameter.

AmiStatus amiGetHoleData

(AmiHoleDescrip* *pHoleDescrip*, **AmiValue&** *diameter*, **AmiValue&** *drillAngle*, **AcGeVector3d&** *dirVec*, **char*&** *psNominalSize*, **AmiUnits&** *units* **)**

Gets the data from a hole descriptor.

Input

pHoleDescrip Pointer to a hole descriptor.

Output

diameter The diameter of the hole.

drillAngle Drill point angle (degrees).

dirVec Hole direction vector.

psNominalSize The nominal size.

units The units of the hole(Ami::kMetric, kEnglish, or kUnknownUnits).

AmiStatus amiGetHoleTapData

(AmiHoleDescrip* *pHoleDescrip*, **Adesk::Boolean&** *isTapped*, **Adesk::Boolean&** *isFull*, **AmiValue&** *majorDiam*, **AmiValue&** *tapDepth*, **AmiValue&** *pitchVal*, **char*&** *psPitchDesc*, **AmiValue&** *tapDrillDiam*, **char*&** *psFitClass***)**

Gets the tap data from a hole feature descriptor.

Input

pHoleDescrip Pointer to a hole feature descriptor.

Output

isTapped If true, the hole is tapped. If false, the hole is not tapped and the rest of the output parameters are meaningless.

isFull If true, the hole is fully tapped and the tap depth parameter is meaningless.

majorDiam The major diameter.

tapDepth The tap depth.

pitchVal The pitch. If units = = Ami::kMetric or kUnknownUnits, this is units per thread (e.g. mm/thread). If units = = Ami::kEnglish, this is threads per inch (tpi).

psPitchDesc The string representation of pitch.

TapDrillDiam Size of the tap drill.

psFitClass String used to specify fit class for the thread.

AmiStatus amiSetCBoreHoleCbDepth

(AmiCBoreHoleDescrip* *pHoleDescrip*, **const AmiValue&** *cbDepth* **)**

Sets the counter bore depth of a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
cbDepth	New counter bore depth value.

AmiStatus amiSetCBoreHoleCbDiam

(AmiCBoreHoleDescrip * *pHoleDescrip*, **const AmiValue&** *cbDiam* **)**

Sets the counter bore diameter of a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
cbDiam	New counter bore diameter value.

AmiStatus amiSetCSinkHoleCsAngle

(AmiCSinkHoleDescrip * *pHoleDescrip*, **const AmiValue&** *csAngle* **)**

Sets the counter sink angle of a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
csAngle	New counter sink angle value (degrees).

AmiStatus amiSetCSinkHoleCsDiam

(AmiCSinkHoleDescrip * *pHoleDescrip*, **const AmiValue&** *csDiam* **)**

Sets the counter sink diameter of a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
csDiam	New counter sink diameter value.

AmiStatus amiSetHoleDiameter

(AmiHoleDescrip * *pHoleDescrip*, **const AmiValue&** *diameter* **)**

Sets the diameter value on a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
diameter	New hole diameter.

AmiStatus amiSetHoleDirectionVec

(AmiHoleDescrip * *pHoleDescrip*, **const AcGeVector3d&** *dirVec* **)**

Sets the direction vector on a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
dirVec	New direction vector.

AmiStatus amiSetHoleDrillAngle

(AmiHoleDescrip * *pHoleDescrip*, **const AmiValue&** *drillAngle* **)**

Sets the drill point angle on a hole descriptor.

Input

pHoleDescrip	Pointer to hole descriptor to set.
drillAngle	New drill point angle value (degrees).

AmiStatus amiSetHoleFitClass

(AmiHoleDescrip* *pHoleDescrip*, **const char*** *psFitClass* **)**

Sets fit class on a feature descriptor for a tapped hole.

Input

PHoleDescrip	Pointer to a hole feature descriptor.

Output

PsFitClass	String used to specify fit class for the thread.

AmiStatus amiSetHoleNominalSize

(AmiHoleDescrip* *pHoleDescrip*, **const char*** *psNominalSize* **)**

Sets nominal size on a hole feature descriptor.

Input

PHoleDescrip	Pointer to a hole feature descriptor.
PsNominalSize	The nominal size.

AmiStatus amiSetHolePitch

(AmiHoleDescrip* *pHoleDescrip*, **const AmiValue&** *pitchVal*, **const char*** *psPitchDesc* **)**

Sets pitch on a feature descriptor for a tapped hole.

Input

PHoleDescrip	Pointer to a hole feature descriptor.

Output

Pitch	The pitch. If units == Ami::kMetric or kUnknownUnits, this is units per thread (e.g. mm/thread). If units == Ami::kEnglish, this is threads per inch (tpi).
PsDescription	The string representation of pitch.

AmiStatus amiSetHoleTapData

(AmiHoleDescrip* *pHoleDescrip*, **Adesk::Boolean** *isFull*, **const AmiValue&** *majorDiam*, **const AmiValue&** *tapDepth* **)**

Sets the hole tap data.

Input

pHoleDescrip	Pointer to hole descriptor
isFull	If true, the hole is fully tapped and the tap depth parameter is meaningless.
majorDiam	The major diameter.
tapDepth	The tap depth.

AmiStatus amiSetHoleTapDrillDiameter

(AmiHoleDescrip *pHoleDescrip, const AmiValue& tapDrillDiam)

Sets tap drill diameter on a feature descriptor for a tapped hole.

Input

PHoleDescrip	Pointer to a hole feature descriptor.

Output

TapDrillDiam Size of the tap drill.

AmiStatus amiSetHoleUnits

(AmiHoleDescrip *pHoleDescrip, AmiUnits units)

Sets the units on a hole feature descriptor.

Input

PHoleDescrip Pointer to a hole feature descriptor.

Units The units of the hole(Ami::kMetric, kEnglish, or kUnknownUnits).

Base Feature (Solid) Descriptors

AmiStatus amiGetBaseData

(AmiBaseDescrip* *pBaseDescrip*, **AcBrBrep*&** *pBrep* **)**

Gets the data from a base feature descriptor. It is the caller's responsibility to delete the returned entity.

Note The B-rep entity returned from this function corresponds to the stock solid used to create this base feature which is not a database entity. Therefore, although the B-rep API can be used to traverse and inspect the solid object, the MCAD API function **amiGetKeyFromBrep** will not work on any of the B-rep elements obtained during traversal.

Input

pBaseDescrip Pointer to a base feature descriptor.

Output

pBrep Pointer to a B-rep API B-rep entity.

AmiStatus amiSetBaseData

(AmiBaseDescrip * *pBaseDescrip*, **AcBrBrep*** *pBrep* **)**

Sets the data on a base feature descriptor. The entity is copied when this call is made.

Input

pBaseDescrip Pointer to a base feature descriptor.

pBrep Pointer to a B-rep API B-rep entity.

Extrusion Descriptors

AmiStatus amiGetExtrusionData

(AmiExtrusionDescrip* *pExtDescrip*, **AcGeVector3d&** *dirVec*, **AmiValue&** *draftAngle* **)**

Gets the data from an extrusion descriptor.

Input

pExtDescrip Pointer to an extrusion descriptor.

Output

dirVec Direction vector for extrusion.

draftAngle Extrusion angle (degrees).

AmiStatus amiGetThinExtrusionData

(AmiThinExtrusionDescrip* *pExtDescrip*, **AmiThicknessType&** *thickType*,**AmiValue&** *thickness1*, **AmiValue&** *thickness2*, **Adesk::Boolean&** *fExtend* **)**

Gets the data from a thin extrusion descriptor.

Input

pExtDescrip Pointer to a thin extrusion descriptor.

Output

thickType Thickness type.

thickness1 Thin thickness.

thickness2 2nd thickness if type is two directions.

fExtend Extend is on (auto to-next termination).

AmiStatus amiSetExtrusionDirectionVec

(AmiExtrusionDescrip* *pExtDescrip*, **const AcGeVector3d&** *dirVec* **)**

Sets the direction vector on an extrusion descriptor.

Input

pExtDescrip Pointer to extrusion descriptor to set.

dirVec New direction vector.

AmiStatus amiSetExtrusionDraftAngle

(AmiExtrusionDescrip* *pExtDescrip*, **const AmiValue&** *draftAngle* **)**

Sets the draft angle on an extrusion descriptor.

Input

pExtDescrip Pointer to extrusion descriptor to set.

draftAngle New draft angle.

AmiStatus amiSetExtrusionProfile

(AmiExtrusionDescrip* *pExtDescrip*, **AmiSketchKey*** *pSketchKey* **)**

Sets the profile to extrude on an extrusion descriptor. The sketch key is copied when this call is made.

Input

pExtDescrip Pointer to extrusion descriptor to set.

pSketchKey Pointer to a sketch key.

AmiStatus amiSetThinExtrusionExtend

(AmiThinExtrusionDescrip* *pExtDescrip*,**Adesk::Boolean** *fExtend***)**

Sets the extend flag on a thin extrusion descriptor.

Input

pExtDescrip Pointer to thin extrusion descriptor to set.

fExtend Extend is on (auto to-next termination).

AmiStatus amiSetThinExtrusionThickness

(AmiThinExtrusionDescrip* *pExtDescrip*,**AmiThicknessType** *thickType*,**const AmiValue&** *thickness1*,**const AmiValue&** *thickness2***)**

Sets the thin thickness on a thin extrusion descriptor.

Input

pExtDescrip Pointer to thin extrusion descriptor to set.

thickType Thickness type.

thickness1 Thin thickness.

thickness2 2nd thickness if type is two directions.

Revolution Descriptors

AmiStatus amiGetRevolveData

(AmiRevolveDescrip* *pRevDescrip*, **AcGeVector3d&** *dirVec*, **AmiLineKey*&** *pLineKey* **)**

Gets the data from a revolve descriptor. It is the caller's responsibility to release the key returned.

Input

pRevDescrip Pointer to a revolve descriptor.

Output

dirVec Revolve direction vector.

pLineKey Axis of revolution.

AmiStatus amiSetRevolveAxis

(AmiRevolveDescrip* *pRevDescrip*, **AmiLineKey*** *pLineKey* **)**

Sets the axis of revolution on a revolve descriptor. The line key is copied when this call is made.

Input

pRevDescrip Pointer to revolve descriptor to set.

pLineKey Pointer to axis of revolution key.

AmiStatus amiSetRevolveDirectionVec

(AmiRevolveDescrip* *pRevDescrip*, **const AcGeVector3d&** *dirVec* **)**

Sets the direction vector on a revolve descriptor.

Input

pRevDescrip Pointer to revolve descriptor to set.

dirVec New direction vector.

AmiStatus amiSetRevolveProfile

(AmiRevolveDescrip* *pRevDescrip*, **AmiSketchKey*** *pSketchKey* **)**

Sets the profile to revolve on a revolve descriptor. The sketch key is copied when this call is made.

Input

pRevDescrip Pointer to revolve descriptor to set.

pSketchKey Pointer to a sketch key.

Chamfer Descriptors

AmiStatus amiGetChamferData

(AmiChamferDescrip* *pChamfDescrip*, **AmiChamferType&** *chamfType*, **AmiValue&** *angle*, **AmiValue&** *distance1*, **AmiValue&** *distance2* **)**

Gets the data from a chamfer descriptor.

Input

pChamfDescrip	Pointer to a chamfer descriptor.

Output

chamfType	The chamfer type.
angle	Angle, if the chamfer is a right or left angle chamfer (degrees).
distance1	Distance from first edge.
distance2	Distance from second edge if the chamfer is a uniform or differential chamfer.

AmiStatus amiSetChamferAngle

(AmiChamferDescrip* *pChamfDescrip,* **const AmiValue&** *angle* **)**

Sets the angle value on a chamfer descriptor.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
angle	New angle value (degrees).

AmiStatus amiSetChamferAngleFace

(AmiChamferDescrip* *pChamfDescrip***, AmiPlaneKey*** *pFaceKey* **)**

Sets the face to which to apply the chamfer angle. The input key is copied when this call is made.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
pFaceKey	Planar face key.

AmiStatus amiSetChamferDist1

(AmiChamferDescrip* *pChamfDescrip,* **const AmiValue&** *distance1* **)**

Sets the first distance value on a chamfer descriptor.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
distance1	New first distance value.

AmiStatus amiSetChamferDist1Face

(AmiChamferDescrip* *pChamfDescrip***, AmiPlaneKey*** *pFaceKey***)**

Sets the face to which to apply the first distance in a two distance chamfer. The input key is copied when this call is made.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
pFaceKey	Planar face key.

AmiStatus amiSetChamferDist2

(AmiChamferDescrip* *pChamfDescrip,* **const AmiValue&** *distance2* **)**

Sets the second distance value on a chamfer descriptor.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
distance2	New second distance value.

AmiStatus amiSetChamferEdges

(AmiChamferDescrip* *pChamfDescrip*, **int** *nEdges*, **const AmiCurveKey**** *pEdges* **)**

Sets the edges on a chamfer descriptor. The array and the keys contained within are copied when this call is made.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
nEdges	Number of edges to set.
pEdges	Array of keys to the edges.

AmiStatus amiSetChamferType

(AmiChamferDescrip* *pChamfDescrip*, **AmiChamferType**, *chamfType* **)**

Sets the chamfer type on a chamfer descriptor.

Input

pChamfDescrip	Pointer to a chamfer descriptor to set.
*chamfTyp*e	New chamfer type value.

Fillet Descriptors

AmiStatus amiGetCubicFilletData

(AmiCubicFilletDescrip* *pFilletDescrip*, **int&** *nRadii*, **AmiValue*&** *pRadii*, **double*&** *pParam* **)**

Gets the data from a cubic variation fillet descriptor. It is the caller's responsibility to delete the returned array.

Input

pFilletDescrip	Pointer to a fillet descriptor.

Output

nRadii	Number of radius values (one for each vertex).
pRadii	Array of radius values.
pParam	Radius placement parameter values.

AmiStatus amiGetFixedFilletData

(AmiFixedFilletDescrip* *pFilletDescrip*, **AmiValue&** *chordLength* **)**

Gets the data from a fixed width fillet descriptor.

Input

pFilletDescrip	Pointer to a fillet descriptor.

Output

chordLength	Fixed width fillet chord length.

AmiStatus amiGetIndFilletData

(AmiIndFilletDescrip* *pFilletDescrip*, **int&** *nRadii*, **AmiValue*&** *pRadii* **)**

Gets the data from an individual radius fillet descriptor. It is the caller's responsibility to delete the array.

Input

pFilletDescrip	Pointer to a fillet descriptor.

Output

nRadii	Number of radius values (one for each edge).

pRadii Array of radius values.

AmiStatus amiGetLinearFilletData

(AmiLinearFilletDescrip* *pFilletDescrip*, **int&** *nRadii*, **AmiValue*&** *pRadii* **)**

Gets the data from a linear variation fillet descriptor. It is the caller's responsibility to delete the array of radius values.

Input

pFilletDescrip Pointer to a fillet descriptor.

Output

nRadii Number of radius values (one for each vertex).

pRadii Array of radius values (one for each vertex).

AmiStatus amiGetUniFilletData

(AmiUniFilletDescrip* *pFilletDescrip*, **AmiValue&** *radius* **)**

Gets the data from a uniform fillet descriptor.

Input

pFilletDescrip Pointer to a fillet descriptor.

Output

radius Fillet radius.

AmiStatus amiSetCubicFilletRadii

(AmiCubicFilletDescrip* *pFilletDescrip*, **int** *nRadii*, **const AmiValue*** *pRadii*, **const** double* *pParam* **)**

Sets the radius values on a cubic fillet descriptor. The arrays are copied when this call is made.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

nRadii Number of radii.

pRadii New radius value array.

pParam Parameter array for radius placement.

AmiStatus amiSetFilletEdges

(AmiFilletDescrip* *pFilletDescrip*, **int** *nEdges*, **const AmiCurveKey**** *pEdges* **)**

Sets the edges on a fillet descriptor. The array and the keys contained within are copied when this call is made.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

nEdges Number of edges to set.

pEdges Array of keys to the edges.

AmiStatus amiSetFixedFilletChordLength

(AmiFixedFilletDescrip* *pFilletDescrip*, **const AmiValue&** *chordLength* **)**

Sets the chord length on a fixed width fillet descriptor.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

chordLength New chord length value.

AmiStatus amiSetIndFilletRadii

(AmiIndFilletDescrip* *pFilletDescrip*, **int** *nRadii*, **const AmiValue*** *pRadii* **)**

Sets the radius values on an individual radius fillet descriptor. The radius array is copied when this call is made.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

nRadii Number of radii.

pRadii New radius value array.

AmiStatus amiSetLinearFilletRadii

(AmiLinearFilletDescrip* *pFilletDescrip*, **int** *nRadii*, **const AmiValue*** *pRadii* **)**

Sets the radius values on a linear fillet descriptor. The array is copied when this call is made.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

nRadii Number of radii.

pRadii New radius value array.

AmiStatus amiSetUniFilletRadius

(AmiUniFilletDescrip *pFilletDescrip*, **const AmiValue&** *radius* **)**

Sets the radius on a uniform fillet descriptor.

Input

pFilletDescrip Pointer to a fillet descriptor to set.

radius New radius value.

Sweep Descriptors

AmiStatus amiGetSweepData

(AmiSweepDescrip* *pSweepDescrip*, **AmiSweepMethod&** *method*, **AmiValue&** *draftAngle* **)**

Gets the data from a sweep descriptor.

Input

pSweepDescrip Pointer to a sweep descriptor.

Output

method Sweep method.

draftAngle Sweep draft angle.

AmiStatus amiSetSweepDraftAngle

(AmiSweepDescrip* *pSweepDescrip*, **const AmiValue&** *draftAngle* **)**

Sets the draft angle on a sweep descriptor.

Input

pSweepDescrip Pointer to sweep descriptor to set.

draftAngle New draft angle.

AmiStatus amiSetSweepMethod

(AmiSweepDescrip* *pSweepDescrip*, **AmiSweepMethod** *method* **)**

Sets the sweep method on a sweep descriptor.

Input

pSweepDescrip	Pointer to sweep descriptor to set.
method	New sweep method.

AmiStatus amiSetSweepPath

(AmiSweepDescrip* *pSweepDescrip*, **AmiSketchKey*** *pSketchKey* **)**

Sets the path to sweep along on a sweep descriptor.

Input

pSweepDescrip	Pointer to sweep descriptor to set.
pSketchKey	Sketch key referring to path.

AmiStatus amiSetSweepProfile

(AmiSweepDescrip* *pSweepDescrip*, **AmiSketchKey*** *pSketchKey* **)**

Sets the profile to sweep on a sweep descriptor. The input key is copied when this call is made.

Input

pSweepDescrip	Pointer to sweep descriptor to set.
pSketchKey	Sketch key referring to profile.

Feature Array Descriptors

AmiStatus amiGetArrayData

(AmiArrayDescrip* *pArrDescrip*, **int&** *nFeatKeys*, **AmiFeatKey**&** *pFeatKeys* **)**

Gets the data from an array descriptor. It is the caller's responsibility to release all the keys in the array and delete the array.

Input

pArrDescrip Pointer to an array descriptor.

Output

nFeatKeys	Number of feature keys being returned.
pFeatKeys	Array of feature keys to the features that have been arrayed.

AmiStatus amiGetPolarArrayData

(AmiPolarArrayDescrip* *pArrDescrip*, **AcGeVector3d&** *revAxis*, **AmiPolarAngleType&** *angleType*, **AmiValue&** *angle*, **AmiValue&** *nInstances*, **Adesk::Boolean&** *fRotateAsCopied*, **AmiPointKey*&** *pPointKey* **)**

Gets the data from a polar array descriptor. It is the caller's responsibility to release the returned point key.

Input

pArrDescrip	Pointer to a polar array descriptor.

Output

revAxis	Axis of revolution (with direction).
angleType	Polar angle type. *kfillAngle:* Where the number of instances are placed within the given angle.

kfullAngle: Where the number of instances are run 360 degrees.

kincrAngle: Where the angle specified is the angle between each instance.

angle The array angle (degrees).

nInstances Number of instances.

fRotateAsCopied Rotate feature as copied flag.

pPointKey Rotate reference point.

AmiStatus amiGetRecArrayData

(AmiRecArrayDescrip* *pArrDescrip*, **AcGeVector3d&** *xVec*, **AcGeVector3d&** *yVec*, **AmiValue&** *nRows*, **AmiValue&** *rowSpacing*, **AmiValue&** *nCols*, **AmiValue&** *colSpacing* **)**

Gets the data from a rectangular array descriptor.

Input

pArrDescrip Pointer to a rectangular array descriptor.

Output

xVec Vector describing X direction of array.

yVec Vector describing Y direction of array.

nRows Number of rows.

rowSpacing The row spacing value.

nCols Number of columns.

colSpacing The column spacing value.

AmiStatus amiSetArrayFeature

(AmiArrayDescrip* *pArrDescrip*, **AmiFeatKey*** *pFeatKey* **)**

Sets the feature to be arrayed on an array descriptor. The input key is copied when this call is made.

Input

pArrDescrip Pointer to array descriptor to set.

pFeatKey Pointer to feature key to be used as base.

AmiStatus amiSetPolarArrayAngle

(AmiPolarArrayDescrip* *pArrDescrip*, **const AmiValue&** *angle* **)**

Sets the angle of revolution on a polar array descriptor.

Input

pArrDescrip Pointer to array descriptor to set.

angle The array angle.

AmiStatus amiSetPolarArrayNumInstances

(AmiPolarArrayDescrip* *pArrDescrip*, **const AmiValue&** *nInstances* **)**

Sets the number of instances on a polar array descriptor.

Input

pArrDescrip Pointer to array descriptor to set.

nInstances Number of instances.

AmiStatus amiSetPolarArrayRotateAsCopied

(AmiPolarArrayDescrip* *pArrDescrip*, **Adesk::Boolean** *fRotateAsCopied*, **AmiPointKey*** *pPointKey* **= NULL**)

Sets the rotate as copied flag and, optionally, the rotate reference point on a polar array descriptor. The input key is copied when this call is made.

Input

pArrDescrip	Pointer to array descriptor to set.
fRotateAsCopied	Rotate feature as copied flag.
pPointKey	Rotate as copied reference work point. This point must be specified if *fRotateAsCopied* is false.

AmiStatus amiSetPolarArrayPolarAngleType

(AmiPolarArrayDescrip* *pArrDescrip*, **AmiPolarAngleType** *angleType* **)**

Sets the angle type on a polar array descriptor.

Input

pArrDescrip Pointer to array descriptor to set.

angleType	Polar angle type.
kFillAngle	Where the number of instances are placed within the given angle.
kFullAngle	Where the number of instances are run 360 degrees.
kIncrAngle	Where the angle specified is the angle between each instance.

AmiStatus amiSetRecArrayColData

(AmiRecArrayDescrip* *pArrDescrip*, **const AmiValue&** *nCols*, **const AmiValue&** *colSpacing* **)**

Sets the column data on a rectangular array descriptor.

Input

pArrDescrip	Pointer to array descriptor to set.
nCols	Number of columns in array.
colSpacing	Column spacing.

AmiStatus amiSetRecArrayRowData

(AmiRecArrayDescrip* *pArrDescrip*, **const AmiValue&** *nRows*, **const AmiValue&** *rowSpacing***)**

Sets the row data on a rectangular array descriptor.

Input

pArrDescrip	Pointer to array descriptor to set.
nRows	Number of rows in array.
rowSpacing	Row spacing.

AmiStatus amiSetRecArrayXVector

(AmiRecArrayDescrip* *pArrDescrip*, **const AcGeVector3d&** *xVec* **)**

Sets the X vector on a rectangular array descriptor.

Input

pArrDescrip	Pointer to array descriptor to set.
xVec	X vector.

AmiStatus amiSetRecArrayYVector

(AmiRecArrayDescrip* *pArrDescrip*, **const AcGeVector3d&** *yVec* **)**

Sets the Y vector on a rectangular array descriptor.

Input

pArrDescrip	Pointer to array descriptor to set.
yVec	Y vector.

Surf Cut Descriptors

AmiStatus amiGetSurfCutData

(AmiSurfCutDescrip* *pSurfCutDescrip*, **AmiSurfKey*&** *pSurfKey*, **AcGeVector3d&** *dirVec* **)**

Gets the data from a surf cut descriptor. It is the caller's responsibility to release the key returned.

Input

pSurfCutDescrip	Pointer to a surf cut descriptor.

Output

pSurfKey	Key to surface used for surf cut.
dirVec	Surf cut direction vector.

AmiStatus amiSetSurfCutDirectionVec

(AmiSurfCutDescrip* *pSurfCutDescrip*, **const AcGeVector3d&** *dirVec* **)**

Sets the direction vector on a surf cut.

Input

pSurfCutDescrip	Pointer to surf cut descriptor.
dirVec	Direction vector.

AmiStatus amiSetSurfCutSurf

(AmiSurfCutDescrip * *pSurfCutDescrip*, **AmiSurfKey*** *pSurfKey* **)**

Sets the surface key on a surf cut descriptor. The input key is copied when this call is made.

Input

pSurfCutDescrip	Pointer to surf cut descriptor.
pSurfKey	Pointer to a surface key. This key must be an AutoSurf surface.

Shell Descriptors

AmiStatus amiGetShellData

(AmiShellDescrip* *pShellDescrip*, **int&** *nOverrides*, **AmiSurfKey**&** *pOverrides*, **AmiValue*&** *pThicknesses*, **int&** *nExcludes*, **AmiSurfkey**&** *pExcludes* **)**

Gets the data from a shell descriptor. It is the caller's responsibility to release all keys returned and delete all arrays.

Input

pShellDescrip	Pointer to a shell descriptor.

Output

nOverrides	Number of thickness overrides.
pOverrides	Keys to faces to thickness overrides.
pThicknesses	Thickness values for each face.

nExcludes Number of excluded faces.

pExcludes Keys to excluded faces.

AmiStatus amiSetShellExcludes

(AmiShellDescrip ****pShellDescrip*, int** *nExcludes***, AmiSurfKey**** *pExcludes* **)**

Sets the excluded faces on a shell descriptor. The array and keys contained within are copied when this call is made.

Input

pShellDescrip Pointer to shell descriptor to set.

nExcludes Number of excluded faces.

pExcludes Array of keys to excluded faces.

AmiStatus amiSetShellOverrides

(AmiShellDescrip* *pShellDescrip***, int** *nOverrides***, AmiSurfKey**** *pOverrides***, AmiValue*** *pThicknesses* **)**

Sets the thickness overrides on a shell descriptor. The array and keys contained within are copied when this call is made.

Input

pShellDescrip Pointer to shell descriptor to set.

nOverrides Number of overrides.

pOverrides Array of keys to faces to override.

pThicknesses The thickness for each override.

Parametric Boolean Descriptors

AmiStatus amiGetParametricBoolData

(AmiParametricBoolDescrip* *pParmDescrip***, AmiCompKey*&** *pCompKey* **)**

Gets the data from a parametric boolean descriptor. It is the caller's responsibility to release the returned key.

Input

pParmDescrip Pointer to a parametric boolean descriptor.

Output

pCompKey Component key to tool body.

AmiStatus amiSetParametricBoolComp

(AmiParametricBoolDescrip* *pParmDescrip***, AmiCompKey*** *pCompKey* **)**

Sets the component key on a parametric boolean descriptor. The input key is copied when this call is made.

Input

pParmDescrip Pointer to parametric boolean descriptor.

pCompKey Pointer to component key.

Constrained Geometry Descriptors

AmiStatus amiCreateConstrGeom

(**AmiConstrGeomDescrip*** *pGeomDescrip*, **AmiGeomKey*&** *pGeomKey*, **Adesk::Boolean** *fGeomKey* **= Adesk::kFalse**)

Creates a work entity on the active part using the constrained geometry descriptor. It is the caller's responsibility to release the returned key.

Input

pGeomDescrip	Constrained geometry descriptor used to create the work entity.
FGeomKey	Set true if a key to the new work entity is to be returned.
pGeomKey	Output geometry key if *fGeomKey* is true.

AmiStatus amiGetConstrGeomData

(AmiConstrGeomDescrip* *pGeomDescrip,* **AmiGeomKey*&** *pGeomKey* **)**

Gets the data from a constrained geometry descriptor. It is the caller's responsibility to release the returned key.

Input

pGeomDescrip	Pointer to a constrained geometry descriptor.

Output

pGeomKey	Key referring to the constrained geometry. If the descriptor was created with *fRetainKey* equal to *kFalse*, this will always be NULL.

AmiStatus amiGetWorkPlaneData

(AmiWorkPlaneDescrip* *pWPDescrip,* **AmiVectorKey*&** *pVecKey,* **AmiWorkPlaneAxisType&** *rAxisType,* **AmiCoordinateSystem&** *rCoordSys* **)**

Gets the data from a work plane descriptor. It is the caller's responsibility to release the returned key. The work plane data comes back in the form of locators. If there is a sketch plane on the work plane, the output parameters to this function are used to describe the sketch plane orientation. The vector key is the vector used to orient the sketch plane while the axis type determines which axis in the sketch plane's coordinate system the vector represents. The rCoordSys parameter describes whether the sketch plane's coordinate system is left or right handed. The rCoordSys value cannot be set by users. It only gets flipped if the part that the work plane is created on gets mirrored.

Input

*pWPDescrip*Pointer to a work plane descriptor.

Output

pVecKey	Axis used to orient the sketch plane. If no sketch plane was created, this key will be NULL.
rAxisType	UCS axis that the vector is used for.
rCoordSys	UCS coordinate system.

Loft Descriptors

AmiStatus amiGetLoftData

(AmiLoftDescrip* *pLoftDescrip***, AmiLoftType&** *loftType***, Adesk::Boolean&** *fMinimizeTwist***, AmiValue&** *rStartWeight***, AmiValue&** *rEndWeight***, Adesk::Boolean&** *fTangentToStart***, AmiValue&** *rStartAngle,* **Adesk::Boolean&** *fTangentToEnd***, AmiValue&** *rEndAngle***, int&** *nXSections***, AmiXSection*&** *pXSections***)**

Gets the data from a loft descriptor. It is the caller's responsibility to delete the array of cross sections.

Input

pLoftDescrip	Pointer to a loft descriptor.

Output

loftType	Loft type.
fMinimizeTwist	Twist minimization flag
alignment	Loft alignment type.
rStartWeight	Weight at starting cross section.
rEndWeight	Weight at ending cross section.
fTangentToStart	If true, loft start angle is tangent to face.
rStartAngle	Start angle if *fTangentToStart* is false.
fTangentToEnd	If true, loft end angle is tangent to face.
rEndAngle	End angle if fTangentToEnd is false.
nXSections	Number of cross sections.
pXSections	Array of cross sections. Cross section can be a closed profile (sketch key), a planar face (plane key), or a work point (point key). Point keys can only be used on the starting and/or ending cross section.

AmiStatus amiSetLoftAngles

(AmiLoftDescrip* *pLoftDescrip*, **Adesk::Boolean** *fTangentToStart*, **const AmiValue&** *rStartAngle*, **Adesk::Boolean** *fTangentToEnd*, **const AmiValue&** *rEndAngle* **)**

Sets the angles at the starting and ending cross sections of a loft.

Input

pLoftDescrip	Pointer to a loft descriptor.
fTangentToStart	If true, loft start angle is tangent to face.
rStartAngle	Start angle if *fTangentToStart* is false.
fTangentToEnd	If true, loft end angle is tangent to face.
rEndAngle	End angle if *fTangentToEnd* is false.

AmiStatus amiSetLoftMinimizeTwist

(AmiLoftDescrip* *pLoftDescrip*, **Adesk::Boolean** *fMinimize***)**

Sets the twist minimization flag on a loft descriptor.

Input

pLoftDescrip	Pointer to a loft descriptor.
fMinimize	Twist minimization flag.

AmiStatus amiSetLoftType

(AmiLoftDescrip* *pLoftDescrip*, **AmiLoftType** *loftType***)**

Sets the type of loft.

Input

pLoftDescrip	Pointer to a loft descriptor.
loftType	Loft type.

AmiStatus amiSetLoftWeights

(AmiLoftDescrip* *pLoftDescrip*, **const AmiValue&** *rStartWeight*, **const AmiValue&** *rEndWeight*)

Sets the weights at the starting and ending cross sections of a loft.

Input

pLoftDescrip	Pointer to a loft descriptor.
rStartWeight	Weight at starting cross section.
rEndWeight	Weight at ending cross section.

AmiStatus amiSetLoftXSections

(AmiLoftDescrip* *pLoftDescrip*, **int** *nXSections*, **AmiXSection*** *pXSections*)

Sets the cross sections on a loft. The cross sections array is copied when this call is made.

Input

pLoftDescrip	Pointer to a loft descriptor.
nXSections	Number of cross sections.
pXSections	Array of cross sections. Cross section can be a closed profile (sketch key), a planar face (plane key), or a work point (point key). Point keys can only be used on the starting and/or ending cross section.

Face Draft Descriptors

AmiStatus amiGetDraftData

(**AmiDraftDescrip*** *pDraftDescrip*, **AmiPlaneKey*** & *pDraftPlane*, **Adesk::Boolean&** *fFlipNormal*, **AmiValue&** *rAngle*, **Adesk::Boolean&** *fIncludeTangents*)

Gets the data from a draft descriptor. It is the caller's responsibility to release the returned key.

Input

pDraftDescrip	Pointer to a draft descriptor.

Output

pDraftPlane	Key to plane or planar face from which to measure the draft angle.
fFlipNormal	If true, the angle is measure opposite of the draft plane normal.
rAngle	Draft angle.
fIncludeTangents	If true, include tangent faces/edges when computing draft.

AmiStatus amiGetEdgeDraftData

(**AmiEdgeDraftDescrip*** *pDraftDescrip*, int& **nFaces**, **AmiEdgeDraftFace*&** *pFaces*)

Gets the data from an edge draft descriptor. It is the caller's responsibility to delete the array of faces.

Input

pDraftDescrip	Pointer to an edge draft descriptor.

Output

nFaces	Number of faces that were drafted.
pFaces	Array of draft face objects.

AmiStatus amiGetPlaneDraftData

(AmiPlaneDraftDescrip* *pDraftDescrip*, **int&** *nFaces*, **AmiSurfKey**&** *pFaces*)

Gets the data from a plane draft descriptor. It is the caller's responsibility to delete the array of keys.

Input

pDraftDescrip Pointer to a plane draft descriptor.

Output

nFaces Number of faces that were drafted.

pFaces Array of keys to drafted faces.

AmiStatus amiGetShadowDraftData

(AmiPlaneDraftDescrip* *pDraftDescrip***, int&** *nFaces***, AmiSurfKey**&** *pFaces***)**

Gets the data from a shadow draft descriptor. It is the caller's responsibility to delete the array of keys.

Input

pShadowDescrip Pointer to a shadow draft descriptor.

Output

nFaces Number of faces that were drafted.

pFaces Array of keys to drafted faces.

AmiStatus amiSetDraftAngle

(AmiDraftDescrip* *pDraftDescrip*, **const AmiValue&** *rAngle***)**

Sets the plane or planar face to measure the angle from on a draft descriptor.

Input

pDraftDescrip Pointer to draft descriptor.

pDraftPlane Key to plane or planar face from which to measure the draft angle.

AmiStatus amiSetDraftTangents

(AmiDraftDescrip ****pDraftDescrip*, **Adesk::Boolean *fIncludeTangents***)**

Sets the include tangent faces/edges flag on a draft descriptor.

Input

pDraftDescrip Pointer to draft descriptor.

fIncludeTangents If true, include tangent faces/edges when computing draft.

AmiStatus amiSetDraftPlane

(AmiDraftDescrip ****pDraftDescrip*, **AmiPlaneKey* *pDraftPlane,* **Adesk::Boolean&,** *fFlipNormal***)**

Sets the plane or planar face from which to measure the angle on a draft descriptor. The input key is copied when this call is made.

Input

pDraftDescrip Pointer to draft descriptor.

pDraftPlane Key to plane or planar face from which to measure the draft angle.

fFlipNormal If true, the angle is measured opposite of the draft plane normal.

AmiStatus amiSetEdgeDraftFaces

(AmiEdgeDraftDescrip* *pDraftDescrip*, **int** *nFaces*, **AmiEdgeDraftFace*** *pFaces***)**

Sets the face info on an edge draft descriptor. The array and keys contained within are copied when this call is made.

Input

pDraftDescrip Pointer to an edge draft descriptor.

nFaces	Number of faces to draft.
pFaces	Array of draft face objects.

AmiStatus amiSetPlaneDraftFaces

(AmiPlaneDraftDescrip* *pDraftDescrip*, **int** *nFaces*, **AmiSurfKey**** *pFaces*)

Sets the faces to draft on a plane draft descriptor. The array and keys contained within are copied when this call is made.

Input

pDraftDescrip	Pointer to a plane draft descriptor.
nFaces	Number of faces to draft.
pFaces	Array of face keys.

AmiStatus amiSetShadowDraftFaces

(AmiShadowDraftDescrip* *pDraftDescrip*, **int** *nFaces*, **AmiSurfKey**** *pFaces*)

Sets the faces to draft on a shadow draft descriptor. The array and keys contained within are copied when this call is made.

Input

pDraftDescrip	Pointer to a shadow draft descriptor.
nFaces	Number of faces to draft.
pFaces	Array of face keys.

Split Descriptors

AmiStatus amiGetFaceSplitData

(AmiFaceSplitDescrip* *pSpltDescrip*, **Adesk::Boolean&** *fAllFaces*, **int&** *size*, **AmiSurfKey**&** *ppFaces*)

Gets the data from a face split feature descriptor. It is the caller's responsibility to release all keys in the output array.

Input

pSpltDescrip	Pointer to split descriptor.

Output

fAllFaces	True if all faces are candidates for split.
size	Size of array of keys to faces (0 if fAllFaces is kTrue).
ppFaces	Array of pointers to keys to faces (NULL if fAllFaces is kTrue).

AmiStatus amiGetPartSplitData

(AmiPartSplitDescrip* *pSpltDescrip*, **Adesk::Boolean&** *fFlipNormal*)

Gets the data from a part split feature descriptor.

Input

pSpltDescrip	Pointer to split descriptor.

Output

fFlipNormal	If false, the normal of the cutting plane was used to determine which half of the resulting split was converted into a new part. If true, the reverse of the normal was used.

AmiStatus amiSetFaceSplitFaces

(AmiFaceSplitDescrip* *pSpltDescrip*, **Adesk::Boolean** *fAllFaces* **= Adesk::kTrue, int** *size* **= 0, AmiSurfKey**** *ppFaces* **= NULL)**

Sets the list of faces to split on a face split descriptor. The array and keys contained within are copied when this call is made.

Input

pSpltDescrip	Pointer to split descriptor.
fAllFaces	If true, all faces will be candidates for the split and no more data will be required (or used).
size	Size of array of keys to faces (must be nonzero if fAllFaces is kFalse).
ppFaces	Array of pointers to keys to faces (must be non NULL if fAllFaces is kFalse).

AmiStatus amiSetPartSplitFlip

(AmiPartSplitDescrip* *pSpltDescrip*, **Adesk::Boolean** *fFlipNormal***)**

Sets the normal flip flag on a part split descriptor.

Input

pSpltDescrip	Pointer to split descriptor.
fFlipNormal	If false, the normal of the cutting plane is used to determine which half of the resulting split will be converted into a new part. If true, the reverse of the normal is used.

AmiStatus amiSetPartSplitName

(AmiPartSplitDescrip* *pSpltDescrip*, **const char*** *pPartName***)**

Sets the name of the part resulting from the split on a part split descriptor.

Input

pSpltDescrip	Pointer to split descriptor.
pPartName	Name of resulting part after split.

AmiStatus amiSetSplitCombinerPartName

(AmiFeatDescrip* *pFeatDescrip*, **const char*** *pPartName* **)**

Sets the part name for a split combiner on a feature descriptor. This function only applies if the combiner type is set to "kSplitPart".

Input

pFeatDescrip	Pointer to a feature descriptor.
pPartName	Name of resulting part after split.

Sketch Descriptors

AmiStatus amiCreateSketchDescrip

(**Ami::SketchType** *sketchType*, **AmiSketchDescrip*&** *pSketchDescrip*, **const char*** *appName* = **Designer)**

Creates an empty sketch descriptor.

Input

sketchType	Type of sketch to describe.

appName Name of application to create descriptor.

Output

pSketchDescrip Pointer to newly created descriptor.

AmiStatus amiCreateSketch

(AmiSketchDescrip *pDescrip*, **AmiSketchkey&** *pSketchKey* **)**

Creates a sketch based on descriptor information.

Input

pDescrip Pointer to a sketch descriptor.

Output

pSketchKey Key to newly created sketch.

AmiStatus amiGetData

(**AmiSketchDescrip*** *pSketchDescrip*, **Ami::SketchAttribute** *keyword*, **<Return Type>** *value*)

Extracts information from the given sketch descriptor. This function is overloaded for all the various data types for attributes

Input

pSketchDescrip Pointer to the sketch descriptor.

keyword Keyword for attribute.

Output

value The value for the specified attribute.

AmiStatus amiSetData

(**AmiSketchDescrip*** *pSketchDescrip*, **Ami::SketchAttribute** *keyword*, **<Return Type>** *value*)

Sets information on the given sketch descriptor. This function is overloaded for all the various data types for attributes.

Input

pSketchDescrip Pointer to the sketch descriptor.

keyword Keyword for attribute.

value The value of the specified attribute.

Table 5: Sketch Attribute Keywords Table

Attribute keyword	Datatype	Applies to	Description
kAttScopeKey	AmiObjectKey*	All	Scope to part where sketch is created.
kAttSketchPlane	AmiSketchPlaneKey*	ClosedProfile Path CutLine SplitLine Polyline Helical Spiral PolylinePath	Key to sketch plane to create the sketch on.
kInferredConstraints	Adesk::Boolean	ClosedProfile Path CutLine SplitLine	Flags whether or not constraints are to be inferred for 2D sketches.

Attribute keyword	Datatype	Applies to	Description
kAttSelectionList	AmiSelection*	ClosedProfile Path CutLine SplitLine Polyline PolylinePath	List of geometry to use to create the sketch.
kAttStartPoint	AcGePoint3d	AxisProfile PolylinePath EdgePath	Start point of work axis sketch
kAttEndPoint	AcGePoint3d	AxisProfile	End point of work axis sketch
kAttEdgeList	AmiCurveKey** Expects an array of curve keys	EdgePath	List of tangent continuous edges used to create edge path.
kAttStartVertex	AmiPointKey*	EdgePath	Start vertex for edge path.
kAttSpline	AmiCurveKey*	SplinePath	Spline curve to create path from.
kAttReverseFlag	Adesk::Boolean	SplinePath	If false, the start of the spline is the start of the path. If true, the path is reversed.
kAttRevolutions	AmiValue	SpiralPath HelicalPath	Number of revolutions on a spiral or helical path.
kAttPitch	AmiValue	SpiralPath HelicalPath	For a spiral path, the change in radius per revolution. For a helical, the change in height.
kAttStartAngle	AmiValue	SpiralPath	The angle from the x-axis of the sketch plane where the helix/spiral should start.
kAttMajorDiameter	AmiValue	SpiralPath HelicalPath	The major diameter of the first revolution.
kAttMinorDiameter	AmiValue	SpiralPath HelicalPath	The minor diameter of the first revolution.
kAttAxis	AmiLineKey*	SpiralPath HelicalPath	The work axis to revolve around.
kAttAxisFace	AmiSurfKey*	SpiralPath	A cylindrical face from which the work axis to revolve around will be created.
kAttClockwiseFlag	Adesk::Boolean	SpiralPath HelicalPath	If true, revolve clockwise. If false, counter clockwise.
kAttDefinitionType	Ami::DefinitionType	HelicalPath	Which values to use to create the helix.
kAttHeight	AmiValue	HelicalPath	The height of the helix.
kAttTaperAngle	AmiValue	HelicalPath	The angle by which the helix will taper along its height.
kAttFlipFlag	Adesk::Boolean	HelicalPath	If false, the helix rises along the sketch plane normal. If true, it moves along the reverse of the

Attribute keyword	Datatype	Applies to	Description
			normal.

Sketch Plane Descriptors

AmiStatus amiGetSketchPlaneDescrip

(AmiSketchPlaneKey* *pSkPlnKey,* **AmiSketchPlaneDescrip*&** *pDescrip,* **Adesk::Boolean** *fReturnInformers* **)**

Gets a descriptor from a sketch plane key.

Input

pSkPlnKey Pointer to a sketch plane key.

fReturnInformers If true, the full description is returned, including all locators. Otherwise, locators will not be available.

Output

pDescrip Pointer to a new descriptor object.

AmiStatus amiCreateSketchPlane

(AmiSketchPlaneDescrip* *pDescrip*, **AmiSketchPlaneKey*&** *pSkPlnKey*, **Adesk::Boolean** *fKey* **= Adesk::kFalse)**

Creates a sketch plane.

Input

pDescrip Pointer to a sketch plane descriptor.

fKey If true, creates and returns a sketch plane key.

Output

pSkPlnKey Pointer to the returned sketch plane key.

AmiStatus amiGetCurrentSketchPlane

(AmiSketchPlaneKey*& *pPlaneKey*, **char*** *appName*=**Designer)**

Returns a key to the current sketch plane.

Input

appName Name of the application that should create the key.

Output

pPlaneKey Pointer to plane key.

AmiStatus amiGetSketchPlaneFromSketch

(AmiSketchKey* *pSketchKey*, **AmiSketchPlaneKey*&** *pPlaneKey* **)**

Given a sketch, returns a key to the sketch's sketch plane.

Input

pSketchKey Pointer to sketch key.

Output

pPlaneKey Pointer to a plane key.

Array Descriptors

AmiStatus amiArrayHasIndependentInstances

(AmiArrayDescrip ****pArrDescrip*,Adesk::Boolean&** *hasIndep*)

Queries a feature array descriptor to see if the array feature has any independent instances.

Input

pArrDescrip Pointer to an array descriptor.

Output

hasIndep TRUE if the array has independent instances.

AmiStatus amiIsArrayInstanceIndependent

(AmiArrayDescrip ****pArrDescrip*, **int *row*,**int** *col*, **Adesk::Boolean&** *isIndep*)

Queries a feature array to see if the instance at a given index is independent.

Input

pArrDescrip Pointer to an array descriptor.

row For rectangular arrays, this is the zero based row index of the instance to be queried. For polar arrays, this is the linear zero based index of the instance to be queried.

col For rectangular arrays, this is the zero based column index of the instance to be queried. For polar arrays, this parameter is ignored.

Output

isIndep TRUE if the instance is independent

Bend Descriptors

AmiStatus amiGetBendData

(AmiBendDescrip ****pBendDescrip*, **AmiBendType *bendType*, **AmiValue&** *angle*, **AmiValue&** *radius*, **AmiValue&** *arcLen*, **AcGeVector3d&** *bendSide*, **AcGeVector3d&** *bendDir* **)**

Gets the data from a bend descriptor.

Input

pBendDescrip Pointer to a bend descriptor.

Output

bendType The bend type specifiez which combination of any two of the angle, radius or arc length values are used to make the bend.

angle The bend angle value.

radius The bend radius value.

arcLen The bend arc length value.

bendSide The bend side vector.

bendDir The bend direction vector.

AmiStatus amiSetBendAngle

(AmiBendDescrip ****pBendDescrip*, **const AmiValue& *angle* **)**

Sets the bend angle on a bend descriptor.

Input

pBendDescrip Pointer to a bend descriptor.

angle The bend angle value.

AmiStatus amiSetBendArcLen

(AmiBendDescrip* *pBendDescrip*, **const AmiValue&** *arcLen* **)**

Sets the bend arc length on a bend descriptor.

Input

pBendDescrip	Pointer to a bend descriptor.
arcLen	The bend arc length value.

AmiStatus amiSetBendDirection

(AmiBendDescrip* *pBendDescrip*, **const AcGeVector3d&** *bendDir* **)**

Sets the bend direction vector on a bend descriptor.

Input

pBendDescrip	Pointer to a bend descriptor.
bendDir	The bend direction vector.

AmiStatus amiSetBendProfile

(AmiBendDescrip* *pBendDescrip*, **AmiSketchKey *** *pSketchKey* **)**

Sets the profile to extrude on an bend descriptor. The sketch key is copied when this call is made.

Input

pBendDescrip	Pointer to bend descriptor to set.
pSketchKey	Pointer to a sketch key.

AmiStatus amiSetBendRadius

(AmiBendDescrip* *pBendDescrip*, **const AmiValue&** *radius* **)**

Sets the bend radius on a bend descriptor.

Input

pBendDescrip	Pointer to a bend descriptor.
radius	The bend radius value.

AmiStatus amiSetBendSide

(AmiBendDescrip* *pBendDescrip*, **const AcGeVector3d&** *bendSide* **)**

Sets the bend side vector on a bend descriptor.

Input

pBendDescrip	Pointer to a bend descriptor.
bendSide	The bend side vector.

AmiStatus amiSetBendType

(AmiBendDescrip* *pBendDescrip*, **AmiBendType** *bendType* **)**

Sets the bend type on a bend descriptor.

Input

pBendDescrip	Pointer to a bend descriptor.
bendType	The bend type specifiez which combination of any two of the angle, radius or arc length values are used to make the bend.

Rib Descriptors

AmiStatus amiGetRibData

(AmiRibDescrip* *pRibDescrip*, **AmiThicknessType&** *thickType*, **AmiValue&** *thickness1*, **AmiValue&** *thickness2*, **AcGeVector3d&** *ribDir* **)**

Gets the data from a rib descriptor.

Input

pRibDescrip Pointer to a rib descriptor.

Output

thickType Thickness type.

thickness1 Rib thickness.

thickness2 2nd thickness if type is two directions.

ribDir Rib direction vector.

AmiStatus amiSetRibDirection

(AmiRibDescrip* *pRibDescrip*, **const AcGeVector3d&** *ribDir* **)**

Sets the rib direction on a rib descriptor.

Input

pRibDescrip Pointer to rib descriptor to set.

ribDir Rib direction vector.

AmiStatus amiSetRibProfile

(AmiRibDescrip* *pRibDescrip*, **AmiSketchKey ***pSketchKey* **)**

Sets the profile to extrude on an rib descriptor. The sketch key is copied when this call is made.

Input

pRibDescrip Pointer to rib descriptor to set.

pSketchKey Pointer to a sketch key.

AmiStatus amiSetRibThickness

(AmiRibDescrip* *pRibDescrip*, **AmiThicknessType** *thickType*, **const AmiValue&** *thickness1*, **const AmiValue&** *thickness2* **)**

Sets the rib thickness on a rib descriptor.

Input

pRibDescrip Pointer to rib descriptor to set.

Output

thickType Thickness type.

thickness1 Rib thickness.

thickness2 2nd thickness if type is two directions.

Pattern Descriptors

AmiStatus amiGetPatternData

(AmiPatternDescrip* *pPatternDesc*, **AmiArray<AmiFeatKey>&** *featKeys*, **Adesk::UInt32&** *nSuppressed*, **AmiPatternInstance*&** *pSuppressedInsts* **)**

Gets the data from a pattern descriptor.

Input

PPatternDesc	Pointer to a pattern descriptor.

Output

FeatKeys	Array containing the list of features that were patterned.
NSuppressed	Number of suppressed pattern instances.
PSuppressedInsts	Array of indices representing suppressed pattern instances. It is the caller's responsibility to delete this array.

AmiStatus amiSetPatternFeatures

(AmiPatternDescrip* *pPatternDesc*, **const AmiArray<AmiFeatKey>&** *featKeys* **)**

Sets the features to be patterned on a pattern descriptor.

Input

PPatternDesc	Pointer to a pattern descriptor.
FeatKeys	Keys to the features to be patterned.

AmiStatus amiSetPatternSuppressedInstances

(AmiPatternDescrip* *pPatternDesc*, **Adesk::UInt32** *nSuppressed*, **const AmiPatternInstance*** *pSuppressed* **)**

Sets the instances to be suppressed on a pattern descriptor.

Input

pPatternDesc	Pointer to a pattern descriptor.
nSuppressed	Number of instances suppressed.
pSuppressed	Array of indices representing suppressed pattern instances.

Axial Pattern Descriptors

Axial patterns inherit from polar patterns—essentially, they are polar patterns with a few additional properties.

The locators are the same for regular polar patterns (specifying a rotational center).

In some cases the 'number of revolutions' needs to be set, otherwise 'angle' needs to be set. If nRevolutions is set then angle needn't be and vice versa.

AmiStatus amiGetAxialPatternData

(AmiAxialPatternDescrip* *pPatternDesc*, **AmiValue&** *nRevolutions*, **AmiPatternSpacing&** *offsetSpacingType*, **AmiValue&** *offsetHeight*, **AcGeVector3d&** *offsetDir* **)**

Gets the data from an axial pattern descriptor.

Input

pPatternDesc	Pointer to an axial pattern descriptor.

Output

nRevolutions	Number of revolutions.
offsetSpacingType	How the offset height value is used.
offsetHeight	Offset height value.
offsetDir	Offset direction.

AmiStatus amiSetAxialPatternNRevolutions

(AmiAxialPatternDescrip* *pPatternDesc*, **const AmiValue&** *nRevolutions*)

Sets number of revolutions on an axial pattern descriptor.

Input

pPatternDesc Pointer to an axial pattern descriptor.

nRevolutions Number of revolutions.

AmiStatus amiSetAxialPatternOffsetDirection

(AmiAxialPatternDescrip* *pPatternDesc*, **const AcGeVector3d&** *offsetDir* **)**

Sets offset direction on an axial pattern descriptor.

Input

pPatternDesc Pointer to an axial pattern descriptor.

offsetDir The offset direction.

AmiStatus amiSetAxialPatternOffsetHeight

(AmiAxialPatternDescrip* *pPatternDesc*, **const AmiValue&** *offsetHeight*)

Sets offset height on an axial pattern descriptor.

Input

pPatternDesc Pointer to an axial pattern descriptor.

offsetHeight The offset height.

AmiStatus amiSetAxialPatternOffsetSpacingType

(AmiAxialPatternDescrip* *pPatternDesc*, **AmiPatternSpacing** *offsetSpacingType* **)**

Sets offset spacing type on an axial pattern descriptor.

Input

pPatternDesc Pointer to an axial pattern descriptor.

offsetSpacingType Offset spacing type.

Polar Pattern Descriptors

Locators are used to indicate rotation center. It is required that a locator is set for all polar patterns. When you select the rotational center using the Mechanical Desktop user interface, you are informed that the valid selections for this center are work point, work axis, cylindrical edge, and cylindrical face. These four types of input work in the same way in the API as they do in the user interface. For work point use the AmiCoincidentLocator, for work axis and edge use the AmiAlignedLocator, and for cylindrical face use the AmiOnPlaneLocator.

AmiStatus amiGetPolarPatternData

(AmiPolarPatternDescrip* *pPatternDesc*, **AmiValue& nInstances, AmiPatternSpacing&** *angleSpacingType*, **AmiValue&** *angle*, **Adesk::Boolean&** *fClockwiseRotDir*, **Adesk::Boolean&** *fMaintainOrientation*, **AmiPointKey*&** *pOrientationWorkPoint* **)**

Gets the data from a polar pattern descriptor.

Input

pPatternDesc Pointer to a pattern descriptor.

Output

nInstances Number of instances.
angleSpacingType How the angle value is used.
angle Angle value.
fClockwiseRotDir Rotation direction. Clockwise when true.
fMaintainOrientation If true, then orientation is maintained.
pOrientationWorkPoint If orientation is maintained, the workpoint used. NULL otherwise.

AmiStatus amiSetPolarPatternAngle

(AmiPolarPatternDescrip* *pPatternDesc*, **const AmiValue&** *angle* **)**

Sets angle on a polar pattern descriptor.

Input

pPatternDesc Pointer to a polar pattern descriptor.
angle The angle.

AmiStatus amiSetPolarPatternAngleSpacingType

(AmiPolarPatternDescrip* *pPatternDesc*, **AmiPatternSpacing** *angleSpacing* **)**

Sets angle spacing type on a polar pattern descriptor.

Input

pPatternDesc Pointer to a polar pattern descriptor.
angleSpacing The angle spacing type.

AmiStatus amiSetPolarPatternNInstances

(AmiPolarPatternDescrip* *pPatternDesc*, **const AmiValue&** *nInst* **)**

Sets number of instances on a polar pattern descriptor.

Input

pPatternDesc Pointer to a polar pattern descriptor.
nInst number of instances.

AmiStatus amiSetPolarPatternOrientation

(AmiPolarPatternDescrip* *pPatternDesc*, **AmiPointKey*** *pOrientationWorkPoint* **)**

Calling this function sets the maintain orietation property on a polar pattern descriptor. The associated workpoint must be passed in.

Input

pPatternDesc Pointer to a polar pattern descriptor.
pOrientationWorkPoint Pointed to the workpoint for orientation.

AmiStatus amiSetPolarPatternRotationDirection

(AmiPolarPatternDescrip* *pPatternDesc*, **Adesk::Boolean** *fClockwise* **)**

Sets rotation direction on a polar pattern descriptor.

Input

pPatternDesc Pointer to a polar pattern descriptor.
fClockwise If true, the rot dir is clockwise; if false, counterclockwise.

Rectangular Pattern Descriptors

Locators are used for column alignment. It is not required that a locator is set. If one is not set, the default of angle = 90 degrees is used. The locators that can be set are:

- an AmiAlignedLocator (containing a curve or line key to either an edge or a work axis)
- an AmiAngledLocator containing the AmiValue of the angle for column placement.

Note that the rowDirection and colDirection vectors correspond to the "flip row (column) direction" in the Mechanical Desktop user ingerface. It's a boolean, so if an input vector does not exactly correspond to one of the two possible directions, it is compared to both and taken to mean the one to which it is closest.

AmiStatus amiGetRectangularPatternData

(AmiRectangularPatternDescrip* *pPatternDesc*, **AmiSketchPlaneKey*&** *pSketchPlane*, **AmiValue&** *nColInstances*, **AmiPatternSpacing&** *colSpacingType*, **AmiValue&** *colSpacing*, **AcGeVector3d&** *colDirection*, **AmiValue&** *nRowInstances*, **AmiPatternSpacing&** *rowSpacingType*, **AmiValue&** *rowSpacing*, **AcGeVector3d&** *rowDirection*, **Adesk::Boolean&** *fAlignedToEdge*, **AmiValue&** *alignmentAngle* **)**

Gets the data from a rectangular pattern descriptor.

Input

pPatternDesc	Pointer to a pattern descriptor.

Output

pSketchPlane	Sketch plane used for the pattern.
nColInstances	Number of column instances.
colSpacingType	How the column spacing value was used.
colSpacing	Column spacing value.
colDirection	Direction of the pattern column.
nRowInstances	Number of row instances.
rowSpacingType	How the row spacing value was used.
rowSpacing	Row spacing value.
rowDirection	Direction of the row column.
fAlignedToEdge	If true, then the columns are Aligned To Edge, and the edge is returned through an aligned locator.
alignmentAngle	If fAlignedToEdge is false, this is the angle used for column alignment; otherwise, this parameter is meaningless.

AmiStatus amiSetRectangularPatternAlignmentAngle

(AmiRectangularPatternDescrip* *pPatternDesc*, **AmiValue&** *alignmentAngle* **)**

Sets the angle for column alignment on a a rectangular pattern descriptor.

Input

pPatternDesc	Pointer to a rect. pattern descriptor.
alignmentAngle	The angle for column alignment.

AmiStatus amiSetRectangularPatternColumnData

(AmiRectangularPatternDescrip* *pPatternDesc*, **const AmiValue&** *nColumnInstances*, **AmiPatternSpacing** *columnSpacingType*, **const AmiValue&** *columnSpacing* **)**

Sets column data on a rect. pattern descriptor.

Input

pPatternDesc	Pointer to a rectangular pattern descriptor.
nColumnInstances	Number of column instances.
columnSpacingType	Column spacing type.
columnSpacing	Column spacing.

AmiStatus amiSetRectangularPatternColumnDirection

(AmiRectangularPatternDescrip* *pPatternDesc*, **AcGeVector3d&** *columnDirection* **)**

Sets the column direction on a rectangular pattern descriptor.

NOTE: Column angle or edge alignment is set separately.

Input

pPatternDesc	Pointer to a pattern descriptor.
columnDirection	The column direction.

AmiStatus amiSetRectangularPatternRowData

(AmiRectangularPatternDescrip* *pPatternDesc*, **const AmiValue&** *nRowInstances*, **AmiPatternSpacing** *rowSpacingType*, **const AmiValue&** *rowSpacing***)**

Sets row data on a rect. pattern descriptor.

Input

pPatternDesc	Pointer to a rect. pattern descriptor.
nRowInstances	Number of row instances.
rowSpacingType	Row spacing type.
rowSpacing	Row spacing.

AmiStatus amiSetRectangularPatternRowDirection

(AmiRectangularPatternDescrip* *pPatternDesc*, **AcGeVector3d&** *rowDirection* **)**

Sets row direction on a rectangular pattern descriptor.

Input

pPatternDesc	Pointer to a rect. pattern descriptor.
rowDirection	The row direction.

AmiStatus amiSetRectangularPatternSketchPlane

(AmiRectangularPatternDescrip* *pPatternDesc*, **AmiSketchPlaneKey*** *pSketchPlane***)**

Sets the sketch plane on a rect. pattern descriptor.

Input

pPatternDesc	Pointer to a rectangular pattern descriptor.
pSketchPlane	Pointer to the sketch plane.

Locators

AmiStatus amiAddAtomLocator

(AmiLocator* *pLoc*, **AmiAtomLocator*** *pAtomLoc* **)**

Adds an atomic locator object to a locator's list of atomic locators. The atomic locator is copied when this call is made.

Input

pLoc Pointer to locator object where atomic locator will be added.

pAtomLoc Pointer to atomic locator to add.

AmiStatus amiCreateAtomLocator

(AmiAtomLocatorType *locType*, **AmiAtomLocator*&** *pAtomLoc*, **char*** *appName*=**Designer)**

Creates a new atomic locator object of the given type. It is the caller's responsibility to delete the atomic locator.

Input

locType Type of atomic locator to create.

appName Name of app that should create the locator.

Output

pAtomLoc Pointer to new atomic locator object.

AmiStatus amiCreateLocator

(AmiLocator*& *pLoc*, **char*** *appName*=**Designer)**

Creates a new empty locator object. It is the caller's responsibility to delete the locator.

Input

appName Name of app that should create the locator.

Output

pLoc Pointer to new locator object.

AmiStatus amiGetAbsoluteLocData

(AmiAbsoluteLoc* *pLoc*, **AmiAbsoluteLocType&** *rType* **)**

Gets the data from an "absolute" locator.

Input

pLoc Pointer to an absolute locator.

Output

rType Absolute locator type.

AmiStatus amiGetAngledLocData

(AmiAngledLoc* *pLoc*, **AmiValue&** *rAngle*, **Adesk::Boolean&** *fFlip* **)**

Gets the data from an "angled" locator.

Input

pLoc Pointer to an angled locator.

Output

rAngle Angle off of the plane.

fFlip True if angled away from normal.

AmiStatus amiGetAtomLocators

(AmiLocator* *pLoc*, **int&** *nLocators*, **AmiAtomLocator**&** *pLocators* **)**

Gets all of the atomic locators from a feature locator. It is the caller's responsibility to delete all the locators in the array and delete the array.

Input

pLoc Pointer to a locator object.

Output

nLocators Number of atomic locators.

pLocators Array of pointers to atomic locator objects.

AmiStatus amiGetAtomLocData

(AmiAtomLocator* *pLoc*, **AmiObjectKey*&** *pObjKey* **)**

Gets the object key from a locator. It is the caller's responsibility to release the key.

Input

pLoc Pointer to a locator object.

Output

pObjKey Pointer to a copy of the locator's object key.

AmiStatus amiGetAtomLocType

(AmiAtomLocator* *pLoc*, **AmiAtomLocatorType&** *locType* **)**

Gets the locator type.

Input

pLoc Pointer to a locator object.

Output

locType Locator type.

AmiStatus amiGetCoordSysLocData

(AmiCoordSysLoc* *pLoc*, **AcGeVector3d&** *rAlignVec*, **AmiNormalOrientnType&** *normDir* **)**

Gets the data from a "coordinate system" locator.

Input

pLoc Pointer to a coordinate system locator.

Output

rAlignVec Alignment vector determines to orientation. The vector is adjusted to run either parallel or perpendicular with the alignment axis and represents the positive X direction of the resulting coordinate system.

normDir Normal orientation.

AmiStatus amiGetDistFromLocData

(AmiDistFromLoc* *pLoc*, **AmiValue&** *dist*, **AcGeVector3d&** *dirVec* **)**

Gets the data from a "distance from" locator.

Input

pLoc Pointer to a distance from locator.

Output

dist The distance value.

dirVec Direction vector.

AmiStatus amiGetLocator

(AmiFeatDescrip* *pFeatDescrip*, **AmiLocator*&** *pLoc* **)**

Gets the locator object from a feature descriptor. It is the caller's responsibility to delete the locator being returned.

Input

pFeatDescrip Pointer to a feature descriptor.

Output

pLoc Pointer to the locator object.

AmiStatus amiGetOffsetLocData

(AmiOffsetLoc* *pLoc,* **AmiValue&** *rOffset,* **Adesk::Boolean&** *positiveNormal* **)**

Gets the data from an "offset" locator.

Input

pLoc Pointer to an offset locator.

Output

rOffset Offset from the plane.

positiveNormal Normal is flipped if false.

AmiStatus amiGetOnPlaneLocData

(AmiOnPlaneLoc* *pLoc,* **AcGePoint3d&** *pos* **)**

Gets the data from an "on-plane" locator.

Input

pLoc Pointer to an on-plane locator.

Output

pos Position on plane.

AmiStatus amiGetTangentLocData

(AmiTangentLoc* *pLoc*, **AcGePoint3d&** *pt* **)**

Gets the data from a "tangent" locator.

Input

pLoc Pointer to a tangent locator.

Output

pt Position on surface.

AmiStatus amiSetAbsoluteLocData

(AmiAbsoluteLoc* *pLoc*, **AmiAbsoluteLocType** *type* **)**

Sets the data on an "absolute" locator.

Input

pLoc Pointer to an absolute locator.

type Absolute locator type.

AmiStatus amiSetAngledLocData

(AmiAngledLoc* *pLoc*, **const AmiValue&** *rAngle,* **Adesk::Boolean** *fFlip* **)**

Sets the data on an "angled" locator.

Input

pLoc Pointer to an angled locator.

rAngle Angle value.

fFlip True if angled away from normal.

AmiStatus amiSetAtomLocData

(AmiAtomLocator* *pLoc*, **AmiObjectKey*** *pObjKey* **)**

Sets the object key on an atomic locator. The key is copied when this call is made.

Input

pLoc Pointer to a locator object.

pObjKey Pointer to an object key.

AmiStatus amiSetCoordSysLocData

(AmiCoordSysLoc* *pLoc*, **const AcGeVector3d&** *rAlignVec*, **AmiNormalOrientnType** *normDir* **)**

Sets the data on a "coordinate system" locator.

Input

pLoc Pointer to a coordinate system locator.

rAlignVec Alignment vector determines to orientation. The vector is adjusted to run either parallel or perpendicular with the alignment axis and represents the positive X direction of the resulting coordinate system. If no alignment axis was specified, this vector is used as the X axis of the new coordinate system.

normDir Normal orientation.

AmiStatus amiSetDistFromLocData

(AmiDistFromLoc* *pLoc*, **const AmiValue&** *dist*, **const AcGeVector3d&** *dirVec* **)**

Sets the data on a "distance from" locator.

Input

pLoc Pointer to a distance from locator.

dist Distance value.

dirVec Direction vector from geometry.

AmiStatus amiSetLocator

(AmiFeatDescrip* *pFeatDescrip*, **AmiLocator*** *pLoc* **)**

Sets the locator object on a feature descriptor. The locator is copied when this call is made.

Input

pFeatDescrip Pointer to feature descriptor to set.

pLoc Pointer to locator object.

AmiStatus amiSetOffsetLocData

(AmiOffsetLoc* *pLoc*, **const AmiValue&** *rOffset*, **Adesk::Boolean** *positiveNormal* **)**

Sets the data on an "offset" locator.

Input

pLoc Pointer to an offset locator.

rOffset Offset value.

positiveNormal Flip normal if false.

AmiStatus amiSetOnPlaneLocData

(AmiOnPlaneLoc* *pLoc*, **const AcGePoint3d&** *pos* **)**

Sets the data on an "on-plane" locator.

Input

pLoc	Pointer to an on-plane locator.
pos	Position on plane.

AmiStatus amiSetTangentLocData

(AmiTangentLoc* *pLoc*, **const AcGePoint3d&** *pos* **)**

Sets the data on a "tangent" locator.

Input

pLoc	Pointer to a tangent locator.
pos	Position on surface.

Terminators

AmiStatus amiCreateTerminator

(AmiTerminatorType *termType*, **AmiTerminator*&** *pTerm*, **char*** *appName*=**Designer)**

Creates a new empty terminator object. It is the caller's responsibility to delete the terminator.

Input

termType	Type of terminator to create.
appName	Name of app that should create the terminator.

Output

pTerm	Pointer to new terminator object.

AmiStatus amiGetBlindTermData

(AmiBlindTerm* *pTerm,* **AmiValue&** *value* **)**

Get the data from a blind terminator.

Input

pTerm	Pointer to a blind terminator object.

Output

value	Terminator value.

AmiStatus amiGetFromToTermData

(AmiFromToTerm* *pTerm,* **int&** *loopIndex1,* **AmiSurfKey*&** *pSurfKey1,* **int&** *loopIndex2,* **AmiSurfKey*&** *pSurfKey2* **)**

Gets the data from a "from to" terminator object. It is the caller's responsibility to release all the keys returned.

Input

pTerm	Pointer to a "from to" terminator object.
OutputloopIndex1	If there are multiple intersections with the face, the index of intersection to start.
pSurfKey1	The face at which the feature starts.
loopIndex2	If there are multiple intersections with the face, the index of intersection to stop.

pSurfKey2 The face at which the feature stops.

AmiStatus amiGetInsideTermData

(AmiInsideTerm* *pTerm*, **AmiValue&** *value* **)**

Gets the data from an inside terminator.

Input

pTerm Pointer to an inside terminator object.

Output

value Terminator value.

AmiStatus amiGetMidPlaneTermData

(AmiMidPlaneTerm* *pTerm*, **AmiValue&** *value* **)**

Gets the data from a midplane terminator object.

Input

pTerm Pointer to a midplane terminator object.

Output

value Terminator value.

AmiStatus amiGetOutsideTermData

(AmiOutsideTerm* *pTerm*, **AmiValue&** *value* **)**

Gets the data from an outside terminator.

Input

pTerm Pointer to an outside terminator object.

Output

value Terminator value.

AmiStatus amiGetTerminator

(AmiFeatDescrip* *pFeatDescrip*, **AmiTerminator*&** *pTerm* **)**

Gets the terminator from a feature descriptor. It is the caller's responsibility to delete the terminator returned.

Input

pFeatDescrip Pointer to a feature descriptor.

Output

pTerm Terminator object.

AmiStatus amiGetTermType

(AmiTerminator* *pTerm*, **AmiTerminatorType&** *tType* **)**

Gets the terminator type.

Input

pTerm Pointer to a terminator object.

Output

tType Terminator type.

AmiStatus amiGetToFaceTermData

(AmiToFaceTerm* *pTerm*, **int&** *loopIndex*, **AmiSurfKey*&** *pSurfKey* **)**

Gets the data from a "to face" terminator object. It is the caller's responsibility to release the key returned.

Input

pTerm Pointer to a "to face" terminator object.

Output

loopIndex If there are multiple intersections with the face, the index of intersection to use.

pSurfKey The face at which the feature terminates.

AmiStatus amiGetToPlaneTermData

(AmiToPlaneTerm* *pTerm*, **AmiPlaneKey*&** *pPlaneKey* **)**

Gets the data from a "to plane" terminator object. It is the caller's responsibility to release the key returned.

Input

pTerm Pointer to a "to plane" terminator object.

Output

pPlaneKey Key to the terminator plane.

AmiStatus amiSetBlindTermData

(AmiBlindTerm* *pTerm*, **const AmiValue&** *value* **)**

Sets the data on a blind terminator.

Input

pTerm Pointer to a blind terminator object.

value Terminator value.

AmiStatus amiSetFromToTermData

(AmiFromToTerm* *pTerm*, **int** *loopIndex1*, **AmiSurfKey*** *pSurfKey1*, **int** *loopIndex2*, **AmiSurfKey*** *pSurfKey2* **)**

Sets the data on a "from to" terminator object. The input keys are copied when this call is made.

Input

pTerm Pointer to a "from to" terminator object.

loopIndex1 If there are multiple intersections with the face, the index of intersection to start.

pSurfKey1 Key to the face at which the feature starts.

loopIndex2 If there are multiple intersections with the face, the index of intersection to stop.

pSurfKey2 Key to the face at which the feature stops.

AmiStatus amiSetInsideTermData

(AmiInsideTerm* *pTerm*, **const AmiValue&** *value* **)**

Sets the data on an inside terminator.

Input

pTerm Pointer to an inside terminator object.

value Terminator value.

AmiStatus amiSetMidPlaneTermData

(AmiMidPlaneTerm* *pTerm*, **const AmiValue&** *value* **)**

Sets the data on a midplane terminator object.

Input

pTerm Pointer to a midplane terminator object.

value Terminator value.

AmiStatus amiSetOutsideTermData

(AmiOutsideTerm* *pTerm*, **const AmiValue&** *value* **)**

Sets the data on an outside terminator.

Input

pTerm Pointer to an outside terminator object.

value Terminator value.

AmiStatus amiSetTerminator

(AmiFeatDescrip* *pFeatDescrip*, **AmiTerminator*** *pTerm* **)**

Sets the terminator object on a feature descriptor. The terminator is copied when the call is made.

Input

pFeatDescrip Pointer to feature descriptor to set.

pTerm Pointer to terminator object.

AmiStatus amiSetToFaceTermData

(AmiToFaceTerm* *pTerm*, **int** *loopIndex*, **AmiSurfKey*** *pSurfKey* **)**

Sets the data on a "to face" terminator object. The input key is copied when this call is made.

Input

pTerm Pointer to a "to face" terminator object.

loopIndex If there are multiple intersections with the face, the index of intersection to use.

pSurfKey Key to the terminating face.

AmiStatus amiSetToPlaneTermData

(AmiToPlaneTerm* *pTerm*, **AmiPlaneKey*** *pPlaneKey* **)**

Sets the data on a "to plane" terminator object. The input key is copied when this call is made.

Input

pTerm Pointer to a "to plane" terminator object.

pPlaneKey Key to the terminator plane.

Table-Driven Versions

These functions give access to the table-driven functionality in Mechanical Desktop Release 3.0, both for parts (design variables and feature suppression) and for global design variables. It relies on file descriptors, which describe the file used to version the design. (it is defined in the header file mifile.h). See the file descriptor section for details on accessing the information in this descriptor.

AmiStatus amiGetCurrentVersion

(char*& *ver*, **AmiObjectKey*** *pVersionedObj =NULL* **)**

Get the current version set on the object referenced by *pVersionedObj*. If *pVersionedObj* is null, the global design variable spreadsheet version is queried. It is the caller's responsibility to delete the version string.

Input

pVersionedObj	Key referencing the table driven object to be queried.

Output

ver	The string name of the current version.

AmiStatus amiGetLinkInfo

(AmiFileDescrip*& *pFileDescrip*, AmiObjectKey* *pVersionedObj* = NULL)

Get the descriptor for the spreadsheet controlling the object referenced by pVersionedObj. If pVersionedObj is null, the global design variable spreadsheet is queried. It is the caller's responsibility to free the descriptor.

Input

pVersionedObj	Key referencing the table driven object to be queried.

Output

pFileDescrip	File descriptor describing the linked file.

AmiStatus amiRemoveLink

(AmiObjectKey* *pVersionedObj* = NULL)

Remove the table driven spreadsheet controlling the object referenced by pVersionedObj. If pVersionedObj is null, the global parameter spreadsheet is unlinked.

Input

pVersionedObj	Key referencing the table driven object to be unlinked.

AmiStatus amiSetCurrentVersion

(char*& *ver*, AmiObjectKey* *pVersionedObj* = NULL)

Set the current version of the object referenced by pVersionedObj. If pVersionedObj is null, the global design variable spreadsheet version is set. This version name is case-sensitive.

Input

pVersionedObj	Key referencing the table driven object to be queried.
ver	The string name of the current version.

AmiStatus amiBuildAndLinkFile

(AmiFileDescrip* *pFileDesc*)

Create and populate the table driven spreadsheet controlling the object referenced by the attribute kAttVersionedObj. If this attribute is null or NAV, the global parameter information is used. The file will be populated using the information in the descriptor including the structure of the spreadsheet and the file. If the file name already exists the link will fail.

Input

pFileDesc	File descriptor describing the file to link.

AmiStatus amiBuildFile

(AmiFileDescrip* *pFileDesc*)

Create and populate the table driven spreadsheet controlling the object by the attribute kAttVersionedObj. If this attribute is null or NAV, the file is built using global parameter

information. The file will be populated using the information in the descriptor including the structure of the spreadsheet and the file. If the file name already exists the link will fail.

Input

pFileDesc File descriptor describing the file to link.

Event Handling

The event handling functions allow applications to register callback functions for specific Mechanical Desktop events. This function will be called every time the specified event occurs in the Desktop, until the callback is unregistered. The event registrations persist across file opens, but during a single session only. They must be re-registered each time the Desktop is loaded.

Each event has a prescribed signature, which the registered callback must fulfill. The signature is typically a key(s) to the affected objects and a context flag. The context flag is used to provide an indication of why the event occurred, e.g. during a copy, during creation of a new file. The Ami::eNullContext indicates that the event occurred during normal modelling operations.

The event reaction is described using an AmiReactDescrip. The make function on this descriptor ensures the appropriately typed function pointer is given for the event keyword (amiEventType) specified. This descriptor is passed to amiRegisterEventReaction, and from that point forward (until the end of session, or the reaction is unregistered), the specified function is called each time the specified event occurs. Argument control for the reaction can be specified using amiSetData, passing in the appropriate keyword and value.

Before registering events, an application must get an application id by calling amiRegisterApp. Given a string (uniquely) identifying the application, this function will return an identifier to be used when registering or unregistering callbacks.

AmiStatus amiReactDescrip::make

(<CallBack Signature Type> *fcn*, **AmiEventType** *type*, **AmiReactDescrip*&** *pRD* **)**

Makes a reaction descriptor to pass into amiRegisterEventReaction. There is one version for each notification function type.

Input

fcn Callback function pointer.

type Event to register the call back for.

Output

pRD Pointer to reaction descriptor. Returns eInvalidArg if the function type is not appropriate for the event.

The following table lists all the events supported and their imposed function pointer type. As new events are supported, they will be added to this table. As other function types are supported for a specific event, they will also be added.

Table 6: Event Keywords

Event Keyword	Function Pointer Type	Argument Order	Description
kOnAttachFile	Ami1FileNotifFcnPtr		Versioning file attached to a part or global design variables (null key returned)
kOnDetachFile	Ami1FileNotifFcnPtr		Versioning file removed from a part or global design variables (null key returned)

kOnNewVersion	AmiFileVerNotifFcnPtr		New version added to part or global design variables table. Note: on table updates all versions are erased and recreated.
kOnEraseVersion	AmiFileVerNotifFcnPtr		Version definition removed. Note: on table updates all versions are erased and recreated.
kOnActivateVersion	AmiKeyVerNotifFcnPtr		Version activated on part (variables or suppression) or global design variables
kOnConflictDetected	Not supported		Conflict detected attaching versioning table
kOnConflictResolved	Not supported		Version table conflict resolved
kOnActivatePart	Ami1KeyNotifFcnPtr		Part activated
kOnNewPart	Ami1KeyNotifFcnPtr		New part created
kOnUpdatePart	Ami1KeyNotifFcnPtr		Part model updated
kOnErasePart	Ami1KeyNotifFcnPtr		Part erased
kOnFeatureAdded	Ami1KeyNotifFcnPtr		Feature added
kOnFeatureRemoved	Ami1KeyNotifFcnPtr		Feature removed
kOnFeatureRenamed	Ami1KeyNotifFcnPtr		Feature renamed
kOnFeatureVersioned	Ami1KeyNotifFcnPtr		Feature excluded through versioning
kOnFeatureSuppressed	Ami1KeyNotifFcnPtr		Feature excluded through suppression
kOnFeatureUnversioned	Ami1KeyNotifFcnPtr		Feature reactivated through change of version
kOnFeatureUnsuppressed	Ami1KeyNotifFcnPtr		Feature reactivated through unsuppression
kOnWorkGeomAdded	Ami1KeyNotifFcnPtr		Work geometry added
kOnWorkGeomRemoved	Ami1KeyNotifFcnPtr		Work geometry removed
kOnWorkGeomRenamed	Ami1KeyNotifFcnPtr		Work geometry renamed
kOnWorkGeomVersioned	Ami1KeyNotifFcnPtr		Work geometry excluded through versioning
kOnWorkGeomSuppressed	Ami1KeyNotifFcnPtr		Work geometry excluded through suppression
kOnWorkGeomUnversioned	Ami1KeyNotifFcnPtr		Work geometry reactivated through change of version
kOnWorkGeomUnsuppressed	Ami1KeyNotifFcnPtr		Work geometry reactivated through unsuppression
kOnFeatureReorder	Ami2KeyBoolNotifFcnPtr	(feature, destinationFeature, after (Boolean), context)	Feature reordered, either before (after = kFalse) or after (after = kTrue) destinationFeature
kOnReplayTruncated	Not supported		
kOnFeatureReplayed	Not supported		
kOnArrayExplode	Not supported		
kOnNewConstraint	Ami1KeyNotifFcnPtr		Constraint Added (Assembly only for now)
kOnEraseConstraint	Ami1KeyNotifFcnPtr		Constraint erased (Assembly only for now)
kOnNewCompDef	Ami1KeyNotifFcnPtr		Component definition created
kOnNewComp	Ami1KeyNotifFcnPtr		New component instanced

kOnNewScene	Ami1KeyNotifFcnPtr		Scene created
kOnNewCompView	Ami2KeyNotifFcnPtr	(comp, scene, context)	View created for component
kOnEraseCompDef	Ami1KeyNotifFcnPtr		Component Definition erased
kOnEraseComp	Ami1KeyNotifFcnPtr		Component erased
kOnEraseScene	Ami1KeyNotifFcnPtr		Scene erased
kOnEraseCompView	Ami2KeyNotifFcnPtr	(comp, scene, context)	View of component erased
kOnRenameComp	Ami1KeyNotifFcnPtr		Component renamed
kOnRenameCompDef	Ami1KeyNotifFcnPtr		Component definition renamed
kOnContextSwitch	Ami2KeyNotifFcnPtr	(newContext, oldContext, editContext)	Switch between active contexts - component def(s) and scene(s).
kOnReorderComp	Ami2KeyBoolNotifFcnPtr	(Component, Target, InsertBefore, EditContext)	Subassembly component order changed
kOnExternalizeCompDef	Ami2KeyNotifFcnPtr	(oldCompDef, newCompDef, editContext)	Component definition externalized – notified at end of process
kOnLocalizeCompDef	Ami2KeyNotifFcnPtr	(newCompDef, oldCompDef, editContext)	Component definition localized – notified at end of process
kOnNewView	Ami1KeyNotifFcnPtr		Drawing view added
kOnViewRemoved	Ami1KeyNotifFcnPtr		Drawing view removed
kOnViewEdit	Ami1KeyNotifFcnPtr		Drawing view edited
kOnViewRegen	Ami1KeyNotifFcnPtr		Drawing view about to be regenerated/updated (this will include recalculation of hidden-lines, if appropriate). Detail views are not supported because they are not generated independently of their parent view.
kOnViewCopied	Ami2KeyNotifFcnPtr		The drawing view has been copied, but the command/transaction has not been closed. If multiple views are copied within a single command, some of the other views might not have been copied yet.
kOnViewPreCopy	Ami2KeyNotifFcnPtr, AmiCloneNotifFcnPtr		The view is in the process of being copied. Some processing has occurred. This notification is an appropriate time for external applications to use an AmiCloneNotifFcnPtr callback and participate in the deep-clone process.

Table 7: Callback Function Pointer Typedefs

Function pointer typedef	Function Prototype	Comments
Ami1KeyNotifFcnPtr	(AmiObjectKey* pObj1Key, AmiEventContextType contextType)	One object key returned.
Ami2KeyNotifFcnPtr	(AmiObjectKey* pObj1Key, AmiObjectKey* pObj2Key, AmiEventContextType contextType)	Two objects involved in the event, both are returned.
Ami3KeyNotifFcnPtr	(AmiObjectKey* pObj1Key, AmiObjectKey* pObj2Key, AmiObjectKey* pObj3Key, AmiEventContextType contextType)	Three objects involved – typically a constraint event.
Ami1KeyBoolNotifFcnPtr	(AmiObjectKey* pObj1Key, Adesk::Boolean flag, AmiEventContextType contextType)	
Ami2KeyBoolNotifFcnPtr	(AmiObjectKey* pObj1Key, AmiObjectKey* pObj2Key, Adesk::Boolean flag, AmiEventContextType contextType)	

Ami3KeyBoolNotifFcnPtr	(AmiObjectKey* pObj1Key, AmiObjectKey* pObj2Key, AmiObjectKey* pObj3Key, Adesk::Boolean flag, AmiEventContextType contextType)	
Ami1FileNotifFcnPtr	(AmiFileDescrip* pFileDescrip, AmiObjectKey* pObjKey, AmiEventContextType contextType)	Versioning file event, possibly in relation to an object. Object key pointer may be NULL.
AmiFileVerNotifFcnPtr	(AmiFileDescrip* pFileDescrip, const char* pVersion, AmiEventContextType contextType)	Version event related to a file.
AmiKeyVerNotifFcnPtr	(AmiObjectKey* pObjKey, const char* pVersion, AmiEventContextType contextType)	Version event related to an object. Object key pointer may be NULL.
AmiCloneNotifFcnPtr	(AmiObjectKey* pKeyPreClone, AmiObjectKey* pKeyPostClone, AcDbIdMapping& idMap, AmiEventContextType contextType)	

Table 8: Event Reaction Attributes

Attribute Keyword	DataType	Read-Only	Applies To	Description
kTriggerDoc	AcApDocument		Any	Only events on this specific document will trigger this notification. If this value is NAV, the notification will be triggered by events happening in any document.
kTriggerObject	AmiObjectKey		Drawing view events	Only events on this specific object will trigger this notification.

AmiStatus amiGetData

(AmiReactDescrip* *pReactDescrip*, **AmiReactAttribute** *keyword*, **<Return Type>&** *value* **)**

Extract an attribute value on the given event reaction descriptor. This function is overloaded for all return types needed to support the current set of reaction attributes. These types are:
Adesk::Boolean
AmiObjectKey

Input

pReactDescrip A pointer to the descriptor.
keyword Keyword for attribute (passed as an enum).

Output

value The value for the specified attribute.

AmiStatus amiRegisterApp

(char * *regAppName*, **AmiAppId&** *appId***)**

Registers an application for reaction registration.

Input

RegAppName Name of application.

Output

appId Application id for use in reaction registration.

AmiStatus amiRegisterEventReaction

(AmiReactDescrip * *pReact*, **AmiAppId** *appId*, **const char*** *appName* **= NULL)**

Registers a callback based on the reaction descriptor passed in.

Input

pReact Reaction descriptor, includes the event and callback fcn.

appId	Application id requesting the registration. The *appId* is used to bulk unregister reactions. You get an application id by calling amiRegisterApp.
appName	Application supporting the event.

AmiStatus amiRemoveEventReaction

(AmiAppId *regAppId***, AmiReactDescrip*** *pReact* **=NULL**, **const char*** *appName* **= NULL)**

Removes callbacks registered as described by *pReact*. All callbacks with the same event type and function pointer registered by the app ID will be removed. If *pReact* is null, all callbacks registered by *regAppId* will be removed. It is the caller's responsibility to delete pReact when finished. You get an application id by calling amiRegisterApp.

Input

regAppId	ID of the Application requesting removal of callbacks
pReact	Optional descriptor specifying which specific callback to remove.
appName	Name of application supporting the events, if needed.

AmiStatus amiSetData

(AmiReactDescrip* *pReactDescrip***, AmiReactAttribute** *keyword***, <Return Type>** *value***)**

Set an attribute value on the given event reaction descriptor. This function is overloaded for all return types needed to support the current set of reaction attributes. These types are:
Adesk::Boolean
AmiObjectKey

Input

pReactDescrip	A pointer to the descriptor.
keyword	Keyword for attribute (passed as an enum).
value	The value for the specified attribute.

Miscellaneous

AmiStatus amiGetDesignMode

(AmiDesignEnvironmentType& *currEnvironment*, **AmiDesignModeType&** *currMode*, **const char*** *appName* = **Designer_Assemblies)**

Returns the current design environment (assembly, single part, proxy, multipart, or mixed) as each environment entails a set of restrictions. Also returns the current mode of the model, drawing or scene.

Input

appName	Application being queried. Currently ignored.

Output

currEnvironment	Current design environment.
currMode	Current design mode.

AmiStatus amiGetKeyFromName

(const char* *name***, AmiKeyType** *keyType***, AmiObjectKey*&** *pKey***, AmiObjectKey*** *pScopeKey* **= NULL, const char*** *appName* **= NULL)**

Get an object key based on an object name. Given an object name, this function attempts to create a key of the given key type. The scope key is used to determine the starting point at which to look for the object. For example, given a key type of "Ami::featKey" and a part key as the scope, the function will look for a feature within that part with the given name and create a feature key to that feature. If

the scope key is NULL, it is assumed that we're looking for an object in the global scope such as a part or component definition. Function delegation is determined first by the scope key, then by the *appName*, and lastly by the key type. It is the caller's responsibility to release the key returned.

Note: only feature, parameter and sketch keys are supported.

Input

name	Name of object to get key for.
keyType	Type of key that is to be created.
pScopeKey	Key to scope object.
appName	Application to create key.

Output

pKey	Pointer to created key.

AmiStatus amiSetDesignMode

(AmiDesignModeType *wantedMode*, **const char*** *appName* = **Designer_Assemblies)**

Sets the design mode to the indicated mode. The mode cannot be set to kUninitialized. This function is meant to be used to switch between modeling, drawing, scene, etc.

Note: amiGetDesignMode returns the design environment in addition to the design mode, but only the mode can be set using this function—the environment is fixed.

Input

wantedMode	Desired design mode.
appName	Application name. (Optional.) Defaults to Designer_Assemblies.

Analysis

The Degrees of Freedom for assembly components and sketches are available through a descriptor object. The attributes on AmiDOFDescrip are listed in the table below. Not all of these attributes are supported in all DOF descriptors; a status of eAttValueNAV is returned for any unset attributes.

The AmiDOFDescrip objects are read-only. They reflect the Degrees for Freedom for the specified object, at the time the descriptor is constructed.

Table 9 : Degrees of Freedom Attribute Keywords

Attribute Keyword	Data Type	Currently supported for	Read-Only	Description
dofAttNumVars	int	sketches, components	Y	The number of variables in the system. For example, for rigid-body motion in 3-space, this value will be six. For sketches, this value will depend upon the nature of the geometry in the sketch.
dofAttNumFreeVars	int	sketches, components	Y	The number of variables that are free (under-constrained). This number will always be less than or equal to dofAttNumVars.
dofAttNumDOF	int	sketches, components	Y	The number of rigid-body degrees of freedom in the system. For assembly components, where the only variables counted in dofAttNumVars are for rigid-body motion, this number will be equal to dofAttNumFreeVars. For descriptors for sketches, this number will be less than or equal to dofAttNumFreeVars.
dofAttNumRDOF	int	components	Y	The number of rotational degrees of freedom.
dofAttNumTDOF	int	components	Y	The number of translational degrees of freedom.
dofAttRotationAxis	AcGeLine3d, array of AcGeLine3d	components	Y	The axes of rotation. If retrieved from the descriptor as a single AcGeLine3d, then the first axis is returned. If queried as an array, then all the axes of rotation are returned.
dofAttTranslationAxis	AcGeLine3d, array of AcGeLine3d	components	Y	The axes of translation. If retrieved from the descriptor as a single AcGeLine3d, then the first axis is returned. If queried as an array, then all the axes of translation are returned.
dofAttRotationPoint	AcGePoint3d	components	Y	A point on the axis of rotation (if needed). For example, if there are 3 RDOF and 1 TDOF in 3D space, then this point will be the pivot point on the part.

AmiStatus amiCheckInterference

(AmiCompKey* *pKey1*, **AmiCompKey*** *pKey2*, **int&** *numClashes*,

AmiCompKey*** *pComps1=(AmiCompKey****)***NULL**,

AmiCompKey*** *pComps2=(AmiCompKey****)***NULL**,

AmiSolidKey*** *pCommonVols=(AmiSolidKey****)***NULL)**

Checks for volume interference between two components. Most of the output parameters are optional, so the caller can control the amount of information returned. If the last argument is non-null, then new solids representing the interferences between the components are created and added to the database, and keys to those solids are passed back.

Input

pKey1 A pointer to the descriptor.

pKey2 Keyword for attribute (passed as an enum).

Output

numClashes The number of common values found.

Input/Output

pComps1,

pComps2 These are pointers to pointers to arrays of pointers to component definition keys. If non-null, the pointer referenced by these pointers are set to point to a newly allocated array of pointers to leaf components, such that the leaf components are part of the two subassemblies passed in, and each pair of components has interfering volume. If null, no component definition keys are created.

pCommonVols This is a pointer to a pointer to an array of pointers to solid keys. If non-null, then the referenced pointer is set to point to a newly allocated array of solid key pointers. These keys are for the newly generated solids that represent the interference between the corresponding pair of leaf components. If null, no solid keys are created

Note: To determine if there is any interference, without generating any keys and without adding any solids to the database, the last three arguments can be omitted and the numClashes argument can be tested for 0.

One AmiCompKey array cannot be used without the other. They are meant to convey 2-tuples of components, such that each pair holds keys to interfering leaf components of the two subassemblies. When solid keys are returned, the solids are created and added to the database, and the last argument should be interpreted as the third part of 3-tuples (comp, comp, solid).

AmiStatus amiCheckInterferenceTrans

(AmiCompKey* *pKey1*, **AmiCompKey*** *pKey2*, **int&** *numClashes*, **AmiCompKey***** *pComps1* **= (AmiCompKey***)NULL, AmiCompKey***** *pComps2* **= (AmiCompKey***)NULL, AcDb3dSolid***** *pTransientVols* **= (AcDb3dSolid***)NULL)**

Checks for volume interference between two components. Most of the output parameters are optional, so the caller can control the amount of information returned. If the last argument is non-null, then new solids are created and added to the database, and keys to those solids are passed back.

Input

pKey1 Key for the first component.

pKey2 Key for the second component.

Output

numClashes The number of common volumes found.

Input/Output

pComps1, pComps2 These are pointers to pointers to arrays of pointers of comp keys. If non-null, the pointer referenced by these pointers is set to point to a newly allocated array of pointers to leaf components, such that the leaf components are part of the two subassemblies passed in, and each pair of comps has interfering volume.

pTransientVols This is a pointer to a pointer to an array of pointers to AcDb3dSolids. If non-null, then the referenced pointer to set to point to a newly allocated array of AcDb3dSolid pointers. These keys are for newly generated solids that represent the interference between the corresponding pair of leaf components.

Note: One AmiCompKey array cannot be used without the other. The caller can ask for all three parts of the 3-tuples (i.e. all three arrays), or no part (i.e. none of the arrays), or just the comp pairs (i.e. the first two arrays), or just the transient solids (i.e. the last array).

AmiStatus amiGetData

(AmiDOFDescrip* *pDescrip*, **AmiDOFAttribute** *keyword*,**<datatype>** *value* **)**

Extracts an attribute value from the given DOF descriptor. This function is overloaded for all return types needed to support the current set of DOF attributes. These types are:

int

AcGePoint3d,

AcGeEntity3d.

The keywords for attributes are defined by the AmiDOFAttribute enum. The following table lists the attribute keywords, the datatype, and brief descriptions.

dofAttNumVars	int	total number of variables
doffAttNumFreeVars	int	number of free variables
dofAttNumDOF	int	number of rigid-body DOF
dofAttNumRDOF	int	number of Rotation DOF
dofAttNumTDOF	int	number of Translation DOF
dofAttRotationAxis	AcGeEntity	first axis of rotation
dofAttTranslationAxis	AcGeEntity	first axis of translation
dofAttRotationPoint	AcGePoint3d	point on rotation axes

If the value for an attribute is NAV, meaning "Not A Value", then eAttValueNAV is returned as the AmiStatus and the content f the 'value' is undefined.

Input

pDescrip A pointer to the descriptor.

attr Keyword for attribute (passed as an enum).

Output

value The value for the specified attribute.

AmiStatus amiGetDataArray

(AmiDOFDescrip* *pDescrip*, **AmiDOFAttribute** ***keyword***, **int&** *numValues*, **<datatype>** *values* **)**

Extracts an attribute value from the given DOF descriptor. This function is overloaded for all return types needed to support the current set of multi-valued DOF attributes. At present, this is limited to

AcGeEntity3d. The keywords for attributes are defined by the AmiDOFAttribute enum. The following table lists the attribute keywords, the datatype, and brief descriptions.

dofAttRotationAxis	AcGeEntity array	(free) axes of rotation
dofAttTranslationAxis	AcGeEntity array	(free) axes of translation

If the value for an attribute is NAV, meaning "Not A Value", then eAttValueNAV is returned as the AmiStatus and the content of the 'value' is undefined.

Input

pDescrip A pointer to the descriptor.

attr Keyword for attribute (passed as an enum).

Output

numValues The number of elements in the array.

values The array of values for the attribute.

AmiStatus amiGetDOFDescrip

(AmiObjectKey* *pKey*, **AmiDOFDescrip*&** *pDescrip* **)**

Returns Degree Of Freedom information for the given object. The descriptor can be queried for individual attribute values. While the input can be any type of object key, this function is only meaningful for a few object types (e.g. components and sketches) and will return an error status for most types. For components, the DOF descriptor holds the Degrees of Freedom for the component in the scope of the directly owning subassembly. Any geometry in the descriptor (e.g. axes) is transformed to reflect the full scope implied by the key.

Input

pKey A pointer to an object key.

Output

pDescrip A pointer to a newly created DOF descriptor. It is the caller's responsibility to delete this object.

AmiStatus amiGetMassProps

(long *size*, **AmiObjectKey**** *pObjKeys*, **AmiMassProps&** *massProps* **)**

Calculates the mass properties for a set of components, parts, AutoCAD solids, or a volume enclosed by surfaces. An assumption is made that all keys in the passed-in array are of the same type. If the first key is not to a part, component, AutoCAD solid, or surface key, an error is returned. The mass prop information is returned in WCS. The mass properties structure (*amiMassProps*) is defined in header file *mibody.h*.

Input

size Number of keys in the **AmiObjectKey** array.

pObjKeys Array of keys.

Output

massProps The total mass properties of the passed-in keys.

AmiStatus amiMinDistance

(AmiCompKey* *pKey1*, **AmiCompKey*** *pKey2*, **double&** *distance*, **AcGePoint3d&** *pt1*, **AcGePoint3d&** *pt2***)**

Finds the minimum distance between two components. Only leaf-node components are currently supported.

Input

pKey1 Key for the first component.

pKey2 Key for the second component.

Output

distance The minimum distance between the two components.

pt1 First end-point of a minimum distance line between the two components.

pt2 Second end-point of a minimum distance line between the two components.

Mechanical Desktop Surface Modeling

These functions handle Mechanical Desktop free form surface modeling.

Geometry Creation

AmiStatus amiCreateSurfFromGeom

(**AmiSurfKey*&** *pSurfKey*, **const AcGeSurface*** *pGeSurf*, **int** *numLoops*, **const AcGeCurveBoundary*** *pLoops*, **const char*** *appName* **= AutoSurf**)

Creates a bounded (trimmed) surface and returns a key to it. It is the caller's responsibility to release the key.

Input

pGeSurf Pointer to GeLib surface used in defining the surface that will be created. For the initial revision only, AcGeNurbsSurface as well as AcGeOffsetSurface based on a NURBS surface are valid inputs. Future revisions are expected to support a broader range of surface types.

numLoops Number of loops in array *pLoops*. If this parameter is set to zero, an untrimmed surface will be created.

pLoops Array of boundary loops. This AcGeCurveBoundary can support model space and parameter space curves that are of type AcGeNurbCurve2d. Future revisions are expected to support using either model space or parameter space trim boundary curves, as well as AcGeCurves of all types.

NOTE: Loop orientation is important. Outer loops must be counterclockwise and inner loops must be clockwise. All Loops must map to closed paths in model space.

appName Name of application module to perform the function. The default application is AutoSurf.

Output

pSurfKey Pointer to the key to the new surface.

AmiStatus amiCreateSurfArrayFromGeom

(**int&** *numKeys*, **AmiSurfKey**&** *pSurfKeys*, **const AcGeSurface*** *pGeSurf*, **int** *numLoops*, **const AcGeCurveBoundary*** *pLoops*, **const char*** *appName* = **AutoSurf**)

Creates a set of bounded (trimmed) surfaces and returns keys to them. It is the caller's responsibility to release the keys.

Input

pGeSurf Pointer to GeLib surface used in defining the surface that will be created. For the initial revision only, AcGeNurbsSurface as well as AcGeOffsetSurface based on a NURBS surface are valid inputs. Future revisions are expected to support a broader range of surface types.

appName Name of application creating the surface.

numLoops Number of loops in array pLoops. If this parameter is set to zero, an untrimmed surface will be created.

pLoops Array of boundary loops. This AcGeCurveBoundary can support model space and parameter space curves that are of type AcGeNurbCurve2d. Future revisions are expected to support using either model space or parameter space trim boundary curves, as well as AcGeCurves of all types.

NOTE: Loop orientation is important. Outer loops must be counterclockwise and inner loops must be clockwise. All loops must map to closed paths in model space.

Output

numKeys Number of keys returned.

pSurfKeys Array of pointers to keys to the new surface.

Drawing Manager

Introduction

The Drawing Manager allows for editing of views and annotations through AmiDescrip descriptor objects. See page **Error! Bookmark not defined.** for a discussion of AmiDescrip and how it is used in the API.

Several attributes can take on values that are reference geometry or non-reference geometry. For example, the orientation plane (vwAttOrientnPlane) of a base view can be a reference to a face of a part or it can simply be the XY-plane in world coordinates. If an attribute value is parametrically associative, then values are passed using geometry keys, otherwise AcGeEntity3d objects are used. If the attribute value of an existing database object is requested as an AmiObjectKey and the query fails with error status eGKeyNotSupported, then this is an indication that the underlying geometry is non-associative and that the same attribute should be requested using the AcGeEntity3d overload. The AcGeEntity3d representation is always available because there is always an AcGeEntity3d representation for reference geometry.

Views

The following table lists all of the view attributes currently defined in the Drawing Manager, along with a brief description of each. These same attributes are then separated by view-type, as several of these attributes are applicable to only some types of view. New keywords will be added to this list as the set of Drawing Manager attributes supported by the MCAD API grows.

Table 10: View Attribute Keywords

Attribute Keyword	Data Type	Read-Only	Applies to	Description
vwAttViewType	AmiViewType	Y	All	The type of view/descriptor.
vwAttParent	AmiObjectKey	Y	All but base	The parent view.
vwAttDisplayType	AmiViewDisplayType		All	Wireframe, hidden-lines. This attribute determines the type of processing used to create the view geometry.
vwAttAHLdisplayOn	Adesk::Boolean		All	Toggle display of hidden-lines (when vwAttDisplayType is not kWireframe). This attribute setting does not affect processing, only the display.
	AmiHiddenLineTreatment		All	Normally hidden-lines are calculated and stored in anticipation of vwAttAHLdisplayOn being set to kTrue. To discard calculated hidden-lines, the AmiHiddenLineTreatment representation of vwAttAHLdisplayOn can be set to kHiddenLinesDiscarded.

vwAttScale	double		All	Scale-factor for the view. If absolute scaling is used, then this scale-factor is applied to the component size. If relative scaling is used, then this scale-factor is applied to the scale of the parent view. The value should always be positive.
vwAttScaleType	AmiViewScaleType		All	Absolute/Relative scaling.
vwAttLocation	AcGePoint3d		All	Center-point of the view.
vwAttExtents	AcGePoint3d array	Y	All	The extents of the viewport box. The array holds two points: the lower-left and upper-right.
vwAttLabel	const char*		All	Label for the view. The label is displayed for Detail views only.
vwAttSectionType	AmiSectionType	Y	All	The type of section. There is a type, kSecNone, to indicate no sectioning.
vwAttHatchOnFlag	Adesk::Boolean		Sectioned	Section hatching on/off flag. This attribute is only applicable to sectioned views.
vwAttHatchAHL	Adesk::Boolean		Sectioned	Normally hatch patterns are not trimmed w.r.t hidden-line calculations. If vwAttHatchAHL is kTrue, then the hatch pattern participates in hidden-line calculations.
vwAttSectionLabel	const char*		Sectioned	Label for the section. This label appears in both the described view and in the parent view. This attribute is read-only for base views with no pre-existing label, as there is no parent view in which is place the label.
vwAttSectionCut	AmiObjectKey	Y	Sectioned	The cutting line/plane/point for a sectioned view.
	AmiObjectKey array	Y	Half sectioned	A pair of cutting planes, when applicable.
	AcGePoint3d	Y	Full and half sectioned	The cutting point. When set using AcGePoint3d, the cutting point will be fixed. Can be queried as AcGePoint3d when fixed and when reference geometry.
	AcGeEntity3d	Y	Full sectioned	The cutting plane. When set using AcGePlane, the cutting plane will be fixed. Can be queried as AcGePlane when fixed and when reference geometry.
	AcGeEntity3d array	Y	Half sectioned	The cutting planes (there are two). Similar to the use of AcGePlane with full-sections.
vwAttSectionCutDir	AmiSectionCutDir		Half sectioned	The cutting direction of the half-section.
vwAttOrientation	AmiOrientationType	Y	Base, Ortho, Iso	Vantage-point. This is an enum used to indicate the standard orientations for ortho and iso views.
	AcGeMatrix3d	Y	All	Orientation transform matrix. This is applicable to any type of view.
vwAttOrientnPlane	AcGeEntity3d, AmiObjectKey, AmiNormalOrientnType	Y	Base, Aux	Base-view orientations are represented as plane-axis-direction. This is the plane component of this triple. When the plane is reference geometry, amiObjectKey is used; when it is fixed geometry AcGePlane is used. Aux-view orientations can be defined in terms of a work-plane, passed as an AmiObjectKey. The AmiNormalOrientnType representation can be used to reverse the normal of the plane.

vwAttOrientnAxis	AcGeEntity3d, AmiObjectKey	Y	Base	The axis component of the plane-axis-direction representation of base-view orientations. When the axis is reference geometry amiObjectKey is used; when it is fixed geometry AcGeLine is used.
vwAttOrientnAxisDir	AmiAxisDirection		Base	The axis direction for the plane-axis-direction representation of base view orientations.
vwAttGap	double		Broken views	The gap in a "broken" view. This attribute is applicable to broken views only.
vwAttRefPoint1	AmiObjectKey	Y	Detail, Aux	Reference Geometry in model-space. For a detail-view, this is the point in the parent view that is being detailed. For an aux-view, this is the first vertex (of two) that defines the axis of rotation for the orientation of the aux-view with respect to the parent view.
vwAttRefPoint2	AmiObjectKey	Y	Aux	Reference Geometry in model-space. For an aux-view, this is the second vertex that defines the axis of rotation for the orientation of the aux-view with respect to the parent view.
vwAttRefBoxUL	Array of double (2 values)	Y	Detail	Upper-left corner of detailed box in parent view. This point is relative to vwAttRefPoint1 in paper-space and is passed as a pair of x, y offsets.
vwAttRefBoxLR	Array of double (2 values)	Y	Detail	Lower-right corner of detailed box in parent view. This point is relative to vwAttRefPoint1 in paper-space and is passed as a pair of x ,y offsets.
vwAttModelEnts	AmiObjectKey array	Y	All	The model entities contained in the view. The model entities can be part keys, solid keys, and scene keys. If a scene key is in the array, then it must be the only key in the array.
vwAttShowParDims	Adesk::Boolean		All	Turns on/off the display of parametric dimensions in the view.
vsAttShowTapLines	Adesk::Boolean		All	Turns on/off the display of tapped-hole annotations in the view.
vsAttLayout	AmiDwLayoutKey, AcDbObjectId		All	The layout or sheet on which the view resides.
vwAttLayout	AmiObjectKey, AcDbObjectId		All	The layout or sheet on which the view is placed.
vwAttShowParDims	Adesk::Boolean		All	Indicates if parametric dimensions are displayed in the view.
vwAttShowTapLines	Adesk::Boolean		All	Indicates if tapped-hole lines are displayed in the view.
vwAttStandardPartRep	Adesk::Boolean		All	Turns on/off Standard Part Representation for 3D standard parts in view. The Standard Part Representation is a 2D representation of a 3D Standard Part that conforms to drafting standards. This attribute has no affect when the MDT Power Pack is not loaded since the Standard Parts functionality is in the Power Pack. Also, this has no affect if there are no Standard Parts in the view or if the view orientation does not allow for a standard 2D representation for the parts (e.g. iso views).
vwAttCenterPt	AcGePoint3d	Y	All	This is the center of the view in model-space coordinates.

When descriptors are used to create new database objects (e.g. new views), some of the read-only attributes can be set. Thus, there are several attributes which are "write-once", meaning that they can

be set when the view is created but thereafter are read-only. Other attributes have values that are derived from other properties of the view and cannot be set even when creating a new view. For example, the matrix representation on the view orientation is derived from the other orientation representations (e.g. AmiViewOrientation) and cannot be set directly, even when creating a new view. Several attributes are mandatory for view creation . The following tables list the attributes applicable to each type of view and describe the roles those attributes play during view creation.

For base views, there is no parent view, the scale-type must be absolute, and the orientation is represented as a triple of plane-axis-direction. The orientation geometry can be parametrically associative or not. If it is associative, then values are passed using geometry keys, otherwise AcGeEntity3d objects are used.

In the tables below, the second column describes any constraints on allowable values for the specified attribute. The third column, labeled 'Role in view creation' indicates if the attribute must have a non-NAV value when the descriptor is used to create a new view. If this entry is blank, then the attribute is optional and can be NAV. The fourth column indicates if the attribute is read-only, write-once, or read/write (this last case is indicated by a blank entry in column four).

Table 11: View Attributes: kBaseVw

Attribute Name	Allowable Values	Role in view creation	Settable through API ?
vwAttViewType	**kBaseVw**	mandatory	Write-once
vwAttDisplayType	**kModelSpace** (unprocessed model space entities) **kWireFrame** (wireframe-no hidden line calculations) **kWireFrameWithSilh** (wireframe with no hidden line calculations, but with silhouette calculations) **kNoOcclusion** (hidden line calculations, but no occlusion) **kNoOcclusionWithTang** (hidden line calculations are on, but no occlusion calculations. Occlusion refers to edges covering other edges (points), as opposed to regular hidden line calculations which involve surfaces covering edges) **kNoOcclusionWithTang** (hidden line calculations are on, occlusion calculations are off, and tangencies are displayed. A tangency is an edge where the adjoining faces have equal normals along the length of the edge. **kSimpleOcclusion** (hidden line calculations with simple occlusion. This means that spline curves are not considered in occlusion calculations.) **kSimpleOcclusionWithTang** (hidden line calculations, with simple occlusion, and with tangencies displayed) **kOcclusion** (hidden line calculations, with full occlusion) **kOcclusionWithTang** (Hidden line calculations, with full occlusion, and tangencies displayed		
vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded		
vwAttScale	Any positive double	mandatory	
vwAttScaleType	kVwSclAbs		Read-only
vwAttLocation	Any AcGePoint3d	mandatory	
vwAttExtents	Pair of AcGePoint3d		Read-only
vwAttSection	See separate table	mandatory	
vwAttLabel	Any string	mandatory	
vwAttOrientation	kBase, AcGeMatrix		Read-only
vwAttOrientnPlane	A geometry key to a reference plane or an AcGePlane3d	mandatory	Write-once
vwAttOrientnAxis	A geometry key to a reference line or an AcGeLine3d	mandatory	Write-once
vwAttOrientnAxisDir	kXAxisPos, kXAxisNeg, kYAxisPos, kYAxisNeg	mandatory	
vwAttModelEnts	A single scene key or a list of keys to parts and solids	mandatory	Write-once

vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

The scale for ortho-views is locked into the scale of the parent. The API permits the scale to be altered within the descriptor, but any changes made to the scale will not be reflected in the view when the descriptor is applied in a view-creation or edit operation.

Table 12: View Attributes: kOrthoVw

vwAttViewType	**kOrthoVw**	mandatory	Write-once
vwAttDisplayType	kWireFrame, kHiddenTangDisp, kHiddenNoTangDisp		
vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded		
vwAttScale	Any positive double		Read-only
vwAttScaleType	kVwSclAbs, kVwSclRel		Read-only
vwAttLocation	Any AcGePoint3d	mandatory	
vwAttExtents	Pair of AcGePoint3d		Read-only
vwAttSection	See separate table		
vwAttLabel	Any string	mandatory	
vwAttOrientation	AmiOrthoOrientation (kLeft, kRight, kTop, kBottom)	mandatory	
vwAttParent	AmiDwVwKey	mandatory	Write-once
vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

Table 13: View: kIsoVw

vwAttViewType	**kIsoVw**	mandatory	Write-once
vwAttDisplayType	kWireFrame, kHiddenTangDisp, kHiddenNoTangDisp		
vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded		
vwAttScale	Any positive double	mandatory	
vwAttScaleType	kVwSclAbs, kVwSclRel	mandatory	
vwAttLocation	Any AcGePoint3d	mandatory	
vwAttExtents	Pair of AcGePoint3d		Read-only
vwAttSection	See separate table		
vwAttLabel	Any string	mandatory	
vwAttOrientation	AmiIsoOrientation (kTopRight, kBottomLeft, kTopLeft, kBottomRight)	mandatory	
vwAttParent	AmiDwVwKey	mandatory	Write-once
vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

For detail-views the vwAttRefPoint1 attribute is used represent the reference point for the "center of focus" of the detailed area. This can be any point reference on the model-space entities of the parent view. The vwAttBoxUL and vwAttBoxUL attributes represent the detailed area. The detailed area is always a rectangle and is represented as X,Y offsets relative to the reference point (vwAttRefPoint1). The detailed area need not include the reference point.

Table 14: View Attributes: kDetVw

vwAttViewType	**kDetVw**	mandatory	Write-once
vwAttDisplayType	kWireFrame, kHiddenTangDisp, kHiddenNoTangDisp		

vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded		
vwAttScale	Any positive double	mandatory	
vwAttScaleType	kVwSclAbs, kVwSclRel	mandatory	
vwAttLocation	Any AcGePoint3d	mandatory	
vwAttExtents	Pair of AcGePoint3d		Read-only
vwAttSection	See separate table		
vwAttLabel	Any string	mandatory	
vwAttRefPoint1	AmiObjectKey to reference point	mandatory	Write-once
vwAttBoxUL	Pair of X,Y offsets from vwAttRefPoint1 to upper-left of detail box.	mandatory	
vwAttBoxLR	Pair of X,Y offsets from vwAttRefPoint1 to lower-right of detail box.	mandatory	
vwAttParent	AmiDwVwKey	mandatory	Write-once
vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

For aux-views, the vwAttRefPoint1 and vwAttRefPoint2 attributes are used hold the reference points that define the axis of rotation for the view orientation. These are reference points on the model-space entities of the parent view. The ref-point representation of the orientation cannot be used in conjunction with the work-plane representation (vwAttOrientnPlane). An aux-view descriptor can hold either one representation or the other, but not both. The AmiOrthoOrientation representation of the vwAttOrientation attribute is used to convey the direction of rotation. The extent of rotation is always 90 degrees.

Table 15: View Attributes: kAuxVw

vwAttViewType	**kAuxVw**	mandatory	Write-once
vwAttDisplayType	kWireFrame, kHiddenTangDisp, kHiddenNoTangDisp		
vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded		
vwAttScale	Any positive double		Read-only
vwAttScale	Any positive double		Read-only
vwAttScaleType	kVwSclAbs, kVwSclRel		Read-only
vwAttLocation	Any AcGePoint3d	mandatory	
vwAttExtents	Pair of AcGePoint3d		Read-only
vwAttSection	See separate table		
vwAttLabel	Any string	mandatory	
vwAttOrientation	AmiOrthoOrientation (kLeft, kRight, kTop, kBottom)	mandatory	
	AcGeMatrix		Read-only
vwAttRefPoint1	AmiObjectKey to reference point		Write-once
vwAttRefPoint2	AmiObjectKey to reference point		Write-once
vwAttOrientnPlane	AmiObjectKey to work-plane		Write-once
vwAttParent	AmiDwVwKey	mandatory	Write-once
vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

At present, the API does not support creation of broken views. A broken view is a type of base view and therefore has no vwAttParent. The vwAttGap attribute holds the size of the break in the broken view.

Table 16: View: kBrokenVw

vwAttViewType	**KBrokenVw**	N/A	
vwAttDisplayType	kWireFrame, kHiddenTangDisp, kHiddenNoTangDisp	N/A	

vwAttAHLdisplayOn	kTrue, kFalse, kHiddenLinesDisplayed, kHiddenLinesNotDisplayed, kHiddenLinesDiscarded	N/A	
vwAttScale	Any positive double	N/A	
vwAttScaleType	kVwSclAbs	N/A	Read-only
vwAttLocation	Any AcGePoint3d	N/A	
vwAttExtents	Pair of AcGePoint3d	N/A	Read-only
vwAttSection	See separate table		
vwAttLabel	Any string	Mandatory	
vwAttOrientation	AcGeMatrix	N/A	Read-only
vwAttGap	Any positive double	N/A	
vwAttLayout	Any Mechanical Desktop layout.	Optional: will default to current layout	
vwAttShowParDims	kTrue, kFalse		
vwAttShowParDims	kTrue, kFalse		

If an API call is made with a descriptor type and attribute type combination that is not listed in the above tables, then and error status is returned and no action is taken (the error status is eNotApplicable, or in some cases, eAttTypeMismatch).

Sectioning information can be applied to base-views and ortho-views. For other types of views, the sectioning settings from the parent view are used in the child view. For those views, only the hatch flags (vwAttHatchAHL, vwAttHatchOnFlag) can be changed independently of the parent view. All other section settings in a descriptor used in a view-creation or edit operation on these other view types should agree with the parent view section or else be set to NAV.

The vwAttSectionType attribute must be set (i.e. cannot be NAV) to create new views. However, the value can be kNone, in which case the view is unsectioned and all the other sectioning attributes listed in Table 17: Drawing Manager View Section Types should be NAV.

The table below lists the attributes for each type of section in Drawing Manager views. Most of the sectioning information is straightforward. However, the representation of cutting information varies depending upon the type of section. Full-section cutting planes can be represented as a single reference point or as a plane. Half-section cutting planes can be represented as a single point or as a pair of planes. Offset-sections, aligned-sections and breakout-sections use a cutting-line, which is passed as a sketch key.

Table 17: Drawing Manager View Section Types

Section Type	Attribute Name	Allowable Values	Role	Settable?	Comments
Unsectioned	vwAttSectionType	**kNone**	mandatory		
Any type of section other than kNone	vwAttHatchOnFlag	kTrue, kFalse			
	vwAttHatchAHL	kTrue, kFalse			
	vwAttSectionLabel	Any string			
Full section	vwAttSectionType	**kSecFull**	mandatory	Write-once	
	vwAttSectionCut	AcGePoint3d, A point key	mandatory	Write-once	The point must be set as a reference point using a geometry key. It can be queried as AcGePoint3d or as a key. Queries return NAV if the cut is a plane instead of a point.
		AcGePlane, A plane key			The plane can be set as a fixed AcGePlane or as a geometry key to a reference plane. The plane can always be queried as AcGePlane. Queries return NAV if the cut is a point instead of a plane.
Half section	vwAttSectionType	**kSecHalf**	mandatory	Write-once	

	vwAttSectionCut	AcGePoint3d, A point key	mandatory	Write-once	Handled in the same manner as a point-cut for a full-section.
		Two AcGePlane, Two Plane keys			A pair of planes, either fixed AcGePlane's or reference planes passed as geometry keys.
	vwAttSectionCutDir	kCutRight, kCutLeft	mandatory		Direction of half-cut cutting line extension.
Offset section	vwAttSectionType	**kSecOffSet**	mandatory	Write-once	
	vwAttSectionCut	Cutting-line sketch	mandatory	Write-once	Cutting-line is passed as a sketch key.
Aligned section	vwAttSectionType	**kSecAligned**	mandatory	Write-once	
	vwAttSectionCut	Cutting-line sketch	mandatory	Write-once	Cutting-line is passed as a sketch key.
Break-out section	kSecBreakOut	Cutting-line sketch, AcGePlane, plane key	mandatory	Write-once	Cutting-line is passed as a sketch key. Both a cutting line and a plane specification must be used.
Radial section	kSecRadial	AcGePoint, point key, AcGePlane, plane key	mandatory	Write-once	Both a plane specification and point specification are needed.

For a descriptor with vwAttSectionType set to kSecNone, all other section attributes should be NAV. Note that sectioning information is mandatory for creation of base views. Even for unsectioned base views, the vwAttSectionType attribute should be set explicitly to kSecNone. For child views (e.g. ortho views), sectioning information is not mandatory for view creation. If the vwAttSectionType is NAV when creating child views, the view will have the same sectioning as the parent.

AmiStatus amiAddDataArray

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewAttribute** *attr*, **int** *numValues*, **<Input Type>*** *values***)**

Adds the given array to the current value of the given attribute within the descriptor. If the current value of the given attribute is NAV, then eAttValueNAV is returned. This function is overloaded for all input types needed to support the current set of multi-valued view attributes where union/append is meaningful. This type is:
AmiObjectKey
The keywords for attributes are defined by the AmiViewAttribute enum. The following table lists the attribute keywords, the datatype, and brief descriptions.

vwAttModelEnts	AmiObjectKey	Model entities

Input

pViewDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).
numValues	The length of the array.
values	The array of new values for the specified attribute.

AmiStatus amiCreateDwView

(AmiDwVwDescrip* *pViewDescrip*, **AmiDwVwKey*&** *pNewViewKey***)**

Create a new view from the descriptor.

NOTE: This function cannot be used in a "transparent" transaction. It should be used in modal transactions. Broken views are not yet supported by this function. It is the caller's responsibility to release the newly created key.

Input

pViewDescrip	A pointer to the descriptor to be used.

Output

pNewViewKey A key for the newly created view.

AmiStatus amiCreateDwVwDescrip

(AmiViewType *viewType*, **AmiDwVwDescrip*&** *pViewDescrip*, **const char*** *appName* = **NULL)**

Returns an empty view descriptor of the given type. An "empty" view descriptor is a descriptor where all values are NAV. It is the caller's responsibility to release the descriptor.

Input

viewType The type for the view descriptor.

appName Name of related application. If NULL is specified then a default application will be used.

Output

pViewDescrip A pointer to an empty view descriptor.

AmiStatus amiDwEditView

(AmiDwVwKey* *pViewKey*, **AmiDwVwDescrip*** *pViewDescrip* **)**

Apply the view descriptor to the given view. The view is updated to reflect the values in the descriptor. Any slots in the view that are NAV are ignored (i.e. treated as "don't care"). Any slots that are read-only, are not NAV, and are different from the current values for the view, will result in an error status (*eAttReadOnly*).

Note To refresh the screen and bring all dependent entities up to date, calls to this function should be followed by AMUPDATE.

Input

pViewKey A pointer to a key for the view to be edited.

PViewDescrip A pointer to the descriptor to be applied.

AmiStatus amiDwVwDescripIsKindof

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewType** *viewType*, **Adesk::Boolean&** *fIsKindOf* **)**

Returns *kTrue* if and only if the given descriptor is of the given type.

Input

pViewDescrip A pointer to a view descriptor.

viewType A view descriptor type.

Output

fIsKindOf A flag indicating if the descriptor is of the given type.

AmiStatus amiFreezeLayers

(AmiDwVwKey* *pView*, **int** *numLayers*, **AcDbOjectId*** *pLayers***)**

Freeze (hide) the specified layers within the specified view.

Input

pView The view for which the layers are to be frozen.

numLayers The number of layers in pLayers array.

pLayers An array of object ids for layers.

AmiStatus amiGetAllDwVws

(int& *numViews*, **AmiDwVwKey**&** *pViews,* **AmiDwLayoutKey*** *pLayout* **= NULL)**

Returns a view key for each view in the current drawing. It is the caller's responsibility to release all keys in the array, and then delete the array.

Input

pLayout	Optional. If non-null, then the output will be confined to the given layout. Otherwise, all active views on all layouts are returned.

Output

numViews	The size of the array
pViews	An array of pointers to newly created keys for all active views.

AmiStatus amiGetData

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewAttribute** *attr*, **<Return Type>&** *value* **)**

Extracts an attribute value from the given view descriptor. This function is overloaded for all return types needed to support the current set of view attributes. These types are:

double
char*
AmiViewType
AmiSectionType
AmiSectionCutDir
AmiHiddenLineTreatment
AmiAxisDirType
Adesk::Boolean
AmiViewDisplayType
AmiOrientationType
AmiViewScaleType
AcGePoint3d
AcGeMatrix3d
AcGeEntity3d
AmiObjectKey*
AcDbObjectID

The keywords for attributes are defined by the *AmiViewAttribute* enum. The View and Annotation Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. Attributes that take on array values are handled by a separate function (*amiGetDataArray*). If the value for the attribute is NAV, meaning "Not A Value", then *eAttValueNAV* is returned as the AmiStatus and the content of the 'value' is undefined. It is the caller's responsibility to free, as appropriate, any returned value.

Input

pViewDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).

Output

value	The value for the specified attribute.

AmiStatus amiGetDataArray

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewAttribute** *attr*, **int&** *numValues*, **<Return Type>*&** *values***)**

Extracts an array of values for a multi-valued attribute. This function is overloaded for all return types needed to support the current set of view attributes. These types are:

AmiObjectKey*

AcGePoint3d*

AcGeEntity3d*

double

The keywords for attributes are defined by the *AmiViewAttribute* enum. The View and Annotation Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. This function supports any of those attributes which have an array datatype. At present those attributes are vwAttModelEnts, vwAttExtents, vwAttRefBoxUL, vwAttRefBoxLR, and vwAttSectionCut. If the value for the attribute is NAV, meaning "Not A Value", then *eAttValueNAV* is returned as the AmiStatus and the content of the 'value' is undefined. It is the caller's responsibility to delete the array. It is the caller's responsibility to free, as appropriate, any returned value.

Input

pViewDescrip A pointer to the descriptor.

attr Keyword for attribute (passed as an enum).

Output

numValues The length of the array.

values The array of values for the specified attribute.

AmiStatus amiGetDwVwDescrip

(AmiDwVwKey* *pViewKey*, **AmiDwVwDescrip*&** *pViewDescrip* **)**

Returns a descriptor for the given view. The descriptor holds information on the current state of the view. The descriptor is de-coupled from the view (it does not change as the view changes). It is the caller's responsibility to release the descriptor.

Input

pViewKey A key for a view.

Output

pViewDescrip A pointer to the descriptor.

AmiStatus amiGetDwVwModelObjs

(AmiDwVwKey* *pViewKey*, **int&** *size*, **AmiObjectKey**&** *pKeys* **)**

Returns an object key for each modeling object in the given view. The object keys returned can be part keys, scene keys, or body keys. It is the caller's responsibility to release the keys in the array and to delete the array.

Input

pViewKey A key for the view for which to create object keys.

Output

size The size of the array.

pKeys An array of pointers to keys for all of the model entities for the view.

AmiStatus amiGetDwVwParent

(AmiDwVwKey* *pViewKey*, **AmiDwVwKey*&** *pParentKey* **)**

This function returns a key to the parent view of the given view. If the given view is a base view, which has no parent, then a error status is returned (*eBaseView*). It is the caller's responsibility to release the key returned.

Input

pViewKey A pointer to a view key.

Output

pParentKey A pointer to a key for the parent of the view.

AmiStatus amiGetNumDwVws

(int& *numViews*, **AmiDwLayoutKey*** *pLayout* **= NULL)**

Returns the number of active views on the given layout (also called a sheet).

Input

pLayout	Optional. If non-null, then the output will be confined to the given layout. Otherwise, all active views on all active layouts are counted.

Output

numViews	The count.

AmiStatus amiSetData

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewAttribute** *attr*, **<Input Type>** *value* **)**

Sets an attribute value from the given view descriptor. This function is overloaded for all input types needed to support the current set of view attributes. These types are:
double
char*
AmiViewType
AmiSectionType
AmiViewDisplayType
AmiSectionCutDir
AmiHiddenLineTreatment
AmiAxisDirType
Adesk::Boolean
AmiOrientationType
AmiViewScaleType
const AcGePoint3d&
AcGeEntity3d
const AcGeMatrix3d&
AmiObjectKey*
AcDbObjectID
The keywords for attributes are defined by the *AmiViewAttribute* enum. . The View and Annotation Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. Attributes that take on array values are handled by a separate function (amiGetDataArray). In addition, the special value Ami:att_NAV can be used to set the specified attribute to "Not A Value". NAV attributes are treated as "unset" or "don't care" values when applying the descriptor to a view in *amiDwEditView*().

Input

pViewDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).
value	The value for the specified attribute.

AmiStatus amiSetDataArray

(AmiDwVwDescrip* *pViewDescrip*, **AmiViewAttribute** *attr*, **int** *numValues*, **<Input Type>*** *values***)**

Sets an attribute value from the given view descriptor. This function is overloaded for all input types needed to support the current set of view attributes. These types are:

AmiObjectKey*

AcGePoint3d*

AcGeEntity3d*

double

The keywords for attributes are defined by the *AmiViewAttribute* enum. The View and Annotation Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. This

function supports any of these attributes which have an array datatype. At present those attributes are vwAttModelEnts, vwAttExtents, vwAttRefBoxUL, vwAttRefBoxLR, and vwAttSectionCut. In addition, the special value AmiDwAtt_NAV can be used to set the specified attribute to "Not A Value". NAV attributes are treated as "unset" or "don't care" values when applying the descriptor to a view in *amiDwEditView*().

Input

pViewDescrip	A pointer to the descriptor.
attr	keyword for attribute (passed as an enum).
numValues	The length of the array.
values	The array of values for the specified attribute.

AmiStatus amiThawLayers

(AmiDwVwKey* *pView*, **int** *numLayers*, **AcDbObjectId*** *pLayers*)

Thaw (display) the specified layers within the specified view.

Input

pView	The view for which the layers are to be thawed.
numLayers	The number of layers in pLayers array.
pLayers	An array of object ids for layers.

Annotations

The following table lists all of the annotation attributes currently defined in the Drawing Manager, along with a brief description of each. New keywords will be added to this list as the set of Drawing Manager attributes supported by the MCAD API grows.

Table 18: Annotation Attribute Keywords

Attribute Keyword	DataType	Read-Only	Applies To	Description
annAttAnnotType	AmiAnnotType	Y	All	The type of descriptor (at present, most descriptors return the generic type kNoteGeneral).
annAttAttachPt	AmiObjectKey		Dimensions, User-defined	The reference geometry in model-space for the attachment point of the annotation. All annotations have at least one attachment point.
annAttLocation	AcGePoint3d		Dimensions	The location of the annotation. This attribute is applicable to dimension annotations only, where it is the location of the dimension text.
annAttComputedVal	double	Y	Dimensions	The value for a dimension annotation.
annAttMeasuredVal	double	Y	Dimensions	The scaled value for a dimension annotation (scaled by the scale factor of the view).
annAttContent	AcDbObjectId array		Dimensions, User-defined	The underlying ACAD entities in the annotation.
annAttGeometry	AcDbObjectId array, AmiGeomKey array		User-defined	The geometry in the annotation. Whereas annAttContent refers to the entire content of the annotation, annAttGeometry refers only to that subset which is geometry. This subset can be read and set independently of the non-geometry subset of annAttContent.
annAttBorder	AcGeEntity3d, AmiObjectKey	Y	Detail Symbol	The “call-out” geometry in parent view for detail views.

AmiStatus amiCreateDwAnnot

(AmiDwAnnotDescrip* *pDescrip*, **AmiDwVwKey*** *pView*, **AmiDwAnnotKey*&** *pAnnotKey*)

Creates a new annotation and adds it to the database. A key to the new annotation is returned. The annotation remains empty; geometry can be added via other functions. This function is limited to AcDmUserDefnNote only. It is the caller's responsibility to release the newly created key.

Input

pDescrip A point to an annotation descriptor holding arguments for the creation operation.

pView The view in which to place the annotation.

Output

pAnnotKey A pointer to a key for the new annotation.

AmiStatus amiCreateDwAnnotDescrip

(AmiAnnotType *annotType*, **AmiDwAnnotDescrip*&** *pAnnotDescrip*, **const char*** *appName* = **NULL)**

Returns an empty annotation descriptor of the given type. An "empty" annotation descriptor is a descriptor where all values are NAV. It is the caller's responsibility to release the descriptor.

Input

annotType The type for the descriptor.

appName Name of related application. If NULL is specified then a default application will be used.

Output

pAnnotDescrip A pointer to an empty descriptor.

AmiStatus amiDwAnnotDescripIsKindof

(AmiDwAnnotDescrip* *pAnnotDescrip*, **AmiAnnotType** *annotType*, **Adesk::Boolean&** *fIsKindOf* **)**

Returns *kTrue* if and only if the given descriptor is of the given type.

Input

pAnnotDescrip A pointer to a annotation descriptor.

annotType An annotation descriptor type.

Output

fIsKindOf A flag indicating if the descriptor is of the given type.

AmiStatus amiDwEditAnnot

(AmiDwannotKey* *pAnnotKey*, **AmiDwAnnotDescrip*** *pAnnotDescrip* **)**

Apply the annotation descriptor to the given view. The view is updated to reflect the values in the descriptor. Any slots in the view that are NAV are ignored (i.e. treated as "don't care"). Any slots that are read-only, are not NAV, and are different from the current values for the view, will result in an error status (*eAttReadOnly*).

Note To refresh the screen and bring all dependent entities up to date, calls to this function should be followed by AMUPDATE.

Input

pAnnotKey A pointer to a key for the annotation to be edited.

pAnnotDescrip A pointer to the descriptor to be applied.

AmiStatus amiGetData

(AmiDwAnnotDescrip* *pAnnotDescrip*, **AmiAnnotAttribute** *attr*, **<Return Type>&** *value* **)**

Extracts an attribute value from the given annotation descriptor. This function is overloaded for all return types needed to support the current set of annotation attributes. The keywords for attributes are defined by the *AmiAnnotAttribute* enum. The View and Annotation Attribute Keywords Table lists the attribute keywords, the datatype, and brief descriptions. If the value for the attribute is NAV, meaning "Not A Value", then *eAttValueNAV* is returned as the *AmiStatus* and the content of the 'value' is undefined. It is the caller's responsibility to free, as appropriate, any returned value.

Input

pAnnotDescrip	A pointer to the descriptor.
attr	The keyword for attribute (passed as an enum).

Output

value	The value for the specified attribute.

AmiStatus amiGetDwAnnotDescrip

(AmiDwAnnotKey* *pAnnotKey*, **AmiDwAnnotDescrip*&** *pAnnotDescrip* **)**

Returns a descriptor for the given annotation. The descriptor holds information on the current state of the annotation. The descriptor is de-coupled from the annotation (it does not change as the annotation changes). It is the caller's responsibility to release the descriptor.

Input

pAnnotKey	A key for an annotation.

Output

pAnnotDescrip	A pointer to the descriptor.

AmiStatus amiGetDwAnnotView

(AmiDwAnnotKey* *pAnnotKey*, **AmiDwViewKey*&** *pOwnerView* **)**

Returns a key for the view to which the given annotation is attached. It is the caller's responsibility to release the key.

Input

pAnnotKey	A pointer to an annotation key.

Output

pOwnerView	A pointer to a new key for the associated view.

AmiStatus amiGetDwVwAnnots

(AmiDwVwKey* *pViewKey*, **AmiAnnotType** *annotType*, **int&** *numNotes*, **AmiDwAnnotKey**&** *pNotes* **)**

Returns an annotation key for each annotation of the given type in the view. *AmiAnnotType* includes a general type that matches all annotation types so this function can be used to get all annotations in the view. It is the caller's responsibility to release all keys in the array and delete the array.

At present, filtering for specific types of annotations in not supported and the annotType argument must be kNoteGeneral.

Input

pViewKey	A view key for the view.
annotType	The type of annotations to be selected.

Output

numNotes	The size of the array returned.
pNotes	An array of ptrs to note keys for this view.

AmiStatus amiMoveDwAnnot

(AmiDwAnnotKey* *pAnnotKey*, **const AcGeVector2d&** *displacement***)**

Moves the annotation by the given displacement vector. Not all annotations can be moved in this way. For annotations that cannot be moved freely, eNotApplicable is returned.

Input

pAnnotKey A pointer to an annotation key.

displacement 2D vector for translation in paper-space.

AmiStatus amiSetData

(AmiDwAnnotDescrip* *pAnnotDescrip*, **AmiAnnotAttribute** *attr*, **<Input Type>** *value* **)**

Sets an attribute value from the given annotation descriptor. This function is overloaded for all input types needed to support the current set of view attributes. The keywords for attributes are defined by the *AmiAnnotAttribute* enum. The View and Annotation Attribute Keywords Table on page 176 lists the attribute keywords, the datatype, and brief descriptions. In addition, the special value Ami:att_NAV can be used to set the specified attribute to "Not A Value". NAV attributes are treated as "unset" or "don't care" values when applying the descriptor to a view in *amiDwEditView()*.

Input

pViewDescrip A pointer to the descriptor.
attr Keyword for attribute (passed as an enum).
value The value for the specified attribute.

Scenes

At present, support for scenes is limited. The explosion factor and the component definition for the scene can be obtained from a scene key. Basic properties for components within scenes (e.g. position and visibility) can be obtained from a descriptor for scene-components. The scene-component descriptor, AmiSceneCompDescrip, is similar to view descriptors and annotation descriptors. The same semantics for NAV is used here.

The following table lists the attributes currently defined for scene-components, along with a brief description of each. New keywords will be added to this list as the set of attributes supported by the MCAD API grows.

Table 19: Scene-Comp Attribute Keywords

Attribute Keyword	Data Type	Read -Only	Description
scpAttTransform	AcGeMatrix	Y	The transformation matrix describing the position and orientation of the component within the scene. For an untweaked component, this matrix will reflect the position of the component within the assembly, as returned by amiGetCompPosition(). For a tweaked component, this matrix will reflect the combined transformation of all of the tweaks on that component.
scpAttIsSuppressed	Adesk::Boolean		The visibility flag for the component in the scene.
scpAttExplodeFactor	double		The explosion factor for the individual component. If negative, then the general explosion factor for the scene will be used.

AmiStatus amiCreateScene

(AmiCompDefKey* *pCompDefKey*, **const char*** *sceneName,* **AmiSceneKey*&** *pSceneKey***)**

Creates a new scene for the given component definition and return a key to the scene. It is the caller's responsibility to release the new scene key.

Input

pCompDefKey Key to a component definition for the new scene.
sceneName Name for the new scene.

Output

pSceneKey Key to the new scene.

AmiStatus amiCreateSceneCompDescrip

(AmiSceneCompDescrip*& *pDescrip***, const char*** *appName* **= NULL)**

Creates an "empty" scene component descriptor. All of the attribute values in the newly created descriptor are attNAV.

Input

appName Application name for delegation.

Output

pDescrip A pointer to the new descriptor.

AmiStatus amiEditSceneComp

(AmiSceneKey* *pSceneKey***, AmiCompKey*** *pCompKey***, AmiSceneCompDescrip*** *pDescrip* **)**

Editing function for scene components. The component key is a modeling key. The corresponding entity within the scene is edited, using the attribute values in the given descriptor. If there is no instance of the given component in the scene, an error status is returned. If an attempt to edit a read-only attribute is made, an error status is returned.

Input

pSceneKey A scene key.

pCompKey A modeling component key.

pDescrip A descriptor holding the new values.

Output

none

AmiStatus amiGetAllScenes

(int& *numKeys***, AmiSceneKey**&** *pKeys***, char** *appName* = **NULL)**

Retrieves all the scenes that currently exist in the database.

Input

appName Name of the application to perform the function (optional).

Output

numKeys Number of keys being returned.

pKeys A pointer to an array of pointers to scene keys.

AmiStatus amiGetData

(AmiSceneCompDescrip* *pDescrip***, AmiSceneCompAttribute** *attr***, <Return Type>&** *value***)**

Extracts an attribute value from the given descriptor. This function is overloaded for all return types needed to support the current set of attributes. These types are:

double

char*

Adesk::Boolean (int)

AcGeMatrix3d

See table for a list of the scene-comp attributes and the corresponding data types.

Input

pDescrip A pointer to the descriptor.

attr Keyword for attribute (passed as an enum).

Output

value The value for the specified attribute.

AmiStatus amiGetSceneCompDef

(AmiSceneKey* *pSceneKey*, **AmiCompDefKey*&** *pCompDefKey* **)**

Returns a pointer to a key for the CompDef for the given scene. It is the caller's responsibility to release the CompDef key returned.

Input

pSceneKey A pointer to a scene key.

Output

pCompDefKey A pointer to a new key for the CompDef of the assembly in the scene.

AmiStatus amiGetSceneCompDescrip

(AmiSceneKey* *pSceneKey*, **AmiCompKey*** *pCompKey*, **AmiCompInSceneDescrip*&** *pDescrip***)**

Creates a descriptor object holding scene component attribute values. The descriptor is de-coupled from the scene and component; it can be edited independently and applied to any component in any scene.

Input

pSceneKey A key for the scene.

pCompKey A key to a model-space component.

Output

pDescrip A pointer to a descriptor for the given view. It is expected that the component corresponds to a component in the compdef for the scene. An error status is returned if it isn't.

AmiStatus amiGetSceneExplodeFactor

(AmiSceneKey* *pSceneKey*, **double&** *factor***)**

Gets the explosion factor to the scene as a whole.

Input

pSceneKey Key to a scene.

Output

factor Explosion factor.

AmiStatus amiGetSceneLock

(AmiSceneKey* *pSceneKey*, **Adesk::Boolean&** *flag***)**

Gets the locking status of the scene.

Input

pSceneKey Key to a scene.

Output

flag kTrue if scene is currently locked, else kFalse.

AmiStatus amiSetData

(AmiSceneCompDescrip* *pDescrip*, **AmiSceneCompAttribute** *attr*,**<Input Type>** *value***)**

Sets an attribute value from the given descriptor. This function is overloaded for all input types needed to support the current set of view attributes. These types are:

double,

char*,

Adesk::Boolean (int),

const AcGeMatrix3d&

See table for a list of the scene-comp attributes and the corresponding data types.

Input

pDescrip	A pointer to the descriptor.
attr	Keyword for attribute (passed as an enum).
value	The value for the specified attribute.

AmiStatus amiSetSceneLock

(AmiSceneCompDescrip* *pSceneKey**, **Adesk::Boolean** *flag* **)**

Sets the locking status of the scene.

Input

pSceneKey	Key to a scene.

Output

flag	If kTrue then the scene will be locked; else it will be unlocked.

Hidden Line (AHL) Calculation

Hidden line (or AHL, for Accurate Hidden Line) calculation is the determination of which lines or edges are covered by other lines or faces in a solid. It is used when making a 2D view of a 3D part. The MCADAPI allows for customization of AHL calculations by allowing external applications to alter the input into the AHL calculations without changing the logical content of Drawing Views. The edits to AHL input can include: hiding 3D entities that are already in the logical content of the view; adding 2D entities; and substituting 2D entities for 3D entities.

The alternate geometry must 2D: AcDbRegion, AcDbCurve, AcDbLine, AcDbArc, etc. Furthermore, these entities must be in model space.

An example of how AHL customization might be used is the substitution of standard 2D drafting schematics for common parts. For example, a 3D screw might be replaced by the common 2D representation, with simplified threads consisting of disconnected straight line segments. This will improve performance as well as allow for use of drafting standards in plotted views.

External applications can use event notification to receive notification when Drawing Views are regenerated (i.e. when AHL calculations are performed). See “Event Handling” on page 225. This allows for edits to AHL settings immediately prior to the AHL calculations.

AmiStatus amiClearCustomAHL

(AmiAppId *appId***, AmiDwVwKey*** *pKey* **)**

Clears the custom AHL settings for the view. All custom AHL settings for the specified view are cleared.

Input

appId	Registered ID for the (external) application that is controlling the AHL settings. You get an application id by calling amiRegisterApp.
pKey	A pointer to a key on the view.

(AmiAppId *appId***, AmiDwVwKey*** *pVwKey***, AmiObjectKey*** *pObjKey* **)**

Clears the hidden line calculation settings for the specified object in the specified view. The third argument can be NULL. The custom hidden line settings normally apply to specific objects in the views. However, for the purposes of hidden line customization, every view has one "null" object and hidden line settings can be applied to these "null" objects. This is useful when there is custom geometry to be added to a view, where the custom geometry does not "replace" any model geometry.

Input

appId Registered ID for the (external) application that is controlling the hidden line calcuation settings. You get an application id by calling amiRegisterApp.

pKey A pointer to a key on the view.

pObjKey A pointer to a component in the view, or NULL.

AmiStatus amiGetCustomAHL

(AmiAppId *appId*, **AmiDwVwKey*** *pDwVwKey*, **AmiObjectKey*** *pObjKey*, **Adesk::Boolean&** *bSectioning*, **int&** *numGeoms*, **AmiGeomKey**&** *paGeomKeys*, **int&** *numCenterlines*, **AcGeEntity3d**&** *paCenterlines***)**

This function gets the recorded custom AHL settings for the given object in the given view.

Input

appId Registered ID for the (external) application that is controlling the AHL settings.

pDwVwKey A pointer to a key to the view in question.

pObjKey A pointer to the object in question. The object is a model-space object associated with the view (i.e. it is part of the view "content"). A NULL pointer can be used to obtain any custom AHL settings that apply to the view in general, and not to any particular object in the view.

Output

bSectioning True if the custom geometry participates in sectioning (if the view is sectioned).

numGeoms The length of the *paGeomKeys* array.

paGeomKeys An array of pointers to geometry keys to the custom geometry.

numCenterlines The length of the *paCenterlines* array.

paCenterlines An array of geometry for custom centerlines to be associated with the replacement geometry in the *paGeomKeys* argument.

AmiStatus amiSetCustomAHL

(AmiAppId *appId*, **AmiDwVwKey*** *pDwVwKey*, **AmiObjectKey*** *pObjKey*, **Adesk::Boolean** *bSectioning*, **int** *numGeoms*, **AmiGeomKey**** *paGeomKeys*, **int** *numCenterlines*, **AcGeEntity3d**** *paCenterlines***)**

This function records custom AHL settings for the given object in the given view. Since Detail-Views use the same graphics as the parent view, AHL settings cannot be customized for Detail-Views independently of the parent; keys to Detail-Views are rejected as input to this function. Any custom centerlines geometry should be in model WCS.

Input

appId Registered ID for the (external) application that is controlling the AHL settings.

pDwVwKey A pointer to a key to the view in question.

pObjKey	A pointer to the object in question. The object is a model-space object associated with the view (i.e. it is part of the view "content"). A NULL pointer can be used if the AHL settings apply to the view in general, and not to any particular object in the view.
bSectioning	If the view is a sectioned view and if this argument is true, then the custom geometry will participate in the sectioning.
numGeoms	The length of the *paGeomKeys* array.
paGeomKeys	An array of pointers to geometry keys.
numCenterlines	The length of the *paCenterlines* array.
paCenterlines	An array of pointers to centerline geometry. It is expected that the centerline geometry will consist of AcGeLineSeg3d and AcGeCircArc3d (for bull's-eyes). Limitations: The custom geometry must be 2D. The custom geometry must be database-resident. The custom geometry must be in model-space.

Layouts

AmiStatus amiGetAllDwLayouts

(int& *numKeys***, AmiDwLayoutKey**&** *pKeys***, const char*** *appName* **= NULL)**

Returns a layout key for each active layout, along with the size of the array.

Input

appName	Optional. Application name for delegation.

Output

numKeys	The size of the array.
pKeys	Array of pointers to keys for all the active layouts.

User Interface Functions

The following functions are used to manipulate the user interface in the Mechanical Desktop.

Browser Display

The following functions allow you to determine whether the browser is currently being displayed, and to turn display of the browser on or off.

AmiStatus amiIsBrowserDisplayed

(Adesk::Boolean& *fDisplay***, const char*** *appName* **= MApiSys)**

Returns a boolean stating whether or not the Mechanical Desktop browser is currently displayed.

Input

appName	Application to perform this action.

Output

fDisplay	If true the browser is on; if false, it is not.

AmiStatus amiDisplayBrowser

(Adesk::Boolean& *fDisplay* =**Adesk::kTrue**, **, const char*** *appName* **= MApiSys)**

Turns display of the Mechanical Desktop browser window on or off.

Input

fDisplay	If true turn on the browser. If false, turn it off.
appName	Application to perform this action.

Browser Customization

The following two functions allow an application to add or remove additional tabs to the Mechanical Desktop browser. By adding custom tabs to the browser, applications can integrate many of their user interface functions with those of the Desktop. As the browser has become the dominant way in which many users interface with the Desktop, adding custom tabs has become increasingly desirable for developers.

AmiStatus amiAddBrowserTab

(const char* *pTabTitle*, **CWnd*** *pPageWnd*, **AmiTabPageDisplay** *pageDisp*, **AcApDocument*** *pdoc* **= NULL, const char*** *appName* **= MApiSys)**

Adds a new tab to the browser.

Input

pTabTitle	Title string to be displayed on the tab.
pPageWnd	Handle to window that will be visible within the tab page. This window will be re-parented and subclassed when this call is made.
pageDisp	Specified when this page should be displayed.
pDoc	Document for which tab is to appear. If this parameter is NULL, the current document is used.
appName	Application to perform this action.

AmiStatus amiRemoveBrowserTab

(CWnd* *pPageWnd*, **AcApDocument*** *pdoc* **= NULL, const char*** *appName* **= MApiSys)**

Removes a tab from the browser.

Input

pPageWnd	Handle to window whose corresponding tab is to be removed from the browser.
pDoc	Document for which tab is to appear. If this parameter is NULL, the current document is used.
appName	Application to perform this action.

File Descriptor Functions

AmiStatus amiAttachFile

(AmiFileDescrip ***pFileToAttach)**

This function adds the file described by *pFileToAttach* as a dependent file. All PDM information is retrieved from the descriptor. The file descriptor must be managed (keyword kAttManaged is true).

Input

pFileToAttach	File descriptor describing the file to attach.

AmiStatus amiCreateFileDescrip

(AmiFileType *fileType*, **AmiFileDescrip*& const char*** *appName* **= Designer)**

Creates an empty file descriptor, with all attributes set to NAV. It is the caller's responsibility to free the descriptor.

AmiStatus amiGetData

(AmiFileDescrip* *pFileDescrip*, **AmiFileAttribute** *attr*, **<Return Type>&** *value* **)**

Extracts an attribute value from the given file descriptor. Returns a code indicating whether the value was successfully retrieved from the descriptor. May return eNAV, indicating there is no value currently set on that attribute. This function is overloaded for all return types needed to support the current set of file attributes. These types are:

Double

Adesk::Boolean

Int

char*

AmiSectionStructure

AmiVersionConfig

AmiDrivingData

AmiFileType

It is the caller's responsibility to free, as appropriate, any returned value.

Input

pFileDescrip	Pointer to the file descriptor to query
Keyword	Enumerator indicating attribute value to query

Output

value	The current value of the specified attribute, appropriately typed

AmiStatus amiGetFile

(AmiFileDescrip * *pFileToUpdate***)**

This function updates the file descriptor with the current path for the file described in *pFileToUpdate*. An error is returned if no corresponding file is registered.

Input

pFileToUpdate	File descriptor describing the file to update.

AmiStatus amiIsPDMRegistered

(Adesk::Boolean& *isRegistered***)**

This function indicates whether a PDM system has registered to handle file management functions.

Output

isRegistered	True if PDM is currently registered.

AmiStatus amiRemoveFile

(AmiFileDescrip * *pFileToRemove***)**

This function removes the file described by pFileToRemove as a dependent file. The correct attachment is determined by the information on the descriptor, including the owner id, and group name.

Input

pFileToRemove	File descriptor describing the file to remove.

AmiStatus amiSetData

(**AmiFileDescrip*** *pFileDescrip*, **AmiFileAttribute** *keyword*, **<type> char*** *value*)

Set an attribute value on the given file descriptor. Returns a value that indicates whether the value was successfully applied to the descriptor. This function is overloaded for all return types needed to support the current set of file attributes. These types are:

Double

Adesk::Boolean

Int

char*

AmiSectionStructure

AmiVersionConfig

AmiDrivingData

AmiFileType

Input

pFileDescrip	Pointer to the file descriptor to modify
Keyword	Enumerator indicating attribute value to modify
value	The new value of the specified attribute, appropriately typed

Attribute Accessors

File Descriptor attributes are all accessed using two functions *amiSetData* and *amiGetData*. The key word enumerator specifies which attribute is to be accessed. The following table enumerates the key words available and provides the supported data types.

Table 20: File Attribute Keywords

Attribute Keyword	DataType	Read-Only	Applies To	Description
kAttFileType	(int)	Write-Once	All	Type of file described
kAttFullPath	const char *		All	Path to specific file described
kAttOwningApp	Double	Y	All	Internal application ID of the application that manages this descriptor
kAttManaged	Int		All	Boolean value indicating if the file is managed by a PDM
kAttGroupName	const char *		Managed	PDM group name
kAttOwnerId	AcDbObjectId		Managed	ID of object that owns or controls this file. Must be unique within the group
kAttProperties	Double		Managed	PDM attribute
kAttVersionedObj	AmiObjectKey*		Versioning	Key to object being versioned by this file. If the global parameters are controlled this is a null key or is NAV.
kAttVersionDir	AmiVersionConfig		Versioning	Whether the versions are across rows or down columns. Must be the same for all sections
kAttHasInfoFor	AmiDrivingData		Versioning	What data is driven from this file, design variable, feature suppression…
kAttStructure	AmiSectionStructure		Versioning	Whether sections are separate or concatenated
kAttFeatSheet	const char *		Versioning	Feature suppression section sheet name
kAttFeatCell	const char *		Versioning	Feature suppression section starting cell (e.g. “A1”, or “B12”)
kAttParamSheet	const char *		Versioning	Parameter value section sheet name
KAttParamCell	const char *		Versioning	Parameter value section starting cell (e.g. “A1”, or “B52”)

If an API call is made with a descriptor type and attribute type combination that is not listed in the above tables, then an error status is returned and no action is taken (the error status is eNotApplicable, or in some cases, eAttTypeMismatch).

Table 21: File Attribute Keywords - continued

File Type	Attribute Name	Allowable Values	Role	Settable?	Comments
All	kAttFullPath	Any string	mandatory		Full path of the file described
	kAttManaged	KTrue, kFalse	optional		A value of kTrue indicates that the file is controlled by a PDM.
Managed	kAttGroupName	Any string	Mandatory		The group name the file should be registered with
	kAttOwnerId	Any object ID, unique within the group	Mandatory		The ID of the object that owns the group and can be used to uniquely identify the file within a group
	kAttProperties	Value established with the PDM as meaningful	Mandatory		This identifies a relationship type within the PDM system and must be established through the PDM's mechanism.
Versioning - Globals	kAttHasInfoFor	kParamData	optional		What kind of data is versioned by this table
	kAttVersionDir	KVersionsAcrossRows, kVersionsDownColumns	mandatory		Are the version names running down the first column, or across the first row
	kAttParamSheet	Any string	mandatory		The sheet name of the parameter section
	kAttParamCell	Any string	mandatory		The upper left corner cell of the parameter table
Versioning – Parts	kAttVersionedObj	Part Key	mandatory		Which part is controlled by this versioning file.
	kAttHasInfoFor	KParamData, KFeatData, kParamAndFeatData	mandatory		Which data is controlled by this versioning file.
	kAttStructure	kConcatenated,, kSeparate	mandatory if kParamAndFeatData		The sections are on a single table, concatenated together, or separate tables with individual starting cells
	kAttVersionDir	KVersionsAcrossRows, kVersionsDownColumns	mandatory		Are the version names running down the first column, or across the first row
	kAttFeatSheet	Any string	If and only if has **separate** feature data		Specifies sheet name if there is feature suppression on a separate table
	kAttFeatCell	Any string	Same as sheet		Specifies the upper left corner cell of the feature suppression table
	kAttParamSheet	Any string	If and only if has parameter data		The sheet name of the parameter section
	kAttParamCell	Any string	Same as sheet		The upper left corner cell of the parameter table

If a descriptor with an invalid combination of attributes is applied to a table link, the error eInvalidDescriptor is returned.

Appendix A
Sample Applications for the MCAD API

The MCAD API includes sample applications that illustrate how MCAD API functions can be used by client applications. These sample applications also allow experimentation with the MCAD API through the commands offered (without forcing the experimenter to write code). Using these sample applications as guidelines, you can move on to writing custom applications that use the MCAD API in various real-world settings.

- The comprehensive sample application, MAISAMP, contains commands that invoke all of the MCAD API functions. It is described below.
- The SmartHole sample illustrates how the original set of Mechanical Desktop features can be broadened through the creation of a custom feature that is built using existing features. A set of commands that work with this new feature is defined in the application. See page 109 for information about how to use it.
- The Hole Projection sample application, MAIPROJHOLE, demonstrates the combined use of the MCAD API, the Geometry Library, and the B-rep API. See page 186 for information about how to use it.
- The MfcBrepTrav sample application allows you to select and traverse an AutoCAD solid, Mechanical Desktop part or B-rep subentity (for example a face or an edge) and display information about how the object is structured in terms of boundary representation. This information is displayed in a tree structure (CTreeCtrl), and you can optionally select an element in this tree structure to highlight the corresponding element in the model and display more information about it. See page 156 for information about how to use it.
- The MyBrowser sample application demonstrates ways to extend and customize the Mechanical Desktop browser. See page 236 for information about how to use it.
- The CountAssemblies sample application counts the number of component definitions and instances and displays the result in a dialog box. See page 237 for information about how to use it.
- The Which_MDT sample application illustrates how to find out where the Mechanical Desktop has been installed and what version it is. See page 238 for information about how to use it.

Comprehensive Sample Application (MAISAMP)

The enclosed C++ sample test application *maisamp.arx* contains commands that invoke all the MCAD API functions. Any key that the MCAD API returns through these sample applications are eligible to be saved into the drawing database persistently (keys are saved with the drawing and can be accessed after a drawing is opened). The MCAD API functions *amiWriteKey()* and *amiReadKey()* are exercised differently so that no MCAD API sample application command exercises the functions. The two functions are used as part of the MaiObject, which is a custom object derived from AcDbObject and whose only purpose is to hold onto keys for saving. The MaiObjects are then saved in an extension dictionary defined by the sample application.

NOTE: If you have questions about a command and what it does, refer to the corresponding MCAD API function.

The Sample Application is loaded manually using the ARX load command in Mechanical Desktop. The file *maisamp.arx* is located in the \SDK\mcadapi\sample\maisamp directory. All of the Sample Applications commands are available while the *maisamp.arx* is loaded and are accessed by typing at the command line. Notice that all of the Sample Application commands are prefixed with "mai." In almost every case, sample application function names are identical to the MCAD API functions they exercises, except that the sample functions' prefix is "mai" instead of "ami." For example, in the sample code below the command *maiCreateKey* exercises the function *amiCreateKey*.

The following example uses the Sample Application to create a key to an AutoCAD line, highlight the key and query the key for its type, copy the key and verify that the copy is the same as the original, and finally erase the object to which the key points and verify that it is erased.

Command: **line**

Specify first point: **<select>**

Specify next point or [Undo]: **<select>**

Specify next point or [Undo]: **<enter>**

Command: **maiCreateKey**

Key Type(ARc/AUgmented/CONE/CUrve/CYlinder/Ellipse/Geom/Line/PLane/

POInt/POLyline/SPHere/SPLIne/SPLSurf/SUrf/Torus/Vector/COMp/ Body/

SOlid/PArt/Feat/sKetch/SCene/Dwgview/ dwgANnotation <Object>): **line**

Restrict the picking to the currently active part instance? Yes/<No>: **no**

Select object: [Last]: **<select>**

Saving key of type: AmiLineKey

Enter name to store (<None>/End): **linekey**

"LINEKEY" added to the MAI_SAMPLE_KEY_DICT dictionary.

Command: **maiHighlight**

Enter key name: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

Hit enter to continue... **<enter>**

Command: **maiGetKeyType**

Enter key name: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

Key type = 4

Command: **maiCopyKey**

Enter key name to copy: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

1. Drawing1.dwg

Which document should this command execute in: **1**

Saving key of type: AmiLineKey

Enter name to store (<None>/End): **linekey2**

"LINEKEY2" added to the MAI_SAMPLE_KEY_DICT dictionary.

Command: **maiAreKeysEquivalent**

Enter name of first object key: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

Enter name of second object key: **linekey2**

MAI_SAMPLE_KEY_DICT Object found in document 1.

The keys are equal.

Command: **maiEraseObject**

Enter object key name: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

Command: **maiObjectWasErased**

Enter key name: **linekey**

MAI_SAMPLE_KEY_DICT Object found in document 1.

The object is erased.

The following is a brief description of the commands in this sample application:

Common

Handling Pick Objects

MAIALLOWGHOST

Exercises: (see MAICREATEKEY and MAIFILLPICKOBJ)
Use this command to set the allowGhost variable used for the *amiPick()* and *amiFillPickObj()* functions. The default is false.

MAICREATEKEY

Exercises: *amiPick, amiGetKeyFromPick, amiGetGeomKey*
This command creates a key where a pick with *amiPick()* must be made. The user is prompted for the key type to create. See also: MAIALLOWGHOST.

MAIFILLPICKOBJ

Exercises: *amiFillPickObj, amiGetKeyFromPick*
This command prompts the user to select entities and then attempts to create a key for each of those entities. This command demonstrates how to effectively use acedSSGet() and acedSSNameX() with the MCAD API. See also: MAIALLOWGHOST.

Creating Keys

MAIGETKEYFROMID

Exercises: *amiGetKeyFromId*
Keys are created via this command from AcDbObjectIds, which are database ids that users can get from an ads_name or from ObjectARX™. This command uses acedEntSel() to prompt the user for a single entity selection and attempts to create a key from the corresponding AcDbObjectId.

MAIGETKEYFROMPATH

Exercises: *amiGetKeyFromPath*
This command attempts to create a key from an AcDbFullSubentPath. The user is prompted for an entity selection and will find an AcDbFullSubEntPath in case the object is either blocked or nested.

Creating Keys

MAICREATEINFERKEY

Exercises: *amiPick*, *amiGetKeyFromPick*, *amiGetGeomKey*
This command creates an inferred geometry key where two (2) picks with *amiPick()* must be made.

Handling Keys

MAIAREKEYSEQUIVALENT

Exercises: *amiAreKeysEquivalent*
This command prompts the user for two keys to check for equivalence. The result is displayed.

MAICOPYKEY

Exercises: *amiCopyKey*
The user is prompted for a key name, and a key copy is created and may be stored in the database. A key copy is a key that refers to the same object as the original key.

MAIGETKEYTYPE

Exercises: *amiGetKeyType*
This command prompts the user for a key name, gives the key's type, and displays the result on the screen.

MAIERASEOBJECT

Exercises: *amiEraseObject*
This command prompts the user for a key name. The command then attempts to erase the geometry that the key refers to.

MAIISKEYKINDOF

Exercises: *amiIsKeyKindOf*
This command prompts the user for a key name, compares that key against all key types in the key hierarchy, and displays the results on the screen.

MAIISKEYNULL

Exercises: *amiIsKeyNull*
The intent of this command is to instantiate each type of key and, since nothing is done with the key variable to 'fill it up', will check to see if it is NULL. Since all of these keys will be NULL, a message displays on the screen stating that, for each key type, the key is NULL.

MAIREFOCUSKEY

Exercises: *amiRefocusKey*
This command prompts the user for an object key name. The command then will refocus this key to the next key given (component key or part key). For example, the user may refocus a geometry key from one component instance to another.

MAIREFOCUSKEY2

Exercises: *amiRefocusKey*
This command prompts the user for a key to refocus and a key to use as a reference. The key to refocus is then refocused to the scope of the reference.

Entity Highlighting

MAIHIGHLIGHT

Exercises: *amiHighlight*
This command exercises highlighting of object keys.

Handling Geometry

MAICREATESURFFROMGEOM

Exercises: *amiCreateSurfFromGeom*
This command creates a simple surface from hard-coded data.

MAICREATESURFARRAYFROMGEOM

Exercises: *amiCreateSurfArrayFromGeom*
This command creates a untrimmed surface from the selected NURBS surface.

MAIGETSKETCHEDSPLINESEGMENTS

Exercises: amiGetSketchedSplineSegments
This command prompts the user for key to a sketch spline. This command then returns geometry keys to the spline line segments of the given sketch spline.

MAIISDIRECTLYEDITABLE

Exercises: *amiIsDirectlyEditable*
This command prompts the user for a key name (a geometry key). The command then checks to see if the geometry that the key refers to may be edited directly (for example, an edge of a Desktop part may not be edited directly).

MAISETGEOMETRY

Exercises: *amiSetGeometry*
This command prompts the user for a geometry key name. The command then performs some hard-coded transformations on the entity if the entity can be edited directly.

Handling Change Notification

MAIDISABLEACTIVENOTIFICATION

Exercises: *amiDisableActiveNotification*

This command prompts the user for a geometry key name for which to disable active notification.

MAIENABLEACTIVENOTIFICATION

Exercises: *amiEnableActiveNotification*
This command prompts the user for a geometry key name for which to enable active notification. When the object is changed, a message displays on the screen stating that the object has been changed. When the object is deleted, a message is also displayed.

MAIENABLEACTIVENOTIFICATION2

Exercises: *amiEnableActiveNotification2*
This command prompts the user for a geometry key name and to select an entity for which the id will be passed into the changed/erased functions. When the object is changed, a message is displayed, and the entity represented by the id passed to the change function is highlighted. When the object is deleted, a message is also displayed, and the entity represented by the id passed to the erased function is deleted.

MAIOBJECTHASCHANGED

Exercises: *amiObjectHasChanged* (see also MAIRESETNOTIFICATION)
This command prompts the user for a geometry key name and checks to see if the object referred to by that key has changed since it was last checked. See also: MAIRESETNOTIFICATION.

MAIOBJECTWASERASED

Exercises: *amiObjectWasErased*
This command prompts the user for a geometry key and then checks to see if the object referred to by that key has been erased. The results are displayed on the screen.

MAIRESETNOTIFICATION

Exercises: (see MAIOBJECTHASCHANGED)
Use this command to set the *resetNotification* variable used for the *amiHasObjectChanged()* function. The default is true.

B-rep API Integration

MAIBREP

Exercises: *amiGetBrepFromKey, amiGetKeyFromBrep*
Use this command to traverse a topological body. The user will have the opportunity to store each object for which a key can be created.

MAIBREPFROMTO

Exercises: *amiGetBrepFromKey, amiGetKeyFromBrep*
This command prompts the user for an object key name. The command then will perform a direct test of *amiGetBrepFromKey()* and *amiGetKeyFromBrep().*

Handling Attributes

MAIADDATTRIBUTE

Exercises: *amiAddAttribute*
This command prompts the user for an attribute name and an object key name to attach the attribute to. The command then attempts to attach the given attribute to the given object *through* the given object key.

MAIATTEDITFLAG

This command sets a value that determines if the user will have the opportunity to edit attributes that are returned during the MAIGETATTRIBUTES command.

MAIATTRASSOCTYPE

This command allows the user to set the type of attribute associativity that is used with *amiAddAttribute()*. Use this command to set the attribute associativity behaviour before using MAICREATEATTRIBUTE and MAIMAKEINSTATT.

MAICREATEATTRIBUTE

Exercises: (see MAIADDATTRIBUTE)
This command creates an instance of the MAISAMPLEATTRIBUTE object and prompts the user for integer data the attribute contains.

MAIGETALLATTRIBUTES

Exercises: *amiGetAllAttributes*
This command returns all of the attributes (of a given type) on the given object.

MAIGETATTRIBUTEHOLDERS

Exercises: *amiGetAttributeHolders*
This command prompts the user for an attribute name and then returns all of the keys to the objects to which the supplied attribute has been attached.

MAIGETATTRIBUTES

Exercises: *amiGetAttributes*
This command prompts the user for an object key name and returns all of the attributes that have been attached to the object identified by the key. The attributes are returned according to the MAILISTINDEX flag. The user will have the option to edit the data as per the flag set by MAIATTEDITFLAG. (See also MAILISTINDEX, MAIATTEDITFLAG.)

MAIHOLDERSFLAG

This command sets a value that determines if MAIGETATTRIBUTES will return the attribute holders or not while cycling through all of the attributes.

MAILISTINDEX

This command sets a value that determines how fields are “looked up” during the use of *amiGetAttributes*. When set to ON (Adesk::kTrue), attribute fields are retrieved with the *amiGetFieldxxxI* functions. Otherwise, fields are retrieved by name. The end result is transparent to the user. (See also MAIGETATTRIBUTES.)

MAIREMOVEATTRIBUTE

Exercises: *amiRemoveAttribute*
This command prompts the user for an attribute name and an object key name to identify an object from which to remove the previously supplied attribute.

MAIOWNERISMCADAPI

Exercises: (see MAICREATEATTRIBUTE)
This command prompts the user to set the *ownerIsMCADAPI* flag for the MAISAMPLEATTRIBUTE class derived from *AmiAttribute*. The default is false.

Instantiable Attributes

MAIDEFINEATTCLASS

Exercises: *amiDefineAttClass*
This command prompts the user for an instantiable attribute class name to define. The command then prompts the user for attribute definition information such as the number of fields, field name(s), field description(s), and field type(s).

MAIGETATTCLASSES

Exercises: *amiGetAttClasses*
This command lists all of the instantiable attribute classes that are currently defined.

MAIGETATTCLASSFIELDS

Exercises: *amiGetAttClassFields*
This command prompts the user for an instantiable attribute class name and then prints out the information that defines the class: field names, descriptions and types.

MAIMAKEINSTATT

Exercises: *amiMakeInstAtt*
This command prompts the user for a key to an object to which to attach a new attribute, and then prompts the user for an attribute class to be used. The user is then requested to enter required data for this instance of the attribute class. The attribute instance is then attached to the specified object.

MAIREADATTCLASSESFROMFILE

Exercises: *amiReadAttClassesFromFile*
This command prompts the user for a file from which to acquire instantiable attribute class definitions and then defines these classes.

MAISTATICFLAG

Exercises: (see MAIDEFINEATTCLASS)
This command prompts the user to turn on or off the static flag for new instantiable attribute class definitions. This command can be invoked transparently by prefixing it with a single quote. It will normally be called inside MAIDEFINEATTCLASS. Follow commands to specify which fields are static. The default is false.

MAIUNDEFINEATTCLASS

Exercises: *amiUndefineAttClass*
This command prompts the user for an instantiable class name and then undefines the class if it is not currently used.

MAIWRITEATTCLASSTOFILE

Exercises: *amiWriteAttClassToFile*
This command prompts the user for an instantiable class name and file name for which to write the attribute class definition to.

MAIWRITEATTCLASSESTOFILE

Exercises: *amiWriteAttClassesToFile*
This command prompts the user for a file name and then writes all defined instantiable classes to that file.

Miscellaneous

MAIENDEXTERNALEDIT

Exercises: *amiEndExternalEdit*

This commands prompts user if he wants to save changes before ending the XRef edit in progress.

MAIGETCURRENTDATABASE

Exercises: *amiGetCurrentDatabase*

This command returns the database that is actively being edited.

MAIGETDBIDSFROMKEY

Exercises: *amiGetDbIdsFromKey*
This command prompts the user for an object key name. The command then attempts to find the AcDbObjectIds that the key refers to and displays the results on the screen.

MAIGETKEYFROMNAME

Exercises: *amiGetKeyFromName*
This command prompts the user for the type of key and scope of the key to be returned. The command returns a new key to the project.

MAIGETOBJECTSTATE

Exercises: *amiGetObjectState*
This command prompts the user for an object to query. The command returns the state of the given object.

MAIGETDESIGNMODE

Exercises: *amiGetDesignMode*
This command returns information about the current modeling/drawing state.

MAISTARTEXTERNALEDIT

Exercises: *amiStartExternalEdit*

This command starts an XRef edit on the given leaf node component for an external part.

Parametric Modeling

Parameter Handling

MAICREATEPARAM

Exercises: *amiCreateParam*
This command allow the user to create a global or part parameter.

MAIEVALPARAMEXPRESS

Exercises: *amiEvalParamExpress*
This command evaluates the validity of a global or local parameter expression.

MAIGETGLOBALPARAMS

Exercises: *amiGetGlobalParams*
This command iterates through each global parameter in the drawing database and displays data associated with the parameter, such as name, expression, and value.

MAIGETPARAMDEPS

Exercises: *amiGetParamDeps*
This command prompts the user for a parameter key name. The command then displays all of the parameter keys that affect the parameter key given.

MAIGETPARAMCOMMENT

Exercises: *amiGetParamComment*
Returns the comment of the parameter key given.

MAIGETPARAMEXPRESS

Exercises: *amiGetParamExpress*
This command prompts the user for a parameter key name. The command then displays the parameter's current expression (formula, if any).

MAIGETPARAMNAME

Exercises: *amiGetParamName*
This command prompts the user for a parameter key name. The command then displays the parameter's name.

MAIGETPARAMUSERS

Exercise: *amiGetParamUsers*
This command prompts the user for a parameter key name. The command then displays all of the parameter keys that are affected by the parameter key given.

MAIGETPARAMVALUE

Exercises: *amiGetParamValue*
This command prompts the user for a parameter key name. The command then displays the parameter's current value.

MAISETPARAMCOMMENT

Exercises: *amiGetParamComment*

Sets the comment of the parameter key given with the expression given.

MAISETPARAMEXPRESS

Exercises: *amiSetParamExpress*
This command prompts the user for a parameter key name. The command then displays the parameter's current expression and prompts the user for a new expression to set that parameter to.

MAISETPARAMVALUE

Exercises: *amiSetParamValue*
This command prompts the user for a parameter key name. The command then displays the parameter's current value and prompts the user for a new expression to which to set the parameter.

Part Handling

MAICREATEEMPTYPART

Exercises: *amiCreateEmptyPart*
This command prompts the user for a name for the part to be created. Then a new part is created with that name.

MAIGETACTIVEPART

Exercises: *amiGetActivePart*
This command returns a part key to the user that refers to the active part, if any.

MAIGETCONTAININGPART

Exercises: *amiGetContainingPart*
This command prompts the user for a key name. The command then attempts to find the part that owns the object referred to by that key (the part given a key to a workplane, for example).

MAIGETMASSPROPS

Exercises: *amiGetMassProps*
This command prompts users to enter key names at the command prompt until they press ENTER to end the prompting. The command then performs a mass properties calculation on all of the object keys supplied and displays some of the related data on the screen.

MAIGETNUMPARTFEATS

Exercises: *amiGetNumPartFeats*
This command prompts the user for a part key name. The command then returns the number of features that the part contains.

MAIGETPARTFEATS

Exercise: *amiGetPartFeats*
This command prompts the user for a part key name. The command then returns, one by one, each feature that the part contains.

MAIGETPARTPARAMS

Exercises: *amiGetPartParams*
This command prompts the user for a part key name. The command then iterates through each local part parameter, displays some associated data, and grants the option to save the key(s).

MAIGETUNUSEDSKETCHESFROMPART

Exercises: *amiGetUnusedSketchesFromPart*
This command prompts the user for a part key name. The command then returns all of the unused sketches for the part.

Regeneration Control

MAIREGEN

Exercises: *amiRegen*
This command prompts the user for a key. Then the command updates that geometry.

MAISETNEEDSREGEN

Exercises: *amiSetNeedsRegen*
This command prompts the user for an object key and then marks that object as needing a regeneration.

Suppression Control

MAISUPPRESSFEAT

Exercises: *amiSuppressFeat*
This command prompts the user for a feature key name. The command then suppresses the feature and all features that depend on that feature.
NOTE: In a suppressed state, some operations, such as AMEDITFEAT, cannot be performed.

MAISUPPRESSFEATSBYTYPE

Exercises: *amiSuppressFeatsByType*
This command suppresses all features (of a given type) on the given part.

MAIUNSUPPRESSFEAT

Exercises: *amiUnsuppressFeat*
This command prompts the user for a feature key name. The command then unsuppresses the feature and all features that depend on that feature.

MAIUNSUPPRESSFEATSBYTYPE

Exercises: *amiUnsuppressFeatsByType*
This command unsuppresses all features (of a given type) on the given part.

MAIUNSUPPRESSPARTFEATS

Exercises: *amiUnsuppressPartFeats*
This command unsuppresses all features on the given part.

Component Definition Handling

MAIADDCOMPTOCOMPDEF

Exercises: *amiAddCompToCompDef*
This command prompts the user for two component definition key names and adds the second definition to the first.

MAICOPYCOMPDEF

Exercises: *amiCopyCompDef*
This command prompts the user for a file name. If valid, the command then creates a component definition (external) and returns the resulting component definition key.

MAICREATECOMPDEFFROMFILE

Exercises: *amiCreateCompDefFromFile*
This command prompts the user for a component definition key to be copied and a new component definition name. A component definition key to the new component definition is created, which may then be stored in the database.

MAIEXTERNALIZECOMP

Exercises: *amiExternalize*
This command prompts the user for a component definition and a file name. Then the command externalizes the given definition.

MAIGETACTIVECOMPDEF

Exercises: *amiGetActiveCompDef*
This command returns the component definition key that represents the current 'edit target'.

MAIGETALLCOMPDEFS

Exercises: *amiGetAllCompDefs*
This command iterates through all of the current drawing's component definitions, displays their names, and returns the keys associated with those definitions along with the total number of component definitions.

MAIGETBODYFROMCOMPDEF

Exercises: *amiGetBodyFromCompDef*
This command prompts the user for a component definition key name and then returns the leaf node body from that component definition.

MAIGETCOMPDEFCHILDREN

Exercise: *amiGetCompDefChildren*
This command prompts the user for a component definition key name. The command then returns all of the definition's child component keys.

MAIGETCOMPDEFNAME

Exercises: *amiGetCompDefName*
This command prompts the user for a component definition key name and returns the component definition's name.

MAIGETMASTERCOMPDEF

Exercises: *amiGetMasterCompDef*
This command returns the component definition key that represents the current drawing database's master component definition, or the definition of the 'overall' assembly.

MAIISCOMPDEFEXTERNAL

Exercises: *amiIsCompDefExternal*

This command prompts the user for a component definition key name. The command then checks to see whether or not that component definition is an external component definition and displays the result.

MAIISCOMPDEFLEAFNODE

Exercises: *amiIsCompDefLeafNode*
This command prompts the user for a component definition key name and then checks to see if that component definition is a leaf node. The result is displayed on the screen.

MAIISCOMPDEFMASTER

Exercises: *amiIsCompDefMaster*
This command prompts the user for a component definition key name and returns whether or not the component definition key given represents the current drawing's master component definition.

MAILOCALIZECOMP

Exercises: *amiLocalize*
This command prompts the user for a component definition. Then the command localizes the given definition.

MAIREMOVECOMPFROMCOMPDEF

Exercises: *amiRemoveCompFromCompDef*
This command prompts the user for a component key name and a component definition key name from which to remove the previously supplied component key.

MAISETACTIVECOMPDEF

Exercises: amiSetActiveCompDef
This command prompts the user for compDef key name and then sets the current edit target component to the referenced by the key entered.

MAISETCOMPDEFNAME

Exercises: *amiSetCompDefName*
This command prompts the user for a component definition key name and a new component definition name. The command then sets the component definition name to the name supplied.

Component Handling

MAICREATECOMPDEF

Exercises: *amiCreateCompDef*
This command prompts the user for a body key name and a component definition name. The command then creates a component definition per the name given and consumes the body given. The resulting component definition key is returned.

MAIGETACTIVELEAFNODE

Exercises: *amiGetActiveLeafNode*
This command returns the active part component to the user.

MAIGETCOMPCHILDREN

Exercises: *amiGetCompChildren*
This command prompts the user for a component key name. The command then displays and returns all of the keys to the components that are the children of the specified component.

MAIGETCOMPDEF

Exercises: *amiGetCompDef*
This command prompts the user for a component key name and returns the key to that component's definition.

MAIGETCOMPINDEF

Exercises: *amiGetCompInDef*
This command prompts the user for a component key name and then returns the corresponding component key that lies within the component's definition.

MAIGETCOMPNAME

Exercises: *amiGetCompName*
This command prompts the user for a component key name. The command then displays the component's name.

MAIGETCOMPOWNER

Exercises: *amiGetCompOwner*
This command prompts the user for a component key name and returns a key to the definition of the component that owns the component passed in.

MAIGETCOMPPOSITION

Exercises: *amiGetCompPosition*
This command prompts the user for a component key name. The command then displays the component's position information.

MAIGETCONTAININGCOMP

Exercises: *amiGetContainingComp*
This command prompts the user for a component key name. The command then attempts to find the component instance that owns the object referred to by that key (the component given a key to the work plane, for example).

MAIISCOMPTOOLBODY

Exercises: *amiIsCompToolBody*
This command prompts the user for a component key to evaluate. Then the command returns whether or not the component is a tool body.

MAISETACTIVELEAFNODE

Exercises: amiSetActiveLeafNode
This command prompts the user for key name. The command then sets the current edit target component to the referenced by the given key.

MAISETCOMPNAME

Exercises: *amiSetCompName*
This command prompts the user for a component key name and a new component name. The command then changes the component specified by the given key to the new name supplied.

MAISETCOMPPOSITION

Exercises: *amiSetCompPosition*

This command prompts the user for a component key name to be changed and the component key name to set the position of the previously supplied component key to.

Constraint Handling

MAICREATEASSMCONSTR

Exercises: *amiCreateConstraints*
This command allows the user to create multiple sketch constraints within a single command.

MAICREATEASSMCONSTR2

Exercises: *amiCreateConstraints*
This command allows the user to create multiple assembly constraints within a single command.

MAICREATESKETCHCONSTR2

Exercises: *amiCreateConstraints*
This command creates assembly constraints defined by the user's input.

MAIGETCONSTRDESCRIPDATA

Exercises: *amiGetConstrDescripData, amiGetData*
This command performs a full constraint query of the constraint key passed in.

MAIGETCONSTREXPR

Exercises: *amiGetConstrExpr*
This command prompts the user for a constraint key name. The command then will print the expression associated with the key.

MAIGETCONSTRPARAM

Exercises: *amiGetConstrParam*
This command prompts the user for a sketch constraint key name and then returns the parameter object associated with the constraint.

MAIGETCONSTRTYPE

Exercises: *amiGetConstrType*
This command prompts the user for a constraint key name and returns the constraint type of that component constraint.

MAIGETCONSTRVALUE

Exercises: *amiGetConstrValue*
This command prompts the user for a constraint key name. The command then displays the constraint's value.

MAIGETCONSTROPERANDS

Exercises: *amiGetConstrOperands*
This command prompts the user for a constraint key name. The command then returns the two geometry keys that are the operands of the given constraint.

MAIGETDIMCONSTRLOC

Exercises: *amiGetDimConstrLoc*
This command prompts the user to enter the name of a constraint (sketch constraint) key that has a parametric dimension associated with it. The command then prints out the dimension's location.

MAISETCONSTREXPR

Exercises: *amiSetConstrExpr*
This command prompts the user for a constraint key name. The command then will prompt the user for a new expression to set the constraint expression to.

MAISETCONSTRVALUE

Exercises: *amiSetConstrValue*
This command prompts the user for a constraint key name and a new value. The command then sets the given constraint's value to the new value given.

Component Constraint Handling

MAIGETCOMPSFROMCONSTR

Exercises: *amiGetCompsFromConstr*
This command prompts the user for a component constraint key and then returns the two components that use that constraint.

MAIGETCONSTRSACTINGONCOMP

Exercises: *amiGetConstrsActingOnComp*
This command prompts the user for a component key name and then returns all of the component constraint keys that affect the positioning of the given component.

MAIGETCONSTRSFROMCOMP

Exercises: *amiGetConstrsFromComp*
This command prompts the user for a component key name and returns the component constraint keys associated with that component.

Sketch Handling

MAICREATESKETCH

Exercises: *amiCreateSketch*
This command prompts the user for the geometry to be used in the sketch and the type of sketch to create. The command then creates a sketch.

MAIGETCURRENTSKETCHPLANE

Exercises: *amiGetCurrentSketchPlane*
This command returns a key to the active sketch plane.

MAIGETEXTSKETCHCONSTRS

Exercises: *amiGetExtSketchConstrs*
This command prompts the user for a sketch key name and then returns all of the constraints that act upon the geometry in the sketch and geometry not in the sketch.

MAIGETFEATSFROMSKETCH

Exercises: *amiGetFeatsFromSketch*
Returns the feature keys that use the same sketch key given.

MAIGETSKETCHCONSTRS

Exercises: *amiGetSketchConstrs*

This command prompts the user for a sketch key name and then returns all of the constraints that act upon the geometry in the sketch.

MAIGETPATHSTARTPOINT

Exercises: *amiGetPathStartPoint*
This command prompts the user for a sketch key and returns a key to the start point.

MAIGETSKETCHCOORDSYS

Exercises: *amiGetSketchCoordSys*
This command prompts the user to enter the name of a sketch key. The command then prints out the sketch's coordinate system, in terms of an AcGeMatrix3d.

MAIGETSKETCHDEFPLANE

Exercises: *amiGetSketchDefPlane*
This command prompts the user for a sketch key and returns a geometry key to the plane the profile is on.

MAIGETSKETCHESFROMGEOM

Exercises: *amiGetSketchesFromGeom*
Returns the sketch keys that share the same piece of geometry key given.

MAIGETSKETCHGEOM

Exercises: *amiGetSketchGeom*
This command prompts the user for a sketch key name and then returns all of the geometry that makes up the sketch.

MAIGETSKETCHPARAMS

Exercises: *amiGetSketchParams*
This command prompts the user for a sketch key name and then returns all of the parameter objects that act upon the sketch.

MAIGETSKETCHPLANE

Exercises: *amiCreateFeatDescrip, amiCreateLocator, amiCreateAtomLocator, amiSetAtomLocData, amiAddAtomLocator, amiSetAtomLocData, amiSetAbsoluteLocData*
This command prompts the user for a line key, plane key, and orientation of sketch plane. The command then creates a sketch plane.

MAIGETSKETCHPLANEDESCRIPDATA

Exercises: *amiGetSketchPlaneDescrip, amiGetFeatData, amiGetDescripType, amiGetLocator, amiGetAtomLocators, amiGetAtomLocType, amiGetAtomLocData, amiGetDistFromLocData, amiGetOnPlaneLocData, amiGetAbsoluteLocData, amiGetAngledLocData, amiGetOffsetLocData, amiGetCoordSysLocData*
This command prompts the user for a key to a sketch plane. Then the command dumps the descriptor information to the command line.

MAIGETSKETCHPLANEFROMSKETCH

Exercises: *amiGetPlaneFromSketch*
This command prompts the user for a sketch key and returns the sketch plane from the sketch.

Sketch Constraint Handling

MAICREATESKETCHCONSTR

Exercises: *amiCreateConstraint*
This command prompts the user for the type of constraint, and the key(s) to use to create the constraint.

MAIGETCONSTRSKETCHES

Exercises: *amiGetConstrSketches*
This command prompts the user for a constraint key name. The command then will return the sketches which are constrained by the specified sketch constraint key.

Feature Handling

MAIGETFEATDESCRIPDATA

Exercises: All of the feature function calls as found in MIFEAT.H (also see MAIRETURNINFORMERS and MAIRETURNPARAMETERS)
This command performs a full feature query of the feature key passed in.

MAIGETFEATDEPFEATS

Exercises: *amiGetFeatDepFeats*
This command prompts the user for the state of the features to return. The command then returns all of the features(in the given state) which are dependent on the given feature.

MAIGETFEATDEPWORKGEOM

Exercises: *amiGetFeatDepWorkGeom*
This command returns all of the work entities dependent on the given feature.

MAIGETFEATEDGES

Exercises: *amiGetFeatEdges*
This command prompts the user for a feature key name. The command then returns to the user all of the curve keys which represent that feature.

MAIGETFEATFACES

Exercises: *amiGetFeatFaces*
This command prompts the user for a feature key name. The command then returns to the user all of the surface keys which represent that feature.

MAIGETFEATPARAMS

Exercises: *amiGetFeatParams*
This command prompts the user for a feature key name. The command then iterates through all of the feature key's associated parameters.

MAIGETFEATSFROMGEOM

Exercises: *amiGetFeatsFromGeom*
This command prompts the user for a geometry key name and then returns all of the features (if any) that share that piece of geometry.

MAIGETFEATSKETCH

Exercises: *amiGetFeatSketch*

This command prompts the user for a feature key name and then returns the sketch key associated with the feature (if one exists).

MAIGETFEATTYPE

Exercises: *amiGetFeatType*
This command returns the type of feature key passed in.

MAIGETFEATVERTICES

Exercises: *amiGetFeatVertices*
This command prompts the user for a feature key name. The command then returns to the user all of the point keys which represent that feature.

MAIISFEATKINDOF

Exercises: *amiIsFeatKindOf*
This command returns the feature type of the given feature.

MAIREORDERFEAT

Exercises: *amiReorderFeat*
This command prompts the user for two feature keys. The command then reorders the first key immediately before the second key in the history tree.

MAIRETURNFEATKEY

Exercises: (also see MAIGETFEATDESCRIPDATA)
This command set the fReturnInformers flag as used by *amiGetFeatDescrip*().

MAIRETURNINFORMERS

This command sets the fReturnFeatKey boolean parameter used with *amiCreateFeature()*. See also MAIFILLET, MAICHAMFER.

MAIRETURNPARAMETERS

Exercises: (also see MAIGETFEATDESCRIPDATA)
This command set the fReturnParameters flag as used by *amiGetFeatDescrip*().

MAISKETCHPLANE

Exercises: *amiGetPickInfo, amiCreateFeatDescrip, amiCreateLocator, amiCreateAtomLocator, amiSetAtomLocData, amiSetCoordSysLocData, amiAddAtomLocator, amiCreateAtomLocator, amiSetAbsoluteLocData, amiSetLocator, amiCreateSketchPlane*
This command prompts the user for the properties of the sketch plane to create. The command then creates the sketch plane.

Feature Descriptor Handling

MAIARRAY

Exercises: *amiCreateFeatDescrip, amiSetArrayFeature, amiCreateLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetAtomLocData, amiSetRecArrayRowData, amiSetRecArrayColData, amiCreateAtomLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetPolarArrayPolarAngleType, amiSetPolarArrayAngle, amiSetPolarArrayRotateAsCopied, amiSetPolarArrayNumInstances, amiSetLocator, amiCreateFeature*
This command prompts for user input of array properties. The command then creates an array feature.

MAIBASE

Exercises: *amiCreateFeatDescrip, amiSetBaseData, amiCreateFeature*
This command creates a base feature given a solid key.

MAICHAMFER

Exercises: *amiCreateFeatDescrip, amiSetChamferType, amiSetChamferDist1, amiSetChamferDist2, amiSetChamferAngle, amiSetChamferEdges, amiSetChamferAngleFace, amiCreateFeature*
This command creates a chamfer feature as per user input.

MAICOMBINE

Exercises: *amiCreateFeatDescrip, amiSetCombinerType, amiSetParametricBoolComp, amiCreateFeature*
This command prompts for user input of combine properties. The command then creates an combine feature.

MAIEXTRUDE

Exercises: *amiCreateFeatDescrip, amiCreateTerminator, amiSetExtrusionDraftAngle, amiSetCombinerType, amiSetExtrusionProfile, amiSetBlindTermData, amiSetMidPlaneTermData, amiSetToPlaneTermData, amiSetToFaceTermData, amiSetFromToTermData, amiSetTerminator, amiSetExtrusionDirectionVector, amiCreateFeature*
This command prompts for user input of extrusion properties. The command then creates an extrusion feature.

MAIFACEDRAFT

Exercises: *amiCreateFeatDescrip, amiSetDraftPlane, amiSetDraftAngle, amiSetPlaneDraftFaces, amiSetShadowDraftFaces, amiSetEdgeDraftFaces, amiCreateFeature*
This command prompts the user for the type of facedraft feature to create. The command then creates a facedraft feature.

MAIFACESPLIT

Exercises: *amiCreateFeatDescrip, amiSetFaceSplitFaces, amiCreateLocator, amiCreateAtomLocator, amiSetAtomLocData, amiAddAtomLocator, amiSetLocator, amiCreateFeature*
This command prompts the user to pick faces to split and an object to use as a split plane. Then the command creates the facesplit operation.

MAIFILLET

Exercises: *amiCreateFeatDescrip, amiSetFilletEdges, amiSetUniFilletRadius, amiSetIndFilletRadii, amiSetFixedFilletChordLength, amiSetCubicFilletRadii, amiSetLinearFilletRadii, amiCreateFeature*
This command creates a fillet feature as per user input.

MAIGETPARTWORKAXES

Exercises: *amiGetPartWorkAxes*
This command queries a given part and returns keys to all of the parts work axes.

MAIGETPARTWORKPLANES

Exercises: *amiGetPartWorkPlanes*
This command queries a given part and returns keys to all of the parts work planes.

MAIGETPARTWORKVERTICES

Exercises: *amiGetPartWorkVertices*
This command queries a given part and returns keys to all of the parts work points.

MAIHOLE

Exercises: *amiCreateFeatDescrip, amiSetHoleTapData, amiSetHoleDiameter, amiSetHoleDrillAngle, amiSetCBoreHoleCbDepth, amiSetCBoreHoleCbDiam, amiSetCSinkHoleCsAngle, amiSetCSinkHoleCsDiam, amiSetHoleDirectionVec, amiSetBlindTermData, amiSetToPlaneTerm, amiSetToFaceTermData, amiCreateLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetLocator, amiSetAtomLocData, amiCreateTerminator, amiSetTerminator, amiCreateFeature*
This command prompts the user to define the type of hole to create and creates the defined hole.

MAILOFT

Exercises: *amiGetKeyType, amiIsKeyKindOf, amiCreateFeatDescrip, amiSetCombinerType, amiSetLoftType, amiSetLoftWeights, amiSetLoftAngles, amiSetLoftXSections, amiCreateFeature*
This command prompts the user for the type of loft feature to create. The command then creates a loft feature.

MAIMODIFIER1

This command sets the first modifier type used in *maiWorkPlane*

MAIMODIFIER2

This command sets the second modifier type used in *maiWorkPlane*

MAIPARTSPLIT

Exercises: *amiCreateFeatDescrip, amiSetPartSplitFlip, amiSetPartSplitName, amiCreateLocator, amiCreateAtomLocator, amiSetAtomLocData, amiAddAtomLocator, amiSetLocator, amiCreateFeature*
This command prompts the user for an object to use as a split plane. Then the command splits the part on this plane.

MAIREVOLVE

Exercises: *amiCreateFeatDescrip, amiCreateTerminator, amiSetCombinerType, amiSetRevolveDirectionVec, amiSetRevolveProfile, amiSetRevolveAxis, amiSetBlindTermData, amiSetMidPlaneTermData, amiSetToPlaneTermData, amiSetToFaceTermData, amiSetFromToTermData, amiSetTerminator, amiCreateFeature*
This command prompts for user input of revolve properties. The command then creates an revolve feature.

MAISHELL

Exercises: *amiCreateFeatDescrip, amiCreateTerminator, amiSetCombinerType, amiSetMidPlaneTermData, amiSetInsideTermData, amiSetOutsideTermData, amiSetShellExcludes, amiSetShellOverrides, amiSetTerminator, amiCreateFeature*
This command prompts for user input of shell properties. The command then creates an shell feature.

MAISURFCUT

Exercises: *amiCreateFeatDescrip, amiSetCombinerType, amiCreateLocator, amiCreateAtomLoca*tor, *amiSetAtomLocData, amiAddAtomLocator, amiSetLocator, amiSetSurfCutSurf, amiSetSurfCutDirectionVec, amiCreateTerminator, amiSetTerminator, amiCreateFeature*

This command prompts for user input of surfcut properties. The command then creates an surfcut feature.

MAISWEEP

Exercises: *amiCreateFeatDescrip, amiCreateTerminator, amiSetToFaceTermData, amiSetFromToTermData, amiSetTerminator, amiSetSweepProfile, amiSetSweepPath, amiSetSweepDraftAngle, amiSetCombinerType, amiCreateFeature*
This command prompts for user input of sweep properties. The command then creates an sweep feature.

MAIWORKAXIS

Exercises: *amiCreateFeatDescrip, amiCreateLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetLocator, amiCreateConstrGeom, amiSetAtomLocData*
This command prompts the user for a surface key name. The command then creates a workaxis.

MAIWORKPLANE

Exercises: *amiCreateFeatDescrip, amiCreateLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetLocator, amiSetAtomLocData, amiCreateConstrGeom*
This command prompts the user for the method of workplane creation. The command then prompts the user to enter the name of keys to use for the creation of the workplane. The command then creates a workplane.

MAIWORKPOINT

Exercises: *amiCreateFeatDescrip, amiCreateLocator, amiCreateAtomLocator, amiAddAtomLocator, amiSetOnPlaneLocData, amiSetLocator, amiCreateConstrGeom*
This command creates a workpoint at a user-defined location.

Array Descriptors

MAIARRAYHASINDEPENDENTINSTANCES

Exercises: *amiArrayHasIndependentInstances*

This command prints out whether or not the array has any independent instances.

MAIISARRAYINSTANCEINDEPENDENT

Exercises: *amiIsArrayInstanceIndependent*

This commands prompts user for row/column spec and prints out whether or not the instance at that location is independent or not.

Table-Driven Versions

MAICREATELINK

Exercises: *amiCreateFileDescrip, amiSetLink, amiSetData*
This command creates a link to the given object in the current drawing.

MAIGETCURRENTVERSION

Exercises: *amiGetCurrentVersion*
This command queries the given object for the name of the current version.

MAIGETLINKDATA

Exercises: *amiGetLinkInfo, amiGetData*
This command queries the given object for its attributes.

MAIREMOVELINK

Exercises: *amiRemoveLink*
This command removes the link for the given object.

MAISETCURRENTVERSION

Exercises: *amiSetCurrentVersion*
This command sets the current version of the given object.

MAISETLINKDATA

Exercises: *amiGetLinkInfo, amiSetLink, amiSetData*
This command sets attributes on the given objects link.

Event Handling

MAIREGISTEREVENTREACTION

Exercises: *amiRegisterApp, amiRegisterEventReaction*
This command prompts the user for the type of event. Then the command registers the event for notification.

MAIREMOVEEVENTREACTION

Exercises: *amiRegisterApp, amiRemoveEventReaction*
This command prompts the user for the type of event to remove. The command then removes registered event callbacks for that event.

Miscellaneous

MAIGETDESIGNMODE

Exercises: *amiGetDesignMode*
This command returns information about the current modeling/drawing state.

MAISETDESIGNMODE

Exercises: *amiSetDesignMode*
This command prompts the user for a new design mode. Then the command sets the design mode to that specified.

Analysis

MAICHECKINTERFERENCE

Exercises: *amiGetDOFDescrip, amiGetData, amiGetDataArray*

This command prompts the user for two component keys and returns interferences between the given components.

MAIGETDOFDESCRIPDATA

Exercises: *amiGetDOFDescrip, amiGetData, amiGetDataArray*

This command prompts the user for an object key. Then the command dumps the descriptor information to the command line.

MAIMINDISTANCE

Exercises: *amiMinDistance*

This command prompts the user for two component keys and returns minimum distance between the given components.

Mechanical Desktop Surface Modeling

MAICREATESURFFROMGEOM

This command creates a simple surface from hard-coded data.

Drawing Manager

MAICREATEDWANNOT

Exercises: *amiCreateDwAnnotDescrip, amiSetDwAnnotData, amiCreateDwAnnot, amiSetDwAnnotGeom*

This command prompts the user for the type of annotation and the attributes to set. The command then creates the drawing annotation.

MAICREATEDWVW

Exercises: *amiCreateDwVwDescrip, amiSetDwVwData, amiCreateDwView*

This command prompts the user for the type of drawing view and the attributes to set. The command then creates the drawing view.

MAICREATEINFERGEOMKEY

Exercises: *amiInferGeomKey*

This command prompts the user for an array of geometry keys and a proximity point. The command then returns an inferred geometry key.

MAIGETALLDWVWS

Exercises: *amiGetAllDwVws*

This command returns keys to all of the drawing views.

MAIGETDWANNOTDATA

Exercises: *amiGetDwAnnotDescrip*

This command prompts the user for an annotation key. Then the command returns the value of that annotation.

MAIGETDWVWANNOTS

Exercises: *amiGetDwVwAnnots*

This command prompts the user for a drawing view key. Then the command returns all of that views annotations.

MAIGETDWANNOTVIEW

Exercises: *amiGetDwAnnotView*
This command prompts the user for an annotation key. Then the command returns the view to which the annotation belongs.

MAIGETDWVWATTRIBUTE

Exercises: *amiGetDwVwDescrip, amiGetData*
This command prompts the user for a drawing view key and the type of attribute to return. The command then returns all attributes of the given type.

MAIGETDWVWPARENT

Exercises: *amiGetDwVwParent*
This prompts the user for a drawing view key and returns the parent view of that key.

maiGetNumDwVws() – Exercises: amiGetNumDwVws
This command returns all of the drawing views in the current drawing.

MAIISDWANNOTKEYKINDOF

Exercises: *amiGetDwAnnotDescrip, amiDwAnnotDescripIsKindof*
This command prompts the user for an annotation key. Then the command returns all of the key types which it derives from.

MAIISDWVWKEYKINDOF

Exercises: *amiGetDwVwDescrip, amiDwVwDescripIsKindof*
This command prompts the user for a drawing view key. Then the command returns all of the key types which it derives from.

MAIMOVEDWANNOT

Exercises: *amiMoveDwAnnot*
This command prompts the user for an annotation key name and displacement value. Then the command moves the annotation to the new location.

MAISETDWANNOTDATA

Exercises: *amiCreateDwAnnotDescrip, amiSetDwAnnotData, amiDwEditAnnot*
This command prompts the user for an annotation key, the attribute to change, and the value to change it to. Then the command sets that attribute.

MAISETDWANNOTVIEW

Exercises: *amiSetDwAnnotView*
This command sets the given annotations owning view.

MAISETDWVWATTRIBUTE

Exercises: *amiCreateDwVwDescrip, amiSetData, amiDwEditView*
This command prompts the user for a drawing view key and type of attribute to set. The command then sets the chosen attribute with the user defined value.

Scenes

MAICREATESCENE

Exercises: *amiCreateScene*
This command prompts the user for a component definition key and the name for the new scene. The command then creates a new scene with the given name.

MAIGETALLSCENES

Exercises: *amiGetAllScenes*
This command returns all of the scenes in a drawing.

MAIGETSCENELOCK

Exercises: *amiGetSceneLock*
This command prompts the user for a scene key and returns the current locking status of that scene.

MAISETSCENELOCK

Exercises: *amiSetSceneLock*
This command prompts the user for a scene key and a locking status to set on that scene.

Hidden Line (AHL) Calculation

MAICLEARCUSTOMAHL

Exercises: amiClearCustomAhl
This command prompts the user for a drawing view key name. Then it gives option of clearing the AHL settings of a view referenced by the key entered earlier or object (key to which needs to be entered) in that view.

MAICUSTOMAHLON

This command prompts the user for a drawing view key name. This function just turns on view-regen notification.

MAIGETCUSTOMAHL

Exercises: amiGetCustomAhl
This command prompts for DwVwKey and the object key in that view. The command then displays recorded custom AHL settings for a given key in a given view. It displays whether custom geometry participates in AHL calculations or not. An array of pointers to geometry keys is also returned.

MAISETCUSTOMAHL

Exercises: amiSetCustomAhl
The command prompts for DwVwKey, object key in that view, AHLflag to specify if the custom geometry participates in AHL calculation, length and array of pointers to geometry keys. This command then records custom AHL settings for the given object in the given view.

Layouts

MAIGETALLDWLAYOUTS

Exercises: amiGetAllDwLayouts
The command returns a layout key for each active layout.

User Interface Functions

MAIISBROWSERDISPLAYED

Exercises: amiIsBrowserDisplayed
The command tells whether or not the Mechanical Desktop browser is currently displayed.

MAIDISPLAYBROWSER

Exercises: amiDisplayBrowser
The command turns the display of Mechanical Desktop browser On or Off.

File Descriptor Functions

MAIISPDMREGISTERED

Exercises: *amiIsPDMRegistered*
This command returns whether or not there is a PDM registered.

MAIREGISTERPDM

Exercises: *amiCreateFileDescrip, amiSetData, amiAttachFile*
This command registers a file with a PDM using a sample app attribute.

MAIUNREGISTERPDM

Exercises: *amiCreateFileDescrip, amiSetData, amiRemoveFile*
This command unregisters a file with a PDM using a sample app attribute.

Utility

MAIDELETEKEY

This command prompts the user for an object key name to delete from the named objects dictionary.

MAIDELETEATTRIBUTE

This command prompts the user for an attribute name to delete from the named object's dictionary.

MAIGETSYSVAR

Exercises: amiGetSysVar
This command prompts the user for a AutoCAD system variable and then displays the value of that AutoCAD system variable.

MAILISTATTRIBUTES

This command lists the names of all of the attributes stored in the named object's dictionary.

MAILISTKEYS

This command lists the names of all of the keys stored in the named object's dictionary.

MAIPRINTATTRIBUTE

This command prompts the user for an attribute name. The command then displays data associated with the attribute given.

MAIPRINTKEY

This command prompts the user for an object key name. The command then displays data associated with the key given, depending on the key type.

MAIPRINTSTATUSINFO

This command switches fPrintStatusInfoFlag which is used in function printStatusInfo for printing the details of the status returned by all McadAPI functions.

MAIRESETATTRIBUTEDICT

This command removes all of the attributes stored in the named object's dictionary.

MAIRESETKEYDICT

This command removes all of the keys stored in the named object's dictionary.

MAISETSYSVAR

Exercises: amiSetSysVar
This command prompts the user for a AutoCAD system Variable name and the value that needs to be assigned to it. This command then sets that value to the AutoCAD system variable that was entered.

SmartHole Sample Application

The SmartHole application demonstrates using the instantiable attribute mechanism to associate features together to simulate an aggregate, or custom feature. The SmartHole is simply a hole feature and an individual radius fillet on all of the holes edges, with attributes attached to each containing a key to a managing object in a dictionary. The SmartHole custom object maintains keys to the hole and fillet feature, as well as the hole diameter and fillet radius values.

Commands

SHCreate

Prompt the user to select two edges of a part, and create a SmartHole feature in the center of the common face. This command demonstrates prompting sequences, various utility functions, hole and fillet feature creation, creation of key containing custom object, and instantiable attribute attachment.

SHEdit

Prompt the user to select a SmartHole feature, and edit the values for the hole diameter and fillet radius. This command demonstrates using attached attributes to control selection and in locating a custom object in a dictionary, as well as editing features through parameters.

SHErase

Prompt the user to select a SmartHole feature and erase it, removing the SmartHole object from the dictionary, and the features on the part. This command demonstrates erasing feature objects, and the proper handling of keys in custom objects during erase and destruction.

SHMatch

Prompt the user to select an assembly component, and if any SmartHole features on the selected component are involved in an 'insert' assembly constraint, allow the user to choose from which SmartHole the other will inherit the hole diameter and fillet radius values from. This command demonstrates accessing assembly constraints, and using the B-Rep API to traverse feature topology.

SHAudit

Iterate over the SmartHole dictionary, and update the SmartHole to the stored values. If the hole feature was erased, erase the SmartHole, otherwise regenerate the fillet to account for any non filleted hole /part intersecting edges. This command demonstrates locating and managing features from a custom object.

Known Issues

In order to erase features with amiEraseObject, the part containing the feature must be the active part. The SetActivePart() utility function is used for this, but only looks for a component instance to activate in the current target. Erasing a SmartHole feature with SHErase or regenerating the fillet on a SmartHole feature on a component that is not instanced into the

current edit target will fail to erase the features. Regenerating the SmartHole's fillets simply erases the existing fillet and recreates it. This means any assembly constraints or dependent features on the fillet will be invalidated by a call to SHAudit. The SmartHole object does not support or prevent manipulation in an externally attached file and wblockClone operations such as localize (xbind), externalize (wblock), and insert. The code is organized for readability by command, and therefore does not necessarily represent a good class design.

Hole Projection Sample Application

This application demonstrates the combined use of the MCAD API, Geometry Library and BREP API in a practical application.

The application, MAIPROJHOLE, simply projects holes (source) through Desktop parts (target). The source may be of any Ami::arcKey from which the target holes will be sized from. The target must be a local part instance. The target(s) must have a valid face parallel with the source arc for which a sketch plane can be created and hole feature added. The holes are then created as hole features created on a workpoint *through* the entire part. The user does have the option to specify whether the holes are created from the closest or from the furthest candidate face on the part(s).

Simple User Interface

1. User selects arcs for holes to be projected and then presses Enter.
2. User then picks all of the parts for which to project the holes through and the holes are created.

Known Issues

The command attempts to reset the active part to that of which was the active part before the command was started. However, if the active part before the command was started was a componentized part, this will not happen.

Methods and Functionality Exercised

MCAD API

amiGetActivePart()
amiHighlight()
amiPick()
amiGetGeomKey()
amiGetGeomData()
amiGetKeyFromBrep()
amiIsKeyKindOf()
amiGetKeyFromId()
Passing keys to acedInvoke()

BREP API

AcBrBrep::setSubentPath()
AcBrBrepFaceTraverser::setBrep()
AcBrBrepFaceTraverser::done()
AcBrBrepFaceTraverser::next()
AcBrBrepFaceTraverser::getFace()
AcBrFace::getSurface()
AcBrFace:: getPointRelationToFace()
AcBrFaceLoopTraverser::setFace()

GeLib

AcGeEntity3d::isOn()
AcGeLine3d::pointOnLine()
AcGeExternalCurve3d::isNativeCurve()

AcGeCircArc3d::getPlane()
AcGeCircArc3d::radius()
AcGeCircArc3d::center()
AcGeExternalBoundedSurface::getBaseSurface()
AcGeExternalSurface::isNativeSurface()
AcGePlanarEnt::isParallelTo()
AcGePoint3d::orthoProject()
AcGePoint3d::distanceTo()

Files

PROJHOLE.DSP, PROJHOLE.DSW	MS Visual C++ version 6 Service Pack 2 project files
PROJREG.CPP	RX entry point and command definition
PROJHOLE.CPP	Main source

MfcBrepTrav Sample Application

The MfcBrepTrav sample application allows you to select an AutoCAD solid, Mechanical Desktop part or B-rep subentity (for example a face or an edge) and display information about how the object is structured in terms of boundary representation.

Depending on the type of object selected, you can apply one of the B-rep API's traversers to it, and the information retrieved by the traverser is displayed in a tree structure (CTreeCtrl). If you check the Highlight box in the MfdBrepTrav dialog, you can select an element in the tree structure and the corresponding element in the model is highlighted. The dialog also displays general information about the selected entity, for example whether a loop forms the interior or exterior loop of the orientation of a face. Finally, it displays the bounding box of the entity so that you can check line and point containment.

To use the MfcBrepTrav sample application, enter BREPINFO at the command line.

MyBrowser Sample Application

The MyBrowser sample application demonstrates ways to extend and customize the Mechanical Desktop browser. The sample is static, which means that the contents of the browser sample is not updated as you modify a Mechanical Desktop drawing. Thus, it is best to enter the command to start the sample application after loading an assembly drawing. The sample retrieves the assembly structure and displays it in its own browser dialog tab. It also shows how to highlight a component when the user moves the mouse over the browser window. The dialog will be removed if you unload the sample application.

To use the MyBrowser sample application, load an assembly drawing and enter MYBROWSER at the command line.

CountAssemblies Sample Application

The CountAssemblies sample application counts the number of component definitions and instances. The result is displayed in a dialog box.

To use the CountAssemblies sample application, enter COUNTASSEMBLIES or ASSYCOUNT at the command line.

Which_MDT Sample Application

The Which_MDT sample application illustrates how to find out where the Mechanical Desktop has been installed and what version it is. This sample is located in

\sdk\mcadapi\sample\Which_MDT

To use the Which_MDT sample application, run the setup.exe located in the above directory as an example. The Installshield file setup.rul (also in the above directory) demonstrates the steps for locating MDT on an end user's system.

Appendix B
How to Use the MCAD API

Understanding the SDK Directory Content

This release of the MCAD API is based on Mechanical Desktop 6 and on AutoCAD 2000i. The included ObjectARX has been upgraded with the latest MCAD API libraries.

Below is a summary of the Mechanical SDK directory structure:

```
/SDK
\---mcadapi
    +---doc                 MCAD API documentation
    +---inc                 MCAD API specific headers
    +---lib                 MCAD API proxy library
    \---sample              Examples demonstrating the use of the MCAD API
        \---CountAssemblies Sample application that counts assemblies
        +---maisamp         General access to MCAD API
        \---MfcBrepTrav     Sample application that traverses B-rep topologies
        +---MyBrowser       Browser sample application
        +---projhole        Hole projection sample application
        \---SmartHole       SmartHole sample application
        +---Which_MDT       Locating MDT on end user machine sample application
|
+---ObjectARX
|   +---arxlabs
|   +---classmap
|   +---docs
|   +---docsamps
|   +---inc                 ObjectARX headers
|   +---lib                 ObjectARX libraries
|   +---redistrib
|   +---samples
|   \---utils
|       \---brep
|           +---inc         BREP headers
|           +---lib         BREP and ACIS dependent geometry libraries
|           \---samples     Examples demonstrating the use of the BREP API
```

The *ObjectARX* Subdirectory

The *ObjectARX* subdirectory contains the header and library files necessary to compile and link applications for the MCAD API.

- The *inc* directory contains ObjectARX headers.
- The *utils\brep\inc* directory contains BREP API headers.
- The *lib* directory contains ObjectARX libraries.
- The *utils\brep\lib* directory contains the ACIS dependent libraries (acbr15.lib and acgex15.lib).

The *mcadapi* Subdirectory

Under the *mcadapi* subdirectory are all of the MCAD API specific files. The *mcadapi* subdirectory contains four subdirectories:

- The *doc* directory contains the following files:
  -*apidev.doc*, which is the developer's guide for the MCAD API in Word 97 format
  -*apidev4.doc*, which contains the same information as *apidev.doc* but is formatted for the A4 paper size used in Europe
  -*wd97vwr32.exe*, which installs a viewer for those not using Word for Office 97
  -*readapi.txt* in ASCII format
  -*apidev.chm, which is a Microsoft HTML help version of the Developer's Guide*

- The *lib* directory contains the files *mcadapi.lib* and *amdt_attributes.lib*, which are the library file that users of the MCAD API need to link against and corresponds to the MCAD API.
- The *sample* directory contains examples of applications written on top of the MCAD API.
    - The maisamp directory contains the application that was used in the MCAD API testing.

      Also included is the executable file for the sample application maisamp.arx, the file defining the functions being exported by the application maisamp.def, and the Visual C++ 6.0 Service Pack 2 project file for the sample application. For more information on this sample application see Appendix A.
    - The smarthole directory contains the application that demonstrates using the instantiable attribute mechanism to associate features together to simulate an aggregate, or custom feature. In this directory are the following source files:
      smarthole.h
      smarthole.cpp
      entryPnt.cpp
      SHAudit.cpp
      SHCreate.cpp
      SHEdit.cpp
      SHErase.cpp
      SHMatch.cpp

      Also included is the executable file for the sample application *smarthole.arx*, the file defining the functions being exported by the application *smarthole.def*, and the Visual C++ 6.0 Service Pack 2 project file for the sample application. For more information on this sample application, see Appendix A.
    - The projhole directory contains the application that is a hole projection application sample. In this directory are the following source files:
      projhole.cpp
      projreg.cpp

      Also included is the executable file for the sample application *projhole.arx*, the file defining the functions being exported by the application *projhole.def*, and the Visual C++ 6.0 Service Pack 2 project file for the sample application. For more information on this sample application, see Appendix A.
    - The CountAssemblies directory contains the application that is an assembly counting application sample. For more information on this sample application, see Appendix A.
    - The MfcBrepTrav directory contains the application that is a B-rep traversal application sample. For more information on this sample application, see Appendix A.
    - The MyBrowser directory contains a browser customization sample appication.. For more information on this sample application, see Appendix A.
    - The Which_MDT directory contains a sample application that demonstrates how to locate an installation and version of MDT on an end user's system. For more information on this sample application, see Appendix A.

Installing the Application

To install the material from the CD onto your system, copy the *SDK* directory tree to your hard disk. Change the read-only attribute on the files in the sample directory if you decide to rebuild the sample application. Update all dependencies for the sample application before rebuilding.

The Registry Structure

To prevent conflicts between installations of Mechanical Desktop 3 and 4, the Desktop's registry structure has been changed for this release. The Mechanical Desktop registry keys for MDT4 are now:
autodesk mechanical\acadm
autodesk mechanical\desktop

Running the Application

ObjectARX supports different ways for loading your application. Please refer to the section labeled "Loading an ARX Application" in the chapter "Writing an ARX Application" in the ObjectARX Developer's Guide.

Guidelines for Building Your Application

When building your application, please observe the following guidelines (in addition to those related to building any ObjectARX application):

- Use Visual C++ 6.0 Service Pack 2 for your development environment (the same platform that is supported for ObjectARX development).
- Include *mi.h* wherever you need to call MCAD API functions.
- Link against *mcadapi.lib*.
- Link against *amdt_attributes.lib*.
- Link against the *acge15.lib* for the geometry library.
- Link against the *acgex15.lib* for the ACIS dependent geometry library.
- Link against the *acbr15.lib* for the B-rep library.

Appendix C
LISP/ADS Support For the Autodesk Mechanical Desktop

Autodesk Mechanical Desktop Commands

Invoking Mechanical Desktop Commands From LISP/ADS

Mechanical Desktop commands that are to be invoked through acedInvoke() should be done so via commands that are acedDefun() registered.

Before invoking a Mechanical Desktop command that has a dialog box, you need to set CMDDIA to zero. After the command is complete, restore CMDDIA to its original value. To support LISP/ADS input, the Mechanical Desktop commands are registered using acedDefun(). The following list shows the different ways a value can be provided for the Mechanical Desktop command prompts when invoked through ADS. From LISP the syntax will be different, but the types shown in this list are still valid. See the end of this section for examples.

Integer (acedGetInt)

- RTSHORT, 2,
- RTSTR, "2",

Value must be in decimal units.

Real (acedGetReal)

- RTREAL, 0.5,
- RTSTR, "0.5",

Value must be in decimal units.

Distance (acedGetDist or acedGetReal)

- RTREAL, 0.5,
- RTSTR, "0.5",

Value must be in decimal units.

Angle (acedGetString)

- RTSTR, "0.5",

The value must be Radians if AUNITS is set to three or Decimal Degrees if AUNITS is set to zero.

Keyword (acedGetString)

- RTSTR, "thru"

String (acedGetString)

- RTSTR, "Section A-A"

Point & Corner (acedGetPoint)

- RTPOINT, pt1
- RT3DPOINT, pt2

Cannot pass a relative or polar value.

Single selection (acedEntSel or acedNEntSelP)

- RTLB

:
:
:
RTLE

This list is the standard list containing the information returned from a call to entsel or nentselp. For more specific information on the information contained in this list, see your AutoCAD documentation. If you manually construct this list, you must include, at least, the entity name and the pick location.

For example: RTLB
RTENAME
RTPOINT or RT3DPOINT
RTLE

For many of the Mechanical Desktop commands it might be necessary to use the point and re-select the entity to determine more specific information about what part of the entity was selected. If this re-selection is necessary, the entity must be visible and the viewport the entity was originally selected in must be the currently active viewport. If not, the command will either fail or result in unexpected behavior.

After you've finished providing single selections to a prompt that continues to loop and prompt you for another selection, use RTNONE, RTNIL or RTSTR and pass an empty string. (The AMFILLET command is an example of this.)

The assembly modeling commands are frequently looking for selections of entities contained within blocks. Thus, use nentselp instead of entsel when providing a selection.

Selection set (acedSSGet)

- RTPICKS, ss

When a selection set is passed to one of the Mechanical Desktop commands, it is the caller's responsibility to make sure the set contains valid entity types that were selected with valid selection methods. Selection sets that contain invalid entity types or entities that were selected with invalid selection methods will cause the command to either fail or create unexpected results.

Many of the Mechanical Desktop commands take advantage of additional information that is only available with an original selection set returned from a call to ssget/acedSSGet. Avoid using ssadd/ads_ssadd to create a new selection set or add entities to an existing selection set that is going to eventually be used in a Mechanical Desktop command. In some situations this will not cause problems, but many of the Mechanical Desktop commands will not produce expected results.

Notes

- When invoked through LISP, all of the Mechanical Desktop prompts will work with interactive prompting. To do this, provide the LISP prompt instead of the value. For example, instead of setting the sketch angle tolerance to a hardcoded value, it can be invoked through LISP so that it prompts you for the value.

  (amskangtol 4.5) This assigns the value 4.5 to amskangtol.
  (amskangtol (getreal)) This prompts you for the value to assign to amskangtol.

- When invoked through LISP or ads, all of the Mechanical Desktop prompts will work if a variable of the appropriate type is provided instead of a hardcoded value. For example, a value can be prompted for earlier in the program, assigned to a variable, the variable can be modified, if needed, and the variable can be used when invoking the Mechanical Desktop command.

  From LISP:

```
:
:
(setq angTol (getreal))
:
:       // Modify angTol if necessary.
:
(amskangtol angTol)
:
:
```

  From Ads:

```
:
ads_real angTol;
status = acedGetReal("Enter value:", &angTol);
:
:    // Modify angTol if necessary.
:
struct resbuf *argList, *rslt;
argList = acutBuildList(RTSTR,     "amskangtol",
                        RTREAL,    angTol,
                        0);
status = acedInvoke(argList, &rslt);
```

Examples Invoking Mechanical Desktop Commands

Here are some examples of how you would invoke one of the Mechanical Desktop commands through LISP or ads.

From LISP:

```
(amhole "drill" "thru" 0.5 "no" "twoEdges" (entsel) (entsel) (getpoint) "1.0" "1.0")
or
(setq e1 (entsel))
(setq e2 (entsel))
(setq p1 (getpoint))
(amhole "drill" "thru" 0.5 "no" "twoEdges" e1 e2 p1 1.0 1.0)
or
(amhole "drill" "thru" 0.5 "no" "twoEdges" '(9.3 8.0) '(11.9 6.7) '(-1 -2) -1 -1)
```

From an ads program:

```
struct resbuf *argList, *rslt;
ads_point pt1, pt2, pt3;
    pt1[0] = 9.3;    pt1[1] = 8.0;    pt1[2] = 0.0;
    pt2[0] = 11.9;   pt2[1] = 6.7;    pt2[2] = 0.0;
    pt3[0] = -1.0;   pt3[1] = -2.0;   pt3[2] = 0.0;

argList = acutBuildList(RTSTR,     "amhole",
                        RTSTR,     "drill",
                        RTSTR,     "thru",
                        RTREAL,    0.5,
                        RTSTR,     "no",
                        RTSTR,     "twoEdges",
                        RT3DPOINT, pt1,
                        RT3DPOINT, pt2,
                        RT3DPOINT, pt3,
                        RTREAL,    -2.0,
                        RTREAL,    -1.0,
                        0);
status = acedInvoke(argList, &rslt);
```

System Variables

To get the value of a Mechanical Desktop system variable through LISP or ads, you need to invoke the system variable command without modifying its value. From LISP, the value of the Mechanical Desktop system variables can be obtained by invoking the system variable command and assigning the return value to the appropriate type. From an ads program, the value of the Mechanical Desktop system variables can be obtained from the result field of acedInvoke(). Below is a list of the system variables and the value types returned. At the end of this section are some examples.

Mechanical Desktop Part Modeling

AMCOMPSV	RTSHORT
AMCONDSPSZ	RTSHORT
AMRULEMODE	RTSHORT
AMSKANGTOL	RTREAL
AMSKMODE	RTSHORT
AMSKSTYLE	RTSTR

Drawing Manager

AMANNOTEPRESERVE	RTSHORT
AMCLCM	RTREAL
AMCLGAP	RTREAL
AMCLOSHT	RTREAL
AMCLPAR	RTSHORT
AMCLTYPE	RTSTR
AMDWGCOLOR	RTSHORT
AMHIDLTYPE	RTSTR
AMHLCALC	RTSHORT
AMLINETHICK	RTREAL
AMPROJTYPE	RTSHORT
AMREUSEDIM	RTSHORT
AMSECLTYPE	RTSTR
AMSECTIONDIM	RTSHORT
AMSTDDTL	RTSTR
AMSTDSCT	RTSTR
AMSTDTAP	RTSTR

AMVANISH	RTSHORT
AMVIEWREFRESH	RTSHORT
AMVPBORDER	RTSHORT

Assembly Modeler

AMAUTOASSEMBLE	RTSHORT
AMINSERTABS	RTSHORT
AMSCENEUPDATE	RTSHORT
AMVIEWRESTORE	RTSHORT

Mechanical Desktop Surfaces Modeling

AMBLENDTOL	RTREAL
AMGRPREFIX	RTSTR
AMINTERPOLY	RTSHORT
AMJOINGAP	RTREAL
AMPAGELEN	RTREAL
AMPFITANG	RTREAL
AMPFITLEN	RTREAL
AMPFITTOL	RTREAL
AMPROJOUTPUT	RTSHORT
AMSFDISPMODE	RTSHORT
AMSFTOL	RTREAL
AMULINES	RTSHORT
AMVECAUG	RTREAL
AMVECSF	RTREAL
AMVLINES	RTSHORT

Part Modeling

AMCMDDIM	RTSHORT

Miscellaneous

ACISOUTVER	RTSHORT

Getting the value of a system variable:

From LISP:

(setq vlines (amvlines ""))

or

(setq gap (amjoingap nil))

From ads:

```
struct resbuf *arglist, *rslt;
arglist = acutBuildList(RTSTR,     “amskangtol”,
                        RTSTR,     “”,
                        0);

retVal = acedInvoke(arglist, &rslt);
```

Integrating the MCAD API with Mechanical Desktop Commands

Implementation of the API is ongoing work, and much effort is targeted at providing extra functionality for developers building Autodesk Mechanical Desktop applications. It is expected that, for some time to come, you can only perform certain operations by calling the commands directly.

Calling Mechanical Desktop Commands

Until now, calling commands has presented serious problems for application developers because the commands are designed primarily for user interaction. Those commands that take entities or subentities as arguments require them to be selected from the screen. This places a large burden on the application, for example, to generate pick points and ensure that the picking will select entities uniquely.

An application already using the MCAD API might have need to call certain commands. The 'Keys' used within the MCAD API have to be queried in order that the application calls the commands with the right data.

Example

An application might have the key of an edge that it wants to fillet. It can only create the fillet by calling the command AMFILLET. This command requires that a pickpoint be specified, identifying the edge to be filleted. When used interactively, the user can change the view and inspect the part to determine how to pick the required edge. This is much harder to achieve within an application. Depending on the view, other edges might be hiding the required edge; thus, it might be difficult, or even impossible, for the application to identify the edge to be filleted.

The problem of integrating the MCAD API with the calling of commands has been solved. It is now possible to call the commands directly using the Keys generated within the MCAD API.

Benefits

The benefits of this enhancement are

- Applications are more reliable.
- Applications can be developed immediately, rather than waiting for future versions of the MCAD API.
- Functionality accessible to developers using the MCAD API is dramatically improved.
- Applications are more efficient.
- Changes to the commands do not affect their use in any other way. If developers are not using the MCAD API, changes made are not visible. Any existing calls to commands continue to work unchanged.
- Certain operations can be performed using the commands. This allows the developers of the MCAD API to concentrate on those aspects for which there is no command alternative. This ensures that the MCAD API developers can best meet the needs of third-party developers.

Usage Details

Using the MCAD API, users can generate Keys to certain entities within Mechanical Desktop. For example, the application might have obtained the Key to an edge, represented as an *AmiCurveKey*. If a Desktop command requires an edge to be specified (e.g., positioning a hole from two edges through

AMHOLE), instead of trying to specify the edge to the command by getting the entity name and a point on the edge using the key and passing this information to the command, the Key can be passed directly to the command. The command would be called using acedInvoke, in the normal way.

acedInvoke takes a resbuf chain, which can be constructed using acutBuildList. Here, instead of identifying the edge using RTLB, RTENAME, pName, RT3DPOINT, pickPt, RTLE, use the sequence RTLB, RTLONG, 0, RTLONG, (long) pKey, RTLE. As stated above, the first way of calling the command continues to be supported as well.

Passing Additional Points

The normal method for calling the commands is to use the key, as described above. Certain commands, as well as requiring the key to identify which entity or subentity, require a point to indicate where on the subentity or entity the command should be applied. When this occurs, it is not sufficient to call the command just with the key, but the user also needs to pass in a point. This has to be done in a specific way.

You cannot simply add another resbuf to the chain containing the point. Instead, the long code that is passed into the command should be 1, rather than 0, and the second resbuf in the list should be a pointer to another resbuf chain, cast to a long. This resbuf chain should contain two resbufs. The first is the key, cast as a long. The second is a 3d Point. For example,

```
struct resbuf*pExtraRb = acutBuildList( RTLONG, (long) pKey,
RT3DPOINT, calculatedPoint, 0);
struct resbuf*pRb = acutBuildList( RTSTR, "AMPARDIM",
    RTLB, RTLONG, 1, RTLONG, (long) pExtraRb, RTLE, ...);
```

As a rule of thumb, if the interactive command depends on the position of the pick, as well as the thing picked, then the command is using the pick point to specify this position, and you should use this alternative method for calling the commands.

NOTE: Passing the additional point, in the fashion described here, is not detrimental, even in the case of commands that do not require the extra information.

Sample Code

The attached sample code lets the user pick a face and then traverses through the edges of the face filleting all edges with the same radius.

```
int facefillet()
{
        // Command which lets the user pick a face, specify radius, and fillets all the edges of the
        // face with the provided fillet.

        AmiStatus amiEs;
        AmiPickObj pickObj;
        amiEs = amiPick("\nPlease select face to fillet\n", Ami::surfKey, NULL, pickObj);
        if( amiEs != Ami::eOk )
                return AcRx::kRetError;

  AmiObjectKey *pKey = NULL;
        AmiKeyType keyType;
```

```
amiEs = amiGetKeyFromPick(&pickObj, keyType, pKey);
if(amiEs != Ami::eOk)
        return AcRx::kRetError;

AmiGeomKey* pGeomKey = AmiGeomKey::cast(pKey);
if (pKey == NULL) {
        // This should never happen
        pKey->release();
        return AcRx::kRetError;
}


AcBrEntity* pBrEnt = NULL;
amiEs = amiGetBrepFromKey(pGeomKey, pBrEnt);
pKey->release();

AcBrFace* pFace = AcBrFace::cast(pBrEnt);
if(pFace == NULL) {
        delete pBrEnt;
        return AcRx::kRetError;
}
if(amiEs != Ami::eOk)
        return AcRx::kRetError;

double radius;
if(RTNORM != acedGetReal("\nPlease enter radius of fillet\n", &radius)) {
        delete pFace;
        return AcRx::kRetError;
}

// now construct the calling list
struct resbuf* pCallingRb = acutBuildList(RTSTR, "", RTREAL, radius, 0);

// Some B-rep error handling omitted
AcBr::ErrorStatus brEs;

AcBrFaceLoopTraverser faLoTrav;
brEs = faLoTrav.setFace(*pFace);
if( brEs != AcBr::eOk ) {
        delete pFace;
        return AcRx::kRetError;
}
// keep the keys somewhere, so they can be released later.
AcDbVoidPtrArray keyArray;

for(;!faLoTrav.done(); faLoTrav.next()) {
        AcBrLoopEdgeTraverser loEdTrav;
        brEs = loEdTrav.setLoop(faLoTrav);
        if(brEs != AcBr::eOk) {
                // maybe an isolated vertex, just continue
                continue;
```

```
                }

                for(;!loEdTrav.done();loEdTrav.next()) {
                        AcBrEdge edge;
                        loEdTrav.getEdge(edge);

                        AmiGeomKey* pEdgeKey = NULL;
                        amiEs = amiGetKeyFromBrep(&edge, pEdgeKey);
                        if(amiEs != Ami::eOk) {
                                acutPrintf("Cannot create Key from edge ... continuing\n");
                                continue;
                        }
                        struct resbuf* pRb = acutBuildList(RTLB, RTLONG, 0,
                                RTLONG, (long) pEdgeKey, RTLE, 0 );
                        pRb->rbnext->rbnext->rbnext->rbnext = pCallingRb;
                        pCallingRb = pRb;
                        keyArray.append( pEdgeKey );
                }
        }
        delete pFace;

        struct resbuf*pRb = acutBuildList(RTSTR, "AMFILLET", 0);

        pRb->rbnext = pCallingRb;
        pCallingRb = pRb;

        struct resbuf* pResRb = NULL;

        if(acedInvoke( pCallingRb, &pResRb ) != RTNORM) {
                acutPrintf("Failure calling acedInvoke\n");
        }

        int nKeys = keyArray.length();
        acutPrintf("Applied Fillet with radius %lf to %d edges\n", radius, nKeys );

        for( int i=0; i< nKeys; i++ )
                ((AmiObjectKey*)(keyArray.at(i)))->release();


  // end of function
        return AcRx::kRetOK;
}
```

Appendix D
Developer Partner Guidelines

The following information is excerpted from *Autodesk Mechanical Solutions Developer Partner Guidelines* Revision 1.0, dated June 26, 1997.

Introduction

Autodesk Mechanical product acceptance and quality are measured by customer perception of not only Autodesk products, but of developer partner products as well. This document presents a set of guidelines and certification requirements to assist our developer partners in their responsibility to deliver high-quality products that integrate well into Autodesk Mechanical products. These guidelines, available to all developer partners, lay the foundation for certification, which allows developer partners to brand their products with a badge symbolizing Autodesk Mechanical product integration and interoperability.

Part 1 outlines the "Product Development Guidelines" that ensure developer partner products work well with the current Mechanical Desktop and AutoCAD Mechanical and with other Autodesk products in the future. Developer partners must follow these guidelines to have their products certified by Autodesk.

Part 2 provides the "Installation Development Guidelines" for developer partners who use the MCAD API in their products.

Autodesk recognizes that each developer partner product is unique in its target market, technical complexity, interaction with the Mechanical Desktop, and packaging. This document, which will change over time with product evolution, is a guideline for all of these products. As new requirements are identified, they will be incorporated into future versions of this document.

If you have any questions about whether your product(s) meet these requirements, contact your standard registered developer support channel.

Part 1

Product Development Guidelines

These guidelines, available to all developer partners, must be followed by those who wish to have their products certified by Autodesk. The guidelines ensure that developer partner products work well with the current Mechanical Desktop, AutoCAD Mechanical, and with other Autodesk mechanical products in the future.

1. Follow All ObjectARX Developer Guidelines

Meaning All Autodesk issued guidelines for development of ObjectARX applications, which are included in the ADN CD collection, must be followed. When a certification process is put in place for ObjectARX applications, all the requirements for that process must be satisfied.

Motivation Your application is an ObjectARX application. Thus, you must follow all basic ObjectARX guidelines as well as all developer partner development and certification guidelines to ensure integration into our environment.

Considerations Autodesk provides extensive documentation, training, and email support to enable developers to achieve this goal.

2. Be Compatible with Autodesk Mechanical Solutions and Other Partner Applications

Meaning Your application should not conflict in any way with the Autodesk Mechanical solutions or with other partner applications. This includes, but is not limited to, installation, de-installation, database coexistence, loading and unloading of files, handling of menus and help files, ability to go back and forth between Autodesk Mechanical solutions and partner applications, and opening of files containing objects created by your application when your application is not present. Applications should have capability to load and unload during an Autodesk Mechanical solution session, allowing the user to control overall memory usage.

Motivation End users want to be able to mix and match different developer partner applications with the Autodesk Mechanical solutions.

Considerations Many of the issues raised here are addressed by the use of commonly known, good programming practices. If you have access to other developer partner applications for the Mechanical Desktop, test your application with these products.

3. Have a Consistent End User Paradigm

Meaning Design your application to look and feel like a natural extension of AutoCAD and the Autodesk mechanical solutions and to be Windows compliant.

Motivation Users want to have seamless integration without drastic paradigm changes as they use AutoCAD, the Autodesk mechanical solutions and developer partner applications.

Considerations In some cases it might make sense to deviate from the established paradigm to enable new functionality or facilitate a task, but this should be the exception rather than the rule and needs to be justified.

4. Use AutoCAD Tools

Meaning Your product should use, as much as possible, tools that are native to the AutoCAD environment. This includes, but is not limited to, utilizing the standard mechanisms for graphics display, manipulation, picking/grips, and highlighting; file handling; entity copy, delete, move, rotate, mirror and scale; grouping and blocking; and rendering, if appropriate. If some of these standard AutoCAD operations are not applicable, it is your responsibility to react to operations accordingly. Note that the use of MFC tools is also encouraged.

Motivation End users want to continue using familiar AutoCAD interactions whenever possible.

Considerations By having your application objects derive from AcDbObject and AcDbEntity, and by implementing the appropriate virtual methods, many AutoCAD native commands automatically work on your entities.

5. Have Complete Geometric Coverage

Meaning Operations that involve geometry should work on all applicable types, including all types of curves and surfaces, geometry that is wireframe, Mechanical Desktop surfaces, a solid, a parametric part, an assembly component, inside block references, and external databases. If an operation is appropriate only for a particular geometry type, the user interface should trap the condition and clearly notify the user.

Motivation Users do not want their applications to work for only certain geometric configurations.

Considerations Full geometric coverage is easily accomplished through the use of the geometric keys provided by the MCAD API. By using these keys, developers are automatically relieved of dealing with the underlying details of the type(s) of geometry that are members of composite

entities such as polylines, blocks, or groups; inferred geometry such as end points, mid points, center points, axes, or planes; and geometry that is externally referenced from other files.

6. Extract All Model Information

Meaning Information that is already present in an Autodesk Mechanical solution model should be extracted automatically by applications that need it. Users should not have to re-enter common attribute information, such as material properties. Other, higher level examples would be that assembly constraints can be extracted automatically from the model and used in defining joints for a kinematics application, loads calculated by kinematics applications can be used by FEA applications, and that design parameters can be used for shape optimization in a CAE or mold flow application.

Motivation Users do not want to spend time reentering information that was previously specified to the system.

Considerations A developer who believes that it is not possible to access the required information should contact Autodesk to remedy the problem.

7. Be Associative

Meaning All your application objects which are related to objects in an Autodesk mechanical solution model should have the ability to update themselves in response to changes in the Autodesk Mechanical solution model.

Motivation Users expect applications to be intelligent in their association to the underlying model across edits to the model.

Considerations The MCAD API provides a variety of tools that make it straightforward for developers to write associative applications in a variety of flavors to fit different application paradigms. By retaining MCAD API keys, an application can update geometric and other entities across model regenerations, add attributes that are associative, and get active and passive notification of changes.

8. Use Standard Attributes and Objects

Meaning Sharing of higher level information requires the use of persistent, sharable attributes created by developer partners. Developer partners should use, whenever applicable, standard attributes, MCAD API instantiable attributes, objects defined by the MCAD API, and additional attribute data defined by developer partner applications instead of defining their own versions. If an application creates objects which have a natural representation within the Mechanical Desktop, that representation should be used. For example, solids should be represented using parts; assembly structures should be made as Mechanical Desktop assemblies.

Motivation The use of commonly defined objects promotes interoperability across different applications. In addition, a standard attribute that is attached by one application can be operated upon by another application, even when the application that planted the attribute is not loaded.

Considerations The instantiable attributes mechanism provided by the MCAD API in combination with standard attribute definitions provide a convenient way for applications to deal with attributes without the burden of having to write supporting code for the classes involved.

9. Use DWG File to Store All Model Data

Meaning Have all the data that defines your model saved with the DWG file format instead of using additional files to save part of the model definition.

Motivation This approach avoids complications when users exchange data. It facilitates support for several operations involving database manipulation (such as undo/redo operations).

Considerations Large data sets could represent an exception to this rule. Examples would be derived NC tool paths, and FEA results. In applications that have large data sets, the user should be able to store the file without this data if so desired.

10. Support Undo/Redo

Meaning All your application objects should fully support undo and redo operations as presented by AutoCAD.

Motivation Users have come to expect this, and it is a key feature of our environment.

Considerations This can be accomplished by following the ObjectARX development guidelines concerning implementation of the appropriate dwgin/out methods for all application classes.

11. Have a High Level of Quality Assurance

Meaning The development of your application should be done in a context of high concern for overall quality.

Motivation The development environment that ObjectARX/MCAD APIs provide is very powerful and complex. Quality problems in your application have the potential to impact other applications as well. Users expect the highest standards of quality from suites of partner applications.

Considerations This is, in a way, an obvious point, but it should not be taken for granted.

12. Support Globalization

Meaning Your application should be localization ready for all the markets where Autodesk mechanical products are sold. The ObjectARX developer guidelines provide information on how to structure your application to achieve this.

Motivation Ours is a global market, and having the applications adapted to the different countries where they are sold is a key user requirement.

Considerations In order to have this process go smoothly, pay attention to globalization throughout the product development. Autodesk has available documentation, strategies, and training material to help developers support globalization in their products. This information can be obtained from your standard registered developer support channels.

13. Stay Current

Meaning Continue to update your application to account for enhancements made to AutoCAD, the Autodesk mechanical solutions, the MCAD APIs, and to common object definitions. The developer partner is responsible for validating the compatibility of its products with all new versions of Autodesk mechanical solutions and the MCAD API that are made available to developers.

Motivation As we continue to make improvements in Autodesk mechanical products it is in the best interest of everyone that these enhancements be reflected in additional functionality in your products. It is also important that all applications are available as soon as possible after a revision of the Mechanical Desktop is available.

Considerations We plan to enhance the Autodesk mechanical products and APIs on an ongoing basis and to keep developers informed of these advances. Many of our future enhancements will

originate from requests and feedback from developer partners and developers in general. ObjectARX and especially the MCAD API provide upward compatibility between Autodesk Mechanical solutions revisions, allowing developer partners to rapidly validate support for new revisions.

14. Pursue Interoperability with Other Partner Applications

Meaning Strive to have your application work synergistically with other certified developer partner applications.

Motivation End users experience great benefit from suites of applications that work well together. For example, an analysis application could use forces calculated by a dynamics application as load conditions.

Considerations To achieve interoperability, partners should interact closely with each other. In addition, Autodesk will continue to promote and assist in these cooperation efforts.

Part 2

Installation Development Guidelines

These guidelines for the design of developer partner applications installations must be followed.

1. What Product to Use for Installation

The Autodesk Mechanical Desktop uses the InstallShield product for its installation. We recommend that developer partners do the same. More information can be found at the InstallShield web site at http://www.installshield.com.

2. Menu Changes

The Autodesk Mechanical Desktop uses a menu replacement method. Developer partners should use the Menuload option when they need to add items to a menu or to add a new pulldown. This will allow the addition of new pulldowns to a menu without editing menus added by the Mechanical Desktop or by other developer partner applications. For more details see chapter 4, "Custom Menus," in the *AutoCAD Customization Guide*.

3. The ACAD.RX and Loading of Applications

It is highly recommended that you consider using demand loading of your application instead of using the acad.rx. Refer to "Loading an ARX Application" in Chapter 3 of the ObjectARX Developer's Guide for information about demand loading.

4. Installation of Your Application in Its Own Directory

A developer partner application should install into its own directory off a root drive. To have your application run with the Mechanical Desktop, your installation will have to edit the *acad.ini* found in the *mcad* or equivalent directory to add your path to the *acad* path statement. This is needed to load your application when the Mechanical Desktop is loaded.

5. Look and Feel of Your Application Installation

The look and feel of a developer partner application installation should match that of the Mechanical Desktop. Using InstallShield automatically ensures this is done.

Reference:
Your Most Frequently Asked Questions

Beginning in January 1998, a list of common questions and answers compiled by the Developer Consulting Group will be available to registered ADN developers at http://www.autodesk.com/adn.

What is a key?
A key is an object that an application uses to reference various kinds of entities.

How do I create a key?
Keys can be created from user pick, from a B-rep API B-rep entity, from data collected in a resbuf, from database ids (and subids), from a full subentity path, or from another key.

Is there a way of creating a key without user interaction?
Yes. Use *amiFillPickObj(), amiGetKeyFromId(), amiGetKeyFromPath(),* and *amiGetKeyFromBrep()* to create keys without user interaction.

Now that I have a key, what can I use it for?
The key allows the application to reference the object referred to persistently. The application stores this key in its data structures, and if it needs to obtain information about the object, it uses the key. The types of information that can be obtained from a key include

- Getting the B-rep entity corresponding to the key
- Obtaining the geometry of the key
- Obtaining the attributes attached to the entity the key refers to

In addition, a key can be used to ask for notification of changes affecting the entity, and highlighting the entity.

Are keys and attributes persistent between drawing sessions?
Yes, the keys and attributes can be stored in the database and are, therefore, persistent.

How are keys stored in the database?
The application using the key is required to store the key in the database. For example, the sample application stores the custom object MaiObject, which contains the key in the NamedObjectDictionary. You are advised to create your own AcDbDictionary within the NamedObjectDictionary, and store your objects in there. To avoid conflicts, the name of your dictionary should start with your developer symbol. Typically, the key will be referenced by some custom object in the owning application. The custom object will need to save its use of the key using the *amiWriteKey* function.

How are attributes stored in the database?
The application using the attribute is required to store the attribute in the database. For example, the sample application stores the attribute in the NamedObjectDictionary.

Are the attributes proxies when my application is not present?
Yes, for application attributes that are instances of a user-defined class derived from *AmiAttribute*. All objects derived from AcDbObject are proxies when the application that defined their class is not present. Application attributes that are instances of the planned instantiable attribute class will not be proxies.

NOTE: Custom objects that might hold keys are also proxies when the application that created them is not present.

How can I see what objects a given attribute is attached to?
The *amiGetAttributeHolders()* function creates an array of keys to objects to which the attribute is attached. These keys will be new keys, not the ones through which the attribute was originally attached. You may want to compare the keys returned to your existing keys to see if they are equal with *amiAreKeysEquivalent().*

Does active notification allow me to stop a change taking place?
In general, no. You can react to the changes, but cannot veto the changes taking place.

Do I need to store all the application data inside the AutoCAD database?
No, this might not make sense, depending on the application. For example, an NC application will certainly want to store objects in the database, associated with particular faces, to ensure that the NC toolpath is calculated associatively. The toolpath data itself, however, could be stored in an external file. This would reduce the size of the drawing, while still preserving the associativity of the NC path.

How do I get the geometry of a curve key?
This is done by obtaining the geometry of the key, and then obtaining the native curve from this, as shown by the following code snippet (the sample application provided with the MCAD API contains detailed code to illustrate this).

```
stat = amiGetGeomData(pCurveKey pGeEntity);
if( stat != Ami::eOk ) {
      acutPrintf("Cannot get geom key %d\n", stat );
      return;
}
int type = pGeEntity->type();
if( type != AcGe::kExternalCurve3d ) {
      acutPrintf("Curve is not external");
      return;
}
AcGeCurve3d* pCu;
AcGeExternalCurve3d *pGeExtCu = (AcGeExternalCurve3d *) pGeEntity;
if( !pGeExtCu->isNativeCurve( pCu ) ) {
      acutPrintf("Curve is not a native curve\n");
      return;
}
if( pCu->type() != AcGe::kCircArc3d ) {
      acutPrintf("Curve is not a circle\n");
      return;
}
```

What happens to geometric keys if the containing Entity is replaced?
Certain operations will cause an Entity to be replaced by another kind of entity. The resulting entity will visually appear the same to the user. For example, calling the command EXPLODE on a Part will cause the Part to be replaced by an AcDb3dSolid. The Solid contains the same B-rep information as the Part but without the Part's parameter and feature data.

If keys existed on any geometry of the Part prior to the explode, they will be lost during the explode, even though the resulting Solid contains subentities that are geometrically identical. If the key has active notification enabled, the notification will be sent that the object referenced by the key has been erased. The following are known cases where geometric keys are lost:

- Creating a component from a Solid or Part, using AMNEW
- Creating a Part from a Solid using AMNEWPART
- Creating a Region from a set of Curves, using REGION
- Exploding an entity, such as Component, Part, Solid, Region, or Body

Does the MCAD API allow me to access the data of Mechanical Desktop objects that are currently proxies because the Mechanical Desktop is not loaded?
No, the MCAD API will not be able to access AutoCAD Designer or AutoSurf entities unless these programs are loaded.

Reference:
Alphabetical Listing of Enum Types

Enum Type	Category/Enum	Additional Information	Header File
AbsoluteLocType: Absolute locator type.	kWorldXY	World XY planar location.	mifeat.h
	kWorldYZ	World YZ planar location.	
	kWorldZX	World ZX planar location.	
	kUCS	Planar location on current UCS.	
	kArbitrary	Arbitrary planar location (error condition).	
AnnotAttribute	annAttAnnotType		midm.h
	annAttLocation		
	annAttComputedVal		
	annAttMeasuredVal		
	annAttBorder		
	annAttContent		
	annAttGeometry		
	annAttAttachPt		
AnnotType	kNoteGeneral	Generic (any note).	midm.h
	kNoteVwLabel		
	kNoteDetLabel		
	kNoteDetSymbol		
	kNoteSecSymbol		
	kNoteDim		
	kNoteHole		
	kNoteTapHole		
	kNoteCtrLine	Centerline	
	kNoteMechanical	Generic (any mechanical note).	
	kNoteBubble	Balloon callout.	
	kNoteWeld	Weld symbol.	
	kNoteSurface	Surface texture annotation.	
	kNoteFeature	Feature ID.	
	kNoteCntrlFrame	Control frame.	
	kNoteDatumTarget	Datum target.	
	kNoteDatumId	Datum identifier.	
	kNotePartList		
	kNoteUserDef	AcDmUserDefnNote annotation.	
	kNoteLabels	Old.	
	kNoteHoles	Old.	
	kNoteRegHole	Old.	
	kNoteTapHoleDisp	Old.	
AtomLocatorType: Locator type enumeration.	kConcentricLoc	Concentric locator.	mifeat.h
	kDistFromLoc	Distance from locator.	
	kAlignedLoc	Aligned locator.	
	kPerpendicLoc	Perpendicular locator.	
	kParallelLoc	Parallel locator.	
	kCollinearLoc	Collinear locator.	
	kTangentLoc	Tangent locator.	
	kHorizontalLoc	Horizontal locator.	
	kVerticalLoc	Vertical locator.	
	kCoincidentLoc	Coincident locator.	
	kOnPlaneLoc	On plane locator.	
	kAbsoluteLoc	Absolute locator.	
	kAngledLoc	Angled locator.	

Enum Type	Category/Enum	Additional Information	Header File
	kOffsetLoc	Offset locator.	
	kCoordSysLoc	Coordinate system locator.	
AttrAssocType	kNoComments	Do not specify how the attribute is handled. The current default behavior is to share the attribute.	miattrib.h
	kCopyAlways	Also copy the attribute and attach the new attribute copy to the new object.	
	kShareAlways	Attach the same attribute to the new object.	
	kIgnoreAlways	Do not attach this attribute to the new object.	
AttributeDataType	attTypeInt		mibase.h
	attTypeDouble		
	attTypeCharPtr		
	attTypeEnum		
	attTypePoint3d		
	attTypeMatrix3d		
	attTypeEntity3d		
	attTypeObjectKey		
	attTypeValue		
	attTypeExternalId		
	attTypeSpecial		
AttrMatchType: When retrieving all attributes, this type is used to qualify the attributes returned by type.	kInstantiable		miattrib.h
	kAmiDerived		
	kInstOrDerived		
AttSpecialValues	attNAV	NAV: Not a value.	mibase.h
AxisDirType	kPositiveX		mibase.h
	kNegativeX		
	kPositiveY		
	kNegativeY		
	kXAxisPos		
	kYAxisNeg		
	kXAxisNeg		
	kYAxisPos		
ChamferType: Chamfer type enumeration.	kEqualDist	Equal distance chamfer.	mifeat.h
	kTwoDist	Two distances chamfer.	
	kDistAngle	Distance and angle chamfer.	
CombinerType: Combiner type enumeration.	kBase	Create base feature operation.	mifeat.h
	kJoin	Join operation.	
	kCut	Cut operation.	
	kIntersect	Intersection operation.	
	kSplitPart	Split operation.	
ConstraintType	eConstraint		miconstr.h
	eCompConstraint	Assembly constraint, no type information	
	eCompMate	Used for creating a mate, e.g. distance constraint, no argument type specified	
	eCompFlush	Used for creating a flush, e.g. distance constraint, no argument type specified.	
	eCompInsert	Used for creating insert constraints between two circular edges or faces	
	eCompAngle	Used for creating an angle constraint between to model edges	

Enum Type	Category/Enum	Additional Information	Header File
	eSketchConstraint	Sketch constraint, no type information	
	eSkHorizontal	Sketch horizontal constraint – query and creation	
	eSkVertical	Sketch vertical constraint – query and creation	
	eSkParallel	Sketch parallel constraint – query and creation	
	eSkColinear	Sketch colinear constraint – query and creation	
	eSkPerpendicular	Sketch perpendicular constraint – query and creation	
	eSkProjection	Sketch projection constraint – query and creation	
	eSkPointCoincidence	Sketch coincident point constraint – query and creation.	
	eSkCoincident	Sketch coincident constraint – query and creation.	
	eSkConcentric	Sketch concentric circles constraint – query and creation	
	eSkTangent	Sketch tangency constraint – query and creation	
	eSkXEqual	Sketch align circle centers along x axis constraint – query and creation	
	eSkYEqual	Sketch align circle centers along y axis constraint – query and creation	
	eSkEqualRadius	Sketch equal radius constraint – query and creation	
	eSkCircleCoincidence	Sketch coincident arc constraint	
	eSkPointCurveCoincidence	Not used	
	eSkCenterPoint	Not used	
	eSkEqualLength	Sketch constraint to maintain that two lengths are equal	
	eSkFixed	Sketch fixed geometry constraint—query and creation	
	eSkMirror	Sketch mirror constraint—query and creation	
	eSkDistance	Sketch distance dimension – general type	
	eskAngle	Sketch angle dimension	
	eSkRadius	Sketch radius dimension	
	eSkHorDistance	Sketch horizontal distance dimension	
	eSkVerDistance	Sketch vertical distance dimension	
	eSkAlignDistance	Sketch aligned distance dimension	
	eSkDiameter	Sketch diameter dimension	
	eSkEllRadiusMajor	Sketch ellipse major radius dimension	
	eSkEllRadiusMinor	Sketch ellipse minor radius dimension	
	eSkAlignDiamDistance	Sketch constrain a line to be a set distance from a point, anywhere around the point	
	eSkOrdXBase	Sketch the base point for x ordinate dimensions (0 point)	
	eSkOrdYBase	Sketch the base point for y ordinate dimensions (0 point)	
	eSkXOrdDistance	Sketch distance along x axis from the x base dimension	
	eSkYOrdDistance	Sketch distance along y axis from the y base dimension	
	eSkParaDistance	Sketch parallel distance dimension	

Enum Type	Category/Enum	Additional Information	Header File
	eSkSize	Used to signify the last member of the enumerated set for used during iteration etc.	
ConstrAttribute	csAttConstrType	AmiConstraintType	miconstr.h
	csAttParametric	Parametric value associated with constraint, AmiValue	
	csAttLocation	Location for the display of the constraint/dimension	
	csAttDisplayable	Read-only flag indicating whether the constraint is displayable in a persistent mechanism	
	csAttDisplayed	Flag indicating (during query) if the constraint is displayed or (during construction) should be displayed	
	csAttDisplayId	The database id of the entity used to display the constraint	
	csAttMaxOperands	Read-only data indicating the maximum number of operands the constraint supports	
	csAttMinOperands	Read-only data indicating the minimum number of operands the constraint requires	
	csAttOperand1	Geometry key referencing the first operand of the constraint	
	csAttOrientation1	AmiNormalOrientnType indicating whether the constraint should be enforced with the existing normal or a flipped normal for the referenced geometry	
	casAttOperand2	Geometry key referencing the second operand of the constraint	
	csAttOrientation2	AmiNormalOrientnType indicating whether the constraint should be enforced with the existing normal or a flipped normal for the referenced geometry	
CoordinateSystem: Work plane coordinate system.	kRightHand	Right handed coordinate system.	mifeat.h
	kLeftHand	Left handed coordinate system.	
DesignEnvironment Type	kPartAssembly	Database contains only components, no parts.	mibase.h
	kSinglePart	Database contains only a single part, no components.	
	kProxy	File was created on a newer version of Mechanical Desktop. This state will never be saved to a file.	
	kMultiPart	Database contains more than one part, no components (legacy data).	
	kMixed	Database contains both components and parts (legacy data).	
	kForeign	No parts or components in file.	
	kUnknown	Mechanical Desktop applications not loaded—file state cannot be determined.	
DesignModelType	kModelMode	Developing the model, whether assembly or part.	mibase.h
	kDrawingMode	Creating drawings.	
	kSceneMode	Creating scenes for assemblies.	
	kUninitialized	When there is no database.	
DOFAttribute:	dofAttNumVars	Number of vars in constraints/equations.	miDOF.h

Enum Type	Category/Enum	Additional Information	Header File
Degree of Freedom (DOF) information. Used to describe classic rigid-body motion in space, or for DOFs associated with systems of equations/constraints, e.g. sketching.	dofAttNumFreeVars	Number of free vars.	
	dofAttNumDOF	Number of rigid-body DOF remaining	
	dofAttNumRDOF	Number of Rotational DOF remaining	
	dofAttNumTDOF	Number of Translational DOF remaining.	
	dofAttRotationAxis	Axis/axes of rotation.	
	dofAttTranslationAxis	Axis/axes of translation.	
	dofAttRotationPoint	Point on rotation axes (if needed).	
DrivingData	kParamData	Table driving parameters.	mifile.h
	kFeatData	Table driving feature suppression.	
	kParamAndFeatData	Table driving both parameters and feature suppression.	
EventContextType: Event context enumeration. Must be synchronized with AcImAppReactor::ImReactorContext.	kNullContext	Event occurred during typical modeling processes	mievent.h
	kNewDrawing	Event occurred during while creating a new drawing	
	kDeepCloneToCurDwg	Event occurred cloning objects into the current drawing	
	kDeepCloneFromCurDwg	Event occurred cloning objects out of the current drawing	
	kDeepCloneWithinCurDwg	Event occurred cloning objects within the current drawing	
EventType	kFirstEvent	Bookkeeping event—always the first event.	mievent.h
	kOnAttachFile	Version/parameter controlling file was attached AmiFileNotifFcnPtr	
	kOnDetachFile	Version/parameter controlling file was detached AmiFileNotifFcnPtr	
	kOnNewVersion	New version definition was added to a version/parameter file. AmiFileVerFcnPtr	
	kOnEraseVersion	Not Supported	
	kOnActivateVersion	Version was activated. AmiKeyVerFcnPtr	
	kConflictDetected	Table notification. Not supported.	
	kConflictResolved	Table notification. Not supported.	
	kOnActivatePart	Designer event, part activated. Ami1KeyNotifFcnPtr	
	kOnNewPart	Designer event, part created. Ami1KeyNotifFcnPtr	
	konUpdatePart	Designer event, part compute. Ami1KeyNotifFcnPtr	
	kOnErasePart	Designer event, part erased. Ami1KeyNotifFcnPtr	
	kOnFeatureAdded	Designer event, feature created. Ami1KeyNotifFcnPtr	
	kOnFeatureRemoved	Designer event, feature erased. Ami1KeyNotifFcnPtr	
	kOnFeatureRenamed	Designer event, feature renamed. Ami1KeyNotifFcnPtr	
	kOnFeatureVersioned	Designer event, feature excluded due to versioning. Ami1KeyNotifFcnPtr	
	kOnFeatureSuppressed	Designer event, feature suppressed. Ami1KeyNotifFcnPtr	
	kOnFeatureUnversioned	Designer event, feature reactivated due to versioning. Ami1KeyNotifFcnPtr	

Enum Type	Category/Enum	Additional Information	Header File
	kOnFeatureUnsuppressed	Designer event, feature unsuppressed. Ami1KeyNotifFcnPtr	
	kOnWorkGeomAdded	Work-geometry created. Ami1KeyNotifFcnPtr	
	kOnWorkGeomRemoved	Work-geometry erased. Ami1KeyNotifFcnPtr	
	kOnWorkGeomRenamed	Work-geometry renamed. Ami1KeyNotifFcnPtr	
	kOnWorkGeomVersioned	Work-geometry excluded due to versioning. Ami1KeyNotifFcnPtr	
	kOnWorkGeomSuppressed	Work-geometry suppressed. Ami1KeyNotifFcnPtr	
	kOnWorkGeomUnversioned	Work-geometry reactivated due to versioning. Ami1KeyNotifFcnPtr	
	kOnWorkGeomUnsuppressed	Work-geometry unsuppressed. Ami1KeyNotifFcnPtr	
	kOnFeatureReorder	Feature moved above destination feature Ami2KeyNotifFcnPtr	
	kOnReplayTruncated	Replay notification. Not supported.	
	kOnFeatureReplayed	Replay notification. Not supported.	
	kOnNewConstraint	Constraint. created Ami1KeyNotifFcnPtr	
	kOnEraseConstraint	Constraint erased. Ami1KeyNotifFcnPtr	
	kOnNewCompDef	Assembly events. Comp Def Created Ami1KeyNotifFcnPtr	
	kOnNewComp	Assembly events. Component added Ami1KeyNotifFcnPtr	
	kOnNewScene	Assembly events. Scene added Ami1KeyNotifFcnPtr	
	kOnNewCompView	Assembly events. Component view added Ami1KeyNotifFcnPtr	
	kOnEraseCompDef	Assembly events. Comp Def erased Ami1KeyNotifFcnPtr	
	kOnEraseComp	Assembly events. Component erased Ami1KeyNotifFcnPtr	
	kOnEraseScene	Assembly events. Scene erased Ami1KeyNotifFcnPtr	
	kOnEraseCompView	Assembly events. View erased Ami1KeyNotifFcnPtr	
	kOnRenameComp	Assembly events. Component renamed Ami1KeyNotifFcnPtr	
	kOnRenameCompDef	Assembly events. Comp def renamed Ami1KeyNotifFcnPtr	
	kOnContextSwitch	Assembly events. Current comp def context changed Ami1KeyNotifFcnPtr	
	kOnReorderComp	Assembly events. Component order changed Ami1KeyNotifFcnPtr	
	kOnExternalizeCompDef	Assembly events. Formerly local Comp def externalized to a file Ami1KeyNotifFcnPtr	
	kOnLocalizeCompDef	Assembly events. Formerly external comp def localized Ami1KeyNotifFcnPtr	
	kLastEvent	Bookkeeping event. Always the last event. Used to control iterations.	
FeatState: Feature state enumeration. Used to create bit	kAll	Used when querying for an array of features means to return all features.	mifeat.h
	kActive	The feature is not currently excluded.	

Enum Type	Category/Enum	Additional Information	Header File
fields telling functions returning features what feature states we're interested in. For example, if you call "amiGetPartFeats" and want all features, you pass in "Ami::kAll". If you want only active features, pass in "Ami::kActive". If you want only active and suppressed features, pass in "ami::kActive | Ami::kSuppressed".	kSuppressed	The feature was excluded via the AMSUPPRESS command or the API function AmiSuppressFeat.	
	kVersioned	The feature was excluded via a spreadsheet.	
	kDirty		
	kComputeFailed		
FeatType: Feature type enumeration.	kHole	General hole feature	mifeat.h
	kDrilledHole	Drilled hole feature	
	kBoreHole	Counter bore hole feature	
	kSinkHole	Counter sink hole feature	
	kExtrusion	Extrusion feature	
	kRevolve	Revolution feature	
	kChamfer	Chamfer feature	
	kFillet	General fillet feature	
	kSweep	Sweep feature	
	kArray	General feature array feature	
	kRecArray	Rectangular feature array feature	
	kPolarArray	Polar feature array feature	
	kBaseFeat	Base feature	
	kSurfCut	Surf cut feature	
	kUniFillet	Uniform radius fillet feature	
	kIndFillet	Individual radius fillet feature	
	kFixedFillet	Fixed radius fillet feature	
	kCubicFillet	Cubic variation fillet feature	
	kLinearFillet	Linear variation fillet feature	
	kShell	Shell feature	
	kParametricBool	Parametric boolean (combine) feature	
	kConstrGeom	General constrained geometry	
	kWorkPoint	Work point	
	kWorkAxis	Work axis	
	kWorkPlane	Work plane	
	kLoft	Loft feature	
	kDraft	Face draft feature	
	kPlaneDraft	Draft from plane feature	
	kEdgeDraft	Draft about edge feature	
	kShadowDraft	Shadow draft feature	
	kSplit	Split feature	
	kFaceSplit	Face split feature	
	kPartSplit	Part split feature	
	kSketchPlane	Sketch plane	
FileAttribute: File descriptor attributes.	kAttFileType		mifile.h
	kAttFullPath	Full path to file on disk	
	kAttManaged	Flag indicating whether the file is managed by the active PDM system	

Enum Type	Category/Enum	Additional Information	Header File
	kAttOwningApp		
	kAttGroupName	For Managed files – the group to associated with the file	
	kAttProperties	For Managed file – the predefined properties flag indicating the type of file for the PDM's use	
	kAttOwnerId	Managed file attributes.	
	kAttVersionDir	Are versions defined across rows or down columns of the spread sheet	
	kAttHasInfoFor	The spreadsheet has information for parameters, suppression or both	
	kAttStructure	Are the sections for parameters and suppression on one sheet (concatenated) or on separate sheets	
	kAttFeatSheet	Name of the sheet holding feature suppression information	
	kAttFeatCell	Cell id for the first cell of feature suppression information	
	kAttParamSheet	Name of the sheet holding parameter definitions	
	kAttParamCell	Cell id for the first cell of parameter definitions	
	kAttVersionedObj	Id of the object controlling this file	
FileType	kVersioningFile	File used in Table Driven Parts to define versions within a model	mifile.h
	kParameterFile	File containing parameter definitions	
	kSpreadsheet	General spreadsheet	
	kExternalInput	External file used for input to a process	
	kExternalOutput	External file created as output from a process	
HiddenLineTreatment: The kHiddenLinesNotDisplayed setting causes hidden lines to be generated and stored, not just displayed. Therefore, the kHiddenLinesDisplayed and kHiddenLinesNotDisplayed settings will exhibit roughly the same performance and file sizes, while kHiddenLinesDiscarded is more efficient when the lines are not expected to be displayed. They can be regenerated when switching to kHiddenLinesDisplayed. Relevant only for ViewDispType !=kWireFrame.	kHiddenLinesDisplayed	Show calculated hidden lines.	midm.h
	kHiddenLinesNotDisplayed	Don't show hidden lines.	
	kHiddenNoTangDisplayed	Don't show and don't store hidden lines.	
KeyType:	objectKey		mibase.h

Enum Type	Category/Enum	Additional Information	Header File
Enumeration of valid key types	geomKey		
	pointKey		
	curveKey		
	lineKey		
	arcKey		
	ellipseKey		
	splineCrvKey		
	polylineKey		
	augLineKey		
	surfKey		
	planeKey		
	coneKey		
	cylinderKey		
	sphereKey		
	torusKey		
	splineSrfKey		
	constrKey		
	compConstrKey		
	sketchConstrKey		
	paramKey		
	sketchKey		
	featKey		
	bodyKey		
	solidKey		
	partKey		
	compKey		
	compDefKey		
	sceneKey		
	vectorKey		
	dwvwKey		
	DwAnnotKey		
	sketchPlaneKey		
	size	Keep this as last enum in the list.	
LoftAlignment: Loft twist type.	kAlignNone		mifeat.h
	kAlignAutomatic		
	kAlignProportional		
	kAlignCorner		
LoftType	kLinear		mifeat.h
	kCubic		
	kClosedCubic		
ModelStateType	kActiveStable	Is the current active part, and is stable.	mibody.h
	kActiveEdited	Edited, needs regen.	
	kInactive	Not the active part, and not under edit.	
	kAffected	Not active, but affected by edits.	
	kMaybeAffected	Maybe affected by current edits, not active.	
	kMaintenance	Active part but in a maintenance mode.	
NormalOrientnType	kRaw	The geometry normal should be interpreted as it is native in the model	mibody.h
	kReversed	The geometry normal should be interpreted as the inverse (or flipped) from how it occurs in the model	
NotifType	kAdded		midm.h
	kEdited		
	kRemoved		

Enum Type	Category/Enum	Additional Information	Header File
OrientationType	kRight	Ortho.	midm.h
	kLeft	Ortho.	
	kTop	Ortho.	
	kBottom	Ortho.	
	kFront	Unused.	
	kBack	Unused.	
	kTopRight	Iso.	
	kBottomLeft	Iso.	
	kTopLeft	Iso.	
	kBottomRight	Iso.	
	kBaseOrientation	Base.	
PolarAngleType: Polar array angle type.	kNone	No angle.	mifeat.h
	kFillAngle	Angle specifies total angle of rotation.	
	kFullAngle	Rotate full circle.	
	kIncrAngle	Angle specifies angle between instances.	
ReactAttribute: Reaction attribute keyword enumeration.	kFirstAttribute	Data type. Bookkeeping attribute: always the first attribute.	mievent.h
	kEvent	AmiEventType	
	kFunction	Pointer to the callback function. Ami1KeyNotifFcnPtr, Ami2KeyBoolNotifFcnPtr	
	kLastAttribute	Bookkeeping attribute: always the last attribute.	
SceneCompAttribute	scpAttTransform	Transformation matrix.	miscene.h
	scpAttIsSuppressed	Visibility within scene.	
	scpAttExplodeFactor	Explosion factor override.	
SectionCutDirection	kCutRight	Cutting direction for kSecHalf.	midm.h
	kCutLeft	Cutting direction for kSecHalf.	
SectionStructure	kConcatenated	Data sections joined together.	mifile.h
	kSeparate	Data sections are separate.	
SectionType	kSecNone		midm.h
	kSecHalf		
SketchType	kWorkPointSketch	Work point.	misketch.h
	kClosedProfile	Closed profile.	
	kPath	2d path.	
	kCutLine	Cut line.	
	kUnknownSketch	Unknown (error condition).	
	kSplitLine	Split line.	
	kAxisProfile	Axis profile.	
	kHelicalPath	3d helical path.	
Status: Error status for the MCAD API.	Codes shared with Acad::ErrorStatus		mibase.h
	eOk	Generic goodness (== Acad::eOk).	
	eMakeProxy	Use in dwgInFields() to force proxy.	
	Codes shared with AcMe::Status		
	eInternalUnsupported	(==AcMe::eNotSupported)	
	General Ami codes		
	eBad	Generic Ami badness	
	eInvalidArg	Invalid argument passed in.	
	eNoImpFunc	No implementation function available.	
	eNoImplementation	Called function does not work with arg type.	
	eOutOfMemory	Out of memory.	
	eNullKey	Key not set.	

Enum Type	Category/Enum	Additional Information	Header File
	eObjectNotEditable	Object cannot be edited.	
	eNoKeySupported	Key creation not supported for a given argument.	
	eGKeyNotSupported	Geom key creation not supported for object.	
	eKeyCreationFailure	Key could not be created.	
	eNotGeom	Implied object is not geometry.	
	eTypeMapFailure	Failure in type mapping.	
	eNotApplicable		
	Components/Constraints		
	eNoConstraints		
	eNoChildren		
	eExternalComponentNotFound		
	Names		
	eInvalidName	Used to be eInvalidComponentName.	
	eNameAlreadyExists	Used to be eComponentNameAlreadyExists	
	Picking Status		
	eUserTermination	User termination.	
	eUserEscape	User-requested cancel.	
	Instantiable Attributes		
	eNoClassRecord	No definition exists for attribute.	
	eClassRecordAlreadyExists	Attribute of the same name already registered.	
	eClassRecordInDb	Attribute of type exists in the current database.	
	eFieldIndexOutOfRange	Field index is out of range.	
	eInvalidFieldName	Field by given name not defined for attribute.	
	eInvalidFieldType	Field type not defined for attribute.	
	eAttributeReadError	Error reading attribute from filer.	
	Sketch Constraints		
	eInvalidSketchConstrType		
	Parameters		
	eParamAlreadyExists		
	eExprEvalFailure		
	eIllegalExpression		
	eParamNotFound		
	eCyclicDependency	Parameter references, at least indirectly, a parameter which uses it.	
	Feature Descriptors		
	eUnsupportedFeatType	Feature type not supported by feature API.	
	eNoCombiner	Feature contains no combiner operation.	
	eNoLocator	Feature contains no locator.	
	eNoTerminator	Feature contains no terminator.	
	Component Containment		
	eUnmatchedLeafObject	The component specifies a leaf object (e.g. part) which does not fit or match the leaf required	
	eNoContainingComp		
	More feature API		
	ePartNotActive	Specified part is not the active part.	
	eAmbiguousPartInfo	Ambiguous part information specified for combine.	

Enum Type	Category/Enum	Additional Information	Header File
	eNoPartInstance	Specified part has no instance.	
	Descriptors		
	eAttReadOnly	Attempted update of read-only attribute.	
	eAttTypeMismatch	Type mismatch on attributes.	
	eAttValueNAV	NAV: "Not a Value"	
	eIllegalDescriptor	Descriptor values inconsistent (wrt type).	
	eAttMultiValued	Multi-valued attribute in single-value context.	
	eAttNoAppend	Cannot append to attribute value (not allowed).	
	DwgMgr		
	eBaseView	Illegal operation on base view (e.g. get parent).	
	eNoCutlineContext	Sectioning operation requires context for cutline.	
	Constraint Creation		
	eInvalidConstrType	Invalid or unsupported constraint type.	
	eArgErased	Key to erased data passed.	
	eBadItemType	Argument geometry type not supported for constraint type.	
	eGeomValidFailure	Geometric validation failed.	
	eNoBaseOrdinate	No base ordinate defined on args.	
	eNoConstrainableGeom	No sketch geometry passed in.	
	eNoCommonSketch	Geometry from different sketches.	
	eNoCommonPart	Geometry from different parts.	
	eCoincidentGeom	Keys point to same geometry.	
	eModelOverConstrained	Constraint would make sketch over-constrained.	
	eModelInconsistent	Constraint inconsistent with existing constraints.	
	eConstraintExists	Constraint already exists.	
	eInternalFailure	Internal failure building constraint.	
	New general Ami codes		
	eObjectIsProxy		
	eInvalidDesignMode		
	eComputeFailure		
SweepMethod	kParallel	Sweep method enumeration.	mifeat.h
	kNormal		
TerminatorType: Terminator type enumeration.	kBlindTerm		mifeat.h
	kThroughTerm		
	kToPlaneTerm		
	kToFaceTerm		
	kFromToTerm		
	kMidPlaneTerm		
	kInsideTerm		
	kOutsideTerm		
VersionConfig	kVersionsAcrossRows	In the table, versions run across rows.	mifile.h
	kVersionsDownColumns	Versions run down columns.	
ViewAttribute	vwAttViewType	General attribute.	midm.h
	vwAttScale	General attribute.	
	vwAttDisplayType	General attribute.	
	vwAttLocation	General attribute.	
	vwAttLabel	General attribute.	
	vwAttHatchOnFlag	General attribute.	
	vwAttAHLDisplayOn	General attribute.	

Enum Type	Category/Enum	Additional Information	Header File
	vwAttHatchOnFlag	General attribute.	
	vwAttAHLDisplayOn	General attribute. Ignored if display type is wireframe.	
	vwAttHatchAHL	General attribute. Ignored if not sectioned and hatched.	
	vwAttModelEnts	General attribute.	
	vwAttExtents	General attribute.	
	vwAttGap	Broken view attributes.	
	vwAttOrientnPlane	Base view attributes.	
	vwAttOrientnAxis	Base view attributes	
	vwAttOrientnAxisDir	Base view attributes	
	vwAttScaleType	Child view attributes	
	vwAttParent	Child view attributes	
	vwAttOrientation	Child view attributes	
	vwAttRefPoint1	Auxiliary and detail view attributes. Model space geom ref.	
	vwAttRefPoint2	Auxiliary and detail view attributes. Model space geom ref.	
	vwAttRefBoxUL	Auxiliary and detail view attributes.	
	vwAttRefBoxLR	Auxiliary and detail view attributes.	
	vwAttSection	Section view attributes.	
	vwAttSectionType	Section view attributes.	
	vwAttSectionLabel	Section view attributes.	
	vwAttSectionCut	Section view attributes.	
	vwAttSectionCutDir	Section view attributes.	
	vwAttShowParDims	View display attributes	
	vwAttShowTapLines	View display attributes	
ViewDispType	kWireFrame		midm.h
	kHiddenTangDisp		
	kHiddenNoTangDisp		
ViewScaleType	kVwSclAbs	Scale factor is absolute.	midm.h
	kVwSclRel	Scale factor is relative to parent.	
ViewType	kBaseVw		midm.h
	kBrokenVw		
	kOrthoVw		
	kAuxVw		
	kIsoVw		
	kDetVw		
	kGenericVw		

Reference:
Glossary

Active (leaf node, part)
An active object is the object currently designated for editing. The active leaf node is the selected component through which changes are made to a specific part. The active part is the definition that is modified by changes made to the active leaf node. Changes to the active part are reflected in all instances of that part.

Active Notification
A mechanism that calls a user-registered function when a monitored object is changed or erased. Active notification is used by applications that need immediate notice of changes in the entity. It allows applications to update themselves as the changes occur. It is typically used when

- Little work is required in updating.
- Updates are relevant to the user causing the change.
- Applications have all the information available at the time of the notification, to be able to update.

API
Acronym for Application Programming Interface. A set of tools that provide programmatic access to the functionality in an application.

Associativity (Attributes)
Indicates whether the attribute should be copied, shared or ignored when the object to which it is attached is copied within a database or across databases.

Atomic Locator
A piece of information that describes a discrete aspect of how a feature was located. For example, a hole located at a distance from two edges would contain a single locator made up of two atomic locators. Each of those would contain information about a single edge and a single distance value or parameter determining the value.

Attribute
Application data associated with a particular entity. This user-defined object is conceptually attached to an application object being referred to by a key and can be retrieved given a key to the application object. It could be meant to be private to the functioning of the application, or data which could be meaningfully shared with other applications.

Attribute Class
A runtime description of a particular class of instantiable attribute.

B-rep
A boundary representation of an object used to represent solids, regions, and other ACIS bodies in AutoCAD, that encapsulates topological information on how different parts of the object are related to one another. B-rep objects are represented by their boundaries, consisting of faces, edges, and vertices.

B-rep API
An API that allows access to the internal structure of the B-rep object. The B-rep API allows applications to traverse through the B-rep and obtain adjacency information. Examples of the use of the B-rep API include obtaining the faces of an edge, and obtaining the loops of edges in a face. See the B-rep API documentation for more information.

Derived Attribute
An instance of a C++ class which derives from AmiAttribute. These attributes may be attached through API keys to entities. They become proxy objects if the application that defined the class is not loaded.

Feature Descriptor
An object containing all the information used in the construction of a feature. Feature descriptors are used to get feature information as well as to create new features.

Filer
An object that manages an I/O stream. Objects in the AutoCAD database, including those relevant to the MCAD API, are written to and from files using filers. The filers are passed in as arguments to the relevant object member functions and encapsulate information about the file that is being used. As well as providing file access, filers are also used for access to database objects, such as by ads_entget. See the *ARX Developer's Guide* for more information.

GeLib
Same as Geometry Library.

Geometry Key
Applications reference entities within the MCAD API using keys. A geometry key is used to reference geometric objects, such as surfaces, curves, and points.

Ghost
A ghost geometric object is an object that has been erased and whose geometry can still be re-created from a key to the object.

Inferred Geometry
A geometry reference that is implied by the entity selected, and not a direct reference to specific geometry. For instance, if a point reference is desired and a circle is selected, the center point of the circle is inferred and referenced through the point key.

Informer
A piece of information common to all feature descriptors describing how the feature was located or terminated.

Instantiable Attribute
An attribute whose data layout is defined at runtime via an attribute description or "class" definition. Since the data layout is saved to the database with the attributes, the attributes can be shared between applications without the creating application being loaded. Also application developers do not need to implement all the methods associated with new persistent classes.

Key
Applications reference entities within the MCAD API using keys. A key is a persistent referencing object that allows application objects to be referenced from a third-party application. It also protects the application from changes in the object. For example, a key, constructed for a face in a Mechanical Desktop part, will remain valid even though a change in a parameter might cause the solid to be regenerated.

Key Copying
The process that, given a key, creates a second key that resolves to the same entity as the first.

Leaf Node
A component definition is a leaf node if it is not a subassembly. A component is a leaf node if its definition is a leaf node.

Locator
A feature informer which specifically describes how a feature was located. A feature locator is made up of one or more atomic locators.

Notification
A mechanism that allows a user to be informed of changes and/or deletions to an object being referred to by a key. Applications using the MCAD API will typically need to reference entities within AutoCAD or the Autodesk Mechanical Desktop. For example, an NC toolpath might reference the faces that it relates to, or a finite element load might be associated with a particular edge. As the referenced entities change, the application may need to know that the entity has changed. This is called notification and is either passive or active.

Object Key
The instantiable base class for all keys. An object key can be used to refer to any application object.

Owner (Attributes)
Indicates whether the database management of the attribute should be done by the API or by some other application. If the API does not "own" the attribute, the application must clone, save and wblock the attribute at the appropriate times, otherwise the attribute will not follow the associativity type used to attach it, and may not be written to the database.

Passive Notification
Used for applications that want to know whether or not the entity referred to by a key has changed at their convenience. Applications would typically use passive notification if they did not need to react immediately to changes in the entity. This might be because

- A large amount of work is done in reacting to the change.
- Changes in the application would not be relevant to the user making the change.
- The application relies on other changes to take place before it can make its updates.
- Changes are likely to be repeated and will result in unnecessary updates.

Pick Object
An object that encapsulates all the information needed to create a key to an object.

Refocus
Refocusing changes a reference from one containing object to the equivalent geometry or feature on a different containing object. This allows one to obtain from a given key another key to the equivalent object in the context of another component or of the defining part.

Scope
A key implicitly has a scope in which it was defined. The scope corresponds to the portion of the assembly structure used in the definition of the key.

Terminator
A feature informer which specifically describes how a feature was terminated. For example a terminator could indicate that a given extrusion was terminated to a given model face.

Unused Sketch
An unused sketch is a profile which has not been consumed by a feature.